CS 6200 — Georgia Tech OMSCS

Graduate Introduction to Operating Systems

Lecture Notes

kinginu

Fall 2026

Contents

1 Introduction to Operating Systems 1
- 1.1 What is an operating system? 1
- 1.2 Abstractions, mechanisms and policies 2
- 1.3 Design principles 3
- 1.4 The user/kernel protection boundary 3
- 1.5 System calls 4
- 1.6 Operating system services 5
- 1.7 Organization of an operating system 6
- 1.8 Linux and Mac OS X 7

2 Processes and Process Management 9
- 2.1 Programs and processes 9
- 2.2 The address space of a process 9
- 2.3 Execution state and the process control block 10
- 2.4 Context switch 11
- 2.5 Process life cycle 12
- 2.6 Process creation 13
- 2.7 The CPU scheduler 14
- 2.8 Inter-process communication 16

3 Threads and Concurrency 18
- 3.1 Processes and threads 18
- 3.2 Why threads are useful 18
- 3.3 Basic thread mechanisms 20
- 3.4 Mutual exclusion 21
- 3.5 Condition variables 22
- 3.6 The readers-writers problem 23
- 3.7 Common pitfalls and spurious wake-ups 25
- 3.8 Deadlocks 26
- 3.9 Kernel-level and user-level threads 27
- 3.10 Multithreading patterns 28

4 Threads Case Study: PThreads 31
- 4.1 POSIX threads 31
- 4.2 Thread creation 31
- 4.3 Detached threads 32
- 4.4 Compiling Pthreads programs 34
- 4.5 Thread creation examples 34
- 4.6 Mutexes 36
- 4.7 Condition variables 37
- 4.8 A producer-consumer example 38

5 Thread Design Considerations 41
- 5.1 Thread data structures 41
- 5.2 Threads in SunOS 5.0 42
- 5.3 Interaction between the library and the kernel 44
- 5.4 Implementation issues 45
- 5.5 Interrupts and signals 46
- 5.6 Handling interrupts and signals 46
- 5.7 Disabling interrupts and signals 48
- 5.8 Interrupts as threads 49
- 5.9 Threads and signal handling 50
- 5.10 Tasks in Linux 51

6 Thread Performance Considerations 54
- 6.1 Comparing threading models 54
- 6.2 Usefulness of threads 55
- 6.3 Performance metrics 55
- 6.4 Multi-process and multithreaded servers 56

6.5 The event-driven model . . . 57
6.6 Blocking operations and helpers . . . 59
6.7 The Flash web server . . . 60
6.8 The Apache web server . . . 61
6.9 Experimental methodology . . . 62
6.10 Experimental results . . . 63
6.11 Designing and running experiments . . . 64

7 Scheduling **66**
7.1 Overview of CPU scheduling . . . 66
7.2 Run-to-completion scheduling . . . 66
7.3 Preemptive scheduling . . . 67
7.4 Priority inversion . . . 68
7.5 Round robin scheduling and timesharing . . . 69
7.6 Choosing the timeslice length . . . 70
7.7 Multi-level feedback queues . . . 72
7.8 The Linux O(1) scheduler . . . 73
7.9 The Linux Completely Fair Scheduler . . . 74
7.10 Scheduling on multiprocessors . . . 75
7.11 Hyperthreading . . . 76
7.12 Scheduling for hyperthreaded platforms . . . 77
7.13 Hardware performance counters . . . 78
7.14 Cycles per instruction as a scheduling metric . . . 78

8 Memory Management **80**
8.1 Goals of memory management . . . 80
8.2 Hardware support . . . 81
8.3 Page tables . . . 81
8.4 Page table entries . . . 83
8.5 Page table size . . . 84
8.6 Multi-level page tables . . . 84
8.7 Speeding up translation: the TLB . . . 85
8.8 Inverted page tables . . . 85
8.9 Segmentation . . . 86
8.10 Page size . . . 86
8.11 Memory allocation . . . 87
8.12 Linux kernel allocators . . . 88
8.13 Demand paging . . . 90
8.14 Page replacement . . . 91
8.15 Copy-on-write . . . 91
8.16 Checkpointing . . . 92

9 Inter-Process Communication **94**
9.1 Inter-process communication . . . 94
9.2 Message-based IPC . . . 94
9.3 Forms of message passing . . . 95
9.4 Shared-memory IPC . . . 96
9.5 Copy versus map . . . 97
9.6 System V shared memory . . . 97
9.7 POSIX shared memory . . . 99
9.8 Shared memory and synchronization . . . 99
9.9 IPC command-line tools . . . 101
9.10 Designing communication over shared memory . . . 101

10 Synchronization Constructs **103**
10.1 Beyond mutexes and condition variables . . . 103
10.2 Spinlocks . . . 103
10.3 Semaphores . . . 104
10.4 Reader-writer locks . . . 105
10.5 Monitors . . . 106
10.6 Other synchronization constructs . . . 107
10.7 Hardware support for synchronization . . . 109

10.8 Shared memory multiprocessors and caches . . . 110
10.9 Spinlock performance . . . 111

11 I/O Management 115
11.1 I/O devices . . . 115
11.2 CPU–device interconnect . . . 115
11.3 Device drivers . . . 116
11.4 Types of devices . . . 116
11.5 CPU–device interactions . . . 117
11.6 Device access: programmed I/O and DMA . . . 118
11.7 Typical device access . . . 119
11.8 Synchronous and asynchronous access . . . 120
11.9 Block device stack . . . 121
11.10 The virtual file system . . . 122
11.11 VFS abstractions . . . 122
11.12 The second extended file system . . . 123
11.13 Inodes . . . 124
11.14 Disk access optimizations . . . 125

12 Virtualization 128
12.1 What is virtualization? . . . 128
12.2 Benefits of virtualization . . . 129
12.3 Virtualization models . . . 129
12.4 Hardware protection levels . . . 131
12.5 Processor virtualization . . . 131
12.6 x86 virtualization before hardware support . . . 132
12.7 Binary translation . . . 132
12.8 Paravirtualization . . . 133
12.9 Memory virtualization . . . 134
12.10 Device virtualization . . . 135
12.11 Hardware support for virtualization . . . 137

13 Remote Procedure Calls 139
13.1 Why remote procedure calls? . . . 139
13.2 RPC requirements . . . 140
13.3 Structure of an RPC system . . . 140
13.4 Interface definition language . . . 141
13.5 Marshalling and unmarshalling . . . 142
13.6 Binding and registries . . . 142
13.7 Pointers as arguments . . . 143
13.8 Partial failures . . . 143
13.9 Sun RPC . . . 144
13.10 Compiling an interface with rpcgen . . . 145
13.11 Registry and binding in Sun RPC . . . 146
13.12 XDR data types and routines . . . 147
13.13 XDR encoding . . . 148
13.14 Java RMI . . . 148

14 Distributed File Systems 151
14.1 Distributed file systems . . . 151
14.2 Remote file service . . . 151
14.3 Stateless and stateful file servers . . . 153
14.4 Caching state in a DFS . . . 154
14.5 File sharing semantics . . . 155
14.6 Files and directories . . . 156
14.7 Replication and partitioning . . . 156
14.8 The Network File System . . . 157
14.9 NFS versions . . . 158
14.10 The Sprite distributed file system . . . 159
14.11 File access operations in Sprite . . . 160

15 Distributed Shared Memory 162
15.1 Peer distributed applications . . . 162
15.2 Distributed shared memory . . . 162
15.3 Hardware and software DSM . . . 163
15.4 Sharing granularity . . . 164
15.5 Access algorithms . . . 164
15.6 Migration and replication . . . 165
15.7 Consistency management . . . 166
15.8 DSM architecture . . . 167
15.9 Implementing DSM . . . 168
15.10 Consistency models . . . 170

16 Datacenter Technologies 173
16.1 Internet services . . . 173
16.2 Scale-out architectures . . . 173
16.3 The motivation for cloud computing . . . 176
16.4 Cloud computing . . . 177
16.5 Deployment and service models . . . 178
16.6 Requirements for the cloud . . . 179
16.7 Cloud enabling technologies . . . 180
16.8 The cloud as a big data engine . . . 181

Bibliography 183

Index 186

Lesson 1

Introduction to Operating Systems

Lecture	P1L2
Textbook	OSTEP [1] ch. 2, 6; Silberschatz et al. [2] ch. 1–2
Keywords	operating system, abstraction, arbitration, mechanism, policy, kernel mode, user mode, trap, system call, signal, monolithic operating system, modular operating system, microkernel

Every application depends on an operating system to use the CPUs, the memory and the devices of the machine on which it runs. This lesson defines what an operating system is and which roles it plays, introduces the elements from which it is built—abstractions, mechanisms and policies—and the principles that guide their design. It then examines the protection boundary between applications and the operating system, which applications cross by means of system calls, and compares the ways in which operating systems are organized, with Linux and Mac OS X as examples. The abstractions introduced here, such as processes, threads, memory pages and files, are the subject of the lessons that follow.

1.1 What is an operating system?

A computer system consists of hardware—one or more CPUs, each possibly with several cores, main memory, storage devices such as hard disks, SSDs and USB flash drives, network interfaces for Ethernet or WiFi, and graphics processors (GPUs)—and of the applications that use it. Several applications usually run at the same time, and all of them need these hardware components. The operating system is the layer of software that sits between the two (Figure 1.1).

Source P1L2, what is an operating system, operating system definition; OSTEP ch. 2; Silberschatz et al. ch. 1.

Definition 1.1 (Operating system). An *operating system* (OS) is a layer of systems software that

- has direct, privileged access to the underlying hardware;
- hides the complexity of the hardware from applications;
- manages the hardware on behalf of one or more applications according to predefined policies; and
- ensures that applications are isolated and protected from one another.

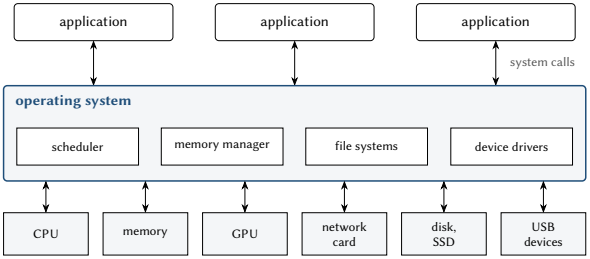


Figure 1.1: The operating system between applications and hardware. Applications request services through system calls; components such as the scheduler, the memory manager, file systems and device drivers carry them out on the hardware.

In short, an operating system *abstracts* the hardware, simplifying what it looks like to applications, and *arbitrates* its use, overseeing and controlling how applications share it. The definition names three roles.

Hiding hardware complexity The operating system presents the hardware to applications through an *abstraction*: a simpler, higher-level interface that hides the details of the devices behind it. An application that stores data does not address the sectors and blocks of a particular disk; it writes to a *file*, and the operating system translates file operations into operations on whatever storage device is present. In the same way a *socket* represents network communication independently of the network interface. Because

One file abstraction spans devices that differ by orders of magnitude: a flash page read takes 25 µs to 75 µs (OSTEP ch. 44), a disk seek 4 ms to 9 ms (OSTEP ch. 37).

the abstraction stays the same when the device changes, applications run unchanged on different hardware.

Resource management The operating system manages the hardware resources by *arbitration*: it decides which application uses which resource and how much of it, allocating memory to applications, scheduling them on the CPUs and controlling their access to devices. The decisions follow policies, for example that access to resources be fair or that the resources used by one application be limited.

Isolation and protection Applications that run at the same time must not interfere with one another. The operating system provides *isolation* by giving each application its own share of the resources, for example its own portion of physical memory, so that its execution is not affected by the others. It provides *protection* by preventing an application from accessing resources that have not been assigned to it, such as the memory of other applications or of the operating system itself. Isolation thus limits the effect of one application on another, while protection limits what an application may touch.

Caution "Operating system" is used loosely: strictly it is the kernel, the part that runs in kernel mode (Section 1.4); a distribution such as Ubuntu also ships the shell, libraries and utilities (Silberschatz et al. ch. 1).

The operating system thus consists of components such as the CPU scheduler, the memory manager, device drivers and file systems. Application programs that merely use these services, such as a spreadsheet or a media player, are not part of it, even when they are distributed together with it.

Example 1.2. A function of the operating system is an abstraction when it simplifies the interface to a device, and arbitration when it divides a resource among applications. Letting a program send a document to a printer without knowing the command language of the printer's model is abstraction; deciding in which order the documents of three programs that print at the same time reach the printer is arbitration. Likewise, offering the same socket interface over Ethernet and over WiFi is abstraction, while limiting the network bandwidth that one application may consume is arbitration.

1.1.1 Examples of operating systems

Source P1L2, operating system examples.

Operating systems differ with the environment they are designed for. On desktop and server machines the main examples are Microsoft Windows and the UNIX-based systems, among them Mac OS X, which derives from BSD UNIX, and Linux. On embedded devices such as smartphones they include Android, which is based on Linux, iOS and Symbian. Each makes design choices suited to its environment. A desktop system favors the responsiveness of the interactive applications of a single user. An embedded system must work within a small memory and a limited battery, often under real-time deadlines. A server system must serve many users and requests with high throughput and must keep running for long periods. The lessons that follow use Linux as their main example.

Android runs the Linux kernel under its own user space—the Bionic C library and the ART runtime—instead of the GNU tools (`source.android.com`).

1.2 Abstractions, mechanisms and policies

Source P1L2, OS elements, OS elements example; Silberschatz et al. ch. 2.

An operating system is built from three kinds of elements. Its abstractions represent executing applications—the process and the thread—and hardware resources—the file, the socket and the memory page. On these abstractions it provides operations, and it decides how the operations are applied.

Definition 1.3 (Mechanism and policy). A *mechanism* is an operation that the operating system provides on its abstractions, such as creating or scheduling a process, opening or writing a file, or allocating memory. A *policy* is a rule that determines how the mechanisms are used to manage the hardware, such as least recently used (LRU) for memory or earliest deadline first (EDF) for scheduling.

On x86-64 a page is 4 KiB by default; huge pages of 2 MiB and 1 GiB also exist. Lesson 8.

Tip To tell a policy from a mechanism, ask whether it could be replaced without changing the code that does the work: LRU could give way to LFU; moving a page to disk is that code.

Memory management shows how the three elements work together (Table 1.1). The abstraction is the memory page, a fixed-size block of memory. The mechanisms allocate a page and map it into the address space of a process, so that the process can use it. Physical memory (DRAM) is usually smaller than the total demand of all processes, so the operating system also moves some pages to disk and brings them back when they are accessed again. The policy decides which pages to move: with LRU, the pages that have not been accessed for the longest time, on the assumption that they are the least likely to be needed soon.

Table 1.1: The elements of an operating system, in general and for memory management.

Element	Examples	Memory management
Abstraction	process, thread, file, socket, memory page	memory page
Mechanism	create, schedule, open, write, allocate	allocate a page, map it into a process
Policy	LRU, EDF	LRU: move the least recently used pages to disk

1.3 Design principles

Two principles guide the design of these elements. The first is the *separation of mechanism and policy*: the operating system implements flexible mechanisms that can support many policies, so that a policy can be chosen or replaced without changing the mechanisms. For choosing the pages to move to disk, the same mechanisms—recording page accesses, moving a page to disk and updating its mapping—support LRU, least frequently used (LFU) and even random selection. Which of them works best depends on the setting in which the system is used.

Source P1L2, OS design principles; Silberschatz et al. ch. 2.

Example 1.4. Physical memory holds four pages, and a fifth page is needed, so one of the four must move to disk. At time 160 ms the recorded accesses are:

Page	A	B	C	D
Last access (ms)	120	40	95	150
Number of accesses	9	25	3	12

LRU moves page B, whose last access lies furthest in the past; LFU moves page C, which has been accessed least often; random selection moves each page with probability 1/4. In all three cases the mechanism executed is the same: the chosen page is written to disk if it has been modified, and its mapping is invalidated.

The second principle is to *optimize for the common case*. The designers must understand where the operating system will be used, which applications its users will run on the machine, and what the requirements of that workload are. They then choose the policies that make sense for this common case and that the underlying abstractions and mechanisms can support. A server that mostly runs a database, for example, performs many random reads and writes, whereas a media server reads large files sequentially, and the policies that suit one workload are a poor fit for the other.

> **Key point**
> Abstractions simplify, mechanisms operate on the abstractions, and policies decide how the mechanisms are used. Keeping mechanisms general lets the same operating system adopt the policies that fit its common case.

1.4 The user/kernel protection boundary

Because the operating system needs direct access to the hardware, it must execute with privileges that applications do not have. Computer platforms therefore distinguish at least two modes of execution.

Source P1L2, user/kernel protection boundary; OSTEP ch. 6; Silberschatz et al. ch. 1.

Definition 1.5 (Kernel mode and user mode). In *kernel mode*, the privileged mode, software may execute every instruction of the processor and access the hardware directly. In *user mode*, the unprivileged mode, instructions that access the hardware directly are forbidden.

The part of the operating system that executes in kernel mode is its *kernel*; applications execute in user mode. The distinction between the two modes is supported by the hardware. The CPU contains a *mode bit* that records the current mode, and an instruction that directly manipulates hardware—a *privileged instruction*—is executed only if the mode bit indicates kernel mode. An attempt to execute a privileged instruction in user mode does not take effect but causes a trap.

Definition 1.6 (Trap). A *trap* is a transfer of control from a running application to the operating system, performed by the hardware: the application is interrupted, the CPU switches to kernel mode, and execution continues at a location in the kernel that the operating system has registered in advance.

x86 processors provide four privilege levels, the protection rings of Lesson 12, and operating systems use at least two of them; ARM calls its modes exception levels EL0–EL3; RISC-V has user, supervisor and machine modes (Arm Architecture Reference Manual; RISC-V Privileged Specification).

A trap is synchronous: it is caused by the instruction that is executing. An interrupt, by

contrast, is raised by a device independently of the running instruction; both transfer control through a table of handler addresses that the operating system registers in advance.

After a trap caused by a privileged instruction or by an access to memory that requires privileges, the operating system determines what caused the trap and whether the access should be granted; if the application attempted something illegal, the operating system may terminate it. Applications

Caution A trap is not an error: every system call is a deliberate trap; only an illegal instruction or memory access leads to termination.

and the operating system interact in three ways (Figure 1.2):

Traps transfer control to the operating system when an application performs an operation that requires privileges.

System calls are requested explicitly. A *system call* is an operation that the operating system exports so that applications can ask it to perform a service or a privileged access on their behalf, for example `open` for a file, `send` for a socket or `mmap` for memory. The set of these operations is the *system call interface*.

Signals travel in the opposite direction. A *signal* is a notification that the operating system passes to an application, for example that a timer the application has set has expired.

The kernel checks for pending signals whenever it returns to user mode; since an interrupt can cause such a return at any point, a handler may run between any two instructions. Signals: Lesson 5.

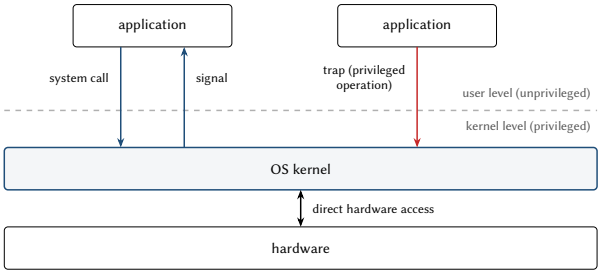


Figure 1.2: The user/kernel protection boundary. Only the kernel accesses the hardware; applications enter the kernel through system calls or traps, and the kernel notifies applications with signals.

1.5 System calls

Source P1L2, system call flowchart; OSTEP ch. 6; Silberschatz et al. ch. 1–2.

Figure 1.3 shows the sequence of events when a user process executes a system call. The process runs in user mode until it needs a service of the operating system and calls the system call. The call traps into the kernel: the mode bit is switched to kernel mode, the execution context changes from that of the user process to that of the kernel, the arguments are passed, and execution jumps to the kernel code that implements the call. The kernel executes the call in kernel mode. When it has completed, the mode bit is switched back to user mode, the execution context of the process is restored, the results are passed back, and the process continues with the instruction after the call.

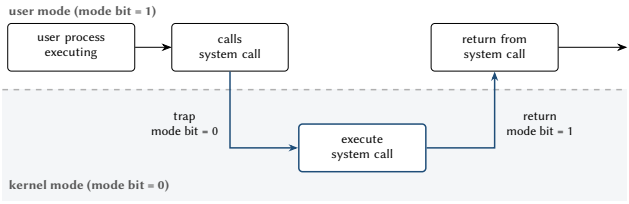


Figure 1.3: Execution of a system call: a trap switches to kernel mode, the kernel executes the call, and the return switches back to user mode. The mode bit follows the convention of Silberschatz et al. [2]: 0 in kernel mode, 1 in user mode.

To make a system call, an application writes the arguments and other relevant data to a well-defined location and invokes the call by its *system call number*. From the number the kernel determines which operation is requested, how many arguments it takes and where to find them. Arguments are passed either directly, as values, or indirectly, as the address of a memory area that holds them. Because such an address may point anywhere, the kernel checks

that it lies in the memory of the calling process before using it; isolation depends on this check. The result of the call, or an error code, is returned in a register. In the synchronous mode described here the process waits until the system call has completed; asynchronous system calls are discussed in later lessons.

Not every call crosses: Linux maps the vDSO into every process, so `clock_gettime` runs in user mode. `vdso(7)`.

Example 1.7. Both calls in the following C program ask Linux to write six bytes to the standard output:

```
#define _GNU_SOURCE
#include <unistd.h>
#include <sys/syscall.h>

int main(void) {
  const char msg[] = "hello\n";
  /* C library wrapper for the system call */
  write(1, msg, sizeof msg - 1);
  /* the same system call, invoked by number */
  syscall(SYS_write, 1, msg, sizeof msg - 1);
  return 0;
}
```

The library function `write` places the number of the `write` system call and its three arguments where the kernel expects them and executes the trap. The file descriptor 1 (standard output) and the length are passed directly; the text is passed indirectly, through its address `msg`.

x86-64 Linux: number in `rax`, arguments in `rdi`, `rsi`, `rdx`; the `syscall` instruction traps.

1.5.1 Crossing the user/kernel boundary

Transitions between user mode and kernel mode are a necessary part of application execution: an application that reads a file, sends data over the network or allocates memory must cross the boundary. The transitions have three properties.

- They are supported by the hardware, through traps on illegal instructions or on memory accesses that require special privileges.
- They require executing a number of instructions to save and switch state; on a 2 GHz machine running Linux a transition takes about 50 ns to 100 ns.[1]
- They switch locality. The kernel executes different code and uses different data than the application, so its instructions replace application data in the processor caches. When the application resumes, part of its data must be fetched from main memory again, which slows it down. The caches know nothing of the boundary: a mode switch by itself flushes nothing, and the application loses its data only because the kernel's accesses displace them. A mode switch keeps the same process and address space; a context switch, which changes the process that runs on the CPU (Lesson 2), also replaces the address space and usually loses more of the cached state.

User/kernel transitions are therefore not cheap.

Source P1L2, crossing the OS boundary.

[1] On a Core i7, entering and leaving the kernel cost 79 and 71 cycles, but a `pwrite` also evicts half of the 32 KiB L1 data cache, and the application needs up to 14 000 cycles to recover [3].

Caution A system call looks like a procedure call in C but costs far more.

1.6 Operating system services

An operating system gives applications access to the hardware by exporting services. At the most basic level the services correspond to hardware components: a scheduler controls access to the CPUs; a memory manager allocates physical memory to the applications that run at the same time and makes sure that they do not overwrite each other's memory; a block device driver gives access to block devices such as disks. Other services provide higher-level abstractions that do not correspond directly to one hardware component, such as the files and directories of a file system. Grouped by purpose, operating systems offer process management, file management, device management, memory management, storage management and security.

Source P1L2, OS services; Silberschatz et al. ch. 2.

Applications reach all of these services through system calls. The scheduler is reached through calls such as `fork`, which creates a process, and `sched_yield`, which gives up the CPU; the memory manager through `mmap` and `brk`; the file system through `open` and `close`; and the block device driver only indirectly, through `read` and `write` on a file. Different operating systems name and structure their calls differently, but they provide comparable functionality, as the Windows and Unix calls in Table 1.2 show.

Process management: Lesson 2; threads: Lesson 3; scheduling: Lesson 7; memory: Lesson 8; I/O and file systems: Lesson 11; distributed file systems: Lesson 14.
Linux 7.0 on x86-64 numbers 386 system calls (`unistd_64.h`); the C library wraps them in a much larger API.

Table 1.2: Examples of Windows and Unix system calls (after Silberschatz et al. [2]).

Category	Windows	Unix
Process control	`CreateProcess()`, `ExitProcess()`, `WaitForSingleObject()`	`fork()`, `exit()`, `wait()`
File manipulation	`CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseHandle()`	`open()`, `read()`, `write()`, `close()`
Device manipulation	`SetConsoleMode()`, `ReadConsole()`, `WriteConsole()`	`ioctl()`, `read()`, `write()`
Information maintenance	`GetCurrentProcessId()`, `SetTimer()`, `Sleep()`	`getpid()`, `alarm()`, `sleep()`
Communication	`CreatePipe()`, `CreateFileMapping()`, `MapViewOfFile()`	`pipe()`, `shm_open()`, `mmap()`
Protection	`SetFileSecurity()`, `InitializeSecurityDescriptor()`, `SetSecurityDescriptorGroup()`	`chmod()`, `umask()`, `chown()`

1.7 Organization of an operating system

Source P1L2, monolithic OS, modular OS, microkernel; Silberschatz et al. ch. 2.

Operating systems differ in how their services are organized. Figure 1.4 shows three basic organizations, and Table 1.3 summarizes their advantages and disadvantages.

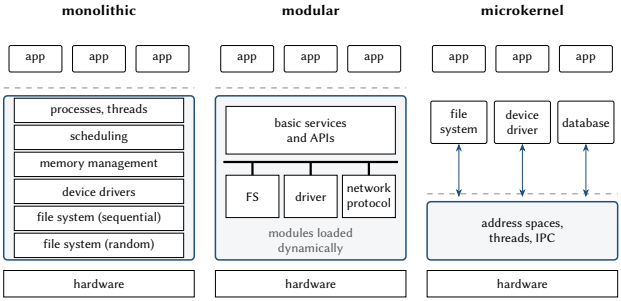


Figure 1.4: Three organizations of an operating system. The dashed line is the user/kernel boundary. A monolithic kernel contains every service; a modular kernel adds services as modules through a common interface; a microkernel keeps only address spaces, threads and IPC, and runs all other services at user level.

1.7.1 Monolithic operating systems

Historically, operating systems were monolithic. In a *monolithic operating system* every service that any application may require and that any type of hardware may demand is part of the operating system: memory management, device drivers, file management, processes and threads, scheduling. Such a system may even include several file systems, one specialized for sequential access and another optimized for random access, as used by databases. Because all of this code is compiled together, the compiler can optimize across the boundaries between components, for example by inlining. The price is a large amount of code and state that is hard to customize, port, maintain, debug and upgrade, and a large memory footprint that reduces the memory left for applications and can therefore degrade their performance.

Silberschatz et al. describe Linux as a monolithic kernel with loadable modules.

1.7.2 Modular operating systems

A *modular operating system*, of which Linux is the standard example, contains a set of basic services and APIs, and everything else can be added as a *kernel module*. The operating system specifies interfaces that a module must implement to become part of it, for example an interface for file systems. Modules can be installed dynamically, according to the workload: a machine that runs a database can load a file system optimized for random access. Since only the modules that are needed are loaded, the system is smaller, easier to maintain and upgrade, and leaves more resources to applications. The indirection through the module interface can cost some performance, although usually not much, and modules may come from entirely different code bases, which makes them a source of bugs.

Tip On Linux `lsmod` lists the loaded modules, `modprobe` loads one, and `uname -r` prints the version of the running kernel.

The Linux source tree passed 40 million lines, comments and blank lines included, in January 2025 (Phoronix); a running kernel loads only a small part.

1.7.3 Microkernels

A *microkernel* provides only the most basic primitives at the operating-system level: address spaces, which represent the memory of executing applications, and threads, which represent their execution contexts. Everything else—file systems, device drivers, databases—runs outside the kernel, at user level, as separate processes.[2] Serving a request therefore involves many interactions between processes, and inter-process communication (IPC) is one of the core abstractions and mechanisms of the microkernel, together with address spaces and threads. A microkernel is small, so the kernel itself has a small memory footprint and, for some applications, lower overhead and better performance; its code is also easier to verify. This makes microkernels attractive for embedded devices and control systems, where correct behavior of the operating system is critical. On the other hand, microkernels are often specialized to the underlying hardware, which limits portability; software development is more complex, because common operating-system components are harder to find; and the frequent crossings of the user/kernel boundary are costly.[3]

[2] The idea goes back to Brinch Hansen's RC 4000 nucleus (1969); Mach (Carnegie Mellon, from 1985) made it widely known. QNX, MINIX 3 and seL4 are current microkernels, and seL4 has a machine-checked proof of functional correctness [4].

[3] How costly is a matter of design: Liedtke's L3 [5] transferred an 8-byte message between address spaces in 5.2 µs on a 50 MHz i486, where Mach needed 115 µs on the same processor; its successor L4 followed. Today seL4 reports 350 to 800 cycles per IPC call on Arm, x86-64 and RISC-V cores (`se14.systems/performance.html`).

Table 1.3: Advantages and disadvantages of the three organizations.

Organization	Advantages	Disadvantages
Monolithic	everything included; optimization across components	hard to customize, port and manage; large memory footprint
Modular	maintainable; smaller footprint; fewer resources needed	indirection through interfaces; bugs from separately developed modules
Microkernel	small kernel footprint; easier to verify	limited portability; complex software development; frequent user/kernel crossings

Example 1.8. Consider a read from a file on disk. In a monolithic or modular kernel the file system and the disk driver run inside the kernel, so the application makes one system call and the request crosses the user/kernel boundary twice, on entry and on return. In a microkernel the application sends an IPC message to the file-system process, which sends one to the driver process; the driver replies to the file system, which replies to the application. Each of the four messages enters the kernel and leaves it again, for eight crossings in total. Assuming 80 ns per crossing and 50 000 reads per second, the monolithic design spends $50\,000 \times 2 \times 80\,\text{ns} = 8\,\text{ms}$ of every second on crossings and the microkernel $50\,000 \times 8 \times 80\,\text{ns} = 32\,\text{ms}$, not counting the switches between the address spaces of the processes involved.

Key point
A monolithic kernel is complete and can be optimized as a whole but is large and hard to manage; a modular kernel adds only the services that are needed, at the cost of indirection; a microkernel is small and verifiable but pays for frequent user/kernel crossings and IPC.

1.8 Linux and Mac OS X

Source P1L2, Linux and Mac OS architectures; Tanenbaum and Bos [6] ch. 10; Silberschatz et al. ch. 2.

A Linux system is organized in layers (Figure 1.5, left). The bottom layer is the hardware. The Linux kernel runs in kernel mode directly on it. Above it, in user mode, the standard library provides the system call interface as functions such as `open`, `close`, `read`, `write` and `fork`; the standard utility programs, such as the shell, editors and compilers, use the library; and users interact with the utility programs and with their applications. The boundaries between these layers are the system call interface, the library interface and the user interface.

Caution `printf` and `malloc` are library functions, not system calls: they call `write` and `brk`/`mmap` only when they must.

The kernel itself (Figure 1.5, right) is entered through the system calls at the top, and at the bottom it handles interrupts and contains the dispatcher (Lesson 7), which switches the CPU between processes. In between lie three components. The I/O component contains the terminal and character device drivers, the sockets, network protocols and network device drivers, and the file systems and block device drivers. The memory management component implements virtual memory (Lesson 8), paging and page replacement, and the page cache (Lesson 11). The process management component handles signals (Lesson 5), creates and terminates processes and threads, and schedules the CPU. Each component can be modified or replaced independently of the others, which is what makes the modular approach possible.

Tip `strace ./prog` lists the system calls a program actually makes; `ltrace` does the same for library calls.

Since Linux 6.6 (2023) the scheduler is EEVDF, which replaced the CFS of Lesson 7 (kernel documentation, `sched-eevdf`).

Mac OS X (now macOS) is built around the Mach microkernel (Figure 1.6). Mach implements the key primitives: memory management, thread scheduling, and IPC, including remote procedure calls (Lesson 13). A BSD component provides Unix interoperability through the BSD command-line interface, support for the POSIX API and network I/O. All application

Mach, BSD, the I/O Kit and kernel extensions run in one kernel address space, so the design is hybrid rather than a pure microkernel (Silberschatz et al. ch. 2).

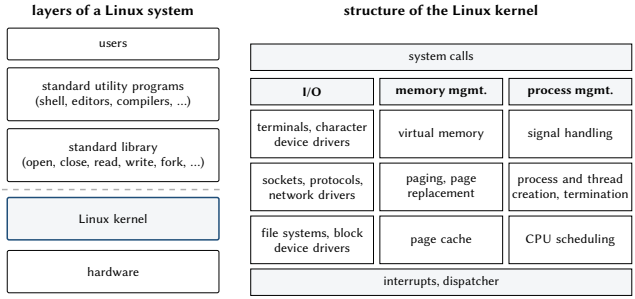


Figure 1.5: Left: the layers of a Linux system; the dashed line is the user/kernel boundary. Right: the components of the Linux kernel (after Tanenbaum and Bos [6]).

environments are layered above these two. Below them, the I/O Kit is an environment for developing device drivers, and kernel extensions are components that are loaded into the kernel dynamically.

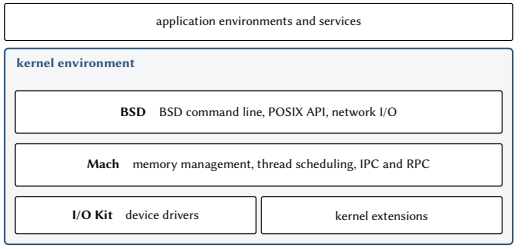


Figure 1.6: The structure of Mac OS X: the Mach microkernel with a BSD layer, the I/O Kit and kernel extensions, all of which run in kernel mode.

Key point — Lesson summary

An operating system is the privileged layer of software between applications and hardware. It hides hardware complexity behind abstractions, arbitrates the use of hardware resources, and isolates and protects applications from one another. It is built from abstractions, mechanisms and policies; separating mechanism from policy and optimizing for the common case guide its design. Hardware-supported user and kernel modes protect the operating system; applications cross the boundary through system calls and traps, which are not cheap, and the operating system notifies applications with signals. Monolithic, modular and microkernel organizations trade completeness, flexibility, size and performance against each other, as the modular Linux kernel and the Mach-based Mac OS X illustrate.

Lesson 2

Processes and Process Management

Lecture	P2L1
Textbook	OSTEP [1] ch. 4–7, 13; Silberschatz et al. [2] ch. 3
Keywords	process, address space, virtual address, page table, process control block, context switch, hot cache, process state, fork, exec, CPU scheduler, timeslice, interprocess communication, shared memory

Lesson 1 described the operating system as the layer that abstracts and arbitrates the hardware on behalf of applications. Its most fundamental abstraction is the process, the representation of an application while it executes. This lesson describes what a process is and what state it comprises, how the operating system represents that state, and how it manages many processes at once: it switches the CPU between them, creates them, schedules them, and lets them communicate with one another.

2.1 Programs and processes

An application stored on disk or in flash memory is a *program*: a static entity, a file of instructions and initial data that does nothing by itself. When the program is launched, the operating system loads it into memory and starts executing it, and it becomes an active entity.

Source P2L1, what is a process; OSTEP [1] ch. 4; Silberschatz et al. [2] ch. 3.

Definition 2.1 (Process). A *process* is an instance of an executing program: the state of a program while it executes, loaded in memory. A process is also called a *task* or a *job*.

Launching the same program twice creates two processes. They execute the same code but each has its own state, for instance different input data or a different position in the program. A process represents the execution state of an *active* application; this does not mean that the process is using the CPU at every moment. It may be waiting for user input, or another process may be running on the CPU while it waits for its turn. Besides its execution state and its data, a process may also require hardware, such as the I/O devices it reads from and writes to.

2.2 The address space of a process

A process encapsulates all the state of the running application: its code, its data, and all the variables that the application creates as it executes. Every element of this state must be uniquely identified by its address.

Source P2L1, what does a process look like, process address space; OSTEP ch. 4, 13.

Definition 2.2 (Address space). The *address space* of a process is the abstraction the operating system uses to encapsulate the state of the process: a range of addresses from V_0 to $V_{\max}$, together with the state stored at those addresses. It is the in-memory representation of the process.

The state in the address space is of several types, laid out as in Figure 2.1:

Text and data The *text segment* holds the code of the program, the instructions produced by the compiler. The *data segment* holds its static data, such as global variables initialized when the program is loaded. Both are static state: they are present from the moment the process is first loaded.

Heap The *heap* holds state that the process creates dynamically during its execution, for example memory allocated with `malloc`. It grows toward higher addresses.

Stack The *stack* is a dynamic part of the address space that grows and shrinks during execution in last-in first-out order. When the process calls a procedure, the state needed to resume the caller, the return address and saved register values, is pushed onto the stack, and the local variables of the called procedure are allocated there; when the procedure returns, all of it is popped again. The stack starts at the high end of the address space and grows toward lower addresses.

`cat /proc/self/maps` prints a live address space, one region per line with its start and end address, permissions and backing file (proc(5)). Default Linux main-thread stack limit: 8 MiB (`ulimit -s`); deeper recursion ends in a segmentation fault (getrlimit(2)).

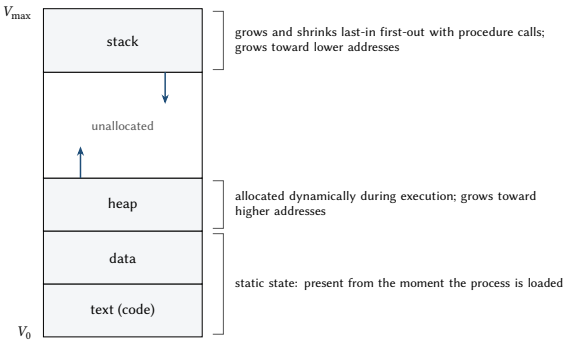


Figure 2.1: Address space of a process. The heap and the stack grow toward each other; the addresses between them are not allocated.

2.2.1 Virtual and physical addresses

Source P2L1, process address space; OSTEP ch. 13.

The addresses V_0 to V_{max} that a process uses need not correspond to actual locations in the physical memory of the machine.

Definition 2.3 (Virtual and physical address). A *virtual address* is an address in the address space of a process, as used by the process itself. A *physical address* is the address of a location in the physical memory (DRAM) of the machine.

The memory-management hardware and the operating system translate every virtual address that the process uses into a physical address. The mapping between the two is recorded in a *page table*, which the operating system maintains for every process. Decoupling the two kinds of address frees the layout of the address space from the layout of physical memory.

Example 2.4. A process stores a variable x at the virtual address `0x00402A10`. Its page table maps this address to the physical address `0x1F3A0A10`. Whenever the process reads or writes x, it uses `0x00402A10`; the translation to `0x1F3A0A10` happens without its involvement.

Tip Translation works on whole pages, 4 KiB on x86, so the low 12 bits of an address pass through it unchanged: both addresses here end in `A10`.

2.2.2 Address space and memory management

Source P2L1, address space and memory management.

The mapping gives the operating system freedom in how it uses physical memory, which it needs for two reasons. First, a process rarely uses its whole address space; large portions, such as the region between the heap and the stack, are not allocated at any given time. Second, there may not be enough physical memory for all the state that is allocated. With 32-bit virtual addresses a single process can address 2^{32} bytes, that is 4 GiB, and several processes together easily need more memory than the machine has.

On x86-64 the hardware translates 48-bit addresses and Linux gives each process the lower half of that range, 128 TiB; the five-level paging of recent processors extends the addresses to 57 bits (Linux *5-level paging* documentation).

The operating system therefore decides dynamically which portions of which address spaces reside where in physical memory. Some portions may temporarily be moved out to disk and brought back into memory when the process needs them (swapping; Lesson 8). For every process, the operating system must know where each part of its address space currently is, in memory or on disk; this is the information held in the page tables. The same information lets it validate memory accesses, that is, check that the process is actually allowed to access the address it uses.

Every process has its own address space, and the address spaces of different processes span the same range of virtual addresses. Two processes running at the same time can therefore use the very same virtual address, and their page tables map it to different physical locations (Figure 2.2). A virtual address identifies a location only together with the process that uses it.

Tip A 32-bit address with 4 KiB pages splits into a 20-bit page number and a 12-bit offset; only the page number is translated (Lesson 8).

2.3 Execution state and the process control block

Source P2L1, process execution state, what is a process control block; OSTEP ch. 4.

When the operating system stops a process, it must record what the process was doing so that it can later resume the process from exactly the same point. A program is compiled into a binary, a sequence of instructions that are not executed strictly in order: jumps, loops and interrupts change the sequence. At every moment the CPU must know where in this sequence

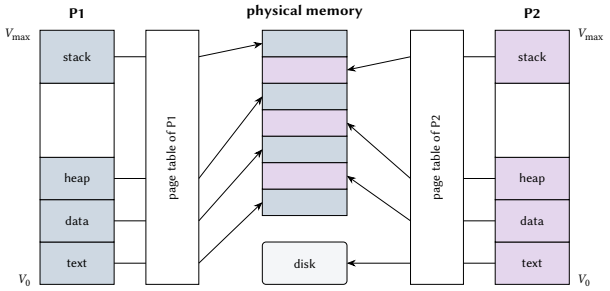


Figure 2.2: Two processes with the same range of virtual addresses. Their page tables map the allocated regions to different physical frames; the text of P2 is currently on disk, and the unallocated regions are not mapped at all.

the process is.

The *program counter* (PC) holds the address of the next instruction to execute; while the process runs, it is kept in a register of the CPU. Other CPU registers hold data values, addresses, and status information that affects the sequence of execution. The *stack pointer* (SP) holds the address of the top of the stack. Together these values form the execution state of the process.

Definition 2.5 (Process control block). A *process control block* (PCB) is a data structure that the operating system maintains for every process. It contains the process state, the process number, the program counter and the other CPU registers, the memory limits and the virtual-to-physical mappings, the list of open files, the priority, the signal mask, and CPU scheduling information.

In Linux the PCB is `struct task_struct`; among its fields are the process identifier (`pid`), the state, `mm`, which describes the memory mappings, and `files`, the table of open files.

The fields of the PCB (Figure 2.3) are maintained at different times:

- The PCB is created and initialized when the process is created; for example, the program counter is set to the first instruction of the program.
- Some fields are updated when the corresponding state changes. When the process requests more memory, the operating system allocates it and updates the memory limits and mappings.
- Other fields change far too often to be kept current in the PCB. The program counter changes with every instruction, so the CPU maintains it in its own register. The operating system collects the values that the CPU keeps for a process and saves them in the PCB whenever the process stops running on the CPU.

Caution One Linux `task_struct` describes one *thread* (Lesson 3). A process is a thread group, the tasks that share one thread-group ID, and its per-process state lives in structures they share: `mm_struct`, `files_struct`, `signal_struct`.

`/proc/PID/status` prints many of these fields: state, parent, number of threads, memory limits, signal masks. On x86-64 each task also owns a 16 KiB kernel stack (Linux *Kernel Stacks* documentation).

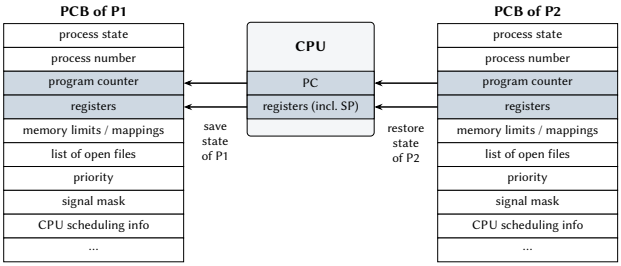


Figure 2.3: Process control blocks of P1 and P2. The shaded fields hold values that the CPU maintains in its registers while the process runs; they are saved into the PCB when the process stops running and restored from it when the process runs again.

2.4 Context switch

Let P1 and P2 be two processes whose PCBs are stored in the memory of the operating system.

Source P2L1, how is a PCB used, context switch; OSTEP ch. 6.

While P1 runs, the CPU registers hold its execution state. When the operating system interrupts P1, it saves the contents of the registers into the PCB of P1 and restores the state of P2 from the PCB of P2 into the registers; P2 then runs, and P1 is idle. When the operating system later interrupts P2, it saves P2's state into its PCB and restores P1's state. P1 resumes from exactly the point at which it was stopped (Figure 2.4). The switch runs in kernel mode, and it also changes the active address space: the operating system loads the page table base register of the CPU (Lesson 8) with the page table of the process that is to run.

Definition 2.6 (Context switch). A *context switch* is the mechanism used by the operating system to switch the CPU from the context of one process to the context of another.

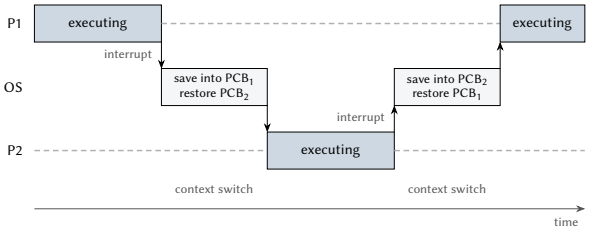


Figure 2.4: Two context switches between P1 and P2. Each process is idle (dashed) while the other runs, and its state waits in its PCB.

Context switches are expensive, for two reasons:

Direct cost the CPU cycles spent executing the load and store instructions that save the state of one process and restore the state of the other, and reloading the page table base register.

Indirect cost the effect on the processor cache. A cache access takes on the order of a few cycles, whereas an access to main memory takes on the order of hundreds of cycles.

Intel Skylake: L1 hit 4–5 cycles, L2 about 12, L3 about 42, DRAM some 50 ns more (7-cpu.com).

While a process runs, much of the data it uses is present in the processor cache. Such a cache is a *hot cache*: most accesses are served from the cache, and the process performs at its best. After a context switch, the next process replaces much of this data with its own. When the first process resumes, it finds a *cold cache*: its data is no longer in the cache, and its accesses are cache misses that must be served from main memory until the cache has been refilled. The operating system should therefore limit how frequently it switches contexts.

Example 2.7. Suppose that saving and restoring the registers takes 1 000 cycles, and that an access costs 4 cycles when the data is in the cache and 200 cycles when it must be read from main memory. If P1 accesses 5 000 data items after it resumes that were evicted from the cache while P2 ran, the first access to each costs $200 - 4 = 196$ extra cycles, in total $5\,000 \times 196 = 980\,000$ cycles. The indirect cost is almost a thousand times the direct cost of the switch.

Li et al. [7] measure 3.8 µs per switch between processes that touch no data, and up to about 1.5 ms per switch once the working set no longer fits in the 512 KiB L2 and is accessed with a large stride (2.0 GHz Xeon, 2007).

Note 2.8. The operating system switches contexts regardless of the state of the cache, for instance when a process of higher priority becomes ready or when the running process has used up the time it was allotted (Section 2.7). The cold cache that the switch leaves behind is part of the price of sharing the CPU.

> **Key point**
> The PCB holds everything the operating system needs to stop a process and resume it later. A context switch saves the CPU state of one process into its PCB and restores another from its PCB; its hidden cost, the cold cache it leaves behind, can far exceed the cost of the save and restore.

2.5 Process life cycle

Source P2L1, process life cycle, process life cycle: states; OSTEP ch. 4.

Caution Idle here means not on the CPU; it is not the kernel's idle task, which runs when no process is ready.

Seen from the CPU, a process alternates between running and not running. A running process that is interrupted and switched out becomes idle, that is, no longer on the CPU, although it is still ready to execute, and it runs again when the scheduler dispatches it to the CPU. A process can be idle for other reasons as well, and the operating system distinguishes these situations

by the *process state*, which it records in the PCB. A process is in one of the following states (Figure 2.5):

New A process that is being created is in the *new state*. The operating system performs *admission control*, deciding whether the process can be accepted; if so, it allocates and initializes the PCB and some initial resources, and the process is *admitted*.

Ready A process in the *ready state* has everything it needs to execute except the CPU. It could run as soon as the scheduler selects it; until then it waits for its turn.

Running A process in the *running state* is executing on the CPU; after the *scheduler dispatch* it continues from the point stored in its PCB. A running process leaves this state in one of three ways: it is interrupted, for instance because its time on the CPU has expired, and returns to the ready state after a context switch; it initiates a long operation, such as reading from disk or waiting for a keyboard or timer event, and enters the waiting state; or it terminates.

Waiting A process in the *waiting state* (also called *blocked*) cannot proceed until some event occurs, such as the completion of an I/O operation. When the event occurs, the process becomes ready again.

Terminated A process that has finished all its operations, or that has detected an error, calls *exit* and returns an exit code that indicates success or an error; a process that causes a fault, such as an invalid memory access, is terminated by the operating system. Either way the process enters the *terminated state*.

Caution An event completion makes a waiting process ready, not running; the CPU is given to it by a dispatch, possibly an immediate one.

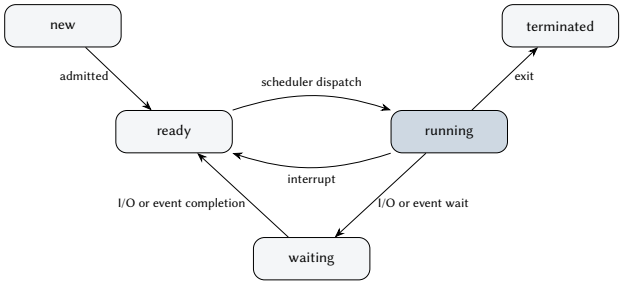


Figure 2.5: Process states and the transitions between them.

Table 2.1 traces one process through its life cycle. Only one transition, the dispatch, gives the process the CPU; every other transition takes it away or leaves the process without it.

Caution Linux does not separate ready from running: both are `TASK_RUNNING`, which `ps` prints as R. S and D are the two kinds of waiting, interruptible and uninterruptible, and T is a stopped process.

Table 2.1: State trace of a process that runs, is interrupted once, reads a file, and exits. The queues it waits in and the dispatch that takes it out of the ready queue are described in Section 2.7.

Event	Transition	State after	Where it is
process is being created	—	new	—
PCB initialized, accepted	admitted	ready	ready queue
scheduler selects it	scheduler dispatch	running	CPU
its time on the CPU expires	interrupt	ready	ready queue
scheduler selects it	scheduler dispatch	running	CPU
it issues a disk read	I/O or event wait	waiting	disk I/O queue
the disk read completes	I/O or event completion	ready	ready queue
scheduler selects it	scheduler dispatch	running	CPU
it calls exit	exit	terminated	—

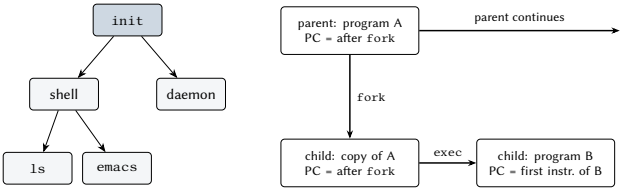
2.6 Process creation

Source P2L1, process life cycle: creation; OSTEP ch. 5; Silberschatz et al. ch. 3.

Processes are created by other processes. When the system boots and the operating system has been loaded, it creates a few initial processes, some of them with privileged access. When a user logs in, a shell process is created for the user, and a program started from the shell, for instance `ls` or `emacs`, runs as a new process created by the shell. A process that creates another process is its *parent process*, and the process it creates is its *child process*. Since every process except the first has a parent, the processes of a system form a tree (Figure 2.6 (a)); on Unix-based systems its root is `init`, the first user process that the kernel starts after booting,

and every other user process descends from it. (On many current Linux distributions the program that runs as this first process is `systemd`.)

`pstree -p` prints the process tree with every PID, including 1 (pstree(1)).



(a) process tree *(b) starting a new program*

Figure 2.6: (a) Processes form a tree rooted in the initial process. (b) A child created by `fork` is a copy of its parent; `exec` replaces its program with a new one.

Most operating systems provide two mechanisms for creating processes, in Unix called `fork` and `exec`:

`fork` With the `fork` system call the operating system creates a new PCB for the child, copies the PCB of the parent into it, and gives the child a copy of the parent's address space. The child is a duplicate of the parent, with the same program and the same program counter, so after the call both parent and child continue at the instruction that follows `fork`.

`exec` With the `exec` system call a process replaces its own image with a new program. The operating system replaces the address space of the process with the new program and updates the PCB to describe it: the program counter now points to the first instruction of the new program, while the process identifier and the open files are kept, except those marked close-on-exec (FD_CLOEXEC, O_CLOEXEC).

To create a process that runs a *different* program, a parent calls `fork`, and the child then calls `exec` (Figure 2.6 (b)). Since parent and child return from the same `fork` to the same place, they tell each other apart by its return value: `fork` returns 0 in the child and the child's process identifier in the parent, or −1 in the parent if no child could be created.

Caution `fork` copies page tables, not memory: a page is duplicated only when one side writes to it (copy-on-write, Lesson 8).

Tip `fork` returns twice, once in each process; 0 means "I am the child".

Android uses `fork` to start applications quickly. Its application processes descend from *Zygote*, a daemon started during boot whose sole purpose is to launch applications; it loads the runtime and the commonly used libraries once.[1] To launch an application, Zygote forks, and the child, which already contains everything Zygote has loaded, becomes the process of the application.

[1] Because `fork` shares the parent's pages copy-on-write, every application process starts with the runtime and the preloaded classes already mapped, in one set of physical pages for as long as no one writes to them: Zygote saves start-up time and memory at once.

Example 2.9. A program that creates a child process with `fork`, runs `ls -l` in it with `exec`, and waits until the child has finished:[2]

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {                 /* child */
        execlp("ls", "ls", "-l", (char *)NULL);
        _exit(127);                 /* only if exec failed */
    } else if (pid > 0) {           /* parent */
        waitpid(pid, NULL, 0);      /* wait for the child */
    }
    return 0;
}
```

[2] A process that has exited keeps its PCB until its parent reads the exit code—here with `waitpid`; until then it is a *zombie* (state Z). A child whose parent exits first is adopted and reaped by its nearest *subreaper* ancestor (PR_SET_CHILD_SUBREAPER, e.g. a systemd user manager) if there is one, otherwise by `init`.

After a successful `execlp` the child executes the code of `ls`; the call never returns.

2.7 The CPU scheduler

Source P2L1, role of the CPU scheduler; OSTEP ch. 6–7.

For the CPU to start executing a process, the process must be ready, and at any time several processes may be ready. The operating system must decide which of them runs next, and for how long.

Definition 2.10 (CPU scheduler). The *CPU scheduler* is the component of the operating system that manages how processes use the CPU. It decides which of the ready processes is dispatched to the CPU and how long it may run.

The processes that are ready wait for the CPU in the *ready queue* (in Linux, the runqueue). To give the CPU to one of them, the operating system performs three operations:

Preempt the running process is interrupted and its context is saved in its PCB; this is called *preemption*;

Schedule the scheduler runs its algorithm to choose the next process from the ready queue;

Dispatch the operating system switches into the context of the chosen process and lets it run on the CPU; this is the *dispatch* of the process.

The CPU is the resource that the processes need, so it should spend as much of its time as possible running them rather than running the operating system. The scheduler therefore needs efficient algorithms, efficient implementations of them, and efficient data structures to represent the waiting processes and their scheduling information, the ready queue in particular.

Linux 2.6.23 replaced the O(1) scheduler by CFS, whose runqueue is a red-black tree: picking the next task is O(1), insertion O(log n) (kernel documentation, sched-design-CFS); Lesson 7.

2.7.1 How often to run the scheduler

Source P2L1, length of process; OSTEP ch. 7.

The more often the scheduler runs, the more CPU time it takes away from the processes. Suppose each process runs for a time T_p before the scheduler, which takes t_{sched}, chooses the next one (Figure 2.7). The time t_{sched} comprises everything the operating system does between two timeslices: running the scheduling algorithm, updating the queue data structures, and the context switch of Section 2.4. Over n such rounds the fraction of the CPU time spent on useful work is

$$\text{useful CPU work} = \frac{\text{total processing time}}{\text{total time}} = \frac{nT_p}{nT_p + nt_{\text{sched}}} = \frac{T_p}{T_p + t_{\text{sched}}}. \tag{2.1}$$

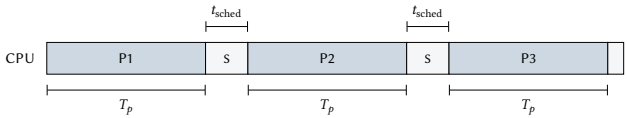


Figure 2.7: Processes run for T_p each; the scheduler (S) runs for t_{sched} between them.

The time T_p that a process is allocated on the CPU is its *timeslice*. The longer the timeslice relative to t_{sched}, the larger the useful fraction. If the timeslice is only as long as a run of the scheduler, half of the CPU time is lost; with a timeslice ten times as long, the useful fraction is $10/11 \approx 91\,\%$.

Example 2.11. Let $t_{\text{sched}} = 0.5\,\text{ms}$. With a timeslice of 2 ms, Equation (2.1) gives $2/2.5 = 80\,\%$ useful work; with a timeslice of 20 ms, $20/20.5 \approx 97.6\,\%$.

Real values: Linux CFS shared a 6 ms period among the runnable tasks, with a floor of 0.75 ms each; Linux 6.6 EEVDF uses a base slice of 0.75 ms (base values, scaled up with the CPU count by default).

A long timeslice is not free, however: while one process holds the CPU, the other ready processes wait for their turn. Choosing a scheduler thus requires design decisions: what timeslice values are appropriate, and which metrics decide the process to run next. Lesson 7 examines these questions.

2.7.2 Processes and I/O

Source P2L1, what about I/O; Silberschatz et al. ch. 3.

A running process may issue an I/O request, for example to read from disk. The operating system delivers the request to the device and places the process on the *I/O queue* associated with that device. The process remains waiting there until the device completes the operation and responds to the request. The process is then ready again: depending on the load of the system it is placed on the ready queue, or, if no other process is waiting, it may be dispatched to the CPU right away.

The wait is long compared with everything else in this chapter: a 4 KiB read takes tens of microseconds from an NVMe SSD and about 10 ms from a rotating disk.

Handling the interrupt is not itself a context switch: the handler runs in the kernel, in the context of whichever process was interrupted (Lesson 5).

Figure 2.8 shows the ways in which a process enters, or re-enters, the ready queue:

- its I/O request has been served by the device;
- its timeslice has expired;
- a new process has been created by a `fork`: the child is placed on the ready queue;
- it was waiting for an interrupt, and the interrupt has occurred.

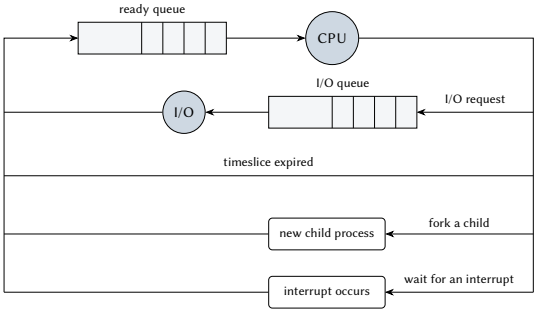


Figure 2.8: Ways in which processes enter the ready queue; every path ends there. On the `fork` path it is the new child, not the process that issued the call, that joins the queue.

A process on an I/O queue is outside the control of the scheduler until the device has completed its request; the scheduler chooses only among the processes in the ready queue, and decides when to switch contexts and to which process.

> **Key point**
> The scheduler preempts the running process, and chooses and dispatches the next one from the ready queue. Because it runs on the same CPU, its own time is overhead (Equation (2.1)), which shrinks as the timeslice grows relative to t_{sched}.

2.8 Inter-process communication

Source P2L1, can processes interact; Silberschatz et al. ch. 3.

Many applications are structured as several processes that must interact. A web application, for example, may consist of a web server process that handles the requests of clients and a database process that stores the data. The operating system, however, isolates processes from one another (Lesson 1): each has its own address space, and no process can access the memory of another.

Definition 2.12 (Inter-process communication). The mechanisms that the operating system provides for processes to interact are called *inter-process communication* (IPC). IPC mechanisms transfer data and information between address spaces, maintain the protection and isolation of the processes, and give applications flexibility and good performance.

Caution IPC is for processes; threads of one process (Lesson 3) share the address space without any of it.

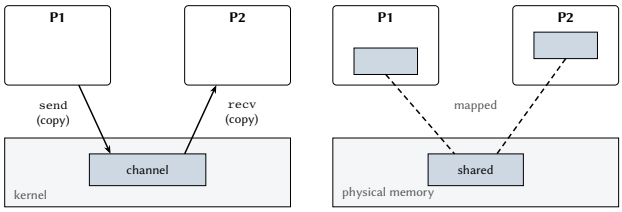
There are two basic kinds of IPC (Figure 2.9):

Message passing In *message passing* the operating system establishes a communication channel, such as a shared buffer in kernel memory. Processes send messages by writing into the channel and receive messages by reading from it, using system calls. The operating system manages the channel and offers the same well-defined interface, such as `send` and `recv`, to all applications. The price is overhead: every piece of information is copied from the address space of the sender into the channel in the kernel, and from there into the address space of the receiver, with a crossing into the kernel on each side.

POSIX names: `pipe`, `mq_open`, `shm_open` with `mmap` (pipe(7), mq_overview(7), shm_overview(7)); Lesson 9.

Shared memory With *shared memory* the operating system establishes a region of physical memory and maps it into the address spaces of both processes. The processes then read and write this memory directly, as they would their own, and the operating system is out of the way. The price is that the operating system no longer defines how the memory is used. There is no fixed, well-defined interface: the processes must agree on the format of the data and coordinate their accesses themselves (synchronization: Lesson 10), developers may have to re-implement this code for every application, and it is more error-prone.

Shared memory moves the cost of communication from the individual exchanges to the setup. Once the region is mapped, an exchange involves no copy and no crossing into the kernel, but creating the mapping is itself costly, so which of the two mechanisms is cheaper follows from how many exchanges the processes make.



(a) message passing *(b) shared memory*

Figure 2.9: (a) Message passing copies every message into a channel in the kernel and out again. (b) With shared memory, the same physical pages are mapped into both address spaces.

Example 2.13. Suppose that sending a message through a kernel channel costs 6 µs (two system calls and two copies), and that exchanging the same data through shared memory costs 1 µs after the mapping has been established at a one-time cost of 300 µs. For n messages, shared memory costs $300 + n$ and message passing $6n$ microseconds. Shared memory is cheaper only when $300 + n < 6n$, that is, for more than 60 messages.

Tip The break-even count is the setup cost divided by the saving per exchange, here $300/(6-1) = 60$ messages.

On current x86 Linux a round trip through a pipe costs a few to about ten microseconds, one through a Unix-domain socket somewhat more, and a minimal system call about a hundred nanoseconds (lmbench-style measurements: `lat_pipe`, `lat_unix`, `lat_syscall`; machine-dependent).

The IPC mechanisms of Unix-based systems and their use are the subject of Lesson 9.

Key point — Lesson summary

A process is an executing program, represented by its address space—text, data, heap and stack, translated from virtual to physical addresses by a page table—and by the execution state that the operating system keeps in its PCB. A context switch saves one process's state into its PCB and restores another's, at a direct cost and at the indirect cost of a cold cache. Processes move between the new, ready, running, waiting and terminated states; they are created with `fork` and `exec`; the CPU scheduler picks the next ready process, and its own overhead depends on the timeslice; and IPC, by message passing or shared memory, lets isolated processes communicate.

Threads and Concurrency

Lecture	P2L2
Textbook	OSTEP [1] ch. 26–32; Silberschatz et al. [2] ch. 4–8
Keywords	thread, multithreading, concurrency, race condition, mutex, condition variable, readers-writers problem, spurious wake-up, deadlock, kernel-level thread, user-level thread, multithreading models, boss-workers pattern, pipeline pattern

A process as described in Lesson 2 has a single execution context, so it can follow only one sequence of instructions at a time. This lesson introduces threads, which give a process several execution contexts that share one address space, and the mechanisms—mutexes and condition variables—with which such threads coordinate their access to shared data. The presentation follows Birrell's tutorial on programming with threads [8]; it then turns to deadlocks, to the relation between user-level and kernel-level threads, and to common patterns for structuring multithreaded applications.

3.1 Processes and threads

Source P2L2, process vs. thread; Birrell [8]; OSTEP ch. 26.

A process is represented by its address space—the code, data and heap, reached through the virtual-to-physical mappings kept in the page table—, by the resources the operating system keeps for it, such as open files, and by its execution context: the values of the CPU registers, including the program counter and the stack pointer, and the stack. The operating system records all of this in the process control block. A process of this kind is single-threaded.

Definition 3.1 (Thread). A *thread* is one execution context within a process. Every thread has its own program counter, stack pointer, register values and stack; all threads of a process share its address space (code, data, heap) and its other resources, such as open files.

Running several threads inside one process is called *multithreading*, and such a process is *multithreaded*. Since the threads share the virtual-to-physical mappings, all of them can access the same code and data. At any moment, however, they may execute different instructions, work on different portions of the input and have different chains of procedure calls in progress. The per-thread state in Figure 3.1 captures exactly these differences: the program counter and the registers record where a thread is and what it is computing, and the private stack holds the local variables and return addresses of its active calls.

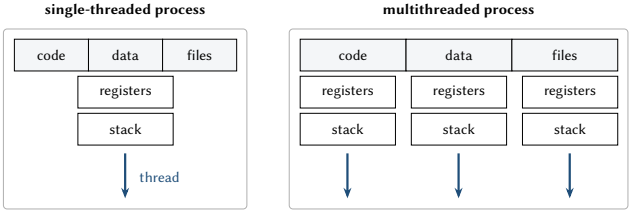


Figure 3.1: A single-threaded and a multithreaded process. The shaded state (code, data and open files) is shared by all threads; every thread has its own registers—including PC and SP—and its own stack.

The operating system's representation of a multithreaded process is correspondingly richer than a PCB for a single-threaded one. It holds the information shared by all threads once, and a separate data structure for each thread that records the thread's execution context.

On Linux a thread is a task created with `clone(2)` and `CLONE_VM`; the kernel keeps no separate thread object (Lesson 5).

3.2 Why threads are useful

Source P2L2, why are threads useful, benefits of multithreading on a single CPU, benefits to applications and OS code; OSTEP ch. 26.

On a machine with several CPUs, threads can speed up an application in two ways.

Same code, different data All threads execute the same code, each on a different portion of the input. In a matrix multiplication, for example, each thread can compute a different

subset of the rows of the result, and the threads run in parallel on different CPUs. Dividing the work of one task in this way is called *parallelization*.

Different code Different threads execute different parts of the program: one thread may process input while another renders the display, or threads may be dedicated to requests of different types. Dedicating threads to particular tasks is called *specialization*.

Specialization has two further benefits. The threads can be managed differently, for example by giving higher priority to the threads that handle more important tasks or customers. And since performance depends on how much of a thread's state is present in the processor cache, a thread that repeatedly executes a small portion of the code works with a hot cache (Lesson 2) more often than a thread that executes everything.

3.2.1 Threads versus processes

The same parallelism could be obtained by running several single-threaded processes. A multithreaded implementation is nevertheless more efficient in three respects:

- **Memory.** The threads share one address space, whereas every process needs its own address space and its own copy of the state. The multithreaded application has lower memory requirements and is therefore less likely to need swapping.
- **Context switching.** Switching between threads of the same process does not switch the address space: the virtual-to-physical mappings stay in place, so the costly part of a process context switch is avoided.
- **Communication.** Threads exchange data through shared variables. Processes need inter-process communication (IPC): message passing enters the kernel for every exchange, and shared memory must first be set up by the operating system, so communication between processes is more costly.

Example 3.2. Suppose an application holds 400 MB of data that each process would have to keep separately (code pages, which the operating system can share between processes, are ignored here) and needs about 1 MB of stack and other per-context state. Run as four processes, each with its own copy of the data, it occupies about $4 \times 401 = 1604$ MB; run as four threads of one process it occupies about $400 + 4 \times 1 = 404$ MB.

On Linux a thread's stack is 8 MiB of address space by default (`RLIMIT_STACK`); only the pages it touches occupy memory, so 10 000 threads reserve about 78 GiB of address space but commit only a few pages each: some hundreds of megabytes in total, including the 16 KiB kernel stack per thread.

3.2.2 Threads on a single CPU

Threads are useful even when there are more threads than CPUs. When a thread issues a request that blocks, such as a disk read, it has nothing to do until the request completes, and the CPU would sit idle for that time t_{idle}. The CPU can instead switch to another thread and switch back when the request completes (Figure 3.2). The other thread then gains $t_{\text{idle}} - t_{\text{ctx}}$ of CPU time, while the blocked thread resumes t_{ctx} later than it would have; the two switches together cost $2\,t_{\text{ctx}}$. Switching pays off when the time gained exceeds the delay, that is, when

$$t_{\text{idle}} \;>\; 2\,t_{\text{ctx}}. \tag{3.1}$$

A disk seek (some 10 ms) or an SSD read (some 100 µs) repays two switches many times over; a wait of the order of a memory access (under 100 ns) never does; for short lock waits spinning beats blocking (Lesson 10).

Because a context switch between threads avoids the address-space switch, its t_{ctx} is smaller than that of a switch between processes, and Equation (3.1) holds for shorter idle periods. Multithreading therefore lets a program hide idle time even on one CPU.

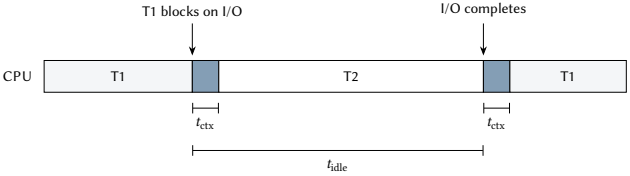


Figure 3.2: T1 blocks on I/O; instead of idling, the CPU switches to T2, and the completion of the I/O triggers the switch back. The switches pay off when they cost less than the idle time (Equation (3.1)).

Example 3.3. Assume a thread context switch costs 0.05 ms and a process context switch 0.5 ms. With threads, every blocking wait longer than 0.1 ms can be used by another thread; with processes only waits longer than 1 ms. A wait of 4 ms gives another thread $4 - 0.05 =$

3.95 ms of CPU time and delays the blocked thread by 0.05 ms; for another process the figures are 3.5 ms and 0.5 ms.

The round figures keep the arithmetic simple; measured costs are microseconds. lmbench (McVoy and Staelin, USENIX 1996) reported 6 µs per context switch on a 200-MHz P6 in 1996, and 2–3 GHz cores stay below a microsecond (OSTEP ch. 6).

3.2.3 Multithreaded applications and kernels

Applications and operating systems both benefit. A multithreaded application achieves parallel speedup on several CPUs and hides idle time on one. A multithreaded operating system kernel runs some of its threads on behalf of applications and others to provide operating-system services such as daemons and device drivers; on a multiprocessor these kernel threads run concurrently on different CPUs.

> **Key point**
> Threads share the address space of their process, which makes them cheaper than processes in memory, context-switch time and communication. The same sharing means that threads can interfere with one another, so they need explicit mechanisms for coordination.

3.3 Basic thread mechanisms

Supporting threads requires three things: a data structure that represents a thread, so that threads can be identified and their resource usage tracked; mechanisms to create and manage threads; and mechanisms that let threads running at the same time in the same address space coordinate safely.

Source P2L2, basic thread mechanisms, thread creation; Birrell [8]; OSTEP ch. 26–27.

Two threads execute concurrently when their executions overlap in time, whether they are interleaved on one CPU or run simultaneously on different CPUs; the resulting *concurrency* creates a problem that processes do not have. The operating system gives different processes different mappings, so a process cannot touch the memory of another. Threads share their mappings, so any thread can legally read and write any data of its process. If two threads update the same data at the same time, or one reads while another modifies it, the data can become inconsistent.

Definition 3.4 (Race condition). A *race condition* exists when the result of a computation depends on the relative timing of threads that access shared data, at least one of them modifying it. Two unsynchronized concurrent accesses to the same data, at least one of them a write, form a *data race*; data races are a common cause of race conditions.

Caution A data race is undefined behaviour in C11 and C++11 (C11 §5.1.2.4): the compiler may reorder or cache the accesses, so there are no "benign" races at the language level.

The mechanisms that threads use to coordinate their access to shared data and their progress relative to one another are called *synchronization* mechanisms. Two basic mechanisms, both described by Birrell, are used throughout this lesson:

- **mutual exclusion**: only one thread at a time may execute a given operation; this is provided by a *mutex* (Section 3.4);
- **waiting**: a thread waits until another thread establishes a specific condition, and is woken up by it; this is provided by a *condition variable* (Section 3.5).

3.3.1 Thread creation

A thread is represented by a thread data structure that contains its identifier, its program counter, stack pointer and other register values, its stack, and attributes the thread library uses to manage it. In Birrell's notation, *thread creation* is performed by the call `t1 = Fork(proc, args)`, which creates a new thread, stores its data structure in `t1`, and starts the new thread executing `proc(args)`. The creating thread continues with the statement after `Fork`, so from this point on both threads run concurrently. Despite its name, `Fork` creates a thread inside the same process; it is unrelated to the Unix `fork` of Lesson 2, which creates a new process.

When the child thread finishes, its result must reach the parent. Either the child stores it in a well-defined location of the shared address space, or the parent obtains it with the *join* operation: `Join(t1)` blocks the calling thread until thread `t1` has completed and returns its result; at that point the resources allocated to the child are freed. Apart from this, parent and child are equals: both can access all resources of the process, and they differ only in their execution contexts.

Drepper and Molnar [9] report that starting and stopping 100 000 threads, which once took 15 minutes on Linux, takes 2 seconds with NPTL and the reworked kernel thread exit.

Example 3.5. A parent thread and a child thread both insert into a shared list:

```
Thread t1;
Shared_list list;

t1 = Fork(safe_insert, 7);  // child inserts 7
safe_insert(9);             // parent inserts 9
Join(t1);                   // optional here
```

Which insertion happens first depends on how the threads are scheduled, so with insertion at the head the list ends up as either $7 \to 9$ or $9 \to 7$. The `Join` is not needed for correctness because the parent does not use a result of the child.

Insertion is not atomic. Inserting element e at the head of a list takes three steps: create e; read the head pointer and store it in e.`next`; write the address of e into the head pointer. If two threads interleave these steps, one insertion can be lost, as the trace in Table 3.1 shows.

Table 3.1: Two unsynchronized insertions into a list that initially holds one element, 3. Both threads read the old head before either writes it, so the write in step 6 overwrites that of step 5 and element 7 is lost.

Step	Thread T1 (insert 7)	Thread T2 (insert 9)	List
1	create e7		3
2	e7.next = head ($\to$ 3)		3
3		create e9	3
4		e9.next = head ($\to$ 3)	3
5	head = e7		$7 \to 3$
6		head = e9	$9 \to 3$

3.4 Mutual exclusion

The race in Table 3.1 disappears if only one thread at a time can execute the three steps of an insertion. With respect to the other threads, the three steps must then behave like one indivisible operation: no other thread can observe or modify the list between them. This requirement is called *mutual exclusion*, and the basic mechanism that enforces it is the mutex.

Source P2L2, mutexes; Birrell [8]; OSTEP ch. 28; Silberschatz et al. ch. 6.

Definition 3.6 (Mutex). A *mutex* (mutual exclusion lock) is a synchronization construct with two operations, lock and unlock. A thread that locks a free mutex acquires it and becomes its owner; a thread that tries to lock a mutex owned by another thread blocks until the mutex is released.

A mutex is represented by a data structure that records at least whether it is locked, which thread owns it, and the list of threads blocked on it. The code executed while a mutex is held is the *critical section*: the part of the program that performs an operation which only one thread at a time may perform, typically an update of shared state.

Birrell's notation makes the critical section a block:

```
Lock(m) {
  // critical section
} // unlock
```

Tip In C++ `std::lock_guard` releases the mutex when its scope ends, reproducing Birrell's block notation below: every path unlocks.

The mutex is released implicitly when the block ends. Most thread interfaces, including the POSIX threads of Lesson 4, instead have separate calls `lock(m)` and `unlock(m)` around the critical section, and the programmer must make sure that every path through the critical section ends with an unlock.

Caution A mutex is not recursive: a thread that locks a mutex it already holds deadlocks against itself. POSIX prescribes this for `PTHREAD_MUTEX_NORMAL`, the behaviour of the default type on Linux; a recursive mutex has to be requested explicitly.

Example 3.7. With a mutex that protects the list, the insertion of Section 3.3.1 becomes safe:

```
list<int> my_list;
Mutex m;

void safe_insert(int i) {
  Lock(m) {
    my_list.insert(i);
  } // unlock
}
```

If the parent locks `m` first, the child blocks in `Lock(m)` until the parent's insertion is complete, and then performs its own. In the scenario of Table 3.1 the list ends as $7 \to 9 \to 3$ or $9 \to 7 \to 3$; an element is never lost.

When the owner releases a mutex on which several threads are blocked, only one of them acquires it; the others stay blocked. The specification of a mutex does not say which one: threads are not guaranteed to acquire the mutex in the order in which they requested it, and a thread that calls `Lock` just as the mutex is released may acquire it ahead of threads that have been waiting for some time. Program correctness must therefore not depend on the order in which waiting threads acquire a mutex.

An uncontended Linux lock is one atomic instruction in user space; the kernel is entered only on contention (`futex(2)`; Drepper, *Futexes Are Tricky*); Lesson 10.

3.5 Condition variables

Mutual exclusion alone is not enough when a thread must wait for a condition that other threads establish. A thread that holds a mutex cannot simply stop and wait for the condition, because while it holds the mutex no other thread can change the shared state. The standard illustration is the *producer-consumer problem*: several producer threads insert items into a shared list, and one or more consumer threads must print and remove all items once the list is full.

Source P2L2, producer/consumer example, condition variable, condition variable API; Birrell [8]; OSTEP ch. 30; Silberschatz et al. ch. 6.

With only a mutex, the consumer has to lock the mutex, check whether the list is full, and if it is not, release the mutex and try again later. This repeated checking wastes CPU time, because most checks find the list not yet full. It would be better if the consumer could sleep and be woken up by the producer whose insertion fills the list.

Definition 3.8 (Condition variable). A *condition variable* is a synchronization construct, used together with a mutex, on which threads wait until some condition on the shared state holds, and through which other threads notify them when it may hold.

With a condition variable `list_full`, the two sides become:

```
// consumer: print_and_clear
Lock(m) {
  while (my_list.not_full())
    Wait(m, list_full);
  my_list.print_and_remove_all();
} // unlock
```

```
// producers: safe_insert
Lock(m) {
  my_list.insert(my_thread_id);
  if (my_list.full())
    Signal(list_full);
} // unlock
```

3.5.1 The condition variable interface

Birrell's interface has three operations on a condition variable `cond`.

`Wait(mutex, cond)` The *wait operation* is called with `mutex` held. It releases the mutex and suspends the thread on the wait queue of `cond`; when the thread is woken up, the operation re-acquires the mutex before it returns.

`Signal(cond)` The *signal operation* wakes up at least one (normally one) thread waiting on `cond`, if there is one.

`Broadcast(cond)` The *broadcast operation* wakes up all threads waiting on `cond`.

A condition variable is represented by a data structure that holds a reference to its mutex and the list of waiting threads. The wait operation proceeds as follows:

```
Wait(mutex, cond) {
  // atomically release mutex and place the
  //   thread on the wait queue of cond
  // ... wait ...
  // remove the thread from the wait queue
  // re-acquire mutex
  // return
}
```

Caution POSIX `pthread_cond_wait` takes (cond, mutex), the reverse of Birrell's `Wait(mutex, cond)`; Lesson 4.

Releasing the mutex while waiting is essential: otherwise no other thread could enter a critical section to change the state and signal. Re-acquiring it before returning guarantees that the waiting thread again holds the mutex when it re-examines the shared state. As a consequence, a woken thread cannot continue until it holds the mutex again; if the mutex is still held, for instance by the signalling thread, the woken thread moves from the wait queue of the condition variable to that of the mutex (Figure 3.3). After a broadcast all woken threads compete for the mutex, so they leave the wait operation one at a time.

The signal operation on a condition variable is unrelated to the Unix signals of Lesson 1.

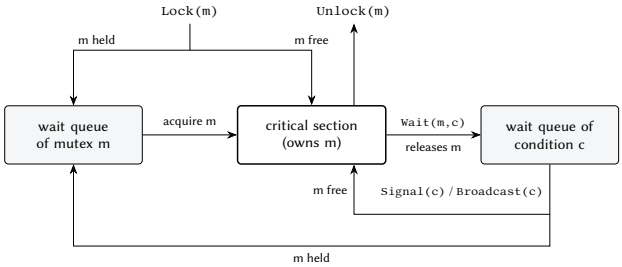


Figure 3.3: A thread waiting on condition `c` releases mutex `m`. When it is signalled it must first re-acquire `m`; if `m` is held, it moves to the wait queue of the mutex.

This is why the consumer tests the list in a `while` loop and not in an `if`. Between the `Signal` and the moment the consumer re-acquires the mutex, other threads may acquire it first and change the list—for example a second consumer that empties it. Signalling a condition variable only says that the condition may hold; the woken thread cannot be guaranteed access to the mutex immediately, and it must re-check the condition once it has.[1]

Key point
Always call `Wait` with the mutex held, inside a `while` loop that re-tests the condition. Signal or broadcast after changing the state that the condition depends on.

[1] Re-testing in a loop reflects Mesa semantics (Lampson and Redell [10]): the signaller keeps the lock. Hoare monitors hand the lock to the woken thread, so `if` would suffice there; monitors are the subject of Lesson 10.

3.6 The readers-writers problem

In the *readers-writers problem* some threads, the readers, only read a shared resource such as a file, and others, the writers, modify it. Any number of readers may access the resource at the same time, but a writer needs exclusive access: at most one writer, and no readers while it writes.

Source P2L2, readers/writer problem and example, critical section structure (with proxy); OSTEP ch. 31; Silberschatz et al. ch. 7.

Protecting the resource with a single mutex is correct but too restrictive, because it also excludes readers from each other. The access rules depend on how many readers and writers are active:

- no active readers and no writer: a reader or a writer may enter;
- one or more active readers: another reader may enter, a writer may not;
- an active writer: neither readers nor writers may enter.

These rules can be expressed with a single counter: the resource is *free* when `resource_counter` $= 0$, being *read* by n readers when `resource_counter` $= n > 0$, and being *written* when `resource_counter` $= -1$ (Figure 3.4).

The counter is an example of a *proxy variable*: a shared variable, protected by a mutex, that encodes the state of the resource. Threads hold the mutex only while they examine and update

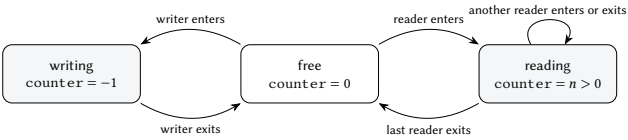


Figure 3.4: The states of the shared resource and the corresponding values of the proxy variable; `counter` is short for `resource_counter`.

the proxy variable; the actual reading and writing happens outside the mutex, under the policy that the proxy variable enforces. The solution uses one mutex and two condition variables:

```
Mutex counter_mutex;
Condition read_phase, write_phase;
int resource_counter = 0;

// READERS
Lock(counter_mutex) {
  while (resource_counter == -1)
    Wait(counter_mutex, read_phase);
  resource_counter++;
} // unlock
// ... read data ...
Lock(counter_mutex) {
  resource_counter--;
  if (resource_counter == 0)
    Signal(write_phase);
} // unlock
```

```
// WRITER
Lock(counter_mutex) {
  while (resource_counter != 0)
    Wait(counter_mutex, write_phase);
  resource_counter = -1;
} // unlock
// ... write data ...
Lock(counter_mutex) {
  resource_counter = 0;
  Broadcast(read_phase);
  Signal(write_phase);
} // unlock
```

Tip Signal when at most one waiting thread can proceed and any of them will do; broadcast when the change can admit several waiters, or when threads waiting on one condition variable wait for different predicates.

The choice of notification in the exit code follows from who can be waiting.

- A **reader** that leaves while other readers remain changes nothing that a waiting thread cares about. When the last reader leaves, only writers can be waiting—readers wait only while a writer is active—and only one of them may enter, so a `Signal` on `write_phase` suffices.
- A **writer** that leaves frees the resource for either kind of thread. All waiting readers may enter together, hence `Broadcast(read_phase)`; only one writer may enter, hence `Signal(write_phase)`. Which thread actually gets in first depends on which one acquires `counter_mutex` first. If a reader gets in first, the other woken readers enter as well, and the writer finds the counter $\neq 0$ in its `while` loop and waits again; if the writer gets in first, the readers find the counter $= -1$ and wait again.

Caution The policy favours readers: while new readers keep arriving the counter never returns to 0, and a waiting writer never enters. glibc's `pthread_rwlock` is reader-preferring by default too, and offers a writer-preferring kind (Lesson 10).

Example 3.9. Table 3.2 traces two readers, a writer that arrives while they are reading, and a third reader that arrives while the writer writes.

3.6.1 Structure of critical sections

The readers-writers solution shows the general shape of code that uses a mutex together with condition variables. A critical section that may have to wait is written as

Table 3.2: A trace of the readers-writers solution, with the value of the proxy variable after each step.

Step	Event	Counter
1	R1 enters	1
2	R2 enters	2
3	W1 finds counter ≠ 0, waits on `write_phase`	2
4	R1 exits; counter ≠ 0, no notification	1
5	R2 exits; counter = 0, signals `write_phase`	0
6	W1 re-acquires the mutex, re-tests, enters	−1
7	R3 finds counter = −1, waits on `read_phase`	−1
8	W1 exits; broadcasts `read_phase`, signals `write_phase`	0
9	R3 re-acquires the mutex, re-tests, enters	1

```
Lock(mutex) {
  while (!predicate_indicating_access_ok)
    Wait(mutex, cond_var);
  update state => update predicate;
  Signal and/or Broadcast(
      cond_var_with_correct_waiting_threads);
} // unlock
```

With a proxy variable, the operation on the resource is split out of the mutex into three parts:

```
// ENTER CRITICAL SECTION
Lock(mutex) {
  while (!predicate_for_access)
    Wait(mutex, cond_var);
  update predicate;
} // unlock

perform critical operation  // e.g. read/write shared file

// EXIT CRITICAL SECTION
Lock(mutex) {
  update predicate;
  Signal/Broadcast(cond_var);
} // unlock
```

Key point
A mutex by itself enforces one policy: one thread at a time. Proxy variables, updated in short critical sections, allow more complex access policies—such as "any number of readers or one writer"—to be built from the same mutex and condition variable primitives.

3.7 Common pitfalls and spurious wake-ups

Source P2L2, common pitfalls, spurious wake-ups; Birrell [8].

Several mistakes recur in programs that use mutexes and condition variables.

- Losing track of which mutex and condition variables belong to which resource. Recording the association where the variables are declared, e.g. `Mutex m1; // protects file1`, avoids this.
- Forgetting to lock or to unlock, or locking and unlocking the wrong mutex. Some compilers and analysis tools can warn about such cases.
- Using more than one mutex for the same resource. If the readers of a file lock `m1` and its writers lock `m2`, reads and writes run concurrently.
- Signalling the wrong condition variable.
- Using `Signal` where `Broadcast` is needed. Only one thread is woken; the others may continue to wait indefinitely.
- Assuming that notifications establish an execution order. The order in which signals are issued does not determine the order in which the woken threads run; if priorities must be respected, another mechanism is needed.

Clang's `-Wthread-safety` checks lock annotations while compiling; ThreadSanitizer (`-fsanitize=thread`) finds races while the program runs, at a typical cost of 5–15× in time and 5–10× in memory.

Two further problems deserve a closer look: spurious wake-ups, which cost performance, and deadlocks (Section 3.8), which stop a program.

3.7.1 Spurious wake-ups

Some notifications wake threads that cannot use them.

Definition 3.10 (Spurious wake-up). A *spurious wake-up* is a wake-up of a thread that cannot make progress when it runs, so that the context switches it causes are wasted.

Caution The lecture calls this a spurious wake-up; Birrell names it a *spurious lock conflict* and uses *spurious wake-up* for waking threads whose predicate is false.

A common cause is signalling while the mutex is still held. In the writer's exit code of Section 3.6 the broadcast is issued inside the critical section. The woken readers leave the wait queue of `read_phase`, but they cannot return from `Wait` before re-acquiring `counter_mutex`, which the writer still holds; they are simply moved to the wait queue of the mutex (Figure 3.3).

When the notifications do not depend on the shared state, the fix is to move them after the unlock:

POSIX also calls a return from wait without any signal a spurious wake-up; the `while` loop handles both.

```
// writer exit, notifications outside the mutex
Lock(counter_mutex) {
  resource_counter = 0;
} // unlock
Broadcast(read_phase);
Signal(write_phase);
```

This is not always possible. In the reader's exit code the decision whether to signal depends on `resource_counter`, which may only be read while holding the mutex, so the `Signal` cannot simply be moved after the unlock; it can be moved only by recording the decision in a local variable inside the critical section and signalling afterwards, which is Birrell's own fix with a local variable `doSignal`. Spurious wake-ups never affect correctness—the `while` loop sends a thread that cannot proceed back to waiting—only performance.

Linux offers `FUTEX_CMP_REQUEUE` for the same purpose: instead of waking the threads waiting on a condition variable it moves them to the wait queue of the mutex, which `futex(2)` describes as avoiding a "thundering herd".

3.8 Deadlocks

Threads that hold some mutexes while requesting others can block each other permanently.

Source P2L2, deadlocks; OSTEP ch. 32; Silberschatz et al. ch. 8.

Definition 3.11 (Deadlock). A *deadlock* is a situation in which two or more competing threads each wait for another to complete, but none of them ever does.[2]

[2] A deadlock needs four conditions at once (Coffman et al., 1971): mutual exclusion, hold and wait, no preemption, circular wait; lock ordering removes the last (OSTEP ch. 32; Silberschatz et al. ch. 8).

The simplest case involves two threads and two mutexes. Both threads need resources A and B to execute `foo(A, B)`, but they lock the protecting mutexes in opposite order:

```
// thread T1                  // thread T2
Lock(m_A) {                   Lock(m_B) {
  Lock(m_B) {                   Lock(m_A) {
    foo(A, B);                    foo(A, B);
  }                             }
}                             }
```

If T1 acquires `m_A` and T2 acquires `m_B` before either reaches its second `Lock`, T1 blocks waiting for `m_B` and T2 blocks waiting for `m_A`. Neither can proceed, and neither will ever release the mutex the other needs.

The situation is described by a *wait graph*, which has an edge from a thread waiting for a resource to the thread that owns the resource. Figure 3.5 shows the example with the mutexes drawn explicitly. A deadlock corresponds to a cycle in the wait graph: for locks, which have a single owner, a cycle is both necessary and sufficient for a deadlock.

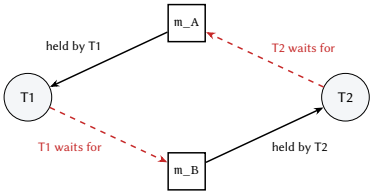


Figure 3.5: Wait graph of the deadlock: following the edges from T1 leads through `m_B`, T2 and `m_A` back to T1.

3.8.1 Avoiding the example deadlock

There are three ways to remove the deadlock from the example.

1. **Unlock A before locking B.** Holding only one mutex at a time (fine-grained locking) makes a cycle impossible, but it does not work here: the threads need both resources at once.
2. **Acquire all locks up front.** A thread that obtains every lock it needs at once before it starts—atomically, all or none, releasing any it holds if one is unavailable—and releases them all at the end cannot deadlock; the extreme case is a single "mega lock" for all resources. For many applications this is too restrictive, because it limits parallelism.
3. **Maintain a lock order.** If every thread acquires the mutexes in the same order, first `m_A` and then `m_B`, the deadlock cannot occur.

The third approach, *lock ordering*, works in general: if the mutexes are numbered and every thread acquires them in increasing order, the numbers would have to increase all the way around a cycle of the wait graph, which is impossible. The difficulty is practical: in a large program with many mutexes it is hard to ensure that every code path respects the order, and tools that check lock acquisition orders help.

Linux *lockdep* (kernel docs, locking/lockdep-design) records the acquisition order of lock classes and warns of inversions before a deadlock occurs; ThreadSanitizer reports lock-order inversions too.

3.8.2 Handling deadlocks in general

Operating systems and applications can deal with deadlocks in three ways.

Prevention With *deadlock prevention* a cycle is never allowed to form. Either the code is changed so that cycles are impossible—a thread releases the locks it holds before it requests another, or all threads follow a lock order—or every lock request is checked at run time and delayed if granting it could later lead to a cycle. The run-time check is expensive, and it requires knowing in advance which locks each thread may request; textbooks call this variant *deadlock avoidance* and reserve *prevention* for the structural rules.

Detection and recovery With *deadlock detection and recovery* the system periodically analyses the wait graph for cycles and, when it finds one, rolls back the execution of some of the threads involved. Rollback requires enough saved state to restore an earlier point, and it is impossible when the threads have already consumed external input or produced external output.

Doing nothing The *ostrich algorithm* ignores deadlocks altogether, on the grounds that they are rare and the other approaches are expensive. If a deadlock does occur, the system is rebooted.

> **Key point**
> A deadlock is a cycle in the wait graph. Acquiring all mutexes in one global order prevents such cycles at no run-time cost; checking every lock request at run time and recovery by rollback are general but expensive.

3.9 Kernel-level and user-level threads

Threads exist at two levels of the system. The operating system kernel can itself be multithreaded, and applications can create threads within their processes; the question is how the two kinds of threads relate to each other.

Source P2L2, kernel vs. user-level threads, multithreading models, scope of multithreading; Silberschatz et al. ch. 4–5.

Definition 3.12 (Kernel-level and user-level threads). A *kernel-level thread* is a thread that is visible to the kernel and managed by kernel components such as the CPU scheduler. A *user-level thread* is a thread as the application sees it, created through a user-level thread library; depending on the multithreading model, the library manages it itself or passes the operations to the kernel.

Some kernel-level threads exist to support the user-level threads of applications; others run operating-system services such as daemons. The CPU scheduler decides which kernel-level threads run on which CPUs. A user-level thread, in turn, can execute only when it is associated with a kernel-level thread and the scheduler runs that kernel-level thread on a CPU.

Tip `ps -eLf` lists the kernel-level threads of every process (LWP column); `top -H` shows them individually.

3.9.1 Multithreading models

How user-level threads are associated with kernel-level threads is described by three multithreading models (Figure 3.6).[3]

[3] The many-to-many model was the ambitious one, and it has largely been abandoned. Solaris 2 implemented it, but the coordination between the two levels proved to cost more than it saved, and Sun made the one-to-one library the default in Solaris 9 (2002); Linux settled on the one-to-one NPTL rather than the many-to-many NGPT (Lesson 5). The model survives in language runtimes, where the scheduler is part of the runtime: a Go program multiplexes goroutines, which start with a stack as small as 2 KiB, onto a few kernel-level threads.

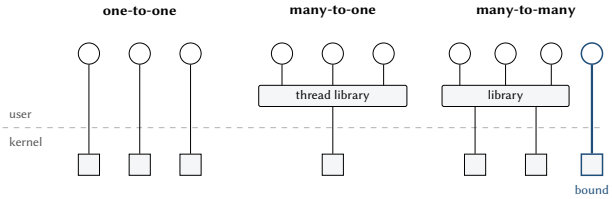


Figure 3.6: Multithreading models. Circles are user-level threads, squares kernel-level threads. In the many-to-many model a bound thread keeps a kernel-level thread of its own.

One-to-one In the *one-to-one model* every user-level thread has its own kernel-level thread. The kernel sees and understands the threads, their synchronization and their blocking, and can schedule them on several CPUs. On the other hand thread creation and every operation that blocks or wakes a thread require a system call, which is expensive (on Linux, uncontended lock operations stay in user space via futexes); the application is limited by the policies and the maximum number of threads that the kernel supports; and it is portable only to kernels that provide this support.

Many-to-one In the *many-to-one model* all user-level threads of a process are mapped onto a single kernel-level thread, and a user-level thread library decides which user-level thread runs on it at any time. Thread management needs no system calls and does not depend on kernel limits or policies, so it is fully portable. But the kernel has no insight into the application's needs, and when one user-level thread blocks on I/O, the kernel-level thread blocks, and with it the whole process.

Many-to-many In the *many-to-many model* some user-level threads are multiplexed onto a set of kernel-level threads while others have a kernel-level thread to themselves. A user-level thread that is permanently associated with a kernel-level thread is a *bound thread*; it can, for example, be given a higher priority or better responsiveness. Because the process has several kernel-level threads, a user-level thread that blocks on I/O blocks only its kernel-level thread; the other user-level threads continue on the remaining ones. The model combines the advantages of the other two, at the price of coordination between the user-level and the kernel-level thread managers.

3.9.2 Scope of multithreading

Thread management happens at two levels, and the level determines the unit among which resources are shared. With *process scope* the user-level thread library manages the threads within a single process and competes for resources only with the other threads of that process; the kernel does not see them individually. With *system scope* threads are managed system-wide by the operating system's thread managers, in particular the CPU scheduler, and compete with all threads in the system.

Linux supports only system scope: `pthread_attr_setscope` with `PTHREAD_SCOPE_PROCESS` fails with `ENOTSUP` (`pthread_attr_setscope(3)`); Lesson 4.

Example 3.13. Process P runs 6 user-level threads and process Q runs 2. With process scope the kernel sees only the two processes and, if it gives each an equal share of the CPU, each thread of P receives $\frac{1}{2} \cdot \frac{1}{6} = \frac{1}{12}$ of the CPU while each thread of Q receives $\frac{1}{4}$. With system scope the kernel sees all 8 threads; dividing the CPU equally among them gives every thread $\frac{1}{8}$, so that P as a whole receives $\frac{3}{4}$.

3.10 Multithreading patterns

Source P2L2, multithreading patterns; thread pools: Silberschatz et al. ch. 4.

Multithreaded applications are commonly organized according to one of three patterns: boss-workers, pipeline and layered.

3.10.1 Boss-workers

In the *boss-workers pattern* one boss thread accepts requests and assigns them to worker threads, and each worker performs an entire task. Every request passes through the boss, so the throughput of the system is limited by the boss: if the boss spends time t_{boss} on each request, the system can accept at most $1/t_{\text{boss}}$ requests per unit time, however many workers there are. The boss must therefore do as little per request as possible.

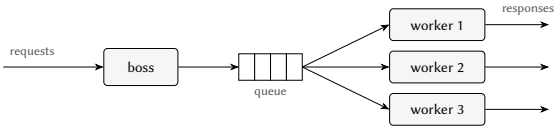


Figure 3.7: Boss-workers with a work queue: the boss only enqueues requests; every worker takes a request from the queue and performs the entire task.

The boss can pass work to workers in two ways (Figure 3.7 shows the second).

- **Directly signalling a specific worker.** The workers need no synchronization among themselves, but the boss must keep track of what each worker is doing and which one is free. This increases t_{boss} and lowers throughput.
- **Placing work in a queue,** from which the workers take it, in a producer-consumer arrangement. The boss need not know anything about the workers, but the workers (and the boss) must synchronize their access to the shared queue. Because it keeps the boss's work small, this option usually gives the better throughput.

Workers can be created on demand, when a request arrives, but creating a thread for every request adds latency. More commonly the application uses a *thread pool*, a set of workers created in advance. The pool can be static, or dynamic, growing—typically by several threads at a time rather than one—when the queue gets long and shrinking when workers are idle.

Tip To keep c CPUs busy with workers that are blocked for a fraction b of their time takes about $c/(1-b)$ workers; more add nothing.

The pattern is simple, but it requires thread pool management, and it ignores locality: the boss does not know which worker has just performed a similar task and would perform the next one with a hot cache. A variant therefore specializes the workers for particular types of requests instead of making all workers equal. Specialized workers have better locality, and the application can provide different quality of service by assigning more threads to more important request types; the cost is load balancing—it is harder to decide how many threads each type should receive.

3.10.2 Pipeline

In the *pipeline pattern* the task is divided into a sequence of subtasks, each performed by a thread (or a group of threads) dedicated to it. Stages communicate through shared buffers, so several requests are processed concurrently, each in a different stage (Figure 3.8). The throughput is determined by the weakest link, the slowest stage. The remedy is to make each stage a thread pool and give slow stages more threads. If stage i takes time t_i per request and has p_i threads, the throughput is

A shell pipeline `a | b | c` is this pattern with processes: stages joined by a kernel buffer of 64 KiB by default on Linux (`pipe(7)`).

$$X = \min_i \frac{p_i}{t_i}. \tag{3.2}$$

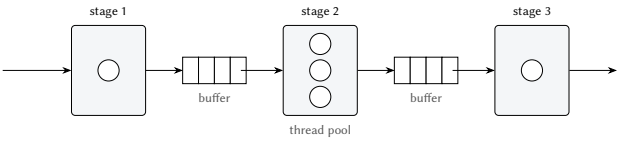


Figure 3.8: A three-stage pipeline. Stages exchange requests through buffers; the slow second stage is a pool of three threads.

The pipeline offers specialization and locality, since every thread repeatedly performs the same subtask; its costs are the effort of keeping the stages balanced and the synchronization overhead of the buffers.

Example 3.14. A three-stage pipeline has stage times 10 ms, 40 ms and 20 ms. With one thread per stage, Equation (3.2) gives $\min(100, 25, 50) = 25$ requests per second, limited by

stage 2. With four threads in stage 2 the throughput becomes $\min(100, 100, 50) = 50$ requests per second, and stage 3 is now the bottleneck.

3.10.3 Layered

A three-tier web application (presentation, logic, data) is a typical instance, each layer with its own thread pool.

In the *layered pattern* related subtasks are grouped into layers, each served by its own threads, and every request passes down through all the layers and back up (Figure 3.9). Like the pipeline it provides specialization, while being less fine-grained. It is not suitable for all applications, and it requires synchronization between each layer and the layers above and below it.

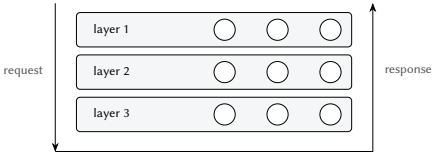


Figure 3.9: The layered pattern: each layer groups related subtasks and has its own threads; a request passes down through the layers and back up.

3.10.4 Comparing boss-workers and pipeline

Lesson 6 compares multi-process, multithreaded and event-driven web servers: Apache's worker MPM is boss-workers with one pool per process (`ThreadsPerChild` = 25 by default), and Flash (AMPED) runs one event loop plus helpers for disk I/O.

Which pattern completes a batch of requests faster depends on the number of requests. Consider n requests, each needing the same total processing time. A boss-workers design with w workers and a negligible boss time t_{boss} completes requests in groups of w, each group taking the full task time t; a pipeline of k equal stages of time s completes the first request after k stage times and then one request per stage time:

$$T_{\text{bw}}(n) = \left\lceil \frac{n}{w} \right\rceil t, \qquad T_{\text{pl}}(n) = (k + n - 1)\, s. \tag{3.3}$$

Caution The formulas compare the completion time of a whole batch. A pipeline does not shorten a single request: it still passes through all k stages, and it waits in a buffer whenever the next stage is busy.

Example 3.15. Both designs use four threads, and a request needs 100 ms of work. The boss-workers design has one boss and $w = 3$ workers with $t = 100$ ms; the pipeline has $k = 4$ stages of $s = 25$ ms. Boss-workers completes 3 requests per 100 ms and the full pipeline 4, so boss-workers can lead only in small batches, where the pipeline's start-up time $k\,s$ still counts. Even there the outcome depends on n, because T_{bw} of Equation (3.3) grows in steps: boss-workers is faster for $n = 2$, 3 and 6, the pipeline for $n = 4$, 7 and 8, the two tie for $n = 1$, 5 and 9, and for every $n \geq 10$ the pipeline wins:

n	Boss-workers (ms)	Pipeline (ms)
3	100	150
4	200	175
6	200	225
12	400	375

Key point — Lesson summary

A thread is an execution context that shares its process's address space; threads provide parallelism, specialization and the use of idle time (Equation (3.1)) more cheaply than processes. Shared data must be protected with mutexes, and threads wait for conditions with condition variables, always re-testing the condition in a `while` loop. Proxy variables build richer policies such as readers-writers; signalling outside the mutex avoids spurious wake-ups; a consistent lock order avoids deadlocks. User-level threads run on kernel-level threads according to the one-to-one, many-to-one or many-to-many model, and applications are structured as boss-workers, pipelines or layers.

Threads Case Study: PThreads

Lecture	P2L3
Textbook	OSTEP [1] ch. 27, 30; Silberschatz et al. [2] ch. 4, 7; Kerrisk [11] ch. 29–30
Keywords	POSIX, POSIX threads, thread creation, thread attributes, joinable thread, detached thread, thread arguments, mutex, trylock, condition variable, safety tips, producer-consumer problem, bounded buffer

Lesson 3 described threads and the mechanisms with which they coordinate in the abstract notation of Birrell [8]: `Fork` and `Join`, mutexes and condition variables. This lesson studies a concrete interface that provides these mechanisms and that most Unix-like operating systems support: POSIX threads, or Pthreads. For each mechanism it presents the corresponding data types and calls, the details that a real interface adds to Birrell's model—thread attributes, detached threads, initialization of synchronization constructs—and rules for using them safely. It ends with a complete producer-consumer program.

4.1 POSIX threads

The *Portable Operating System Interface* (POSIX) is a family of IEEE standards that specifies the interface an operating system provides to applications. It defines the names, arguments and semantics of system calls and library functions, but not how an operating system implements them. Its purpose is portability: a program written against POSIX can be compiled and run on every operating system that complies with the standard, not only on the one for which it was first written.

Source P2L3, lesson preview; POSIX.1 [12]; OSTEP ch. 27.

Definition 4.1 (POSIX threads). The part of POSIX that specifies the interface for creating and managing threads, together with the synchronization constructs with which threads coordinate—mutexes and condition variables—, is called *POSIX threads*, or *Pthreads*.

POSIX.1 is maintained jointly by IEEE and The Open Group and forms the core of the Single UNIX Specification; this lesson cites the 2017 edition [12].

Pthreads is part of the base POSIX.1 standard. Like the rest of POSIX, it leaves the implementation open: a Pthreads library may, for example, map its threads onto kernel-level threads according to any of the multithreading models of Lesson 3. The names of its functions and types begin with `pthread_` and those of its constants with `PTHREAD_`; the data types end in `_t` and are opaque, so a program manipulates them only through the functions of the interface.

Linux's Pthreads library is NPTL, which gives each thread its own kernel task (`clone(2)`), i.e. the one-to-one model; Lesson 5 examines it.

The interface closely follows the mechanisms of Lesson 3. Table 4.1 lists the Pthreads counterpart of each of Birrell's operations; the following sections take them in turn.

Table 4.1: Birrell's mechanisms (Lesson 3) and their Pthreads counterparts. `t`, `m` and `c` denote a thread, a mutex and a condition variable.

Operation	Birrell	Pthreads
thread type	`Thread`	`pthread_t`
create	`Fork(proc, args)`	`pthread_create(&t, &attr, proc, arg)`
join	`Join(t)`	`pthread_join(t, &status)`
mutex type	`Mutex`	`pthread_mutex_t`
lock	`Lock(m) {`	`pthread_mutex_lock(&m)`
unlock	`}`	`pthread_mutex_unlock(&m)`
condition type	`Condition`	`pthread_cond_t`
wait	`Wait(m, c)`	`pthread_cond_wait(&c, &m)`
signal	`Signal(c)`	`pthread_cond_signal(&c)`
broadcast	`Broadcast(c)`	`pthread_cond_broadcast(&c)`

4.2 Thread creation

Source P2L3, PThread creation; OSTEP ch. 27; Kerrisk ch. 29.

Caution Opaque also means that two identifiers are compared with `pthread_equal(t1, t2)` and not with `==`: POSIX allows `pthread_t` to be a structure.

In Pthreads a thread is represented by a variable of type `pthread_t`, the counterpart of Birrell's `Thread`. It identifies the thread, and through it the library reaches the thread's data structure with its identifier, its execution state and the other information needed to manage it. Threads are created and joined with two functions:

```c
#include <pthread.h>

int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
int pthread_join(pthread_t thread, void **status);
```

A create-and-join round trip costs on the order of 10 µs on current x86-64 Linux; a thread pool (Lesson 3) pays it once per worker.

`pthread_create` corresponds to `Fork(proc, args)`. The new thread starts by executing `start_routine(arg)`, and the identifier of the new thread is stored in `*thread`. The start routine takes and returns a `void *`, so that a pointer to data of any type can be passed to the thread; several arguments are packed into a structure and passed as a pointer to it. The second argument, `attr`, has no counterpart in Birrell's `Fork`; it is described below. The call returns 0 when the thread has been created and an error number otherwise, for instance when the system lacks the resources for another thread.

`pthread_join` corresponds to `Join(t)`: it blocks the caller until `thread` has terminated. A thread terminates when its start routine returns or when it calls `pthread_exit(status)`. The value it returned, or passed to `pthread_exit`, is stored in `*status`; a caller that does not need the result passes `NULL`. Like `pthread_create`, `pthread_join` returns 0 on success and an error number on failure.

Caution Joining the same thread twice, or joining a detached thread, is undefined behaviour in POSIX, not merely an error return (POSIX.1).

4.2.1 Thread attributes

Definition 4.2 (Thread attribute). A property of a thread that is set when the thread is created—such as its stack size, whether it can be joined, its scheduling policy and priority, or its scope—is a *thread attribute*. The attributes of a new thread are passed to `pthread_create` in an attribute object of type `pthread_attr_t`.

Default stack: the `RLIMIT_STACK` soft limit, usually 8 MiB of address space (Lesson 3); the minimum is `PTHREAD_STACK_MIN`, 16 KiB on x86-64 (`pthread_attr_setstacksize(3)`).

Passing `NULL` instead of an attribute object creates a thread with default attributes. An attribute object is itself opaque and is manipulated only through functions:

```c
int pthread_attr_init(pthread_attr_t *attr);
int pthread_attr_destroy(pthread_attr_t *attr);
/* for each attribute xxx: */
int pthread_attr_setxxx(pthread_attr_t *attr, ...);
int pthread_attr_getxxx(const pthread_attr_t *attr, ...);
```

`SCHED_OTHER` is the fair scheduler of Lesson 7 (CFS, replaced by EEVDF in Linux 6.6); `SCHED_FIFO` and `SCHED_RR` are real time, priorities 1–99; only `SCHED_RR` has a quantum, 100 ms by default (`sched_rr_get_interval(2)`).

Linux supports only system scope: `pthread_attr_setscope` rejects `PTHREAD_SCOPE_PROCESS` with `ENOTSUP`, so the return value of every attribute setter must be checked too.

`pthread_attr_init` fills the object with the default values; it must be called before any other function is applied to the object, and `pthread_attr_destroy` releases it. The object is consulted only while `pthread_create` runs, so it can be destroyed, or reused for further threads, as soon as the thread exists. Table 4.2 lists the attributes that matter in this course. The scope attribute selects between the system scope and the process scope of Lesson 3: with `PTHREAD_SCOPE_SYSTEM` the thread competes for the CPU with all threads in the system, with `PTHREAD_SCOPE_PROCESS` only with the threads of its own process.

Table 4.2: Thread attributes and the functions that set them. Each has a matching `get` function.

Attribute	Set with	Meaning
stack size	`pthread_attr_setstacksize`	size of the thread's stack
detach state	`pthread_attr_setdetachstate`	joinable (default) or detached (Section 4.3)
scope	`pthread_attr_setscope`	system or process scope
inheritance	`pthread_attr_setinheritsched`	inherit policy and priority from the creator, or take them from the object
scheduling policy	`pthread_attr_setschedpolicy`	e.g. `SCHED_OTHER`, `SCHED_FIFO`, `SCHED_RR`
priority	`pthread_attr_setschedparam`	priority within the policy

4.3 Detached threads

Definition 4.3 (Joinable thread). A *joinable thread* is a thread that another thread can join: when it terminates, its identifier and exit status are kept until a `pthread_join` collects them. A Pthreads thread is joinable unless it is created detached or detached later.

Source P2L3, detaching PThreads; Kerrisk ch. 29.

The threads of a process are peers: any of them, not only the thread that created it, may join a joinable thread. The usual arrangement is nevertheless that a parent thread creates children and joins them later, and the joins do more than collect results. A joinable thread that has

terminated but has not been joined remains a *zombie* thread: its identifier and exit status are kept until some thread joins it, and if none ever does, they occupy resources for as long as the process lives. A main thread must in addition not return from `main` before its children have finished, because returning ends the whole process and with it every thread that is still running.

The same accounting as for zombie processes (Lesson 2); a server that never joins its workers eventually cannot create new ones.

Definition 4.4 (Detached thread). A *detached thread* is a thread that cannot be joined. When it terminates, its resources are released immediately.

A thread can be detached after its creation, by itself or by another thread, or it can be created detached by setting the detach state in its attribute object:

```
int pthread_detach(pthread_t thread);

pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
```

Tip A thread that nobody will join detaches itself with `pthread_detach(pthread_self())` as its first statement.

A detached thread cannot be made joinable again. Detaching removes the need to join: nobody has to collect the thread's status, and its resources are reclaimed as soon as it terminates. It does not change the relation between the threads, which are peers in either case. A thread ends without ending the process by calling `pthread_exit(status)`: it terminates only the calling thread, and the process continues until its last thread has terminated, whether the remaining threads are joinable or detached (Figure 4.1).

Caution Returning from `main` or calling `exit` ends the whole process, detached threads included.

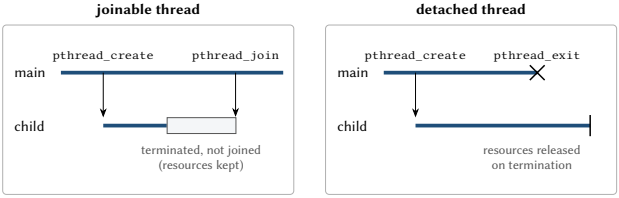


Figure 4.1: Left: a joinable child that terminates before it is joined keeps its resources until `pthread_join` collects its status. Right: a detached child releases its resources itself; the main thread ends with `pthread_exit`, which leaves the process running whether the remaining threads are joinable or detached.

Example 4.5. The main thread starts a detached logging thread with system scope and exits, leaving the logger to finish its work. The logger is an ordinary start routine:

```
void *logger(void *arg) {            /* thread main */
  printf("logger: flushing records\n");
  pthread_exit(NULL);
}
```

The main thread sets up the attribute object, creates the logger and exits:

```
int main(void) {
  pthread_t tid;
  pthread_attr_t attr;

  pthread_attr_init(&attr);          /* required */
  pthread_attr_setdetachstate(&attr,
                              PTHREAD_CREATE_DETACHED);
  pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
  pthread_create(&tid, &attr, logger, NULL);
  pthread_attr_destroy(&attr);
  pthread_exit(NULL);                /* not: return 0 */
}
```

Had `main` ended with `return 0`, the process could terminate before the logger prints anything.

Key point
A Pthreads thread is joinable unless it is detached. Join every joinable thread, or detach threads whose result nobody needs; a main thread that must not end the process calls `pthread_exit` instead of returning.

4.4 Compiling Pthreads programs

Source P2L3, compiling PThreads; OSTEP ch. 27.

A program that uses Pthreads needs three things.

1. **The header.** Every source file that uses Pthreads includes `<pthread.h>`, which declares the types, constants and functions.
2. **The compiler flag.** The program is compiled and linked with `-pthread` or `-lpthread`:

   ```
   gcc -o main main.c -pthread
   gcc -o main main.c -lpthread
   ```

   On GNU/Linux `-pthread` also defines `_REENTRANT`; since glibc 2.34 the Pthreads functions are part of `libc` itself.

   `-lpthread` only links the Pthreads library, whereas `-pthread` also sets the preprocessor definitions that the platform requires for multithreaded code; it is the more portable choice.
3. **Checked return values.** Pthreads functions report failure through their return value, 0 on success and an error number otherwise, rather than through `errno`. A caller that cannot continue reports the number, e.g. with `strerror`, and exits. A program that ignores the result of `pthread_create` may later join a thread that was never created, using an identifier that was never set.

The race detector `-fsanitize=thread` used below must be given at link time too, or the link fails on undefined `__tsan` symbols; `-g` adds source lines to its reports.

Caution Like the other `posix_*` functions, Pthreads does not follow the system-call convention (return −1, set `errno`); `errno` is unreliable, so `perror` may print an unrelated value.

4.5 Thread creation examples

Source P2L3, PThread creation examples 1–3; OSTEP ch. 27.

Three small programs show how threads are created and joined and how data reach a new thread.

4.5.1 Creating and joining several threads

The first program creates a fixed number of threads with default attributes and waits for all of them:

```c
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 3

void *greet(void *arg) {            /* thread main */
  printf("Hello from a worker\n");
  return NULL;
}

int main(void) {
  pthread_t tid[NUM_THREADS];
  int i;

  for (i = 0; i < NUM_THREADS; i++)  /* create */
    pthread_create(&tid[i], NULL, greet, NULL);
  for (i = 0; i < NUM_THREADS; i++)  /* join */
    pthread_join(tid[i], NULL);
  return 0;
}
```

The lines do not interleave: POSIX has each stdio call lock its stream (`flockfile(3)`), so the `printf`s need no mutex.

The program has two phases: the first loop creates the threads, the second joins them. Each worker prints one line. The workers share no data, so they need no synchronization, and because all of them print the same text, the output does not depend on the order in which they are scheduled. The joins keep `main` from returning, and thereby terminating the process, before every worker has printed its line.

4.5.2 Passing an argument

Each worker is to print its own index. The start routine reads the index through its argument:

An alternative passes the index by value, as `(void *)(intptr_t)i`, and casts it back: no shared variable, but the conversion is implementation-defined.

```
void *show_index(void *arg) {       /* thread main */
  int *p = (int *)arg;
  int my_num = *p;
  printf("Thread %d\n", my_num);
  return NULL;
}
```

A first attempt passes it the address of the loop variable:

```
for (i = 0; i < NUM_THREADS; i++)
  pthread_create(&tid[i], NULL, show_index, &i);
```

The program does not reliably print 0, 1 and 2. Every thread receives the same address, that of the single variable `i` in the stack frame of `main` (Figure 4.2, left), and a thread reads the value only when it executes `*p`. By then `main` may already have incremented `i`. Because `i` is visible to all threads that hold its address, a change made by one thread is seen by all of them, and a thread that reads `i` while `main` modifies it is in a data race (Lesson 3). A worker can therefore print an index that belongs to another worker, or even a value that `i` takes only after the creation loop: 3 when the loop ends, or a value set by the join loop, which reuses `i`. Table 4.3 shows one possible execution.

Tip Race detectors find this reliably: build with `-fsanitize=thread`, or run under `valgrind --tool=helgrind`.

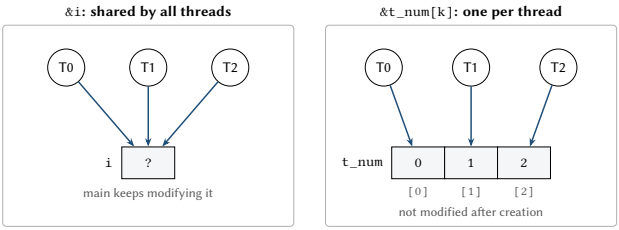


Figure 4.2: Left: all threads receive the address of the loop variable `i`, whose value changes while they start. Right: each thread receives the address of its own array element, which does not change after the thread has been created.

Table 4.3: One execution of the program that passes `&i` with three threads. T0 reads `i` after `main` has incremented it and T2 reads it after the join loop has reset it; only T1 prints its own index, and the program prints 1, 1 and 0.

Step	Main thread	i	Workers
1	create T0 with `&i`	0	
2	`i++`	1	
3	create T1 with `&i`	1	
4		1	T0 reads `*p`: prints 1
5		1	T1 reads `*p`: prints 1
6	`i++`, create T2 with `&i`	2	
7	`i++`, creation loop ends	3	
8	join loop: `i = 0`, join T0	0	
9		0	T2 reads `*p`: prints 0

4.5.3 Passing private data

The race disappears if each thread receives the address of data that belongs to it alone and that no other thread modifies:

When the creator cannot wait, the usual arrangement is one argument structure per thread, allocated with `malloc` and freed by the thread itself.

```
int t_num[NUM_THREADS];

for (i = 0; i < NUM_THREADS; i++) {
  t_num[i] = i;
  pthread_create(&tid[i], NULL, show_index, &t_num[i]);
}
```

Each worker now reads its own element of `t_num`, whose value was set before the thread was created and never changes afterwards (Figure 4.2, right). Every index is printed exactly once. The *order* of the lines, however, is still not determined: it depends on how the threads are scheduled, and any permutation of the three lines can occur. The array lives in the stack frame of `main`, which does not return before it has joined all threads, so the memory remains valid for as long as the threads use it.

Three lines admit 3! = 6 orders, so a run that prints them in order says nothing about the other five.

Caution Never pass a thread the address of a local variable of a function that may return before the thread has read it.

> **Key point**
> An argument passed by address is shared, not copied. Give every thread its own data—an array element, or a structure allocated for it—that outlives the thread's use of it and that no other thread modifies.

4.6 Mutexes

Pthreads mutexes solve the mutual exclusion problem among concurrent threads, as mutexes do in Lesson 3. A mutex has type `pthread_mutex_t`, and the two basic operations are

Source P2L3, PThread mutexes; OSTEP ch. 27–28; Silberschatz et al. ch. 7.

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

The semantics are those of Birrell's mutex: a thread that locks a free mutex becomes its owner, and a thread that locks a mutex owned by another thread blocks until the mutex is released.[1] The difference is syntactic, but it matters. Birrell's `Lock(m) { ... }` releases the mutex implicitly at the end of the block; in Pthreads the critical section is the code between the two calls, and the program must call `pthread_mutex_unlock` explicitly on every path out of it, including early returns and error paths.

[1] Locking a free mutex costs one atomic read-modify-write instruction in user space: glibc enters the kernel, through `futex(2)`, only when a thread must block or must wake another. An uncontended lock therefore costs tens of nanoseconds and a contended one a system call and a context switch. Drepper, "Futexes Are Tricky" [13].

Example 4.6. The safe list insertion of Lesson 3 in C with Pthreads:

```
struct node { int value; struct node *next; };

struct node *head = NULL;          /* shared list */
pthread_mutex_t list_m = PTHREAD_MUTEX_INITIALIZER;

void safe_insert(int value) {
  struct node *e = malloc(sizeof *e);
  e->value = value;

  pthread_mutex_lock(&list_m);
  e->next = head;                  /* critical section */
  head = e;
  pthread_mutex_unlock(&list_m);
}
```

Only the two statements that read and write the shared head pointer are in the critical section; allocating and filling the new element involves no shared data and is done before the lock is taken, which keeps the critical section short.

4.6.1 Other mutex operations

Three further operations complete the basic mutex interface:

The types are small—40 B for `pthread_mutex_t` and 48 B for `pthread_cond_t` on x86-64 glibc—so a lock can sit inside each structure it protects.

```
int pthread_mutex_init(pthread_mutex_t *mutex,
                       const pthread_mutexattr_t *attr);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

The type is an attribute too: relocking a default mutex is undefined in POSIX (glibc, which maps it to `NORMAL`, deadlocks; Lesson 3), `PTHREAD_MUTEX_ERRORCHECK` returns `EDEADLK`, `PTHREAD_MUTEX_RECURSIVE` counts.

`pthread_mutex_init` initializes a mutex before its first use. Its attribute object, of type `pthread_mutexattr_t`, specifies the behaviour of the mutex, for instance whether it may be shared among processes (Lesson 9); `NULL` selects the defaults. A mutex with default attributes can also be initialized statically, as in Example 4.6, with `PTHREAD_MUTEX_INITIALIZER`.

`pthread_mutex_trylock` locks the mutex if it is free and returns 0. If the mutex is locked, it does not block but returns immediately with the error `EBUSY`. The calling thread can then do something else and try again later.

`pthread_mutex_destroy` frees the data structures of a mutex that is no longer needed. The mutex must not be locked.

`pthread_mutex_timedlock` (POSIX.1) blocks only up to an absolute deadline and then returns `ETIMEDOUT`.

Example 4.7. A thread updates optional statistics only when the mutex happens to be free, rather than delaying its main work:

```
if (pthread_mutex_trylock(&stats_m) == 0) {
  stats.requests += local_count;
  pthread_mutex_unlock(&stats_m);
  local_count = 0;
}  /* else: keep counting locally, retry later */
```

Calling `trylock` in a loop turns it into a spinlock, which burns CPU while another thread holds the mutex; Lesson 10 compares spinning with blocking.

4.6.2 Mutex safety tips

The most common errors with mutexes are avoided by a few rules.

- **One mutex per piece of shared data.** All accesses to a shared variable must go through the same mutex. If different code paths protect the same data with different mutexes, they exclude nothing from each other.
- **A mutex must be visible to all threads that use it.** It is therefore declared at global scope, or in a structure that all these threads reach. A mutex declared as a local variable of the thread function is a different mutex in every thread.
- **Lock mutexes in a global order.** If every thread acquires mutexes in the same order, the deadlocks of Lesson 3 cannot occur.
- **Always unlock, and unlock the correct mutex.** A forgotten unlock blocks every other thread that needs the mutex; unlocking the wrong mutex leaves one critical section unprotected and another one locked.

Tip Number the mutexes and always lock in increasing order; a thread that then needs a lower-numbered one releases what it holds and starts again.

4.7 Condition variables

Pthreads condition variables, of type `pthread_cond_t`, provide Birrell's three operations.

Source P2L3, PThread condition variables; OSTEP ch. 27, 30.

```
int pthread_cond_wait(pthread_cond_t *cond,
                      pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

The semantics are those of Lesson 3. `pthread_cond_wait` is called with `mutex` held; it atomically releases the mutex and places the calling thread on the wait queue of `cond`, and when the thread is woken up, it re-acquires the mutex before it returns. `pthread_cond_signal` wakes up at least one of the waiting threads, `pthread_cond_broadcast` all of them; both have no effect when no thread is waiting.

`pthread_cond_timedwait` takes an absolute deadline, on `CLOCK_REALTIME` by default (`pthread_condattr_setclock`), not a relative timeout (POSIX.1).

Caution The argument order is the reverse of Birrell's `Wait(m, c)`: `pthread_cond_wait` takes the condition variable first.

POSIX allows `pthread_cond_wait` to return even though no thread has signalled, and calls such a return a spurious wake-up too; it is a different case from the one examined in Lesson 3, where a thread is woken by a notification that it cannot use. Both leave the woken thread unable to proceed, and both are handled by the same rule: a woken thread tests its predicate again, so the `while` loop of Lesson 3 is not merely good practice but required:[2]

```
pthread_mutex_lock(&m);
while (!predicate)
  pthread_cond_wait(&c, &m);
/* predicate holds and m is held */
update_shared_state();
pthread_mutex_unlock(&m);
```

[2] The rationale of POSIX.1 [12] explains the licence: requiring that a thread never return from a wait in vain would make condition variables slower on a multiprocessor, and an implementation may legitimately implement `pthread_cond_signal` as a broadcast. As the predicate has to be tested anyway, the loop costs nothing and makes a program tolerant of extra notifications.

Like mutexes, condition variables are initialized and destroyed explicitly:

```
int pthread_cond_init(pthread_cond_t *cond,
                      const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

The attribute object of type `pthread_condattr_t` specifies the behaviour of the condition variable, for instance whether it is shared among processes; `NULL` selects the defaults, which the static initializer `PTHREAD_COND_INITIALIZER` also provides.

Sharing one between processes requires `PTHREAD_PROCESS_SHARED` on both the condition variable and its mutex, and both must lie in shared memory (Lesson 9).

4.7.1 Condition variable safety tips

- **Do not forget to notify waiting threads.** Whenever a thread changes the state that a predicate depends on, it must signal or broadcast the condition variable on which the threads that wait for this predicate are blocked.

With n waiters a broadcast costs n wake-ups and n mutex acquisitions, $n-1$ of them wasted—the thundering herd of Lesson 3.

- **When in doubt, broadcast.** Broadcasting where a signal would do is still correct, because every woken thread re-tests its predicate and waits again if it cannot proceed; it costs performance, since these threads are woken up and scheduled for nothing.
- **A mutex is not needed to signal or broadcast.** When the decision to notify does not depend on shared state that can only be read under the mutex, the notification can be issued after the mutex has been released: the woken threads then do not have to wait for a mutex that the signalling thread still holds, which is the cause of spurious wake-ups examined in Lesson 3. Returns from `pthread_cond_wait` without any notification remain possible, so this does not make the `while` loop unnecessary.

> **Key point**
> Pthreads mutexes and condition variables have Birrell's semantics; what they add is explicit unlocking, explicit initialization and destruction, and attribute objects. Wait in a `while` loop, notify whenever the predicate may have changed, and issue the notification after unlocking where the decision to notify allows it.

4.8 A producer-consumer example

Source P2L3, producer and consumer example using PThreads; OSTEP ch. 30.

`num` can be dropped by leaving one slot empty: the buffer is then empty when `add == rem` and full when `(add + 1) % BUF_SIZE == rem`.

The producer-consumer problem of Lesson 3 can now be solved with a complete Pthreads program. A producer thread inserts values into a shared *bounded buffer*, an array of `BUF_SIZE` slots used as a circular queue, and a consumer thread removes them in the order in which they were inserted. Four variables describe the buffer: the array `buffer`, the index `add` of the next slot to fill, the index `rem` of the next slot to empty, and the number `num` of values in the buffer. Both indices advance modulo `BUF_SIZE`, so the queue wraps around the end of the array (Figure 4.3).

One mutex `m` protects these variables; the producer waits on `c_prod` while the buffer is full, the consumer on `c_cons` while it is empty. All shared state is global, as the mutex safety tips require, and initialized statically:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define BUF_SIZE  4     /* slots in the shared buffer */
#define NUM_ITEMS 10    /* values to produce          */

int buffer[BUF_SIZE];   /* shared circular buffer     */
int add = 0;            /* next slot to fill          */
int rem = 0;            /* next slot to empty         */
int num = 0;            /* values in the buffer       */

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c_cons = PTHREAD_COND_INITIALIZER;
pthread_cond_t c_prod = PTHREAD_COND_INITIALIZER;

void *producer(void *param);
void *consumer(void *param);
```

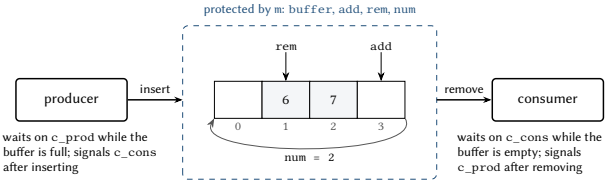


Figure 4.3: The producer-consumer program with `BUF_SIZE` = 4: one mutex protects all buffer state, and each thread waits on its own condition variable.

The main thread creates and joins the two threads:

This is the bounded buffer of OSTEP ch. 30; OSTEP ch. 31 builds the same program with semaphores.

```
int main(void) {
  pthread_t tid1, tid2;

  if (pthread_create(&tid1, NULL, producer, NULL) != 0 ||
      pthread_create(&tid2, NULL, consumer, NULL) != 0) {
    fprintf(stderr, "cannot create threads\n");
    exit(1);
  }
  pthread_join(tid1, NULL);   /* wait for producer */
  pthread_join(tid2, NULL);   /* wait for consumer */
  printf("parent done\n");
  return 0;
}
```

The producer inserts the values $1, \ldots,$ NUM_ITEMS:

Tip With BUF_SIZE a constant power of two the compiler already avoids the division; declaring add and rem unsigned, or writing & (BUF_SIZE - 1), reduces the wrap to a single AND.

```
void *producer(void *param) {
  int i;
  for (i = 1; i <= NUM_ITEMS; i++) {
    pthread_mutex_lock(&m);
    while (num == BUF_SIZE)         /* full: wait  */
      pthread_cond_wait(&c_prod, &m);
    buffer[add] = i;                /* not full    */
    add = (add + 1) % BUF_SIZE;
    num++;
    pthread_mutex_unlock(&m);
    pthread_cond_signal(&c_cons);   /* not empty   */
    printf("produced %d\n", i);
  }
  return NULL;
}
```

The consumer is symmetric:

Each thread has its own i and n on its own stack (Lesson 3); buffer, add, rem and num are the only shared state.

```
void *consumer(void *param) {
  int i, n;
  for (n = 0; n < NUM_ITEMS; n++) {
    pthread_mutex_lock(&m);
    while (num == 0)                /* empty: wait */
      pthread_cond_wait(&c_cons, &m);
    i = buffer[rem];                /* not empty   */
    rem = (rem + 1) % BUF_SIZE;
    num--;
    pthread_mutex_unlock(&m);
    pthread_cond_signal(&c_prod);   /* not full    */
    printf("consumed %d\n", i);
  }
  return NULL;
}
```

The program applies the rules of this lesson and of Lesson 3.

- Each thread tests its predicate in a `while` loop while holding `m`. When the wait returns, the thread holds `m` again, and it re-tests the predicate before it touches the buffer.
- Each thread signals after unlocking, so the woken thread does not have to wait for `m`. A signal when nobody waits has no effect; this is harmless, because a thread tests the predicate before it waits and therefore never waits for a change that has already happened.
- Because producers wait only on `c_prod` and consumers only on `c_cons`, a notification always reaches a thread of the kind that can use it.
- The consumer stops after NUM_ITEMS values. A consumer that loops forever would never terminate, and the second `pthread_join` in `main` would never return.

The `printf` is outside the critical section for the same reason: writing to a terminal can block, and a thread that blocks while holding `m` stops the other one.

Caution With several producers or consumers sharing a single condition variable, a `signal` could wake a producer when only a consumer can proceed; such a design needs `broadcast`.

Example 4.8. Table 4.4 traces one execution of the program in which the producer runs ahead and fills the buffer before the consumer starts. Removed values are shown as `-`.

Table 4.4: A trace of the producer (P) and consumer (C) with `BUF_SIZE` = 4. The buffer column lists slots 0–3; the values of `add`, `rem` and `num` are those after each step.

Step	Event	buffer	add	rem	num
1	P inserts 1	1 - - -	1	0	1
2	P inserts 2, 3, 4	1 2 3 4	0	0	4
3	P finds buffer full, waits on `c_prod`	1 2 3 4	0	0	4
4	C removes 1, signals `c_prod`	- 2 3 4	0	1	3
5	P re-acquires `m`, re-tests, inserts 5	5 2 3 4	1	1	4
6	C removes 2, 3, 4	5 - - -	1	0	1
7	C removes 5	- - - -	1	1	0
8	C finds buffer empty, waits on `c_cons`	- - - -	1	1	0
9	P inserts 6, signals `c_cons`	- 6 - -	2	1	1
10	C re-acquires `m`, re-tests, removes 6	- - - -	2	2	0

Key point — Lesson summary

POSIX threads implement Birrell's mechanisms with concrete types and calls (Table 4.1). `pthread_create` starts a thread with one `void *` argument and an attribute object that sets, for example, its stack size, scope and detach state; joinable threads must be joined, detached threads release their resources themselves, and `pthread_exit` ends a thread without ending the process. Programs include `<pthread.h>`, are built with `-pthread` and check return values, and give every thread private argument data. Each shared resource has one globally visible mutex, locked in a global order and always unlocked; condition variables are waited on in a `while` loop and notified whenever a predicate may have changed, after unlocking where the decision to notify allows it, as in the producer-consumer program.

Lesson 5

Thread Design Considerations

Lecture	P2L4
Textbook	OSTEP [1] ch. 6, 26–28; Silberschatz et al. [2] ch. 4; Kerrisk [11] ch. 20–22, 28
Keywords	hard process state, light process state, lightweight process, red zone, adaptive mutex, reaper thread, interrupt handler, signal handler, signal mask, top half, bottom half, real-time signal, task, NPTL

Lesson 3 introduced kernel-level and user-level threads and the models that map one kind onto the other, and Lesson 4 the POSIX threads interface through which applications use them. This lesson looks inside the implementation: which data structures the kernel and a user-level thread library keep, how the two levels coordinate their decisions, and how interrupts and signals interact with multithreading. The presentation follows two papers on the multithreaded kernel of SunOS 5.0 [14] and its user-level thread library [15], and closes with the way Linux represents threads.

5.1 Thread data structures

Source P2L4, thread data structures: single CPU, at scale; hard and light process state; rationale for data structures; Eykholt et al. [14]; OSTEP ch. 26.

Supporting threads requires data structures at two levels: in the kernel for kernel-level threads, and in the user-level library for user-level threads. A process whose user-level threads all run on one kernel-level thread (the many-to-one model of Lesson 3) is the simplest case. The kernel describes the process by its PCB, which holds the virtual address mappings together with the execution context of the kernel-level thread: its registers, stack pointer and stack. The user-level library keeps a data structure for each user-level thread, with the thread's identifier, its saved registers and its stack. By loading one of these contexts onto the kernel-level thread, the library decides which user-level thread runs.

The PCB is Lesson 2; here its contents are split, so that the threads of one process share the address-space part.

When a process has several kernel-level threads, one PCB can no longer hold everything, because every kernel-level thread needs its own execution context. Replicating the complete PCB for each of them would also replicate the virtual address mappings, which are the same for all threads of the process. The kernel therefore keeps a separate kernel-level thread data structure with the registers, stack pointer and stack of each kernel-level thread, and the PCB keeps the information that is common to the process.

On x86-64 Linux every task carries its own 16 KiB kernel stack (four pages), while the address space it points to is shared. Kernel documentation, `x86/kernel-stacks`.

With many processes, many threads and several CPUs, the relationships among these structures must be recorded as well (Figure 5.1):

- the user-level library tracks all user-level threads of its process and the kernel-level threads on which it can run them;
- the PCB tracks the kernel-level threads that execute on behalf of the process;
- each kernel-level thread points to the PCB of its process, so that the kernel finds the address space when it runs the thread;
- a data structure for each CPU records, among other things, the thread that currently runs on it, and a running kernel-level thread points to its CPU.

5.1.1 Hard and light process state

When the kernel switches between two kernel-level threads of the same process, the virtual address mappings remain valid, and only the state that belongs to the individual thread has to be saved and restored. Some of that state concerns the user-level threads: which signals are masked and the arguments of a system call in progress belong to the user-level thread that currently runs on a kernel-level thread, not to the whole process. The state of a process thus divides into two parts.

Tip A field is hard state if it is valid for every thread of the process, light state if it is valid only for the thread currently executing.

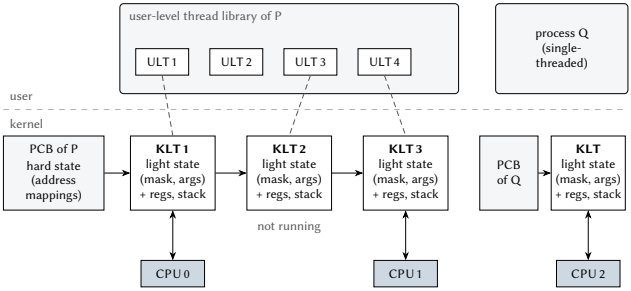


Figure 5.1: Thread data structures at scale. The PCB of a process keeps the state shared by all its threads and a list of its kernel-level threads (KLTs); each KLT keeps its own state and, while it runs, is linked to a CPU. Dashed lines show which user-level threads (ULTs) the library has currently placed on which KLTs. Back pointers from KLTs to PCBs are omitted.

Definition 5.1 (Hard and light process state). The *hard process state* is the part of the state of a process that is relevant to all user-level threads executing within it, such as the virtual address mappings. The *light process state* is the part that is relevant only to the subset of user-level threads currently associated with a particular kernel-level thread, such as the signal mask and the system call arguments.

Caution The two terms come from the lecture; textbooks speak of per-process and per-thread state.

5.1.2 Why several data structures

A single PCB and a set of smaller linked structures lead to different costs (Table 5.1). The single PCB is a large contiguous structure, private to every execution context even where its contents are shared; it is saved and restored in full on every context switch, and any change of state updates it. Smaller structures that refer to one another through pointers keep shared state once, let a context switch save and restore only what changes, and let the user-level library update the part of the state it manages without involving the kernel. The split improves scalability, performance and flexibility, and reduces overhead.

Table 5.1: A single PCB compared with several smaller data structures.

	Single PCB	Several data structures
Structure	one large, contiguous structure	small structures linked by pointers
Sharing	private to each execution context	shared state kept once
Context switch	saved and restored in full	only the part that changes
Updates	every change modifies the PCB	the library updates only its part
Consequences	poor scalability, high overhead, lower performance, little flexibility	better scalability, lower overhead, higher performance, more flexibility

5.2 Threads in SunOS 5.0

Source P2L4, SunOS 5.0 threading model, user-level thread data structures, kernel-level data structures; Eykholt et al. [14]; Stein and Shah [15].

SunOS 5.0, the kernel of Solaris 2, was designed for shared-memory multiprocessors, and the kernel itself is multithreaded. Its threading model (Figure 5.2) has the following elements:

- processes may be single-threaded or multithreaded; the user-level library supports both the many-to-many and the one-to-one model, including bound threads, and a process may run all its user-level threads on a single lightweight process;
- every kernel-level thread that executes user-level threads is associated with a lightweight process;
- other kernel-level threads have no lightweight process; they perform work of the kernel itself;
- the kernel-level scheduler schedules the kernel-level threads onto the CPUs.

Definition 5.2 (Lightweight process). In SunOS 5.0 a *lightweight process* (LWP) is the kernel data structure associated with a kernel-level thread that executes user-level threads; it holds the light process state of that execution context. To the user-level library an LWP represents a virtual CPU onto which it schedules user-level threads.

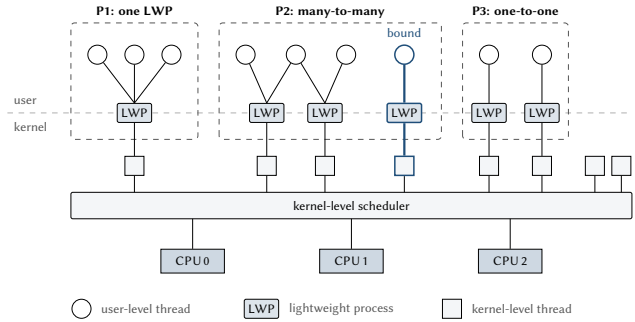


Figure 5.2: The SunOS 5.0 threading model. Each kernel-level thread that runs user-level threads has an LWP; the two kernel-level threads on the right perform kernel work and have none. P1 runs all its user-level threads on a single LWP. The kernel-level scheduler places kernel-level threads on CPUs.

5.2.1 User-level thread data structures

When the library of Stein and Shah creates a thread, it returns a thread identifier. The identifier is not a pointer to the thread data structure but an index into a table of pointers to the structures. The indirection protects against stale identifiers: if a thread has exited and the memory of its structure has been reused, a direct pointer would silently refer to unrelated data, whereas the table entry can record the state of the thread so that the library returns a meaningful error.

The thread data structure contains the execution context (the registers, including the stack pointer), the signal mask, the priority, the *thread-local storage*, and the stack. Thread-local storage holds the variables that the program declares as private to each thread. Their total size is computed by the run-time linker when the program starts, and the library allocates the thread-local storage of every new thread at the base of its stack; the thread structure itself is kept in thread-local storage. The stack size is set by a library default or by the application. Since the size of all parts is known when the thread is created, the library can allocate the structures of successive threads next to one another in contiguous memory, which gives good locality.

Caution Lecture vs paper: the lecture says that thread-local storage is known at compile time, so that space can be reserved for it in every thread. In Stein and Shah its size is fixed only at program start, by the run-time linker.

Contiguous allocation has a danger. The library does not control how far a stack grows, and the operating system does not know about user-level threads at all. A thread whose stack overflows therefore writes into the data structure of the neighbouring thread, and the damage shows only when that other thread runs and fails—far from the cause.

Definition 5.3 (Red zone). A *red zone* is a region of virtual address space between a thread's stack and the neighbouring thread's data structure that is not mapped to memory, so that any access to it causes a fault.

Caution An unrelated meaning: in the x86-64 ABI the *red zone* is the 128 B below the stack pointer that a leaf function may use without adjusting it.

A stack that grows into its red zone causes a fault at once, in the thread whose stack overflowed, which makes the error easy to diagnose (Figure 5.3).

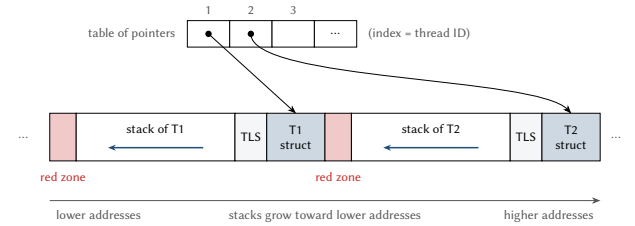


Figure 5.3: User-level thread data structures allocated contiguously. The thread identifier indexes a table of pointers to the structures. An overflow of the stack of T2 runs into the red zone before it can reach the structure of T1.

Example 5.4. A library gives each thread a block of 64 KiB: a 4 KiB red zone at the bottom, a 56 KiB stack, and 4 KiB at the top for the thread structure and its thread-local storage. Measured in KiB from the start of the region, block k occupies $[64k, 64k + 64)$. Thread 3 thus has its red zone at $[192, 196)$, its stack at $[196, 252)$ and its structure at $[252, 256)$, while the structure of thread 2 lies at $[188, 192)$. If thread 3 needs more than 56 KiB of stack, its next push lands in $[192, 196)$ and faults; without the red zone it would overwrite the structure of thread 2. For 1024 threads the region spans 64 MiB of virtual address space, of which the 4 MiB of red zones use no physical memory.

On x86, NPTL keeps the same layout: the thread descriptor sits at the top of the stack block, and glibc puts one guard page below the stack—4 KiB by default, `pthread_attr_setguardsize`.

5.2.2 Kernel-level data structures

The kernel of SunOS 5.0 keeps four kinds of structures for processes and threads (Table 5.2). The LWP resembles the data structure of a user-level thread, but it is visible to the kernel. Its contents are needed only while the process runs, so the LWP can be swapped out together with the process. The kernel-level thread structure holds information that the kernel needs even when the process is not running, such as scheduling information, and it is never swapped out. The CPU structure identifies the thread currently running on each CPU.

On SPARC processors, which have spare registers, SunOS keeps the pointer to the current thread in a dedicated register. Linux keeps one run queue, `struct rq`, per CPU and a per-CPU pointer to the current task (`kernel/sched`); the SunOS CPU structure plays the same role.

Table 5.2: Kernel data structures for processes and threads in SunOS 5.0.

Structure	Contents
Process	list of kernel-level threads, virtual address space, user credentials, signal handlers
LWP	user-level registers, system call arguments, resource usage information, signal mask; swappable
Kernel-level thread	kernel-level registers, stack pointer, scheduling information, pointers to the associated LWP, process and CPU; not swappable
CPU	current thread, idle thread, dispatching and interrupt-handling information

5.3 Interaction between the library and the kernel

The user-level library does not know what happens in the kernel, and the kernel does not know what happens in the library. System calls and special signals let the two levels coordinate. The application tells the library how many of its threads should run at the same time—its *concurrency level*, set with `pthread_setconcurrency`—and the library asks the kernel for kernel-level threads through system calls, requesting more when it needs them; the kernel can notify the library of events that affect its threads, such as a kernel-level thread blocking. (On Linux `pthread_setconcurrency` has no effect: with a one-to-one library there is nothing to choose, and POSIX lets an implementation ignore the hint.)

Source P2L4, basic thread management interactions, thread management visibility and design; Stein and Shah [15]; Silberschatz et al. ch. 4.

Example 5.5. A process has six user-level threads, of which at most three need to run at the same time. Table 5.3 traces how the library and the kernel adjust the number of kernel-level threads.

Caution Lecture vs paper: the lecture speaks of blocking I/O in general. SunOS sends `SIGWAITING` only when every LWP of the process is in an indefinite wait, such as a read on a terminal or a socket, not a disk read, and the library catches it only while it has more threads than LWPs.

Table 5.3: Coordination between a user-level library and the kernel for a process with six user-level threads (U1–U6).

Step	Event	KLTs
1	The library asks the kernel for three kernel-level threads; the kernel creates K1–K3.	K1–K3
2	U1–U3 run on K1–K3; U4–U6 wait on a user-level condition.	3 running
3	U1 and U2 issue blocking reads on a terminal or socket (indefinite waits), and K1 and K2 block in the kernel. Meanwhile U4 becomes ready.	2 blocked, 1 running
4	U3 also issues such a read, so no kernel-level thread is left to run U4 and the whole process would stall. When K3 blocks, the kernel therefore notifies the library—in SunOS 5.0 with the signal `SIGWAITING`; the library requests another kernel-level thread, the kernel creates K4, and U4 runs on it.	3 blocked, 1 running
5	The I/O completes and U1–U3 continue. K4 then stays idle for a long time, and the library destroys it.	K1–K3

5.3.1 Visibility of thread management

Each level sees only part of the picture.

The kernel sees the kernel-level threads, the CPUs and the decisions of the kernel-level scheduler.

The user-level library sees the user-level threads and the kernel-level threads assigned to its process.

The library may bind a user-level thread to a kernel-level thread, as for the bound threads of Lesson 3, and the kernel may pin a kernel-level thread to a CPU; neither decision is visible at the other level.

In POSIX a bound thread is `PTHREAD_SCOPE_SYSTEM`, the only scope NPTL supports; pinning a kernel-level thread to CPUs is the cache affinity of Lesson 7.

The lack of visibility causes problems. Suppose a user-level thread acquires a mutex implemented by the library, and the kernel then preempts the kernel-level thread on which it runs, for instance because its timeslice has expired. The other user-level threads that need the mutex cannot proceed, even when the kernel runs the kernel-level threads on which they are scheduled: nothing can change until the kernel runs the preempted kernel-level thread again, and the kernel, which does not see the mutex, has no reason to do so soon. In the many-to-many model, user-level scheduling decisions and changes to the mapping of user-level threads onto kernel-level threads are visible only to the library. The one-to-one model avoids many of these problems, because the kernel sees every thread.[1]

Being ordinary code in the address space of the process, the library's scheduler runs only when execution enters the library:

- a user-level thread explicitly yields;
- a timer set by the library expires;
- a user-level thread calls a library function, such as locking or unlocking a mutex, that can change which threads are runnable;
- a blocked user-level thread becomes runnable;
- a signal arrives from the kernel, for example on behalf of a timer.

[1] The general remedy for the many-to-many model is to make the kernel report every event that changes the number of running threads. Anderson et al. [16] called such an upcall a *scheduler activation*: the kernel hands the library a fresh activation whenever a thread blocks, unblocks or is preempted, so that the library can reschedule at once; `SIGWAITING` is a narrow form of the same idea. NetBSD and FreeBSD implemented activations and later removed them—like the many-to-many model itself, they cost more than they returned.

5.4 Implementation issues

Multiprocessors raise further problems for the library, and the cost of creating threads suggests reusing them.

Source P2L4, issues on multiple CPUs, synchronization-related issues, destroying threads; Eykholt et al. [14]; OSTEP ch. 28.

5.4.1 User-level threads on multiple CPUs

On a multiprocessor the library runs on several kernel-level threads at the same time, and a decision taken on one CPU can require an action on another. Let T1, T2 and T3 be user-level threads with priorities T3 > T2 > T1, run by the library on two kernel-level threads, K1 and K2, on two CPUs (Figure 5.4). T2 holds mutex `m` on CPU 1. T3 runs on CPU 2 and tries to lock `m`; it blocks, and the library schedules T1 on K2. When T2 unlocks `m`, the library, executing on CPU 1, finds that T3 is runnable again and has a higher priority than T1, which is running. T1 must be preempted, but the library code on CPU 1 cannot modify the registers of CPU 2. It sends a signal to K2; the signal makes K2 execute the library on CPU 2, which preempts T1 and schedules T3.

The signal reaches CPU 2 as an interrupt: if K2 is running there, the kernel sends an inter-processor interrupt to that CPU, which enters the kernel and delivers the signal (in Linux, `kick_process`).

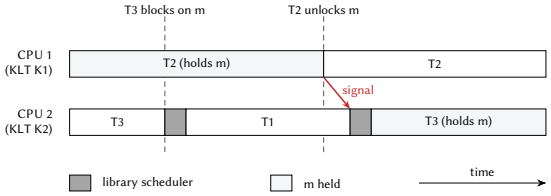


Figure 5.4: User-level threads with priorities T3 > T2 > T1 on two CPUs. When T2 unlocks `m`, the library on CPU 1 signals K2 so that the library on CPU 2 preempts T1 in favour of T3.

5.4.2 Adaptive mutexes

A multiprocessor also changes the best way to wait for a mutex. Suppose one thread holds a mutex on CPU 1 when another thread, running on CPU 2, tries to lock it. Normally the second

thread is placed on the wait queue of the mutex and blocks, which costs a context switch now and another one when it is woken up. If the critical section is short and its owner is running on another CPU, the owner is likely to release the mutex before these context switches would complete, and it is cheaper for the second thread to *spin*: to stay on its CPU and repeatedly check whether the mutex is free. If the owner is not running, spinning is useless, since the mutex cannot be released before the owner is scheduled again.

Tip Spin only if the expected wait is shorter than two context switches; otherwise block.

Definition 5.6 (Adaptive mutex). An *adaptive mutex* is a mutex whose lock operation spins while the owner of the mutex is running on another CPU, and blocks the calling thread otherwise.

Adaptive mutexes make sense only on multiprocessors—on a single CPU the owner cannot be running while another thread spins—and only for short critical sections; long critical sections are better served by blocking. To decide between spinning and blocking, the lock operation must find out whether the owner is running, so the mutex data structure records its owner. Lesson 10 returns to spinning as the basis of the spinlock.

OSTEP calls a lock that first spins for a while and then sleeps a *two-phase lock*; the mutex of the Linux kernel spins only while the owner is running, exactly as here.

5.4.3 Destroying threads

Creating a thread allocates and initializes its data structures and its stack, and destroying it frees them. Because applications create and destroy threads frequently, reusing these structures pays off. When a detached thread exits, its structures are not freed immediately; the thread is placed on a list called the *death row*. (A thread that is not detached waits on a list of zombies until another thread joins it.) A *reaper thread* destroys the threads on the death row and frees their data structures; it runs when the library has idle LWPs or when the death row reaches a limit. The stacks it frees go to a cache, from which the library takes the stack of a new thread, which saves the cost of allocating and initializing it.

glibc does the same for NPTL threads: a freed stack stays in a cache of up to 40 MiB (tunable `glibc.pthread.stack_cache_size`) and is handed to the next `pthread_create`.

Caution Lecture vs paper: in the lecture a new thread reuses the structure of a thread still on the death row, before the reaper runs; in Stein and Shah reuse goes through the reaper's stack cache.

> **Key point**
> The kernel and the user-level library each see only their own threads. System calls and signals let them coordinate, but decisions such as preempting a lock owner or preempting a thread on another CPU need explicit notification across the levels. Several CPUs also change how a thread waits—an adaptive mutex spins only while the owner runs—and threads that exit are kept for reuse instead of being freed at once.

5.5 Interrupts and signals

Source P2L4, interrupts and signals intro; OSTEP ch. 6; Kerrisk [11] ch. 20.

Two kinds of events can divert a thread from its normal execution: interrupts and signals. Signals were introduced in Lesson 1 as notifications that the operating system passes to an application. Interrupts are the corresponding notifications that the hardware delivers to the CPU.

Definition 5.7 (Interrupt). An *interrupt* is an event generated externally to the CPU, by a component such as an I/O device, a timer or another CPU, that notifies the CPU that some condition requires attention.

Which interrupts can occur is determined by the physical platform, and interrupts appear asynchronously, independently of what the CPU is executing. Signals, in contrast, are events triggered by the CPU and the software running on it; which signals exist is determined by the operating system, and they can appear synchronously or asynchronously. The two mechanisms nevertheless have much in common (Table 5.4): both have a unique identifier, defined by the hardware or by the operating system; both can be masked, that is, disabled or suppressed; and when enabled, both trigger a handler. Interrupt handlers are set by the operating system for the entire system, signal handlers by each process for itself.

5.6 Handling interrupts and signals

Source P2L4, interrupt handling, signal handling; OSTEP ch. 6, 36; Kerrisk ch. 20–21.

When a device needs attention, it sends an interrupt to the CPU over the interconnect that links them, such as PCI Express. Early systems used a dedicated physical line for every interrupt; modern devices send the interrupt as a message over the interconnect itself, and the content of the message identifies the interrupt and thereby the device. An interrupt raised by sending such a message instead of asserting a dedicated line is a *message signaled interrupt* (MSI).

Table 5.4: Interrupts and signals.

	Interrupt	Signal
Generated by	devices, timers, other CPUs	the CPU and the software running on it
Defined by	the hardware platform	the operating system
Occurrence	asynchronous	synchronous or asynchronous
Masked	per CPU	per execution context
Handler set	by the operating system, system-wide	by each process

Definition 5.8 (Interrupt handler). An *interrupt handler* is the routine that the operating system executes in response to a particular interrupt. The *interrupt handler table* maps each interrupt number to the start address of its handler.

The CPU uses the interrupt number to look up the interrupt handler table, sets its program counter to the start address found there, and executes the handler (Figure 5.5a). Which interrupts occur is specific to the hardware; how they are handled is specific to the operating system.

The table is also called the interrupt vector table; on x86 it is the interrupt descriptor table.

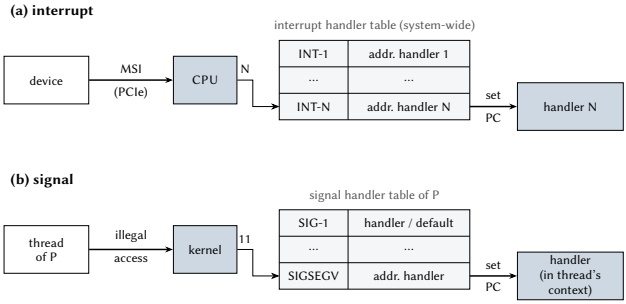


Figure 5.5: (a) An interrupt raised by a device selects a handler from the system-wide interrupt handler table. (b) An illegal memory access makes the kernel raise SIGSEGV, which selects a handler from the signal handler table of the process.

Signals are generated by the operating system. When a thread accesses memory that is not allocated to its process, for example, the operating system raises SIGSEGV (signal 11 on Linux). For each process it keeps a *signal handler table* that maps every signal number to the action to be taken (Figure 5.5b). The set of signals is operating-system specific; POSIX defines a standard set. Every signal has a default action, which is one of the following: terminate the process, ignore the signal, terminate the process and dump core, stop the process, or continue a stopped process.

Linux defines 31 standard signals and a block of real-time ones; glibc reserves the first two of those for NPTL, so SIGRTMIN is 34 and SIGRTMAX 64 (`signal(7)`).

Definition 5.9 (Signal handler). A *signal handler* is a function that a process installs to be executed, in the context of the receiving thread, when a particular signal is delivered.

A process installs handlers with `signal` or, more portably, with `sigaction`. Most signals can be caught in this way; a few cannot, notably SIGKILL and SIGSTOP, whose default actions always apply.

Example 5.10. A program, using the headers `signal.h` and `unistd.h`, that stops its work loop when a timer expires:

Caution A handler may assign only to `volatile sig_atomic_t` objects and call the functions listed in `signal-safety(7)`; `printf` and `malloc` are not among them.

```
static volatile sig_atomic_t expired = 0;

static void on_alarm(int signo) {
    (void)signo;
    expired = 1;            /* keep handlers minimal */
}

int main(void) {
    struct sigaction sa;
```

```
    sa.sa_handler = on_alarm;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGALRM, &sa, NULL);
    alarm(2);               /* SIGALRM after 2 s */
    while (!expired)
        do_work();
    return 0;
}
```

Signals are classified by how they relate to the thread that receives them.

Definition 5.11 (Synchronous and asynchronous signals). A *synchronous signal* occurs in response to an action of a specific thread and is delivered to that thread, for example SIGSEGV on an access to protected memory or SIGFPE on a division by zero. An *asynchronous signal* occurs independently of what the receiving thread executes, for example SIGKILL sent by another process with `kill`, or SIGALRM on the expiration of a timer.

A signal can also be sent explicitly to one thread, for example with `pthread_kill`; like a synchronous signal, it is then delivered to that thread only.

A signal reaches user space only when the receiving thread returns from the kernel (Lesson 1); the kernel consults the pending signals that are not blocked on every such return.

5.7 Disabling interrupts and signals

Interrupt and signal handlers execute in the context of the thread that was interrupted: the program counter of that thread is set to the handler, and the handler runs on its behalf, without the scheduler being involved—a signal handler on the thread's own stack, an interrupt handler on a kernel stack. This creates a problem when the handler accesses shared state. If the interrupted thread holds a mutex `m` and the handler tries to lock `m`, the handler blocks. The owner of `m` is the interrupted thread itself, which cannot continue until the handler returns: the thread deadlocks with its own handler.

Source P2L4, why disable interrupts or signals, more on signal masks, interrupts on multicore systems, types of signals; Kerrisk ch. 20–22.

There are two ways to avoid this. Handler code can be kept simple, never locking a mutex; this is often too restrictive. Or the code can control when handlers may run, by disabling particular interrupts or signals while it holds mutexes that their handlers need.

Definition 5.12 (Interrupt mask and signal mask). An *interrupt mask* or a *signal mask* is a sequence of bits with one bit per interrupt or signal; the value of the bit indicates whether the interrupt or signal is enabled (1) or disabled (0).

A thread that disables a signal around a critical section follows this pattern:

```
clear_field_in_mask(mask);  // disable
lock(mutex) {
  ...                       // handler cannot run here
} // unlock
reset_field_in_mask(mask);  // enable
```

An interrupt or signal that occurs while it is disabled is not lost: it remains *pending*, and its handler runs as soon as the mask enables it again. If the same signal occurs several times while it is disabled, the handler normally runs only once; generating a signal several times does not guarantee that the handler runs several times (Section 5.7.3).

Caution POSIX signal sets list the *blocked* signals: a signal that is a member of the set is disabled, the opposite of the bit convention above.

5.7.1 Setting the masks

Interrupt masks are kept per CPU. If the mask of a CPU disables an interrupt, the hardware interrupt-routing mechanism does not deliver that interrupt to the CPU. The kernel sets the mask by writing to the hardware. Signal masks are kept per execution context: the kernel does not interrupt a kernel-level thread whose mask disables the signal, and where user-level threads run on top of kernel-level threads the kernel sees only the kernel-level masks, which Section 5.9 returns to. A single-threaded process changes its signal mask with `sigprocmask`, a thread of a multithreaded process with `pthread_sigmask`; both take one of the operations SIG_BLOCK, SIG_UNBLOCK or SIG_SETMASK.

Example 5.13. The pattern above written with POSIX calls, for a handler of `SIGALRM` that locks the same mutex:

```
sigset_t set, old;
sigemptyset(&set);
sigaddset(&set, SIGALRM);
pthread_sigmask(SIG_BLOCK, &set, &old);    // disable
pthread_mutex_lock(&m);
/* ... critical section ... */
pthread_mutex_unlock(&m);
pthread_sigmask(SIG_SETMASK, &old, NULL);  // restore
```

Caution `pthread_mutex_lock` is not async-signal-safe; a handler should rather set a flag, and every thread that may run it must block the signal.

5.7.2 Interrupts on multicore systems

On a multicore system an interrupt can be directed to any CPU that has it enabled. Enabling a given interrupt on a single core only, so that only that core handles it, avoids the overhead and the perturbation that handling it would cause on all the other cores.[2]

> **Key point**
> Handlers run in the context of the interrupted thread. Code that holds a mutex needed by a handler must disable the corresponding interrupt or signal while it holds the mutex; an event that occurs meanwhile stays pending until it is enabled again.

[2] On Linux `/proc/interrupts` lists the count of every interrupt source, one column per CPU, and `/proc/irq/N/smp_affinity` holds the mask of the CPUs to which interrupt *N* may be routed. The `irqbalance` daemon, where it runs, adjusts these masks.

5.7.3 Types of signals

Signals differ in what happens when several instances arrive before the handler has run.

One-shot signals With a *one-shot signal*, *n* pending instances are equivalent to one: the handler is guaranteed to run at least once, but not *n* times. On some systems the handler is also reset to the default action after it has run, so the process must re-install it explicitly to catch the signal again.

Real-time signals With a *real-time signal*, instances are queued: if the signal is raised *n* times, the handler is called *n* times. POSIX reserves the range `SIGRTMIN` to `SIGRTMAX` for them.

Example 5.14. A thread blocks both `SIGUSR1`, a standard signal, and the real-time signal `SIGRTMIN`. While they are blocked, each signal is raised three times; then the thread unblocks them. The handler of `SIGUSR1` runs once, because the three instances collapse into one pending signal; the handler of `SIGRTMIN` runs three times.

Caution Queuing depends on the signal number, not on the call: a real-time signal queues whether it is sent with `kill` or `sigqueue`, a standard signal never has more than one pending instance; `sigqueue` with `SA_SIGINFO` only adds a data value (`signal(7)`).

5.8 Interrupts as threads

SunOS 5.0 avoids the deadlock between a thread and its handler in a different way: it handles interrupts as threads. If an interrupt handler runs as a thread of its own, it can block on a mutex like any other thread; it waits until the owner, possibly the interrupted thread, releases the mutex, and the scheduler runs it afterwards. Creating a new thread for every interrupt would be too expensive, so interrupt threads are created and initialized in advance, and whether one of them becomes a full thread is decided at run time. An interrupt switches onto the stack of such a preallocated thread, while the interrupted thread stays pinned beneath it and cannot run; a handler that does not block finishes there, and the pinned thread resumes. Only if the handler blocks on a mutex does the interrupt thread become a complete thread, which the scheduler treats like any other, and the pinned thread continues at once.

Source P2L4, interrupts as threads, performance of threads as interrupts; Eykholt et al. [14].

A related design, which SunOS 5.0 did not adopt, divides the work of interrupt handling into two parts (Figure 5.6). Eykholt et al. reject it because it requires restructuring the device drivers and always costs a full context switch, whereas an interrupt thread becomes a full thread only when its handler blocks.

Of the Linux deferral mechanisms, only work queues and threaded handlers may block.

Definition 5.15 (Top half and bottom half). The *top half* of interrupt handling runs immediately when the interrupt occurs; it does a minimal amount of processing and must be fast and non-blocking. The *bottom half* is the deferred part; it can be of arbitrary complexity and, when a thread executes it, it is scheduled like any other thread and may block.

Handling interrupts as threads is not free. In SunOS 5.0 it adds about 40 SPARC instructions to every interrupt. It also saves work elsewhere: since a handler that needs a mutex can now

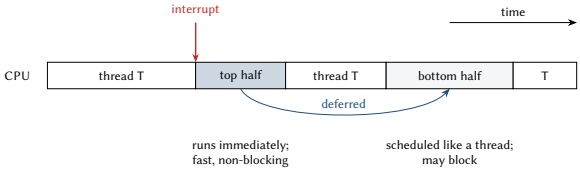


Figure 5.6: Interrupt handling divided into a top half, which interrupts thread T, and a bottom half, which the scheduler runs later.

block, the mutex operations no longer have to change the interrupt mask to keep handlers from running while a mutex is held, which saves about 12 instructions for each mutex operation. With c instructions added per interrupt and s saved per mutex operation, the scheme pays off when

$$c\,N_{\text{int}} < s\,N_{\text{mutex}} \iff \frac{N_{\text{int}}}{N_{\text{mutex}}} < \frac{s}{c}, \tag{5.1}$$

where N_{int} and N_{mutex} are the numbers of interrupts and of mutex operations. The model counts only the fixed cost and assumes that handlers rarely block: converting an interrupt thread into a full thread costs much more, a flush of the SPARC register windows, and Eykholt et al. claim the net gain only as long as interrupts do not block too frequently. Mutexes are locked and unlocked far more often than interrupts occur, so the balance is positive.

Linux makes the same trade: booting with `threadirqs` runs handlers in threads, and under `PREEMPT_RT`, mainline since Linux 6.12, that is the default for every handler not marked `IRQF_NO_THREAD` or per-CPU.

Example 5.16. A system handles 2 000 interrupts and performs 150 000 mutex operations per second. With $c = 40$ and $s = 12$, interrupt threads cost $40 \times 2\,000 = 80\,000$ instructions per second and save $12 \times 150\,000 = 1\,800\,000$, a net saving of 1 720 000 instructions per second. By Equation (5.1) the scheme would lose only with more than $12/40 = 0.3$ interrupts per mutex operation, that is, more than 45 000 interrupts per second at this mutex rate.

> **Key point**
> Optimize for the common case. Interrupts as threads make the rare event, an interrupt, slightly more expensive, in exchange for making the frequent event, a mutex operation, cheaper.

5.9 Threads and signal handling

Source P2L4, threads and signal handling, cases 1–4; Stein and Shah [15].

In a many-to-many implementation, signal masks exist at two levels. The user-level library keeps a mask for each user-level thread in its thread data structure, and the kernel keeps a mask for each kernel-level thread, in its LWP. A user-level thread changes its own mask through the library, without a system call; the kernel sees only the masks of the kernel-level threads. When a signal arrives, the kernel interrupts a kernel-level thread whose mask enables it, whatever the mask of the user-level thread currently running on it.

With NPTL the two levels coincide and the mismatch disappears: `pthread_sigmask` is a system call (`rt_sigprocmask`) that sets the kernel's mask of that thread directly.

The SunOS library resolves the mismatch by installing its own routine as the handler of the signals in the signal handler table of the process. This routine runs first, examines the user-level masks, and invokes the application's handler in a suitable user-level thread. Four cases arise (Figure 5.7).

Case 1 The signal is enabled both in the kernel-level thread and in the user-level thread running on it. The kernel interrupts the kernel-level thread; the library routine finds the signal enabled in the current user-level thread and runs the handler there.

Case 2 The running user-level thread has the signal disabled, but another user-level thread, runnable but not running, has it enabled. The kernel interrupts the kernel-level thread, whose mask enables the signal. The library routine sees that the current thread has the signal masked, schedules the runnable thread on this kernel-level thread, and the handler runs in it.

Case 3 The user-level thread with the signal enabled is running on a different kernel-level thread. The library routine on the interrupted kernel-level thread knows where that thread runs and makes a system call that sends a *directed* signal to the other kernel-level thread. The kernel interrupts it, and the library routine there finds the signal enabled and runs the handler.

Case 4 No user-level thread has the signal enabled, but the masks of the kernel-level threads

still enable it. The library routine on the interrupted kernel-level thread makes a system call that disables the signal in the mask of that kernel-level thread, and re-raises the signal for the process. The kernel delivers it to another kernel-level thread that still has it enabled, where the same steps follow, until the signal is disabled in all kernel-level threads and remains pending. When a user-level thread later enables the signal, the library makes a system call to enable it in a kernel-level thread again, and the pending signal is delivered.

Tip One rule generates the four cases: run the handler wherever a user-level mask enables the signal; if no thread does, clear the kernel-level mask and re-raise.

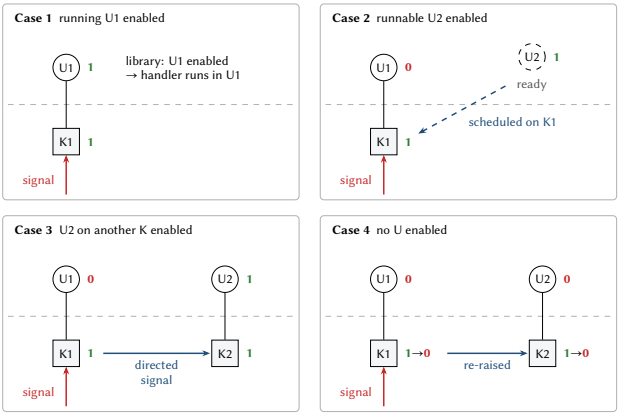


Figure 5.7: The four cases of signal handling with user-level threads (U) and kernel-level threads (K). The digit beside each thread is its mask bit for the signal: 1 enabled, 0 disabled.

The design follows the same principle as interrupts as threads. Signals occur far less often than threads change their signal masks, typically around every critical section. Changing a user-level mask is a cheap library operation, whereas keeping the kernel-level mask in step would require a system call on every change. In return, delivering a signal becomes more expensive—the library routine runs first and may have to redirect the signal or make system calls—but this happens rarely.

5.10 Tasks in Linux

Source P2L4, tasks in Linux; Silberschatz et al. ch. 4; Kerrisk ch. 28.

The Linux kernel does not distinguish between processes and threads. Both are built from a single kind of execution context, and a multithreaded process is a group of such contexts that share their resources.

Definition 5.17 (Task). In the Linux kernel a *task* is the basic execution context, a kernel-level thread represented by a `struct task_struct`. A single-threaded process consists of one task, a multithreaded process of several tasks.

The main fields of the structure are:

```
struct task_struct {        // abridged, Linux 3.x
    // ...
    pid_t pid;              // task identifier
    pid_t tgid;             // thread group identifier
    int prio;
    volatile long state;
    struct mm_struct *mm;          // address space
    struct files_struct *files;    // open files
    struct list_head tasks;
    int on_cpu;
    cpumask_t cpus_allowed;
    // ...
};
```

Every task has its own identifier, `pid`. The tasks of one process form a thread group whose identifier, `tgid`, is the `pid` of the leader of the group; this is the process identifier that `getpid` returns. The task structure does not contain the process's resources: `mm` and `files` point to separately allocated structures for the address space and the open files. As in Section 5.1, the state is split into structures that several tasks can share.

Caution The field named `pid` is the thread identifier: `getpid` returns `tgid`, and `gettid` returns `pid`. `ps -L` prints the two as `PID` and `LWP`.

A task is created with the *clone* system call:

```
clone(function, stack_ptr, sharing_flags, args);
```

The new task executes `function(args)` on the stack `stack_ptr`; each bit of `sharing_flags` selects whether a part of the state is shared with the creating task or copied (Table 5.5). With all these flags set, together with `CLONE_THREAD`, which places the new task in the thread group of its creator, the new task is a thread of the creating process; with all of them cleared, it is a new process, and `fork` is implemented in this way.

Table 5.5: Sharing flags of `clone`.

Flag	When set	When cleared
`CLONE_FILES`	share the file descriptor table	copy the file descriptor table
`CLONE_FS`	share file system information	copy file system information
`CLONE_SIGHAND`	share the signal handler table	copy the signal handler table
`CLONE_VM`	share the address space	new address space (copy)

POSIX threads in current Linux systems are implemented by the *Native POSIX Thread Library* (NPTL). Its design [9] rests on a few points:

- the one-to-one model: every user-level thread is a task, so the kernel sees every thread with its information, such as its signal mask, and the coordination problems of Sections 5.3 and 5.9 do not arise;
- synchronization built on futexes (Lesson 4), which enter the kernel only to wait or to wake;
- POSIX signal semantics implemented in the kernel: a signal sent to the process goes to one of its threads that does not block it, a fatal signal terminates the whole process, and stop and continue signals act on all its threads;
- no manager thread: `clone` was extended so that a thread starts and terminates without help, and at its exit the kernel clears the thread identifier in user memory and wakes a futex on it, which `pthread_join` waits for;
- scalability: the kernel scheduler of the time ran in constant time whatever the number of threads, and new kernel support for thread-local storage removed the limit of 8192 threads per process that LinuxThreads had on IA-32.

In Drepper and Molnar's benchmark NPTL creates and destroys threads several times faster than LinuxThreads and NGPT; the 15 minutes against 2 seconds of Lesson 3 came from the reworked kernel exit handling.

LinuxThreads is sometimes described as many-to-many; the many-to-many library of that era was NGPT.

The drawbacks that once argued against the one-to-one model have faded: traps into the kernel have become much cheaper, and memory and the range of thread identifiers are large enough for many kernel-level threads. NPTL replaced LinuxThreads, an earlier library that also created one task per thread but did not conform to POSIX in areas such as signal handling; the many-to-many library developed at the same time, Next Generation POSIX Threads (NGPT), had the coordination problems of Section 5.3 and was abandoned in favour of NPTL.

Key point — Lesson summary
Thread state is divided into hard process state shared by all threads, light process state per kernel-level thread, and user-level structures with red zones. In SunOS 5.0 an LWP links the two levels, which coordinate through system calls and signals. Handlers run in the interrupted thread's context, so masks prevent deadlock; interrupts as threads, top and bottom halves and user-level signal masks optimize the common case. Linux makes every thread a task created with `clone`, mapped one-to-one by NPTL.

Lesson 6

Thread Performance Considerations

Lecture	P2L5
Textbook	OSTEP [1] ch. 26, 33; Silberschatz et al. [2] ch. 4–5; Kerrisk [11] ch. 49–50, 63; Pai et al. [17]
Keywords	performance metric, execution time, throughput, response time, multi-process model, multithreaded model, event-driven model, event dispatcher, asynchronous I/O, helper, AMPED, Flash, workload, baseline

Lessons 3–5 introduced threads, their programming interface and their implementation, and argued that multithreading makes applications faster and more efficient. This lesson asks how such a claim is evaluated. It shows that which design is better depends on the performance metric and on the workload, compares the multi-process, multithreaded and event-driven architectures of a web server, and follows the evaluation of the Flash web server of Pai et al. [17] as a case study in the design of experiments.

6.1 Comparing threading models

Source P2L5, which threading model is better.

Lesson 3 compared the boss-workers and pipeline patterns by the total time they need to complete a batch of requests. That is only one of several reasonable criteria, and a different criterion can reverse the verdict. A server with five threads receives a batch of n requests, each of which needs 125 ms of processing:

- in the boss-workers configuration one thread is the boss, whose own time per request is negligible, and $w = 4$ threads are workers, each of which processes a whole request in $t = 125$ ms;
- in the pipeline configuration each of the $k = 5$ threads is a stage, and every stage takes $s = 25$ ms.

With boss-workers the requests complete in groups of w, so request i completes at time $\lceil i/w \rceil t$; in the pipeline request i completes at $(k + i - 1)\,s$. Two metrics can be derived from these completion times. The time to complete the whole batch is the completion time of the last request. The *average completion time* of a request is

Caution The n requests are all present at time 0 and completion is counted from there; with arrivals spread over time the relevant metric is response time (Section 6.3).

$$\overline{T}_{\text{bw}}(n) = \frac{t}{n}\sum_{i=1}^{n}\left\lceil \frac{i}{w} \right\rceil, \qquad \overline{T}_{\text{pl}}(n) = k\,s + \frac{(n-1)\,s}{2}. \tag{6.1}$$

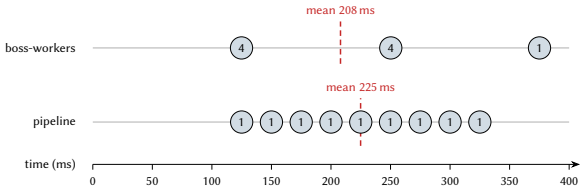


Figure 6.1: Completion times of nine requests. Each circle gives the number of requests completing at that time: boss-workers with four workers completes four requests every 125 ms, the five-stage pipeline one request every 25 ms after the first 125 ms. Dashed lines mark the average completion times.

Example 6.1. For a batch of $n = 9$ requests (Figure 6.1), boss-workers completes four requests at 125 ms, four at 250 ms and the last at 375 ms; the pipeline completes the first request at 125 ms and then one every 25 ms, the last at 325 ms. The pipeline finishes the batch sooner. By Equation (6.1), however, the average completion time is $(4 \cdot 125 + 4 \cdot 250 + 375)/9 \approx 208.3$ ms for boss-workers and $125 + 8 \cdot 25/2 = 225$ ms for the pipeline, so boss-workers serves the average request sooner. For $n = 40$ the pipeline wins by both metrics: it finishes the batch at 1100 ms rather than 1250 ms, with an average of 612.5 ms rather than 687.5 ms.

Boss-workers completes w requests at once after t, so many requests finish early, whereas the pipeline delivers only one request per stage time. In steady state, however, the pipeline

completes $1/s = 40$ requests per second against $w/t = 32$ for boss-workers, and in a large batch this higher rate dominates. Which model is better thus depends both on the metric and on the number of requests.

6.2 Usefulness of threads

Source P2L5, are threads useful; OSTEP ch. 26.

The earlier lessons gave several reasons for using threads.

- **Parallelization**: on several CPUs, threads that work on parts of the same task speed it up.
- **Specialization**: threads dedicated to particular tasks work with a hot cache and can be given different priorities.
- **Efficiency**: threads share an address space, so they need less memory than processes and communicate and synchronize more cheaply.
- **Hiding idle time**: even on a single CPU, another thread can run while one thread waits for I/O.

Whether these benefits make a thread-based design useful depends on what is measured, and different parties measure different things. For an application that multiplies matrices, the relevant measure is its execution time. For a web service it is the number of client requests handled per unit time, and the time a client waits for a response—which may be reported as the average, the maximum or a percentile, such as the 95th percentile. For the owner of the hardware it is the utilization of resources such as the CPU. Three measures thus recur for any system that serves requests: how many requests it completes per unit time, how long a request takes to be answered, and how much of the capacity of the system is in use.

Tip Throughput times average response time is the number of requests in the system, Little's law of Section 6.9: a quick check on any measured pair.

Example 6.2. A server handles 200 requests: 180 receive a response after 20 ms, and 20 after 400 ms. The average response time is $(180 \cdot 20 + 20 \cdot 400)/200 = 58$ ms. The 95th percentile is the smallest value that at least 95 % of the requests do not exceed; in sorted order the 190th response time is 400 ms, so the 95th percentile is 400 ms. The average hides the fact that one request in ten waits twenty times longer than most.

Caution A percentile is not an average of the slowest requests: the 95th percentile ignores the worst 5 % entirely.

6.3 Performance metrics

A statement that one design performs better than another is meaningful only with respect to a metric.

Source P2L5, performance metrics; really are threads useful; Silberschatz et al. ch. 5.

Definition 6.3 (Performance metric). A measurement standard for a system, that is, a measurable, quantifiable property of the system of interest that can be used to evaluate its behaviour, for instance to compare it with alternative implementations, is a *performance metric*.

Ideally a metric is obtained by experiments with real software deployed on real machines under real workloads. This is often infeasible—the machines may not be available, the software may not be finished, and real users cannot be asked to wait for an experiment—so measurements come instead from small experiments designed to be representative of realistic settings, or from simulation. An environment of this kind, set up to run experiments and obtain metrics, is called a *testbed*.

The most common metrics are the following.

Execution time The *execution time* of a task is the time from its start to its completion. It is the natural metric for a batch computation such as a matrix multiplication.

Throughput The *throughput* of a system is the number of tasks, such as requests, that it completes per unit time. It differs from the *request rate*, the number of requests that arrive per unit time: a system that cannot keep up with the request rate still completes requests only at its throughput, and the backlog grows.

Response time The *response time* of a request is the time from its issue until the client receives the response, including any time the request waits before it is processed. Over many requests it is summarized by an average, a maximum or a percentile.

Wait time The *wait time* of a task is the time from its submission until it starts to execute. It matters when users care about when their jobs start rather than when they finish.

CPU utilization The *CPU utilization* is the fraction of time during which the CPU is busy; utilization metrics for other resources are defined in the same way.

Many further metrics relate performance to what it costs or to what a particular party cares about (Table 6.1). Providers of a service often commit to performance targets in a *service level agreement* (SLA), for example that 99 % of requests receive a response within 200 ms; the

percentage of requests that violate the agreement is then itself a metric.

Table 6.1: Performance metrics and the parties that are typically interested in them.

Metric	Measures	Of interest to
execution time	time to complete a task	users of batch jobs
throughput, request rate	tasks completed, requests arriving per unit time	operators
response time	time until a request is answered	clients
wait time	time until a task starts	users
CPU utilization	fraction of time the CPU is busy	operators
platform efficiency	throughput relative to the resources used	datacenter operators
performance per dollar, per watt	performance relative to cost or power	operators, buyers
SLA violations	percentage of requests that miss the agreed target	providers, clients
client-perceived performance	quality as seen by a client, e.g. the frame rate of a video	clients
aggregate performance	a metric averaged over many tasks or clients	operators
average resource usage	e.g. the memory used over time	operators

6.3.1 Dependence on metric and workload

The question whether threads are useful therefore has no single answer. The answer depends on the metric, as Section 6.1 showed for average and total completion time, and it depends on the workload (Section 6.9): the better of two threading models changed with the number of requests. The same holds for most design decisions in operating systems. Different kinds of graphs favour different shortest-path algorithms, different patterns of file access favour different file system designs, and different request loads favour different server architectures.

> **Key point**
> No design is best in general. A comparison is meaningful only for a stated metric and a stated workload, and which of two designs is better typically depends on both.

6.4 Multi-process and multithreaded servers

Source P2L5, multi-process vs. multi-threaded; Silberschatz et al. ch. 4.

The remainder of the lesson compares ways of structuring a concurrent server, using as the example a web server that serves static files. A client sends a request, the server processes it in the steps of Figure 6.2, and it responds by sending the file. Some steps, such as parsing the request and computing the response header, need only CPU time. Others may block: accepting a connection waits for a client, reading the request and sending the data wait for the network, and finding and reading the file may wait for the disk.

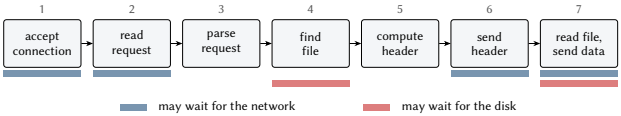


Figure 6.2: Processing steps of a simple web server for static content. Bars below a step show which resource it may have to wait for.

A server that executes these steps for one request at a time serves a single client and keeps all others waiting, even while it is itself blocked. The simplest remedy is to run several copies of it.

Definition 6.4 (Multi-process model). In the *multi-process model* (MP) a server consists of several processes, each of which runs a single-threaded server that executes all the steps for one request at a time (Figure 6.3a).

The multi-process model is easy to program, since each process is an ordinary sequential program. Its costs follow from the separate address spaces. Many processes need much

memory, which leaves less memory for caching files. Switching between processes is costly. State that the processes should share, such as a cache of file data or of response headers, is hard and expensive to maintain, because it requires IPC or explicitly shared memory. Finally, all processes must accept connections on the same port, which requires some care in setting them up; typically a parent process creates the listening socket and then forks the server processes, which inherit the socket and all call `accept` on it.

Definition 6.5 (Multithreaded model). In the *multithreaded model* (MT) a server is a single process whose threads, sharing one address space, each process requests concurrently (Figure 6.3b).

Caution "Multithreaded" here is the server model of this lesson (Pai et al.), not the one-to-one or many-to-one mapping models of Lesson 3.

The threads can be organized in several ways: every thread may execute all the steps, accepting connections itself; or, in the boss-workers pattern of Lesson 3, a boss thread accepts connections and passes them to workers; or the steps may be the stages of a pipeline. In every organization the threads share the address space, so shared state such as a cache is directly accessible, the memory footprint is smaller, and context switches between threads are cheap. In return, the implementation is not simple: the shared state requires synchronization, with the overhead and the risk of errors that come with it. The model also relies on the operating system to support kernel-level threads, which was not universal when these architectures were first compared but is no longer a concern.

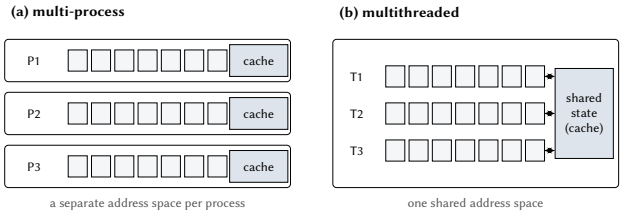


Figure 6.3: (a) The multi-process model: every process executes all processing steps (cells) of one request and keeps its own copy of state such as a cache. (b) The multithreaded model: threads in one address space execute the steps and share the state.

6.5 The event-driven model

A third architecture achieves concurrency without multiple execution contexts.

Source P2L5, event-driven model; OSTEP ch. 33; Kerrisk [11] ch. 63.

Definition 6.6 (Event-driven model). An application that runs as a single process with a single thread of control in a single address space, and performs its work in response to events, follows the *event-driven model*.

At the centre of such an application is the *event dispatcher*, a loop that repeatedly waits for events and, depending on the event, invokes one or more of the handlers registered for it (Figure 6.4). For a web server the events include the arrival of a connection request, the arrival of request data on a connection, the completion of a send, which means that the connection can take more data, and the completion of a disk read. Pai et al. call a server built in this way an instance of the *single-process event-driven model* (SPED).

An *event handler* is the code executed in response to one event; invoking it is simply a call to that code. A handler runs to completion. If it would have to block, it does not wait: it initiates the blocking operation, records how far the request has progressed, and returns control to the dispatcher, and the completion of the operation later arrives as another event. Dispatcher and handlers together form a state machine. The information that a thread would keep implicitly, in its program counter and on its stack, is kept explicitly in a record for each request.

Caution Running to completion admits no blocking call at all, not even one hidden in a library function such as the name lookup `getaddrinfo`: it stops every connection, not one.

OSTEP calls this explicit bookkeeping *manual stack management*.

6.5.1 Concurrent execution

Although it has a single thread, an event-driven server processes many requests concurrently. Every request in progress is at some step of its processing. The dispatcher executes one step of one request, then turns to the next event, which may belong to a different request, so that the processing of the requests is interleaved. Table 6.2 traces three connections; after step 6, for

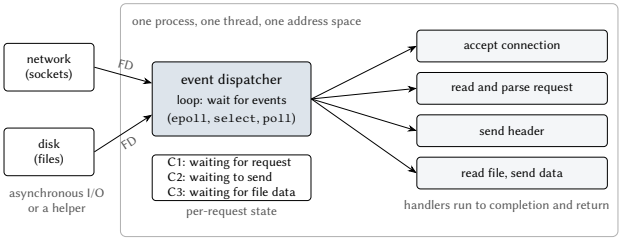


Figure 6.4: An event-driven web server. The dispatcher waits for events on file descriptors (FDs)—sockets that are ready, and disk reads whose completion is reported by asynchronous I/O or by a helper (Section 6.6)—and invokes the handler for each event. The state of every request in progress is kept in an explicit record.

instance, one thread has three requests in progress, each at a different step. The trace assumes that the completion of a file read arrives as an event, which requires asynchronous I/O or the helpers of Section 6.6; finding the file may have to wait for the disk as well.

Table 6.2: Trace of an event-driven server with three connections. States: R waiting for the request, F waiting for file data, S waiting until the connection can take more data, — not yet connected.

Step	Event	Handler action	C1	C2	C3
1	connection request	accept C1	R	—	—
2	connection request	accept C2	R	R	—
3	request on C1	parse, find file, start read	F	R	—
4	connection request	accept C3	F	R	R
5	request on C2	parse, find file, start read	F	F	R
6	file data for C1	send header and data	S	F	R
7	request on C3	parse, find file, start read	S	F	F
8	C1 can take data	send the rest, close	done	F	F
9	file data for C2	send header and data	done	S	F

6.5.2 Why the model works

Lesson 7 revisits the CPU scheduler whose time-slicing is counted here as context-switch overhead; Lesson 2 gives the price: about 3.8 µs of direct work per switch, more than 1000 µs once the caches have to be refilled [7].

Lesson 3 showed that on a single CPU, switching to another thread while a thread waits for I/O pays off when the idle time exceeds the cost of the two context switches, $t_{\text{idle}} > 2t_{\text{ctx}}$. When there is no idle time, context switches are pure overhead: they consume CPU cycles that could have processed requests. The event-driven model obtains the benefit without the cost. It processes a request until the request has to wait, then processes another request, and no context switch is involved. On a machine with several CPUs, the model is applied once per CPU: several event-driven processes, each with its own dispatcher, share the incoming connections.

6.5.3 Implementation

The event sources of a web server are accessed through file descriptors: sockets for the network, files for the disk, and pipes to the helpers introduced below. For a socket or a pipe an event is the readiness of its descriptor for input or output; for a regular file, readiness says nothing about whether a read must wait for the disk, a limitation to which Section 6.6 returns. To find out which descriptors are ready, the dispatcher uses one of three system calls. `select` and `poll` receive the whole set of descriptors of interest on every call; the kernel checks all of them, and the application scans the result, although typically only a few are ready. Their cost therefore grows with the number of open connections. `epoll`, available on Linux, eliminates most of this overhead: descriptors are registered once, and each call returns only those that are ready.[1]

Caution `select` is capped as well: on Linux it handles only descriptors below `FD_SETSIZE` = 1024. `poll` has no such limit, and the limit of `epoll` is `/proc/sys/fs/epoll/max_user_watches` (select(2), epoll(7)).

[1] Serving ten thousand connections from one machine was known as the *C10K problem*, after Kegel's 1999 survey of the techniques available for it. The scalable interfaces answered it: `kqueue` in FreeBSD 4.1 (2000) and `epoll` in Linux 2.5.44 (2002), generally available with Linux 2.6.

Example 6.7. The skeleton of a dispatcher built on `epoll`. The state of each request is kept in a `struct conn`, which holds the descriptor, the buffers, the file and the offset reached, and in `handler` a pointer to the function for the next step:

```
int ep = epoll_create1(0);   /* fds added: epoll_ctl */
```

```
for (;;) {                                  /* dispatcher */
    struct epoll_event ev[64];
    int n = epoll_wait(ep, ev, 64, -1);     /* wait */
    for (int i = 0; i < n; i++) {
        struct conn *c = ev[i].data.ptr;
        c->handler(c);            /* runs to completion */
    }
}
```

The handler of the listening socket accepts a connection, allocates a `struct conn` whose handler reads the request, and registers the new descriptor. Every handler performs one step and sets `handler` to the step that follows.

With EPOLLET `epoll` reports a descriptor once per state change; a handler must then read or write until it sees EAGAIN.

The benefits of the model follow from its single address space and single flow of control. A request in progress costs only its state record, not a thread with its stack, so the memory requirement is small. There are no context switches between execution contexts. And since handlers never run concurrently, no synchronization is needed.

> **Key point**
> An event-driven server interleaves the processing of many requests in one thread: a dispatcher waits for events on file descriptors and runs a handler for each. It saves memory, context switches and synchronization, provided that no handler blocks.

6.6 Blocking operations and helpers

Source P2L5, asynchronous I/O, helpers; OSTEP ch. 33; Pai et al.

The event-driven model has one serious problem: a handler that blocks blocks the entire process. While the single thread waits for the disk, the dispatcher does not run, and no other request makes progress, not even a request whose data are ready to be sent. Readiness notification does not help with disk reads: `select` and `poll` always report a regular file as ready, even when reading it must wait for the disk, and `epoll` does not accept regular files at all.

Definition 6.8 (Asynchronous I/O). With *asynchronous I/O* a system call that starts an I/O operation returns before the operation has completed, and the calling thread continues to execute. The kernel obtains from the call all the information it needs to carry out the operation, and either learns where to deliver the results or tells the caller where to retrieve them later.

Asynchronous I/O requires support from the kernel, for example kernel-level threads that carry out the blocking part of the operation, or from the device, for example a device that transfers the data by direct memory access (DMA, Lesson 11) and signals completion with an interrupt. It fits the event-driven model well: the completion of an asynchronous operation is one more event for the dispatcher.

POSIX defines `aio_read` and `aio_write`; glibc implements them with threads. On Linux `io_uring` (2019) is the interface that also covers buffered file I/O.

Asynchronous calls are not always available, however, and where they exist they may not be supported for every kind of device. An event-driven server then delegates its blocking operations.

Definition 6.9 (Helper). A *helper* is a thread or process that an event-driven server uses only to execute blocking operations on behalf of its dispatcher.

The dispatcher passes an operation to a helper through a pipe, and the helper reports completion through a second pipe, since a pipe carries data in one direction only; a socket pair serves both directions at once. Because pipes and sockets are file descriptors, the dispatcher waits for these reports with `select` or `poll` like for any other event. The helper blocks; the dispatcher, and with it all other requests, does not. An event-driven process combined with helper processes forms the *asymmetric multi-process event-driven model* (AMPED). The model is *asymmetric* because the helpers perform only particular operations, unlike the equivalent processes or threads of the MP and MT models. Pai et al. used helper processes because, at the time, not every operating system supported kernel-level threads. With helper threads instead of processes, the server follows the *asymmetric multithreaded event-driven model* (AMTED).

Helpers resolve the portability problem of the basic event-driven model: the server does not depend on asynchronous I/O support in the kernel. They also keep the memory footprint smaller than that of a worker for every request, since helpers are needed only for the operations that are blocked at the same time, not for every connection. On the other hand, the approach applies only to certain classes of applications, and on a multiprocessor it complicates the

routing of events: each event, including the completion reported by a helper, must reach the dispatcher that handles the corresponding request.

Example 6.10. A server has 2 000 open connections, of which at most 25 wait for the disk at any time. In the multithreaded model each connection needs a thread; with 64 kB per thread for its stack and data structures, the threads use 128 MB. An AMPED server keeps a state record of, say, 2 kB per connection (4 MB) and 25 helper processes, each of which costs, say, 1 MB for its private pages, page tables and kernel structures (25 MB), about 29 MB in total. The difference can be used to cache file data, which in turn reduces the number of requests that must wait for the disk.

That is the resident cost: a Pthreads stack reserves 8 MiB of address space but uses pages only as it touches them (Lesson 4).

Table 6.3 summarizes the four architectures.

Table 6.3: Architectures of a concurrent web server.

	MP	MT	SPED	AMPED
Execution contexts	a process per concurrent request	a thread per concurrent request	one	one, plus helpers
Memory per request	address space	thread stack and structures	state record	state record
Context switches	between processes	between threads	none	only for blocking operations
Synchronization	through IPC	locks on shared state	none	none
Blocking disk I/O	blocks one process	blocks one thread	blocks the server	blocks one helper

6.7 The Flash web server

Source P2L5, Flash; Pai et al.; Kerrisk ch. 49–50.

Pai et al. built an event-driven web server based on the AMPED model. In their server, *Flash*, helpers perform the disk operations that may block, that is, translating the path name of a request into a file, which may have to read directories, and reading file data; they communicate with the dispatcher through pipes. Flash accesses file data through memory-mapped files (Lesson 9), so a page of a file is read from disk when it is first accessed.

Tip `mincore` answers with one byte per page: with the 4 KiB pages of x86-64, a 4 MiB mapping needs a vector of 1024 bytes.

Before a handler sends file data, the dispatcher uses the system call `mincore` to test whether the pages of the data are in memory (Figure 6.5). If they are, a handler sends the data at once, and this normally does not block (the pages can still be evicted between the test and the send). If they are not, the dispatcher passes the request to a helper through a pipe. The helper accesses the pages, which brings them into memory and may block, and then reports completion to the dispatcher through a pipe. The dispatcher receives the report as an event and invokes the handler, which now finds the data in memory. Most requests to a web server are for popular files whose data are already in memory; for these requests Flash avoids the helpers and the cost of communicating with them, which can be a large saving.

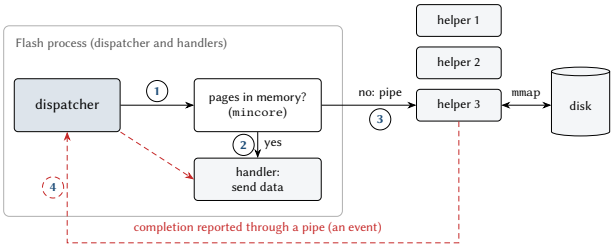


Figure 6.5: Serving file data in Flash. (1) The dispatcher tests with `mincore` whether the data are in memory. (2) If they are, a handler sends them directly. (3) Otherwise a helper is asked through a pipe to access the pages, reading them from disk. (4) Its completion report is an event, after which the handler sends the data.

Example 6.11. The test in C, using the headers `sys/mman.h`, `unistd.h` and `stdbool.h`, for file data mapped at a page-aligned address:

```
static bool in_memory(void *addr, size_t len)
{
    size_t pg = (size_t)sysconf(_SC_PAGESIZE);
    size_t n = (len + pg - 1) / pg;
    unsigned char vec[n];           /* 1 byte per page */
    if (mincore(addr, len, vec) != 0)
        return false;
    for (size_t i = 0; i < n; i++)
        if (!(vec[i] & 1))          /* page not resident */
            return false;
    return true;
}

/* dispatcher, before file data are sent */
if (in_memory(c->map, c->len))
    send_data(c);                   /* rarely blocks */
else
    ask_helper(c);                  /* through a pipe */
```

6.7.1 Additional optimizations

Flash combines the architecture with optimizations that reduce the work per request.

Application-level caching Flash caches both data and the results of computations: the translation of the path name in a request to a file, the response headers, which depend only on the file, and the mapped files themselves. A hit in the cache of path names also saves a request to a helper and back.

Gather I/O The response header and the file data are sent with a single vector I/O operation, `writev`, which passes several memory regions at once and so saves a system call. The kernel still copies the regions from user space into its socket buffers; a network interface with scatter-gather DMA can then gather these kernel buffers without assembling them in one, but it does not avoid the copy.

Linux goes one step further: `sendfile` (2.2, 1999) copies file data into the socket inside the kernel, without a trip through user space.

Alignment The copy of the regions given to `writev` into the kernel is slower when the regions are not aligned. Flash therefore aligns every response header on a 32-byte boundary and pads its length to a multiple of 32 bytes, by lengthening variable-length header fields, to match the 32-byte cache lines of the machines it targeted; the file data that follow the header are then copied at an aligned position.

Caution Despite its usual name, *alignment for DMA*, this optimization makes the copy into kernel buffers cheaper rather than avoiding it.

These optimizations are common in web servers today. They are independent of the server architecture, a fact that matters for the evaluation in Section 6.9.

6.8 The Apache web server

Source P2L5, Apache web server.

The open-source *Apache* HTTP server shows how a production server combines the models. Its *core* provides the basic server skeleton, which accepts connections and manages the execution contexts—in Apache 2 through a module of its own, as the note at the end of the section describes. The functionality that processes a request is implemented by *modules*, each of which registers handlers at *hooks*, the stages of request processing (Figure 6.6a). As a request progresses, the core invokes the handlers registered for each stage, a flow of control that resembles the event-driven model; unlike event handlers, however, these handlers run on the thread assigned to the request and may block.

The execution contexts combine the multi-process and multithreaded models (Figure 6.6b). In the model of the lecture, Apache runs several processes, and each process is a boss-workers server with a dynamic thread pool. Both the number of threads in each pool and the number of processes are adjusted to the load, for instance by the number of idle workers or of pending connections. The real `worker` module described below scales differently: each child process creates a fixed number of worker threads (`ThreadsPerChild`) and one listener thread, and the load is followed by creating or terminating child processes, so that the number of idle threads stays between `MinSpareThreads` and `MaxSpareThreads`.

Note 6.12. In Apache 2 the skeleton is itself a module: accepting connections and managing the processes and threads is the task of a *multi-processing module*, which selects the structure

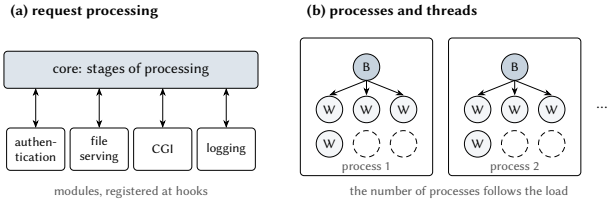


Figure 6.6: The Apache web server. (a) Modules register handlers at the stages of request processing that the core provides. (b) Several processes, each a boss (B) with a pool of workers (W); dashed circles are the workers that the model of the lecture adds as load increases, whereas the `worker` module keeps a fixed number per process.

of the execution contexts. `prefork` uses processes only, `worker` combines processes with a fixed number of threads in each, and `event` adds an event loop that watches idle connections.

Apache httpd 2.4 defaults to the `event` MPM on Linux; `prefork` and `worker` remain selectable (Apache docs, "Multi-Processing Modules").

6.9 Experimental methodology

Source P2L5, experimental methodology; Pai et al.

An experimental comparison must answer three questions: which systems are compared, with which inputs, and by which metrics. The evaluation of Flash illustrates each.

6.9.1 Comparison points

Flash was compared with servers of every architecture discussed in this lesson (Table 6.4). The MP, MT and SPED servers were built from the code of Flash and differ from it only in their architecture; they share its optimizations. Differences among them can therefore be attributed to the architecture. Two independent servers show where Flash stands among the servers in use at the time.

Table 6.4: Servers compared in the evaluation of Flash.

Server	Architecture
MP	multi-process: each process has a single thread and handles one request at a time
MT	multithreaded, boss-workers
SPED	single-process event-driven, without helpers
Zeus	version 1.30, a commercial SPED server, run with two processes in the trace-based experiments
Apache	version 1.3.1, multi-process
Flash	AMPED

6.9.2 Workloads

Definition 6.13 (Workload). The *workload* of an experiment is the set of inputs presented to the system under test; for a web server, the sequence of requests with their arrival times and the files they request.

A workload should be realistic, reproducing for instance the distribution of web page requests over time that real servers see, and it should be controlled and reproducible, so that every server is measured with the same input. A *trace*, a record of the requests observed by a real system that the experiment replays, meets both requirements. A *synthetic workload* is generated by a program from chosen parameters; it is less realistic, but it lets each parameter be varied systematically. Flash was evaluated with two traces collected at web servers of Rice University: the *CS trace* of the computer science department, with a large set of files, and the *Owlnet trace* of a server hosting the personal pages of students, with a smaller set. A synthetic workload repeatedly requested a single file whose size was varied.

The server machine was a 333 MHz Pentium II with 128 MB of memory and several 100 Mbit/s interfaces, running Solaris 2.6 and FreeBSD 2.2.6 [17].

6.9.3 Metrics

A web server is measured by the data it delivers and by the clients it serves, in two metrics:

$$\begin{aligned} \text{bandwidth} &= \frac{\text{total bytes transferred}}{\text{total time}}, \\ \text{connection rate} &= \frac{\text{total client connections}}{\text{total time}}. \end{aligned} \tag{6.2}$$

Both were evaluated as functions of the file size. A larger file lets the server amortize the fixed cost of a connection over more bytes, which raises the bandwidth, but it also means more work per connection, which lowers the connection rate. A simple model makes this precise. If serving a file of size S costs a fixed time c per connection plus b per byte, a server that handles one connection at a time achieves

$$R(S) = \frac{1}{c + bS}, \qquad B(S) = S \cdot R(S) = \frac{S}{c + bS} \;<\; \frac{1}{b}, \tag{6.3}$$

where R is the connection rate and B the bandwidth.

Example 6.14. With $c = 1\,\text{ms}$ and $b = 0.02\,\text{ms}$ per kilobyte, a file of 10 kB takes 1.2 ms: $R \approx 833$ connections per second and $B \approx 8.3\,\text{MB/s}$. A file of 100 kB takes 3 ms: $R \approx 333$ connections per second, but $B \approx 33.3\,\text{MB/s}$. As the file grows, B approaches $1/b = 50\,\text{MB/s}$ while R keeps falling.

Tip Little's law relates the two views: a rate R with mean response time T keeps RT connections open at once—1000 per second at 100 ms is 100 at a time.

6.10 Experimental results

The results of the evaluation depend above all on whether the requested data are in memory. Table 6.5 summarizes the findings described below.

Source P2L5, experimental results, summary of performance results; Pai et al.

Tip Read each row of the table as "workload: who wins" and ask which resource limits that workload: the CPU for cached data, the disk for large data sets.

Table 6.5: Summary of the performance results of the Flash evaluation.

Workload	Ranking	Reason
data in cache	SPED > Flash SPED, Flash > MT, MP	Flash tests for memory presence unnecessarily synchronization and context-switching overhead
disk-heavy	Flash > SPED Flash > MT, MP	SPED blocks, lacking asynchronous I/O Flash is more memory-efficient and switches contexts less

Single file (best case) With the synthetic workload every request is for the same file, which is always cached. The bandwidth, n requests times the file size divided by the time, was measured for file sizes up to 200 kB. All servers show a similar trend. SPED performs best. Flash comes close, but it pays for the `mincore` test on every request, which here is never needed. On FreeBSD, Zeus shows an anomaly for files of 10–100 kB, which Pai et al. suspect stems from the alignment problem described above. MT and MP are slower because of synchronization and context switching, and Apache, which lacks the optimizations, is slowest.

Owlnet trace The data set is small and mostly fits in the cache, so the trends resemble the best case. Occasionally, however, a request needs the disk. SPED then blocks entirely, while the helpers of Flash keep it serving other requests.

CS trace The data set is larger, and many requests need disk I/O. Among the servers built from the Flash code, SPED performs worst, because it lacks asynchronous I/O. MT is better than MP: its smaller memory footprint leaves more memory for caching files, which reduces disk I/O, and sharing state between threads is cheaper than the IPC that processes need. Flash performs best: its small footprint leaves more memory for caching, so fewer requests need blocking I/O, and it needs no synchronization. Apache is again the slowest of all the servers measured.

Optimizations Measurements of Flash with various combinations of the caches of path names, response headers and mapped files show that each optimization contributes substantially. Apache would have benefited from them too.

Pai et al. report that Flash "can match or exceed the performance of existing Web servers by up to 50 % across a wide range of real workloads" [17].

Key point
When the data are in memory, the event-driven servers win, and the extra test of Flash costs a little. When the workload needs the disk, the helpers of Flash avoid blocking, and its small memory footprint leaves more memory for caching, which reduces disk I/O. Flash performs well across workloads rather than best for each.

6.11 Designing and running experiments

Source P2L5, advice on designing experiments, advice on running experiments.

An experiment is worth running only if it is relevant: it must support a statement about a solution that others believe in and care about.

6.11.1 Relevance and metrics

[2] Clients are quoted percentiles rather than averages because the slowest reply decides: if a user request waits for 100 servers and one call in 100 takes longer than a second, the request is slow with probability $1 - 0.99^{100} \approx 63\,\%$. Dean and Barroso call this the tail at scale [18].

Relevance depends on the stakeholders. For a web server, clients care about response time, while the operator of the server cares about throughput and costs.[2] A new design that improves both response time and throughput is clearly valuable; one that improves one metric without degrading the other is still an improvement; one that improves one metric at the expense of the other may be useful to some stakeholders, provided the degraded metric remains acceptable. Another possible goal is to keep the response time unchanged as the request rate grows. The goals thus determine which metrics to measure and how to configure the experiments.

Metrics should be chosen accordingly. Standard metrics reach a broader audience, because readers know how to interpret and compare them; for the same reason, experiments often use a *benchmark*, a standardized workload with rules for running it and reporting the results that a community has agreed upon. Beyond that, the metrics should answer the questions of who cares about the result, what they care about and why: client performance calls for response time or the number of requests that time out, and operator costs for throughput and costs.

6.11.2 Configuration space

The behaviour of a system depends on many factors, which together form its configuration space:

- **system resources**: hardware, such as the CPUs and memory, and software, such as the number of threads or the sizes of queues;
- **workload**: for a web server, the request rate, the number of concurrent requests, the file sizes and the access pattern.

Since the whole space cannot be explored, an experiment chooses a subset of the configuration parameters, picks a range of values for each parameter that is varied, and uses a relevant workload. The best-case and worst-case scenarios should be included. The combinations of factors should be useful: many combinations merely repeat the same point.

Two rules govern comparisons. First, compare *like with like*: two configurations whose results are compared must differ only in the factor under study. If one server is measured with a light workload on a large machine and another with a heavy workload on a small machine, no conclusion about the servers is possible, because the difference may come from the workload, from the resources or from the servers. Second, compare against a *baseline*: a reference system whose performance is understood, such as the state of the art, the most common practice, or an ideal best-case or worst-case scenario. A claim that a design is better is a claim that it is better than a baseline.

Caution The baseline must be tuned as carefully as the new design; measuring against a default configuration is the commonest way to overstate an improvement.

6.11.3 Running experiments

Once the experiments are designed, each test case is run many times, the metrics are computed over the runs, for example as averages together with their spread, and the results are represented in a form, typically a graph, that makes the point clearly. Finally, conclusions must be drawn: measurements that are not interpreted with respect to the goals of the experiment do not support any statement.

Example 6.15. A team wants to show that a new AMPED server reduces the 95th-percentile response time of a photo-sharing site without lowering throughput. The hardware is fixed at one server machine, and the workload is a trace of the site replayed at increasing request rates of 1 000, 2 000, 4 000, 8 000 and 16 000 requests per second. A best case, with every file in memory, and a worst case, with a data set four times larger than memory, bound the range. The

baseline is the multithreaded server currently in use. With two servers, five rates, two cases and five repetitions each, the study consists of $2 \cdot 5 \cdot 2 \cdot 5 = 100$ runs, and it reports for every configuration the mean of the percentile and of the throughput together with their spread.

Key point — Lesson summary

Which design is better depends on the metric and on the workload. A concurrent server can use processes (MP), threads (MT) or one event-driven thread (SPED); helpers turn blocking operations into events when asynchronous I/O is unavailable (AMPED, AMTED). Flash, an AMPED server, comes close to SPED when data are cached and matches or exceeds the alternatives when the workload needs the disk; Apache combines processes with pools of threads. Sound experiments choose relevant metrics and workloads, compare configurations that differ in the factor under study, compare against a baseline, repeat runs and draw conclusions.

Lesson 7

Scheduling

Lecture	P3L1
Textbook	OSTEP [1] ch. 7–10; Silberschatz et al. [2] ch. 5; Fedorova et al. [19]
Keywords	runqueue, FCFS, SJF, priority inversion, round robin, timeslice, MLFQ, O(1) scheduler, CFS, cache affinity, NUMA, simultaneous multithreading, hardware performance counter

Lesson 2 introduced the CPU scheduler, which selects the next process to run from the ready queue, and showed that the length of the timeslice trades the overhead of scheduling against the time that ready processes wait. This lesson studies how the selection is made: scheduling algorithms from simple run-to-completion policies to preemptive, priority-based and timesharing schedulers, the runqueue data structures that implement them, and the two schedulers of the Linux kernel. It then turns to the questions raised by multiprocessors and by hyperthreaded processors, following the paper of Fedorova et al. [19]. The performance metrics of Lesson 6 serve to compare the algorithms.

7.1 Overview of CPU scheduling

Source P3L1, scheduling overview and the CPU scheduler; OSTEP ch. 7; Silberschatz et al. ch. 5.

The CPU scheduler decides how and when processes, and the threads within them, access the shared CPUs. It schedules tasks that run user-level processes and threads as well as the kernel's own kernel-level threads. Following the Linux terminology of Lesson 5, this lesson calls every such schedulable entity a *task*.

Even the simplest scheduler embodies a goal. Three elementary strategies show the range of possible goals:

- dispatch tasks immediately, in the order in which they arrive: the scheduling decision is trivial and its overhead is low;
- dispatch simple, short tasks first: this finishes the largest number of tasks early, and so maximizes the throughput, the number of tasks completed per unit time, over any interval shorter than the whole workload;
- dispatch complex, long tasks first: this keeps the resources occupied and maximizes the utilization of the CPU, the devices and memory.

The scheduler runs whenever the CPU may have to be given to a different task:

- the CPU becomes idle, for example because the running task blocks waiting for I/O;
- a new task becomes ready, for example a newly created task or a task whose I/O operation has completed; the scheduler must determine whether the new task is more important than the running one and should interrupt it;
- the timeslice of the running task expires.

A timer tick notices the expiry: Linux kernels are built with `CONFIG_HZ` of 100, 250, 300 or 1000, a tick every 10, 4, 3.3 or 1 ms.

The scheduler then selects one of the ready tasks and dispatches it: it performs a context switch to the selected task, enters user mode, and sets the program counter to the next instruction of the task, which continues to execute from there.

Which task is selected is determined by the scheduling policy, or algorithm. How the selection is carried out depends on the data structure that holds the ready tasks.

Linux names its main policies `SCHED_OTHER` (the fair scheduler), `SCHED_FIFO`, `SCHED_RR` and `SCHED_DEADLINE` (sched(7)).

Definition 7.1 (Runqueue). The data structure in which the scheduler keeps the tasks that are ready to run is its *runqueue*.

The ready queue of Lesson 2 is such a runqueue. Despite the name, a runqueue need not be a FIFO queue: many schedulers use trees, or several queues. The runqueue and the scheduling algorithm are tightly coupled. An algorithm can be implemented efficiently only with a runqueue that makes its selection cheap, and a runqueue is designed for the algorithm that it serves.

7.2 Run-to-completion scheduling

Source P3L1, run to completion scheduling, SJF performance; OSTEP ch. 7; Silberschatz et al. ch. 5.

The first schedulers to consider never take the CPU away from a task. We analyze them under simplifying assumptions: a group of tasks is to be scheduled, the execution time of every task is known in advance, there is no preemption, and there is a single CPU. The algorithms are compared by throughput, average completion time, average wait time and CPU utilization

(Lesson 6). The completion time of a task is measured from its arrival until it completes, and its wait time from its arrival until it starts to run.

Definition 7.2 (Run-to-completion scheduling). Under *run-to-completion scheduling* a task, once dispatched, runs on the CPU until it completes; it is never preempted.

Definition 7.3 (FCFS). A scheduler that follows *first-come first-served scheduling* (FCFS) runs the tasks in the order in which they arrive.

The runqueue of FCFS is a FIFO queue: an arriving task is appended at the tail, and the scheduler takes the task at the head. Both operations take constant time.

Definition 7.4 (SJF). A scheduler that follows the policy *shortest job first* (SJF) runs the tasks in the order of their execution times, shortest first.

The runqueue of SJF must be ordered by execution time. An ordered queue lets the scheduler take the first task in constant time, but inserting a task requires a search for its position, which takes time linear in the length of the queue. A balanced tree ordered by execution time inserts a task in logarithmic time, and the scheduler takes its leftmost node.

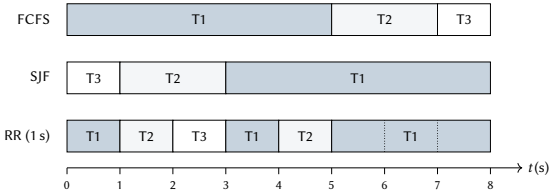


Figure 7.1: Three tasks that arrive at time 0 in the order T1, T2, T3, with execution times of 5 s, 2 s and 1 s, under FCFS, SJF and round robin with a timeslice of 1 s (Section 7.5). Dotted lines separate consecutive timeslices of the same task.

Example 7.5. Three tasks arrive at time 0 in the order T1, T2, T3, with execution times of 5 s, 2 s and 1 s (Figure 7.1). Under FCFS they complete at 5, 7 and 8 s. The throughput is $3/8 = 0.375$ tasks per second, the average completion time is $(5 + 7 + 8)/3 \approx 6.67$ s, and the average wait time is $(0 + 5 + 7)/3 = 4$ s. SJF runs T3, T2, T1, which complete at 1, 3 and 8 s. The throughput is still 0.375 tasks per second, but the average completion time falls to $(1 + 3 + 8)/3 = 4$ s and the average wait time to $(0 + 1 + 3)/3 \approx 1.33$ s.

The total work is the same in both orders, so the throughput measured over the whole batch does not change. By running short tasks first, however, SJF has more tasks finished at any instant before the end of the batch, and it lowers both averages. In fact, for tasks that arrive together and run to completion, no other order yields a smaller average wait time.

Caution SJF can starve long tasks if short ones keep arriving; Section 7.3 returns to starvation.

7.3 Preemptive scheduling

In practice tasks arrive at different times, and a task that arrives may deserve the CPU more than the task that is running. It may be shorter, or more important. A run-to-completion scheduler makes it wait until the running task completes, however long that takes.

Source P3L1, preemptive scheduling: SJF + preempt, priority; OSTEP ch. 7–8; Silberschatz et al. ch. 5.

Definition 7.6 (Preemptive scheduling). Under *preemptive scheduling* the scheduler may interrupt a running task before it completes and give the CPU to another task.

SJF becomes preemptive as follows. Whenever a task arrives, the scheduler compares its execution time with the remaining execution time of the running task. If the new task is shorter, the running task is preempted, returned to the runqueue, and the new task runs.

The execution time of a task is rarely known in advance. The scheduler therefore estimates it from the history of the task, using heuristics such as the time the task ran the last time it was on the CPU, or a windowed average of the times it ran the last n times.

This policy is also known as *shortest remaining time first*; OSTEP calls it *shortest time-to-completion first* (STCF).

Silberschatz et al. use an exponential average, $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$: only τ_n need be kept; with $\alpha = 1/2$ every older burst counts half as much as the one after it.

7.3.1 Priority scheduling

Tasks also differ in importance. Kernel-level threads that perform critical work for the operating system may be more important than threads of user processes, and among user processes some

are more urgent than others.

Definition 7.7 (Priority). The *priority* of a task is a value that expresses its importance to the scheduler. A scheduler that uses *priority scheduling* runs the ready task of highest priority next, and preempts the running task when a task of higher priority becomes ready.

The runqueue of a priority scheduler can consist of one queue per priority level, from which the scheduler selects the first task of the highest non-empty level; or it can be a tree ordered by priority.

Example 7.8. Three tasks arrive as follows (Figure 7.2):

Task	Arrival (s)	Execution time (s)	Priority
T1	0	7	medium
T2	1	2	low
T3	2	3	high

With preemptive SJF, T1 runs until T2 arrives at 1 s; T2 is shorter than the 6 s that remain of T1 and preempts it. When T3 arrives at 2 s, only 1 s of T2 remains, so T2 continues and completes at 3 s. T3 is shorter than the rest of T1 and runs next, completing at 6 s; T1 completes at 12 s. Measured from their arrivals, the tasks take 12, 2 and 4 s, 6 s on average. With preemptive priority scheduling, T2 does not preempt T1, but T3 does: T3 runs from 2 to 5 s, T1 then completes at 10 s, and T2 only at 12 s.

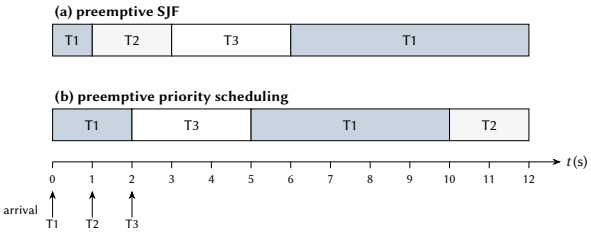


Figure 7.2: The tasks of Example 7.8 under (a) preemptive SJF and (b) preemptive priority scheduling.

7.3.2 Starvation and aging

Priority scheduling has a danger. As long as tasks of higher priority keep arriving, a task of low priority remains in the runqueue.

Definition 7.9 (Starvation). A ready task that is indefinitely denied the CPU, because the scheduler always prefers other tasks, suffers *starvation*.

The remedy is *priority aging*. The priority used by the scheduler is not the task's actual priority alone but a function of the actual priority and of the time the task has spent in the runqueue. The longer a task waits, the higher its effective priority becomes, until it exceeds that of the arriving tasks and the task runs. Aging thus prevents starvation.

7.4 Priority inversion

Source P3L1, priority inversion; Silberschatz et al. ch. 5–6.

Priorities interact with synchronization. Take three tasks with priorities T1 > T2 > T3, of which T1 and T3 lock the same mutex `m` (Figure 7.3a). Each task needs 3 s of CPU time. T3 holds `m` for the first 2 s of its run, and T1 holds it for 1 s after it has run for 1 s. T3 arrives at time 0 and locks `m` at once. At time 1, T2 arrives and preempts T3; at time 2, T1 arrives and preempts T2. At time 3, T1 tries to lock `m`, which is held by T3, and blocks. Of the two remaining tasks, T2 has the higher priority, so T2 runs, and it runs to completion before T3 can continue. Only then does T3 finish its critical section and unlock `m`, and T1 can proceed. T2 completes at time 5, before T1, which completes at 8; T3 completes last, at 9.

Definition 7.10 (Priority inversion). In a *priority inversion* a task of high priority waits for a resource held by a task of lower priority, while tasks of intermediate priority, which do not

need the resource, run in the meantime. The high-priority task is delayed as if the priorities were inverted.

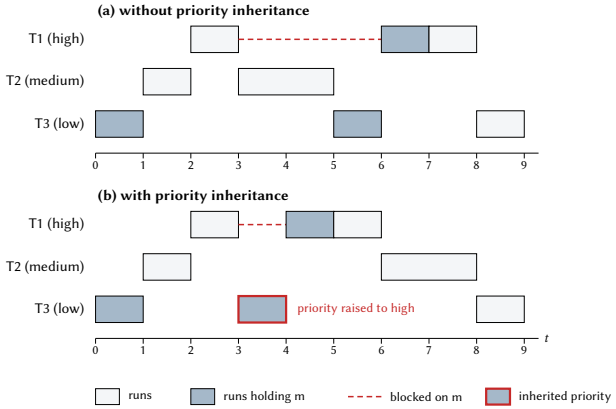


Figure 7.3: T1 and T3 share mutex `m`. (a) While T1 is blocked on `m`, the medium-priority T2 runs to completion ahead of it. (b) With priority inheritance, T3 runs at the priority of T1 until it unlocks `m`, and T1 completes before T2.

The solution is to raise the priority of the owner of the mutex temporarily (Figure 7.3b). When T1 blocks on `m`, T3 is given the priority of T1, so that T3, not T2, runs and finishes its critical section. When T3 unlocks `m`, its priority is lowered back to its own, T1 acquires the mutex and runs, and T1 completes at time 6, ahead of T2, which completes at 8.

Definition 7.11 (Priority inheritance). With *priority inheritance* the owner of a lock temporarily runs at the priority of the highest-priority task that is blocked on the lock, and returns to its own priority when it releases the lock.

To boost the owner, the lock operation must know who the owner is; this is another reason, besides the adaptive mutexes of Lesson 5, for recording the owner in the mutex data structure (Lesson 3).[1] POSIX threads (Lesson 4) provide priority inheritance as a protocol attribute of a mutex:

```
pthread_mutexattr_t attr;
pthread_mutex_t m;

pthread_mutexattr_init(&attr);
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&m, &attr);
```

[1] In July 1997 the Mars Pathfinder lander reset itself repeatedly: a low-priority meteorological task held the mutex of the shared information bus, the high-priority bus task blocked on it, and medium-priority communication ran instead until a watchdog timer reset the spacecraft. JPL reproduced the inversion on the ground and uploaded a patch that enabled priority inheritance on that mutex.

Key point

Preemption lets a scheduler serve short or important tasks first. Priorities bring the risks of starvation, avoided by aging, and of priority inversion, avoided by letting a lock owner inherit the priority of the tasks blocked on its lock.

Caution POSIX also defines `PTHREAD_PRIO_PROTECT`, the priority-ceiling protocol: the owner runs at a ceiling priority declared for the mutex, whether or not anyone is blocked. The lecture treats only inheritance.

Source P3L1, round robin scheduling, timesharing and timeslices; OSTEP ch. 7.

7.5 Round robin scheduling and timesharing

A scheduler for tasks whose execution times are unknown can simply let them take turns. Such a scheduler needs neither estimates of execution times nor, in its basic form, priorities.

Definition 7.12 (Round robin). A scheduler that uses *round robin scheduling* (RR) takes the task at the head of the runqueue, like FCFS. Unlike FCFS, a task need not run to completion: it may yield the CPU, for example to wait for I/O, and when it is ready again it is placed at the tail of the runqueue.

Round robin can be combined with priorities: tasks of equal priority take turns, and a task

of higher priority that becomes ready preempts the running one. In its most common form, round robin interleaves the tasks even when they do not yield, by limiting how long each task may run.

The timeslice of Lesson 2 is the maximum amount of uninterrupted time given to a task; it is also called the time quantum. A task may run for less than its timeslice. It may block, waiting for I/O or on a synchronization construct, and be placed on the corresponding queue; or a task of higher priority may become ready and preempt it. When the timeslice expires, the running task is preempted and returns to the tail of the runqueue.

Silberschatz et al. ch. 5 give a typical quantum of 10–100 ms, far longer than a context switch of a few microseconds (Lesson 2); Linux still gives its real-time round-robin class a fixed quantum: 100 ms by default, set with the sysctl `sched_rr_timeslice_ms`.

Definition 7.13 (Timesharing). The interleaving of tasks by means of timeslices is called *timesharing* of the CPU: every task runs for at most one timeslice at a time, so that a CPU-intensive task is preempted when its timeslice expires and cannot hold the CPU until it completes.

Example 7.14. The tasks of Example 7.5 run under round robin with a timeslice of 1 s, ignoring the cost of context switches (Figure 7.1). The order is T1, T2, T3, T1, T2, T1: T3 completes at 3 s, T2 at 5 s and T1 at 8 s, after running alone for its last three timeslices. The metrics are compared in Table 7.1. Without knowing any execution time, round robin lowers the average completion time from 6.67 s to 5.33 s, halfway from FCFS to the 4.00 s of SJF, and since a task waits for at most one timeslice of each task ahead of it before it first runs, its average wait time is even shorter than that of SJF.

Table 7.1: Metrics of the three schedules of Figure 7.1.

	FCFS	SJF	RR (1 s)
Throughput (tasks/s)	0.375	0.375	0.375
Average completion time (s)	6.67	4.00	5.33
Average wait time (s)	4.00	1.33	1.00

Timesharing has several benefits: short tasks complete sooner; the system is more responsive, since every ready task runs within a bounded time; and tasks that initiate lengthy I/O operations reach them sooner, so that the operations can proceed while other tasks compute. It also has a cost that the example ignores. At the end of every timeslice the timer interrupts the running task, the scheduler runs, and a context switch takes place. To keep this overhead small, the timeslice must be much longer than the time taken by the scheduler and the context switch, as Equation (2.1) showed for the time of the scheduler alone.

7.6 Choosing the timeslice length

Source P3L1, how long should a timeslice be, CPU bound timeslice length, I/O bound timeslice length, summarizing timeslice length; OSTEP ch. 7.

How long a timeslice should be depends on how the tasks use it. The timeslice must balance the benefits of timesharing against its overhead, and the balance differs between tasks that mostly compute and tasks that mostly wait for I/O.

7.6.1 CPU-bound tasks

A task that spends most of its time computing and rarely blocks is a *CPU-bound task*.

Caution The 0.2 s is an illustration, not a measurement: Lesson 2 puts the direct cost of a switch at a few microseconds, so a real system loses far less per timeslice than this example suggests.

Example 7.15. Two CPU-bound tasks, T1 and T2, each need 6 s of CPU time and arrive at time 0; a context switch takes 0.2 s. With a timeslice of 1 s the tasks alternate twelve times, with eleven context switches between the timeslices. T2 starts at 1.2 s. The last timeslice of T1 starts after ten timeslices and ten context switches, at 12 s, so T1 completes at 13 s and T2, after one more context switch, at 14.2 s. With a timeslice of 3 s the tasks alternate four times, with three context switches: T2 starts at 3.2 s, T1 completes at 9.4 s and T2 at 12.6 s. With an infinite timeslice T1 runs to completion at 6 s and T2 runs from 6.2 to 12.2 s. Table 7.2 lists the resulting metrics.

For CPU-bound tasks, a longer timeslice improves throughput and average completion time, because fewer context switches are spent. Only the wait time grows, and for CPU-bound tasks, such as batch computations, it matters little: their users see the time at which a computation completes, not the time at which it starts. In the limit, an infinite timeslice, that is, run to completion, is best for them.

Table 7.2: Two CPU-bound tasks of 6 s with a context switch of 0.2 s (Example 7.15).

Timeslice	Throughput (tasks/s)	Avg. wait (s)	Avg. completion (s)
1 s	$2/14.2 \approx 0.141$	$(0 + 1.2)/2 = 0.6$	$(13 + 14.2)/2 = 13.6$
3 s	$2/12.6 \approx 0.159$	$(0 + 3.2)/2 = 1.6$	$(9.4 + 12.6)/2 = 11.0$
∞	$2/12.2 \approx 0.164$	$(0 + 6.2)/2 = 3.1$	$(6 + 12.2)/2 = 9.1$

7.6.2 I/O-bound tasks

A task that frequently blocks for I/O and computes only briefly between its I/O operations is an *I/O-bound task*. If all tasks are I/O-bound, the length of the timeslice hardly matters, as long as it exceeds their short bursts of computation: each task then blocks for I/O before its timeslice expires and leaves the CPU of its own accord. The length matters when I/O-bound tasks share the CPU with CPU-bound ones.

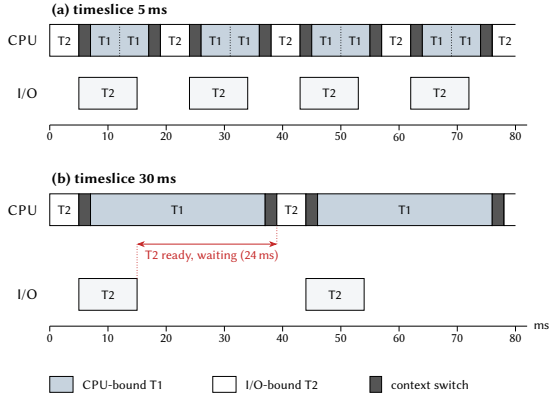


Figure 7.4: A CPU-bound task T1 and an I/O-bound task T2 under round robin (Example 7.16). When a timeslice of T1 expires while T2 is still waiting for its I/O, T1 simply continues.

Example 7.16. A CPU-bound task T1 shares the CPU with an I/O-bound task T2, which computes for 5 ms and then issues an I/O operation that takes 10 ms; a context switch takes 2 ms (Figure 7.4). With a timeslice of 5 ms, the I/O operation of T2 completes during the second timeslice of T1, and T2 waits 4 ms before it runs again. The pattern repeats every 19 ms: T2 issues an I/O operation every 19 ms, and the device is busy for $10/19 \approx 53\,\%$ of the time. With a timeslice of 30 ms, T2 waits 24 ms after each I/O operation, the pattern repeats every 39 ms, and the device is busy only $10/39 \approx 26\,\%$ of the time. T1, in turn, receives 10 ms of every 19 ms ($\approx 53\,\%$) with the short timeslice but 30 ms of every 39 ms ($\approx 77\,\%$) with the long one.

7.6.3 Summary

The two kinds of tasks prefer opposite timeslice lengths.

CPU-bound tasks prefer longer timeslices. Fewer context switches limit the overhead and keep CPU utilization and throughput high.

I/O-bound tasks prefer shorter timeslices. An I/O-bound task runs soon after its I/O completes and issues its next I/O operation earlier, which keeps the devices busy and gives better user-perceived performance, since interactive tasks are typically I/O-bound.

The 10 ms of the example is a disk figure: a 7200 rpm drive averages about 4 ms of rotation and a seek of similar length, whereas an NVMe SSD answers a read in tens of microseconds.

7.6.4 CPU utilization

The fraction of CPU time spent on useful work, Equation (2.1), extends to a mix of tasks with different behaviour. Choose a recurring interval of the schedule in which every task has had its opportunity to run. Within that interval, let t_{run} be the total time the CPU spends running tasks and t_{ctx} the total time it spends on context switches. Then

$$\text{CPU utilization} = \frac{t_{\text{run}}}{t_{\text{run}} + t_{\text{ctx}}}. \tag{7.1}$$

Tip For one switch of cost c per timeslice q the fraction is $q/(q + c)$: to keep the loss below 1 %, give a timeslice at least a hundred times the cost of a context switch.

Example 7.17. Three I/O-bound tasks each compute for 2 ms and then issue an I/O operation of 5 ms; two CPU-bound tasks never block; a context switch takes 0.5 ms; all tasks run for a long time. With a timeslice of 2 ms, every task, CPU-bound or not, leaves the CPU after 2 ms, and every run is followed by a context switch, so the utilization is $2/(2 + 0.5) = 80\,\%$. With a timeslice of 20 ms, one round consists of $3 \cdot 2 + 2 \cdot 20 = 46$ ms of computation and five context switches; the utilization is $46/(46 + 2.5) \approx 94.8\,\%$. Both rounds, of 12.5 and 48.5 ms, are longer than an I/O operation, so the I/O-bound tasks are always ready when their turn comes.

Caution Equation (7.1) ignores idle time: a stretch in which every task waits for I/O adds to neither t_{run} nor t_{ctx}, so the ratio can read 100 % while the CPU often has nothing to run.

> **Key point**
> Timesharing makes a system responsive without knowledge of execution times, at the price of an interrupt, a scheduler run and a context switch per timeslice. CPU-bound tasks favour long timeslices, I/O-bound tasks short ones.

7.7 Multi-level feedback queues

Source P3L1, runqueue data structure; OSTEP ch. 8; Corbató et al. [20].

To give I/O-bound and CPU-bound tasks different timeslices, the scheduler can keep all tasks in one runqueue and check the type of each task when it is selected, or it can keep the two kinds of tasks in different structures. A runqueue of several queues takes the second approach (Figure 7.5):

- the first queue holds the most I/O-intensive tasks, has the highest priority, and gives its tasks a timeslice of 8 ms;
- the second queue holds tasks that mix I/O and computation, with a timeslice of 16 ms;
- the last queue holds the CPU-intensive tasks, has the lowest priority, and gives them an infinite timeslice, that is, it schedules them FCFS.

The I/O-bound tasks receive the benefits of timesharing, and the CPU-bound tasks avoid its overhead.

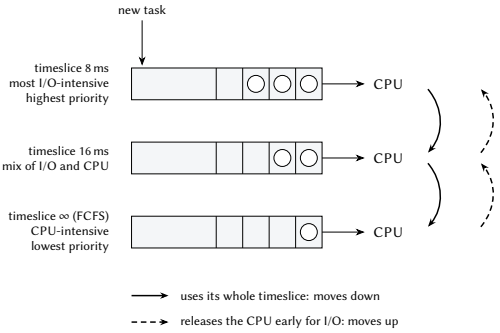


Figure 7.5: A multi-level feedback queue with three levels. Each level serves its tasks with its own timeslice; tasks move between levels according to how they use their timeslices.

This design raises the question of how the scheduler knows whether a task is I/O-intensive or CPU-intensive. Heuristics based on the history of a task do not help for a new task, nor for a task that changes its behaviour. The solution is to let the scheduler learn the type of a task from the way the task uses its timeslices:

1. a new task enters the topmost queue;
2. if the task yields the CPU before its timeslice expires, the choice of level was good, and the task stays at this level;
3. if the task uses its entire timeslice, it is more CPU-intensive than the level assumes, and it is moved down one level;
4. a task in a lower queue that releases the CPU early because it waits for I/O receives a priority boost and is moved up.

Caution OSTEP's MLFQ differs in two details: every level runs its tasks round robin, the lowest one included, and after a period S all tasks are moved back to the top level, so that none of them starves (OSTEP ch. 8).

Definition 7.18 (Multi-level feedback queue). A *multi-level feedback queue* (MLFQ) is a runqueue of several levels, each with its own scheduling policy, in which tasks move between the levels according to their observed use of the CPU.

An MLFQ is not merely a set of priority queues. Its levels treat tasks differently, with timeslices of different lengths and FCFS at the lowest level, and the feedback mechanism adjusts the level of each task over time. The design goes back to the CTSS timesharing system of Fernando Corbató [20], who received the Turing Award in 1990.

Example 7.19. A new task enters the 8 ms level and computes for 30 ms without blocking. It uses its whole first timeslice and moves to the 16 ms level, uses that timeslice as well and moves to the lowest level, where it completes its remaining 6 ms. A task that computes for 3 ms between I/O operations never uses up an 8 ms timeslice and stays at the top level.

7.8 The Linux O(1) scheduler

The $O(1)$ *scheduler* of Linux selects a task to run and adds a task to its runqueue in constant time, regardless of the number of tasks in the system. It is a preemptive, priority-based scheduler with 140 priority levels, numbered so that a smaller number means a higher priority. Levels 0 to 99 are for real-time tasks, and levels 100 to 139 for timesharing tasks.

Source P3L1, Linux O(1) scheduler; Bovet and Cesati [21] ch. 7.

A user process has the default priority 120. It can be adjusted through the process's *nice value*, which ranges from −20 to 19 and is added to the default, so that the priorities of timesharing tasks range from 100 to 139. A process can lower its own priority, for example with the `nice` system call; raising it requires privileges.

Tip `nice -n 5 cmd` starts a command with nice value 5; `renice` changes the value afterwards; `chrt` selects a real-time policy.

The timeslice of a task depends on its priority: tasks of low priority receive the shortest timeslices, and tasks of high priority the longest. The scheduler also uses feedback, based on the *sleep time* of a task, the time it spends waiting or idling:

- a task that sleeps for long periods is interactive, and its priority is boosted by up to 5 levels (its number decreases by up to 5);
- a task that sleeps little is compute-intensive, and its priority is lowered by up to 5 levels (its number increases by up to 5).

Example 7.20. In the Linux 2.6 kernels described by Bovet and Cesati, a timesharing task whose static priority, the priority given by its nice value, is p receives a timeslice of $(140 - p) \times 20$ ms if $p < 120$ and $(140 - p) \times 5$ ms otherwise. A process with nice value 5 has priority 125 and a timeslice of $15 \times 5 = 75$ ms; a process with nice value 0, 100 ms; a process with nice value -10, $30 \times 20 = 600$ ms. If the process with nice value 5 is interactive, the feedback lets it run at a priority as high as 120; if it computes without sleeping, at a priority as low as 130.

Caution The two numberings run in opposite directions: a larger priority number, and hence a larger nice value, means a lower priority, while a larger `sched_priority` (1–99), the value used for `SCHED_FIFO` and `SCHED_RR`, means a higher one.

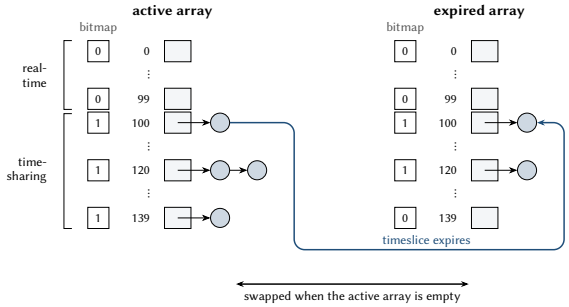


Figure 7.6: The runqueue of the $O(1)$ scheduler. Each array has one list per priority level and a bitmap of the non-empty lists. A task whose timeslice expires moves to the expired array; when the active array is empty, the two arrays are swapped.

The runqueue consists of two arrays of task lists, with one list for each priority level (Figure 7.6). The *active array* is the one from which the scheduler picks the next task. A bitmap records which of its 140 lists are non-empty; the scheduler finds the first set bit and takes the first task of that list, and adding a task appends it to a list and sets a bit. Both operations take constant time, because the number of levels is fixed. A task remains in the active array until its timeslice expires: a task that blocks before then returns to the active array, with the rest of its timeslice, when it becomes ready again.

The *expired array* holds the tasks whose timeslices have expired and that are therefore inactive. When the timeslice of a task expires, the scheduler recomputes its priority and

timeslice and moves it to the expired array. When no more tasks remain in the active array, the scheduler swaps the two arrays, which requires only exchanging two pointers. This arrangement also gives tasks of low priority a chance to run: a task of high priority, although it receives a long timeslice, moves to the expired array once it has used that timeslice, and waits there until every task in the active array, including those of low priority, has used its own.

In the Linux 2.6 kernels a task judged interactive was normally returned to the active array instead, unless the tasks in the expired array had already waited too long.

The $O(1)$ scheduler was introduced by Ingo Molnar in the Linux 2.5 development series and became the scheduler of Linux 2.6. As workloads became more interactive, with applications such as streaming media, its jitter became a problem, and in Linux 2.6.23 it was replaced by the Completely Fair Scheduler, also written by Molnar.

7.9 The Linux Completely Fair Scheduler

Source P3L1, Linux CFS scheduler; OSTEP ch. 9.

The $O(1)$ scheduler has two problems. First, the performance of interactive tasks is poor: once a task is in the expired array, it is not scheduled until all tasks in the active array have used their timeslices, so an interactive task may wait an unpredictable time before it runs. Second, it makes no guarantee of fairness, in the sense that within a given time interval every task should run for an amount of time proportional to its priority.

The *Completely Fair Scheduler* (CFS) addresses both problems. It was the default Linux scheduler for non-real-time tasks from version 2.6.23 to 6.5. Its runqueue is a *red-black tree*, a self-balancing binary search tree in which no path from the root to a leaf is more than twice as long as any other, so that all paths have approximately the same length. The tree is ordered by the virtual runtime of the tasks (Figure 7.7).

Definition 7.21 (Virtual runtime). The *virtual runtime* (`vruntime`) of a task is the time the task has spent running on the CPU, measured with nanosecond granularity and scaled according to the priority of the task.

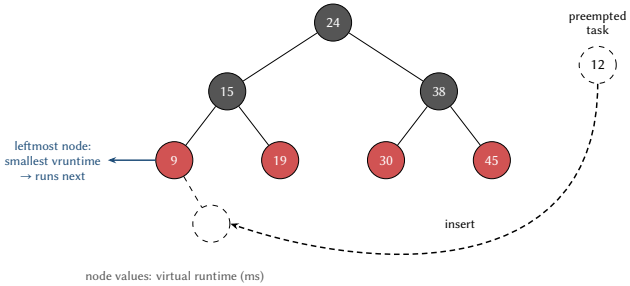


Figure 7.7: The runqueue of CFS, a red-black tree of the ready tasks ordered by virtual runtime (grey nodes are black, red nodes red). The leftmost task runs next; a preempted task is inserted at the position given by its virtual runtime.

Since Linux 6.6 the fair scheduling class selects tasks by the EEVDF algorithm, which retains the virtual runtime accounting of CFS.

The algorithm of CFS is simple:

- it always schedules the task of the leftmost node, which has the smallest virtual runtime, that is, the task that has so far received the least CPU time;
- while the task runs, CFS periodically increases its virtual runtime and compares it with the virtual runtime of the leftmost task in the tree;
- if the running task's virtual runtime is still smaller, the task continues to run; if it is larger, the task is preempted and inserted into the tree at the position its virtual runtime dictates, and the leftmost task runs.

Priorities are expressed by the rate at which virtual runtime progresses. For tasks of low priority, virtual runtime advances faster than real time, so they soon lose the CPU; for tasks of high priority it advances more slowly, so they run longer. A single tree thus holds the tasks of all priorities. Selecting a task takes constant time, since the scheduler keeps a pointer to the leftmost node, and adding a task takes $O(\log N)$ time for N tasks. Equivalently, CFS divides a scheduling period shared by all runnable tasks into timeslices proportional to the tasks' weights (OSTEP ch. 9).

Example 7.22. CFS derives the rate from a weight for each nice value: nice value 0 has weight 1024, and each step of the nice value changes the weight by a factor of about 1.25; nice value

5 has weight 335. A task of weight w that runs for a time Δt advances its virtual runtime by $\Delta t \cdot 1024/w$. If task A (nice 0) and task B (nice 5) each run for 6 ms, the virtual runtime of A grows by 6 ms but that of B by $6 \cdot 1024/335 \approx 18.3$ ms. Because CFS keeps the virtual runtimes of the tasks close, B runs about a third as long as A, and the two receive $1024/1359 \approx 75\,\%$ and 25 % of the CPU.

Tip The factor is chosen so that one nice level is worth about 10 % of the CPU: one level apart gives $1.25/2.25 \approx 55 : 45$, five levels apart the 75:25 here.

Table 7.3: The two Linux schedulers.

	$O(1)$ scheduler	CFS
Linux versions	2.6.0–2.6.22	2.6.23–6.5
Runqueue	active and expired arrays of 140 lists	red-black tree ordered by virtual runtime
Select a task	$O(1)$	$O(1)$
Add a task	$O(1)$	$O(\log N)$
Priorities	selection order, that is, the highest non-empty list, and timeslice length; sleep-time feedback adjusts the priority	rate of virtual runtime
Interactive tasks	may wait for the whole active array	run as soon as they have the least virtual runtime

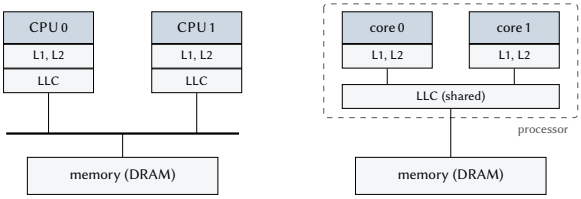
7.10 Scheduling on multiprocessors

Source P3L1, scheduling on multiprocessors; OSTEP ch. 10; Silberschatz et al. ch. 5.

Scheduling becomes more complex when there are several CPUs. Two architectures must be distinguished (Figure 7.8).

Definition 7.23 (Shared memory multiprocessor). A *shared memory multiprocessor* has several CPUs, each with its own private caches, all of which share main memory.

Definition 7.24 (Multicore processor). A *multicore processor* contains several cores, each of which is a CPU with its own private caches; the cores share the last-level cache (LLC) of the processor and main memory.



(a) shared memory multiprocessor (b) multicore processor

Figure 7.8: (a) A shared memory multiprocessor: every CPU has its own caches. (b) A multicore processor: the cores have private caches and share the last-level cache.

To the operating system, each core of a multicore processor is a CPU, and a system may combine both architectures, with several multicore processors.

7.10.1 Cache affinity and load balancing

The performance of a task depends on the state of the caches. While a task runs on a CPU, the CPU's caches fill with the task's data. If the task runs on the same CPU the next time, it finds a hot cache (Lesson 2); if it runs on a different CPU, it starts with a cold cache and runs more slowly until that cache has been filled.

Definition 7.25 (Cache affinity). The benefit that a task gains from running on the CPU whose caches hold its data is its *cache affinity*. A scheduler preserves it by keeping each task on the same CPU as much as possible.

A task that ran within the last `sched_migration_cost_ns`, 500 µs by default, counts as cache-hot, and the load balancer leaves it where it is. Since Linux 5.13 the tunable is in debugfs: `/sys/kernel/debug/sched/migration_cost_ns`.

Schedulers for multiprocessors therefore use a hierarchical architecture. Each CPU has its own runqueue, and a scheduler for that CPU selects tasks only from it, so that a task keeps running on the same CPU. Above the per-CPU schedulers, a *load balancing* component moves tasks between the runqueues so that no CPU is overloaded while another has too little to do. It acts on the lengths of the runqueues, or when a CPU becomes idle and takes tasks from the runqueues of other CPUs.

On Linux an application can restrict a thread to a set of CPUs with `sched_setaffinity`.

7.10.2 NUMA-aware scheduling

A multiprocessor may also have several memory nodes (Figure 7.9).

Definition 7.26 (NUMA). In a system with *non-uniform memory access* (NUMA) main memory consists of several memory nodes, each of which is close to a socket of one or more processors. Access from a CPU to its local memory node is faster than access to a remote node, which passes through an interconnect between the sockets, such as Intel's UPI (formerly QPI) or AMD's Infinity Fabric.

`numactl --hardware` shows the nodes and their distances; `numactl --cpunodebind=0 --membind=0 cmd` pins a program to one node (numactl(8)).

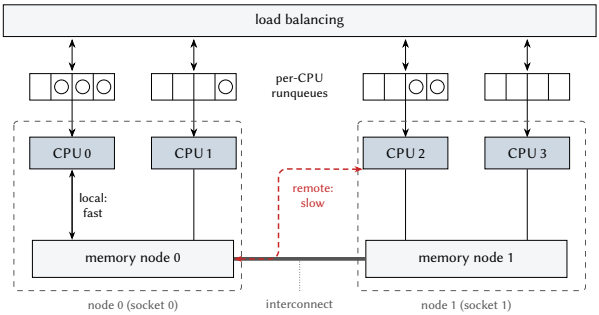


Figure 7.9: Hierarchical scheduling on a NUMA system. Every CPU schedules tasks from its own runqueue, and load balancing moves tasks between the runqueues. A CPU reaches the memory of its own node directly and the memory of the other node through the interconnect.

With *NUMA-aware scheduling* the scheduler keeps a task on a CPU close to the memory node that holds the task's state, so that its memory accesses remain local. Load balancing must take this into account: moving a task to a CPU on another node may balance the runqueues but slows down every access of the task to main memory, which then crosses the interconnect.

Example 7.27. A task spends 40 % of its execution time accessing main memory, and a remote access takes 1.5 times as long as a local one. If the load balancer moves the task to a CPU of the other node while its memory stays where it is, the task's execution time grows by a factor of $0.6 + 0.4 \times 1.5 = 1.2$, that is, by 20 %.

7.11 Hyperthreading

The CPUs must be multiplexed among tasks because the execution context of a task, its register values such as the program counter and the stack pointer, is held in the registers of a CPU while the task runs. To switch to another task, the context must be saved and the other context loaded. Hardware can instead provide several sets of registers, and thus several hardware-supported execution contexts, on one CPU. The CPU remains a single CPU, with one set of execution resources shared by the contexts, but passing from one context to another needs no saving and restoring of state.

Source P3L1, hyperthreading; Silberschatz et al. ch. 5; Fedorova et al. [19].

Definition 7.28 (Simultaneous multithreading). In *simultaneous multithreading* (SMT) a CPU provides several hardware execution contexts, or hardware threads, each with its own register state, which share the CPU's pipeline and caches.

The technique is also called hardware multithreading, hyperthreading, the name used by Intel, or chip multithreading. More precisely, *chip multithreading* (CMT) refers to processors that combine several cores with several hardware threads per core, the platforms studied by

Fedorova et al.; in practice the names are often used interchangeably. Processors with up to eight hardware threads per core have been built, but two is the most common configuration, and SMT can be enabled or disabled at boot time.

Designs differ in how closely the contexts share the pipeline. The simplest interleave them, giving the pipeline to another context as often as every cycle; in simultaneous multithreading proper, such as Intel's Hyper-Threading, instructions of several hardware threads occupy the stages of the pipeline in the same cycle.

Intel's first implementation cost less than 5 % of the die area and gave up to 30 % more performance on common server application benchmarks (Marr et al., *Intel Technology Journal*, 2002).

Each hardware thread appears to the operating system as a separate virtual CPU, on which the scheduler can place a task. The scheduler must therefore decide which tasks to co-schedule on the hardware threads of the same CPU. A context switch between hardware threads takes on the order of a few cycles, whereas loading data from memory after a cache miss takes on the order of 100 cycles. By the argument of Lesson 3, switching pays off when the idle time exceeds twice the cost of a switch (Equation (3.1)); it pays off by a wide margin here. With SMT, a CPU can therefore hide the latency of memory accesses, just as multithreading on a single CPU hides the latency of I/O.

7.12 Scheduling for hyperthreaded platforms

The question is which tasks should share a CPU; a simple model of a multithreaded CPU, in the spirit of the platforms studied by Fedorova et al., answers it. It rests on four assumptions.

Source P3L1, scheduling for hyperthreading platforms; Fedorova et al. [19].

1. A thread issues an instruction on every CPU cycle, and the pipeline executes at most one instruction per cycle.
2. A memory access of a memory-bound thread takes 4 cycles, during which the thread issues no instructions. The small value keeps the timing diagram short; it is the delay of a load served by the L1 cache rather than the hundred or so cycles of an access that reaches memory.
3. Switching between hardware threads is instantaneous.
4. The CPU has two hardware threads.

In this model, a CPU-bound task needs the pipeline on every cycle. A *memory-bound task* is one whose progress is limited by memory accesses: it spends most of its cycles waiting for data to arrive from memory. Figure 7.10 shows the three possible pairings.

The cores simulated by Fedorova et al. interleave four hardware threads cycle by cycle.

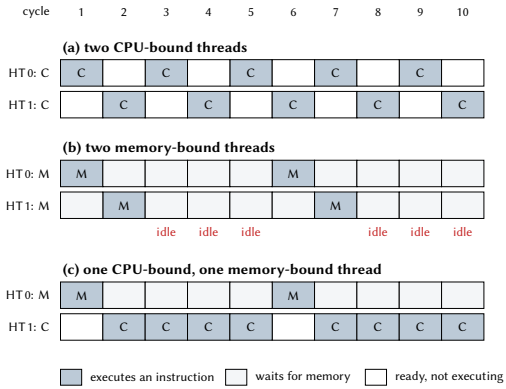


Figure 7.10: Co-scheduling two threads on the hardware threads (HT) of one SMT CPU, with C for a CPU-bound and M for a memory-bound thread. Each column is a cycle of the shared pipeline.

Two CPU-bound threads. Both threads are ready on every cycle, but the pipeline executes only one instruction per cycle. The threads interfere, contending for the pipeline: each progresses at half the speed it would have alone, a performance degradation by a factor of 2. The memory component meanwhile is idle.

Two memory-bound threads. Both threads spend most of their cycles waiting for memory, and during 6 of the 10 cycles in the figure neither has an instruction to execute. These CPU cycles are wasted.

A CPU-bound and a memory-bound thread. The CPU-bound thread uses the cycles in

which the memory-bound thread waits: it executes on 8 of 10 cycles, while the memory-bound thread progresses as fast as it would alone. No cycle is idle, and both the CPU and the memory are well utilized. Some interference remains, since the CPU-bound thread loses one cycle in five, but it is small.

> **Key point**
> On an SMT CPU, co-scheduling a CPU-bound with a memory-bound task avoids both contention for the pipeline and idle cycles. The scheduler should therefore mix the two kinds of tasks on the hardware threads of each CPU.

7.13 Hardware performance counters

Source P3L1, CPU bound or memory bound, scheduling with hardware counters; Fedorova et al.

Mixing CPU-bound and memory-bound tasks requires the scheduler to know which is which. The history-based heuristics used for I/O-bound tasks do not work. Sleep time, as used by the $O(1)$ scheduler, does not reveal a memory-bound task, because a thread that waits for memory is not sleeping: it waits inside the processor, invisible to the operating system. Nor can software keep track of individual memory accesses, which would take far too much time. The information must come from the hardware.

Definition 7.29 (Hardware performance counter). A *hardware performance counter* is a register of the processor that is updated as the processor executes, and counts events relevant to performance.

Processors typically provide several counters, and the counters available differ between architectures. They count, for example:

- cache misses at each level of the cache hierarchy, L1, L2 and so on up to the LLC;
- instructions and cycles, from which the instructions per cycle (IPC) or its inverse, the cycles per instruction, are derived;
- power and energy consumption, on some processors.

Software interfaces and tools make the counters accessible, for example `oprofile`, whose documentation lists the counters available on different architectures, and the Linux `perf` tool.

Tip The command `perf stat -e cycles,instructions,LLC-load-misses` *cmd* reports these counts for a program *cmd*.

From counter values the scheduler can estimate what kind of resources a task needs. A task with few LLC misses finds its data in the cache and is likely CPU-bound; a task with many LLC misses keeps waiting for memory and is likely memory-bound. A single counter is rarely conclusive. Schedulers therefore combine several counters in models with thresholds tuned for each architecture, based on well-understood characteristics of workloads.

Definition 7.30 (Cycles per instruction). The *cycles per instruction* (CPI) of a task is the number of CPU cycles it takes per instruction executed, the ratio of the cycle count to the instruction count.

Example 7.31. During an interval of 100 ms, the counters report that task A executed 4.2×10^8 instructions in 4.6×10^8 cycles, with 0.2 LLC misses per 1000 instructions, and that task B executed 0.9×10^8 instructions in the same number of cycles, with 25 LLC misses per 1000 instructions. A has a CPI of about 1.1 and B of about 5.1. Both indicators classify A as CPU-bound and B as memory-bound, and a scheduler following Section 7.12 would place them on the two hardware threads of the same CPU.

7.14 Cycles per instruction as a scheduling metric

Source P3L1, CPI experiment, CPI experiment results; Fedorova et al.

Fedorova et al. proposed to distinguish the tasks by a single counter-derived metric, the CPI. The idea is attractive: a memory-bound task spends many cycles waiting for memory and has a high CPI, while a CPU-bound task has a low one: 1 in a model that executes one instruction per cycle, and below 1 on a superscalar processor. A scheduler could then mix tasks with different CPIs on the hardware threads of each core.

Caution A high CPI can also come from branch mispredictions or pipeline contention, so a high CPI does not by itself prove a memory-bound thread.

To test the idea, they simulated a chip multithreading processor with four cores of four hardware threads each, sixteen hardware threads in total. The workload was synthetic and consisted of threads with widely different CPIs. The threads were assigned to the cores in several ways, ranging from assignments in which every core ran a mix of CPIs to assignments in which every core ran threads of the same CPI. The lecture compares the assignments by aggregate IPC (at most 4 on four single-issue cores); the paper reports aggregate throughput in transactions per second.

The assignments with mixed CPIs performed best, and those in which the threads on each core had similar CPIs worst. With these workloads, scheduling by CPI is clearly beneficial, and the experiment confirms that contention for the processor pipeline matters. Realistic workloads, however, behave differently. The CPIs measured for real applications, such as those of standard benchmark suites, are not nearly as distinct: they cluster in a narrow range.[2] CPI therefore cannot tell the tasks of a real workload apart well enough to guide the scheduler, and what works for the synthetic workload does not work in practice.

[2] The workshop paper itself was optimistic about CPI: it reports that measured CPI varies by an order of magnitude between applications, and that the near-term CPI of a thread can be predicted. The limitations it names are different ones: CPI reveals neither contention for the caches nor contention for the floating-point unit, and it is distorted when measured on a busy system.

Note 7.32. The study nevertheless leaves three lessons. Tasks on SMT processors contend for the processor pipeline; hardware counters can characterize a workload; and schedulers must be aware of resource contention, not merely balance the load. Later work by the same group [22] found the use of the last-level cache over time to be a better indicator of contention than CPI.

Key point — Lesson summary

A scheduler selects the next task from its runqueue, and the runqueue structure is designed together with the algorithm. FCFS and SJF run tasks to completion; preemption and priorities serve urgent tasks but bring starvation, avoided by aging, and priority inversion, avoided by priority inheritance. Round robin with timeslices makes a system responsive; CPU-bound tasks favour long timeslices and I/O-bound tasks short ones, which a multi-level feedback queue provides by learning the type of each task. Linux replaced its $O(1)$ scheduler, with active and expired arrays, by CFS, which runs the task of least virtual runtime from a red-black tree. On multiprocessors, per-CPU runqueues preserve cache affinity, load balancing spreads the tasks, and NUMA-aware scheduling keeps tasks near their memory. On SMT processors, CPU-bound and memory-bound tasks should share a CPU; hardware performance counters help to tell them apart, although CPI alone is not a sufficient metric.

Lesson 8

Memory Management

Lecture	P3L2
Textbook	OSTEP [1] ch. 13, 15–23; Silberschatz et al. [2] ch. 9–10; Bovet and Cesati [21] ch. 2, 8, 17
Keywords	virtual memory, paging, segmentation, page table entry, page fault, multi-level page table, TLB, large page, buddy allocator, slab allocator, demand paging, page replacement, copy-on-write, checkpointing

Lesson 7 described how the operating system shares the CPU among processes. This lesson turns to the second resource that every process needs, physical memory. The operating system gives every process its own virtual address space (Lesson 2), keeps in physical memory only the parts that are in use, and brings other parts in from secondary storage when they are needed. We examine the two ways of organizing this mapping, pages and segments; the hardware and the data structures that make address translation fast and compact; the allocators that manage physical memory; the policies that decide which pages leave memory; and two services, copy-on-write and checkpointing, that the memory management hardware makes possible.

8.1 Goals of memory management

Source P3L2, lesson preview, memory management: goals; OSTEP [1] ch. 13, 15; Silberschatz et al. [2] ch. 9.

The operating system manages the physical memory (DRAM) of the machine on behalf of the processes that run on it. Processes do not use physical addresses: each has a virtual address space whose range can be far larger than physical memory. With 32-bit virtual addresses every process can address 4 GiB, and the address spaces of all processes together easily exceed the memory that is installed.

Definition 8.1 (Virtual memory). The abstraction with which the operating system decouples the virtual address spaces of processes from physical memory is called *virtual memory*. The operating system maps the parts of every address space that are in use onto physical memory, keeps other parts on secondary storage, and translates and validates every address that a process uses.

On Linux x86-64 user space gets the lower half of the 48-bit space, 128 TiB; the kernel uses the upper half (Documentation/arch/x86/x86_64/mm.rst).

A process does not need all of its state at once; at any time it uses only part of its address space. The operating system can therefore keep only that part in physical memory, as long as the state that the process needs next can be made available quickly. Two tasks follow.

Allocation The operating system decides which portions of which address spaces reside in physical memory and where, and it establishes the mapping from virtual to physical addresses. Since physical memory may be smaller than the memory the processes use, some state must be moved to disk and brought back when it is needed; allocation therefore includes *replacement*, the choice of the state that leaves memory to make room.

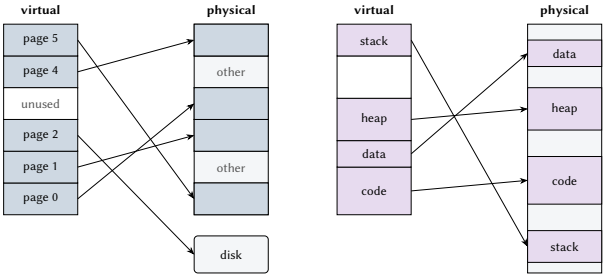
Arbitration Every memory access must be translated from a virtual into a physical address, and validated: the address must belong to an allocated region of the address space, and the access must be permitted. The conversion of a virtual address into the physical address to which it maps is called *address translation*. It happens on every memory access and must therefore be fast.

The operating system can divide the address space into units in two ways (Figure 8.1).

Page-based memory management With *paging* the virtual address space is divided into fixed-size blocks, each called a *page*, and physical memory into blocks of the same size, each called a *page frame*. Allocation maps pages to page frames; arbitration uses page tables.

Segment-based memory management With *segmentation* the address space is divided into blocks of flexible size, each called a *segment*, which correspond to logical components of the process, such as its code, heap or stack. Allocation maps segments to regions of physical memory of the same size; arbitration uses segment registers.

Paging is the dominant approach in current operating systems. Segmentation is supported on some architectures, notably x86, where it is combined with paging (Section 8.9).



(a) paging (b) segmentation

Figure 8.1: (a) Paging maps fixed-size pages to page frames; page 2 is on disk, and the unused page has no mapping. (b) Segmentation maps segments of different sizes to regions of physical memory of the same sizes.

8.2 Hardware support

Translating and validating every memory access in software would be far too slow. Memory management therefore relies on hardware in the CPU package, which handles every access, while the operating system sets up the data structures and handles the exceptions (Figure 8.2).

Source P3L2, memory management: hardware support; OSTEP ch. 15, 18–19.

Definition 8.2 (Memory management unit). The *memory management unit* (MMU) is the hardware component of the CPU package that translates the virtual addresses issued by the CPU into physical addresses and that raises a fault when a translation is not possible.

The MMU reports a fault to the operating system in three situations: an access to a virtual address that has not been allocated (an illegal access); an access that violates the permissions, such as a write to read-only memory; and an access to a page that is not present in physical memory and must first be brought in from disk. The MMU uses registers that point to the page table of the running process, or, with segmentation, that hold the base address, the limit and further information of the segments. Because walking a page table in memory is slow, the MMU also contains a cache of valid translations, the TLB (Section 8.7).

The translation itself is performed entirely in hardware. The operating system creates and maintains the data structures that the hardware uses, such as the page tables, but the hardware dictates their format and therefore which memory management mechanisms are possible: for example, the page sizes and the layout of a page table entry.

Caution Not every architecture translates in hardware alone: on MIPS and SPARC a TLB miss raises an exception and a kernel handler fills the TLB in software. OSTEP ch. 19.

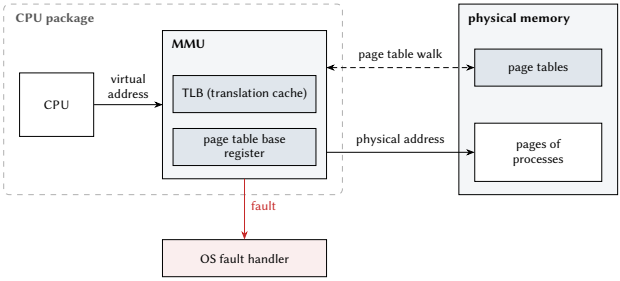


Figure 8.2: Hardware support for memory management. The MMU translates virtual addresses with the help of its TLB, walks the page tables in physical memory when a translation is not cached, and raises a fault to the operating system when an access is not possible.

8.3 Page tables

A page table (Lesson 2) maps every page of a virtual address space to the page frame that holds it. Because pages and page frames have the same size, the page table need not map every

Source P3L2, page tables; OSTEP ch. 18; Silberschatz et al. ch. 9.

address individually: the position of an address within its page is the same in virtual and in physical memory, so only the page must be translated.

For pages of 2^d bytes, a virtual address is split into two parts.

Virtual page number The high-order bits form the *virtual page number* (VPN), which indexes the page table.

Offset The d low-order bits form the *offset*, the position of the address within its page, which is not translated.

Each entry of the page table is a *page table entry* (PTE). Among other information (Section 8.4), it holds the *physical frame number* (PFN) of the frame that contains the page. The physical address combines the PFN with the unchanged offset (Figure 8.3):

Caution A PFN need not be as wide as a VPN. With physical address extension (PAE), 32-bit x86 maps 20-bit VPNs onto 24-bit PFNs and so addresses 64 GiB of physical memory.

$$\text{VPN} = \left\lfloor \frac{VA}{2^d} \right\rfloor, \qquad \text{offset} = VA \bmod 2^d, \qquad PA = \text{PFN} \cdot 2^d + \text{offset}. \tag{8.1}$$

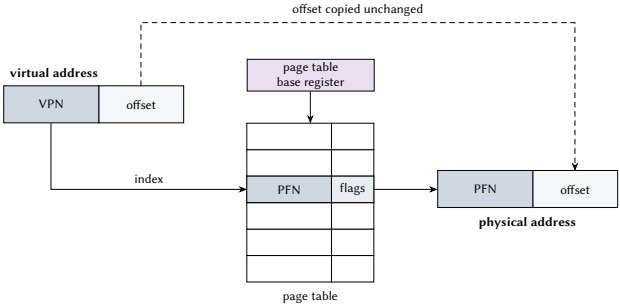


Figure 8.3: Address translation with a page table. The VPN selects the entry of the page table of the running process; its PFN, combined with the unchanged offset, forms the physical address.

Example 8.3. With 32-bit virtual addresses and 4 KiB pages ($d = 12$), a VPN has 20 bits and an offset 12 bits, i.e., three hexadecimal digits. The virtual address `0x00035A7C` has the VPN `0x00035` and the offset `0xA7C`. If entry `0x35` of the page table holds the PFN `0x001C2`, the physical address is `0x001C2A7C`. Every address from `0x00035000` to `0x00035FFF` is translated with the same entry.

8.3.1 Allocation on first touch and reclaiming

Physical memory is not allocated when a process reserves a range of virtual addresses, for example for a large array, but only when the process accesses the memory. This policy is called *allocation on first touch*: the first access to a page faults, and the operating system then allocates a page frame for the page and establishes the mapping in the page table.

Example 8.4. The following code reserves 256 MiB but receives page frames only as the loop touches the $2^{28}/2^{12} = 65\,536$ pages one after another:

```
size_t n = 256UL << 20;          /* 256 MiB            */
char *buf = malloc(n);           /* no page frames yet */
for (size_t i = 0; i < n; i += 4096)
    buf[i] = 1;                  /* first touch faults */
```

Conversely, when a page has not been used for a long time, the operating system may reclaim its frame: it moves the contents of the page to disk and marks the mapping as invalid (Section 8.4). If the process accesses the page again, the MMU raises a fault and traps into the operating system. The operating system verifies that the access is allowed, brings the contents back into memory, possibly into a different page frame, and re-establishes the mapping; the process then continues without noticing anything but the delay.

The operating system maintains one page table for every process. The MMU finds the page table of the running process through the *page table base register*, a CPU register that holds the address of that page table; on x86 it is the register CR3. On a context switch, the operating system loads the address of the page table of the next process into this register.

Writing CR3 flushes all non-global TLB entries unless process-context identifiers (PCID) are in use (Intel SDM vol. 3A, §4.10).

8.4 Page table entries

Besides the PFN, every PTE contains flags, which the MMU checks on every access and which the operating system uses to manage memory. The most important flags are the following.

Valid bit The *valid bit*, also called the *present bit*, indicates whether the mapping is valid, i.e., whether the page is present in physical memory in the frame that the entry names.

Dirty bit The hardware sets the *dirty bit* whenever the page is written. Pages that cache the contents of a file, for example, need to be written back to the file only if their dirty bit is set.

Access bit The hardware sets the *access bit* (referenced bit) whenever the page is accessed, for reading or for writing.

Protection bits The *protection bits* specify which operations are permitted on the page: read, write, execute.

The operating system reads the dirty and access bits and clears them when it has acted on them.

Without PAE, a 32-bit x86 PTE has 32 bits (Figure 8.4). The upper 20 bits hold the PFN; among the lower bits are the present bit P, the dirty bit D and the accessed bit A. R/W is a protection bit: with 0 the page is read-only, with 1 it may also be written. U/S restricts the privilege level: with 0 the page is accessible only in supervisor (kernel) mode, with 1 also in user mode. This format has no execute permission: the no-execute bit exists only in the 64-bit entries of PAE and x86-64. PWT and PCD carry caching-related information (write-through, cache disabled), and bits 9–11 are unused by the hardware and available to the operating system.

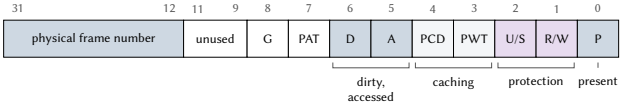


Figure 8.4: A 32-bit x86 page table entry. The global bit G and the page attribute bit PAT are not discussed here.

8.4.1 Page faults

The MMU uses a PTE both to translate an address and to check the access.

Definition 8.5 (Page fault). A *page fault* is the exception that the MMU raises when an access cannot be performed with the current page table entry: the entry is not valid, or the access violates its protection bits.

On a page fault the CPU places an error code on the kernel stack and traps into the kernel, which runs its page fault handler. On x86 the error code tells whether the page was not present or the access violated the protection bits, whether the access was a read or a write, and whether it came from user or kernel mode; the faulting virtual address is saved in the register CR2. From this information and from its own records of the address space, the handler determines the action (Table 8.1).

Table 8.1: How the operating system handles an access, depending on the page table entry and on its own records.

Page table entry	Situation	Action
valid	access permitted	none: the MMU translates
not valid	allocated, never accessed	allocate a frame (first touch)
not valid	page is on disk	bring it in (Section 8.13)
not valid	address not allocated	illegal access: `SIGSEGV`
valid	access violates protection	`SIGSEGV`, or copy (Section 8.15)

A process that makes an illegal access receives the signal `SIGSEGV` (Lesson 5), whose default action terminates it. All other faults are resolved by the operating system, after which the faulting instruction is executed again.

Source P3L2, page table entry; OSTEP ch. 18, 21; Silberschatz et al. ch. 9.

Caution The lecture, like x86, merges two notions in one bit. OSTEP (ch. 18, 21) keeps them apart: the valid bit says whether a translation is allocated, the present bit whether the page is in physical memory.

On x86-64 a PTE is 8 bytes: bit 63 is NX (no execute), bits 12–51 hold the PFN (Intel SDM vol. 3A, §4.5).

A fault that needs no I/O is a *minor* fault, one that reads from disk a *major* fault; Linux counts both per process (`minflt`, `majflt` in `/proc/<pid>/stat`).

8.5 Page table size

Source P3L2, page table size; OSTEP ch. 20.

A flat page table has one entry for every virtual page number, whether the page is used or not. For n-bit virtual addresses, pages of 2^d bytes and page table entries of e bytes, the number of entries and the size of the table are

$$N_{\text{PTE}} = \frac{2^n}{2^d} = 2^{n-d}, \qquad S_{\text{PT}} = 2^{n-d} \cdot e. \tag{8.2}$$

On a 32-bit architecture with 4 KiB pages and 4-byte entries a page table has 2^{20} entries and occupies 4 MiB. With 64-bit addresses and 8-byte entries it would have 2^{52} entries and occupy 2^{55} bytes, 32 PiB, for a single process. Since every process has its own page table, the cost is multiplied by the number of processes. Yet a process uses only a small part of its address space, so almost all of these entries would describe pages that are never used.

Tip Larger pages mean fewer entries: doubling the page size halves N_{PTE} (Section 8.10).

Example 8.6. The x86-64 architecture uses 48-bit virtual addresses. With 4 KiB pages and 8-byte entries, a flat page table would need 2^{36} entries and 2^{39} bytes, 512 GiB, per process.

8.6 Multi-level page tables

Source P3L2, multi-level page tables; OSTEP ch. 20; Silberschatz et al. ch. 9.

Flat page tables are therefore impractical for large address spaces. Since most of an address space is unused, page tables are instead organized hierarchically, so that no page table entries are needed for the pages of large unused regions.

Definition 8.7 (Multi-level page table). A *multi-level page table* is a tree of tables. The outer table, the *page directory*, holds pointers to inner page tables, and the inner page tables hold the PFNs. An inner page table is allocated only for a region of the address space that contains valid pages.

In a two-level table the virtual address is split into three parts (Figure 8.5): p_1 indexes the page directory, p_2 the inner page table that the directory entry points to, and the offset d selects the address within the frame. If p_2 has k bits, one inner page table covers 2^{k+d} bytes of the virtual address space. A gap in the address space of this size needs no inner page table at all; its directory entry is simply marked invalid. If the process later allocates memory in that region, the operating system allocates an inner page table for it and fills in the directory entry.

Tip A table of 2^9 entries of 8 bytes is exactly 4 KiB: an index of 9 bits per level makes every table fit in one page frame.

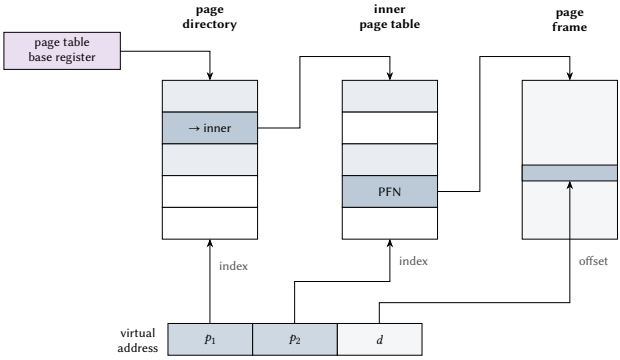


Figure 8.5: A two-level page table. White directory entries have no inner page table; the translation takes one access to the page directory and one to the inner page table.

Example 8.8. Consider 20-bit virtual addresses and 1 KiB pages ($d = 10$). A flat page table has $2^{10} = 1024$ entries. A two-level table with p_1 of 4 bits and p_2 of 6 bits has a page directory of 16 entries, and each inner table of 64 entries covers 64 KiB. A process uses the lowest 100 KiB (text, data and heap, pages 0–99) and the highest 20 KiB (stack, pages 1004–1023). It needs inner tables for pages 0–63, 64–127 and 960–1023, in total $16 + 3 \cdot 64 = 208$ entries, about a fifth of the flat table.

x86-64 uses four levels of 9 bits each above a 12-bit offset: $4 \cdot 9 + 12 = 48$ address bits.

The hierarchy can be extended with further levels. A third level adds a table of pointers to page directories, and a fourth level a table of pointers to those. Additional levels matter most

on 64-bit architectures, whose address spaces are both larger and sparser: the larger the region covered by a single entry high in the tree, the larger the gaps that need no tables below it.

The price is translation latency. Every level adds one memory access to the translation of an address, since the entries of all levels are read from memory in turn. More levels save more memory for the page tables but make each translation slower.

x86-64 implements only 48 of the 64 virtual address bits. Intel's 5-level paging adds a fifth level and raises this to 57 bits (128 PiB); Linux has supported it since 4.14.

8.7 Speeding up translation: the TLB

With a single-level page table every memory reference requires two memory accesses, one to the page table entry and one to the data; with four levels it requires five. The MMU avoids most of these accesses with a cache.

Source P3L2, speed of translation: TLB; OSTEP ch. 19; Silberschatz et al. ch. 9.

Definition 8.9 (Translation lookaside buffer). A *translation lookaside buffer* (TLB) is a cache of recently used address translations integrated into the MMU. On a TLB hit the address is translated without accessing the page table; on a TLB miss the page table is walked, and the translation is then stored in the TLB.

A TLB entry also holds the protection and validity bits of the translation, so that the access can be checked without the page table. A small number of entries suffices for a high hit rate because memory accesses exhibit *locality*: a page that has just been accessed is likely to be accessed again soon (temporal locality), and addresses close to it, which usually lie in the same page, are likely to be accessed next (spatial locality). An Intel Core i7 processor of the Sandy Bridge generation, for example, has per core a data TLB of 64 entries, an instruction TLB of 128 entries, and a second-level TLB of 512 entries shared by both kinds of translations (all for 4 KiB pages; Intel 64 and IA-32 Architectures Optimization Reference Manual).

The MMU also caches entries of the upper levels (paging-structure caches), so a miss usually costs fewer than L accesses. Intel SDM vol. 3A, 4.10.3.

With TLB lookup time t_{TLB}, memory access time t_{mem}, hit rate h and an L-level page table, the effective time of a memory access is

$$t_{\text{eff}} = h\,(t_{\text{TLB}} + t_{\text{mem}}) + (1-h)\,\big(t_{\text{TLB}} + (L+1)\,t_{\text{mem}}\big). \tag{8.3}$$

A context switch makes TLB entries stale; they are flushed unless tagged with their address space (Lesson 12).

Example 8.10. Let $t_{\text{mem}} = 80\,\text{ns}$, $t_{\text{TLB}} = 1\,\text{ns}$ and $L = 4$. Without a TLB, every access costs $5 \cdot 80 = 400\,\text{ns}$. With a hit rate of 99 %, Equation (8.3) gives $0.99 \cdot 81 + 0.01 \cdot 401 \approx 84.2\,\text{ns}$, only 5 % more than an untranslated access. With a hit rate of 90 % it gives $0.9 \cdot 81 + 0.1 \cdot 401 = 113\,\text{ns}$.

Tip Equation (8.3) rearranges to $t_{\text{eff}} = t_{\text{TLB}} + \big(1 + L(1-h)\big)\,t_{\text{mem}}$: the lookup is always paid, the walk only on a miss.

> **Key point**
> Flat page tables are proportional to the virtual address space and far too large. Multi-level page tables allocate tables only for the regions in use, at the price of one memory access per level; the TLB, which exploits locality, avoids those accesses for almost every translation.

8.8 Inverted page tables

A different way to keep page tables small is to organize them by physical rather than by virtual memory. Physical memory is much smaller than the virtual address spaces of all processes together, and every page frame holds at most one page at a time.

Source P3L2, inverted page tables; OSTEP ch. 20; Silberschatz et al. ch. 9.

Definition 8.11 (Inverted page table). An *inverted page table* has one entry for every page frame of physical memory. The entry of frame i records the process identifier and the virtual page number of the page that frame i holds.

The system needs a single inverted page table, whose size is proportional to physical memory and independent of the number of processes and of the size of their address spaces. A virtual address consists of the process identifier (pid), the page number p and the offset d. To translate it, the table is searched for the entry that matches (pid, p); the index i of that entry is the frame number, and the physical address is formed from i and d (Figure 8.6 (a)).

Caution One entry per frame means one (pid, p) pair per frame: a frame that several processes share does not fit the structure, which makes shared memory awkward.

A linear search of the table is slow. Most translations are served by the TLB, but the search on a miss must still be fast, and it is accelerated with hashing. In a *hashed page table* a hash function applied to the page number, together with the process identifier, selects a bucket, which heads a linked list of the entries whose keys hash to that bucket (Figure 8.6 (b)). Each element holds the process identifier, the page number, the frame number and a pointer to the next element, so a translation searches only one short list.

Hashed inverted tables: 64-bit UltraSPARC and PowerPC (Silberschatz et al. ch. 9); Itanium's hashed VHPT. x86 has always used multi-level tables.

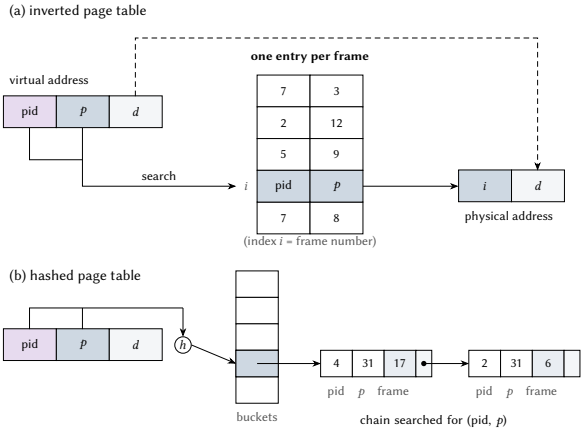


Figure 8.6: (a) An inverted page table is searched for (pid, p); the index of the matching entry is the frame number. (b) A hashed page table searches only the chain of the bucket selected by the hash function h.

Example 8.12. A machine with 16 GiB of physical memory and 4 KiB frames has $2^{34}/2^{12} = 2^{22}$ frames. With entries of 16 bytes its inverted page table occupies 2^{26} bytes, 64 MiB, whether the machine runs ten processes or a thousand.

8.9 Segmentation

Source P3L2, segmentation; OSTEP ch. 16; Silberschatz et al. ch. 9; Bovet and Cesati ch. 2.

With segmentation the address space is divided into segments of arbitrary size that correspond to logically meaningful components of the process, such as its code, data, heap and stack. A virtual address, on x86 called a *logical address*, consists of a *segment selector* and an offset within the segment. In its simplest form a segment is a contiguous region of physical memory described by its *base*, the address at which it starts, and its *limit*, its size. The physical address is base + offset, and it is valid only if offset < limit; otherwise the access faults.

Example 8.13. A process has a code segment with base `0x4000` and limit `0x1800`, and a stack segment with base `0x9000` and limit `0x0800`. The logical address (code, `0x0A20`) translates to `0x4A20`, since `0x0A20` < `0x1800`. The logical address (stack, `0x0900`) faults, since its offset exceeds the limit of the stack segment.

On x86 the selector is held in a segment register, such as CS for code, DS for data and SS for the stack. It indexes a *segment descriptor table*—the global descriptor table (GDT) or a local descriptor table (LDT)—whose entry, the segment descriptor, holds the base, the limit and the permissions of the segment. The sum of base and offset is a *linear address*. Segmentation is combined with paging: the linear address is translated into a physical address by the page tables (Figure 8.7).

A descriptor is 8 bytes; a selector holds a 13-bit index—hence the 8192 entries—a bit choosing GDT or LDT, and a 2-bit privilege level.

- On 32-bit x86 (IA-32) both mechanisms are active. A descriptor table holds up to 8192 segment descriptors. Linux, like most operating systems, defines segments that start at address 0 and span the whole address space, and it relies on paging for translation and protection.
- On x86-64, segmentation is retained only for backward compatibility. In 64-bit mode the processor treats the segment bases as 0 (except for FS and GS) and does not check limits, so paging alone translates addresses.[1]

[1] The two exceptions are in use. Linux keeps the thread-local storage of the running thread at the FS base and the per-CPU data of the kernel at the GS base; the instruction `swapgs` exchanges the user and the kernel value of the latter when a system call enters the kernel.

8.10 Page size

Source P3L2, page size; OSTEP ch. 20, 23; Silberschatz et al. ch. 9.

The size of a page is determined by the number of bits of its offset. An offset of 10 bits gives pages of 1 KiB, an offset of 12 bits pages of 4 KiB, which is the default on x86. Many architectures support several page sizes at the same time.

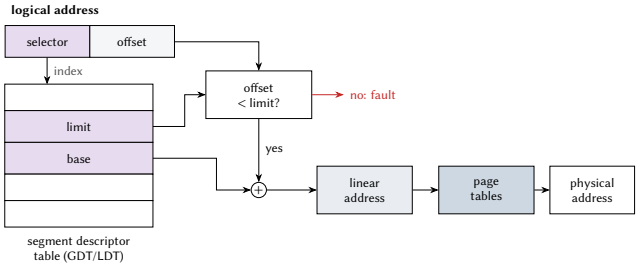


Figure 8.7: Segmentation combined with paging on x86. The selector indexes the segment descriptor table; an offset within the limit is added to the base, and the resulting linear address is translated by the page tables.

On x86-64, two larger page sizes are available besides the default. A *large page* has a size of 2 MiB, and its offset has 21 bits. A page of 1 GiB, whose offset has 30 bits, is called a *huge page*. Table 8.2 compares the three sizes.

Table 8.2: Page sizes on x86 and the page table entries needed to map 1 GiB of memory.

Page size	Offset bits	Entries for 1 GiB	Reduction
4 KiB (default)	12	262 144	1
2 MiB (large)	21	512	$2^9 = 512$
1 GiB (huge)	30	1	$2^{18} = 262\,144$

Larger pages have two benefits. They need fewer page table entries, so the page tables are smaller; and each TLB entry covers more memory, so more accesses are TLB hits. Their drawback is wasted memory.

Caution Terminology: the lecture's *large* (2 MiB) and *huge* (1 GiB) pages are both huge pages in Linux (Transparent Huge Pages, `/sys/kernel/mm/transparent_hugepage/`).

Definition 8.14 (Internal fragmentation). Memory that is allocated but unused because an allocation is rounded up to a whole unit, here to a whole page, is lost to *internal fragmentation*.

A large page is mapped as a whole even if the process uses only part of it, so the larger the pages, the more memory internal fragmentation can waste.

Example 8.15. A process maps 1 GiB of memory. With 4 KiB pages this takes 262 144 entries, and a TLB of 64 entries reaches only 256 KiB of it; with 2 MiB pages it takes 512 entries, and the same TLB reaches 128 MiB. A second region of 5222 KiB, however, occupies 1306 pages of 4 KiB, wasting 2 KiB, but three pages of 2 MiB, wasting $6144 - 5222 = 922$ KiB.

8.11 Memory allocation

Address translation applies a mapping from virtual to physical addresses. Page tables record the mapping and the MMU checks every access against it, but some other component must decide what the mapping is: which physical memory backs each page of each process, and which memory is still free.

Source P3L2, memory allocation, memory allocation challenges; OSTEP ch. 17; Silberschatz et al. ch. 9–10.

Definition 8.16 (Memory allocator). A *memory allocator* determines which physical memory backs a region of virtual memory, and thereby the mapping from virtual to physical addresses. Page tables and the MMU only record this mapping and check the validity of accesses and their permissions.

Allocators exist at two levels.

Kernel-level allocators allocate memory regions, i.e., pages, for the state of the kernel, such as the process control blocks, and for the static state of processes: their code, stack and initialized data. They also keep track of the free memory in the system.

User-level allocators manage the dynamic state of a process, its heap, through calls such as `malloc` and `free`. The kernel provides the process with a region of memory, and the user-level allocator divides it among the allocations without involving the kernel again. Examples are dlmalloc, jemalloc, Hoard and tcmalloc.

The main challenge of an allocator is fragmentation of the free memory.

glibc's `malloc` grows the heap with `brk`, but serves each request above the `mmap` threshold with an `mmap` of its own. The threshold is initially 128 KiB; glibc raises it dynamically (up to 32 MiB on 64-bit) unless `M_MMAP_THRESHOLD` is set.

Definition 8.17 (External fragmentation). Free memory suffers from *external fragmentation* when it is split into non-contiguous holes, so that a request for a contiguous region cannot be satisfied although enough memory is free in total.

Contiguous physical memory is needed, for example, for large pages, for buffers that devices access directly, and for data structures of the kernel. An allocator counters external fragmentation in two ways: it places allocations so that free memory stays contiguous, and when memory is freed, it coalesces the freed region with adjacent free regions into a larger one.

Example 8.18. A pool of 12 page frames receives requests for A (3 frames), B (2), C (3) and D (2); later A and C are freed, and a request for E needs 4 contiguous frames (Figure 8.8). If A and C were placed apart, 8 frames are free, but in holes of 3, 3 and 2 frames: E fails. If A and C were placed next to each other, freeing them leaves a contiguous hole of 6 frames, and E succeeds.

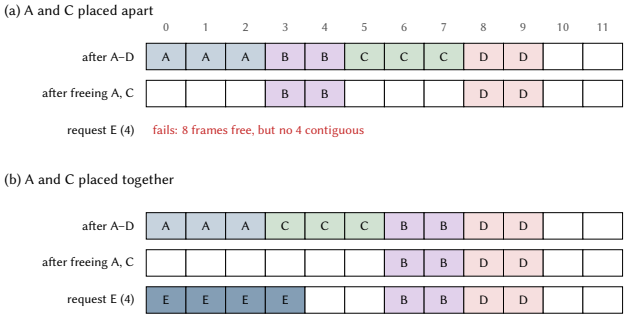


Figure 8.8: External fragmentation in a pool of 12 frames. The same requests and frees leave (a) three small holes or (b) one large hole, depending on where the allocator placed A and C.

8.12 Linux kernel allocators

Source P3L2, Linux kernel allocators; OSTEP ch. 17; Silberschatz et al. ch. 10; Bovet and Cesati ch. 8; Bonwick [23].

The Linux kernel combines two allocators: a buddy allocator for page frames and a slab allocator for kernel objects on top of it.

8.12.1 Buddy allocator

Definition 8.19 (Buddy allocator). A *buddy allocator* manages free memory in blocks whose sizes are powers of two. A request is rounded up to a power of two and served from the smallest free block that is large enough; a larger block is split in halves, called buddies, until a block of the requested size exists. When a block is freed and its buddy is also free, the two are merged, and merging continues up the tree.

Fragmentation can still occur, but merging works well and is fast: the buddy of the block of 2^k frames that starts at frame a starts at frame $a \oplus 2^k$, i.e., the two addresses differ in a single bit, so the allocator finds a buddy without searching. The rounding to powers of two, however, causes internal fragmentation.

Linux keeps free lists for the orders 0–10, i.e. 1 to 1024 frames or 4 KiB to 4 MiB; `/proc/buddyinfo` counts the free blocks of each order.

Example 8.20. A buddy allocator manages 16 page frames. Table 8.3 traces five operations, and Figure 8.9 shows the blocks after the third. The request for 3 frames is rounded up to 4 and that for 7 frames to 8, wasting one frame each. When B is freed, its buddy 6–7 is free, so the two merge into 4–7; the buddy of 4–7, block 0–3 with $0 = 4 \oplus 4$, is also free, so the merge continues to 0–7, and it stops there because the buddy 8–15 is allocated.

8.12.2 Slab allocator

Objects of the kernel have sizes that are not powers of two, and the buddy allocator would waste much of the memory it gives them. The Linux kernel therefore allocates its objects with a slab allocator [23].[2]

[2] Bonwick wrote the first slab allocator for Solaris 2.4. Linux has had three implementations of the idea—SLAB, SLUB and the compact SLOB; SLUB, the default since 2.6.23, is the only one left since SLOB was removed in Linux 6.4 and SLAB in 6.8.

Table 8.3: Trace of a buddy allocator with 16 page frames.

Step	Operation	Splits or merges	Free blocks after
0	—	—	0–15
1	allocate A (3 → 4)	split 0–15, 0–7; A = 0–3	4–7, 8–15
2	allocate B (2)	split 4–7; B = 4–5	6–7, 8–15
3	allocate C (7 → 8)	C = 8–15	6–7
4	free A	buddy 4–7 is split: no merge	0–3, 6–7
5	free B	merge 4–5 with 6–7, then with 0–3	0–7

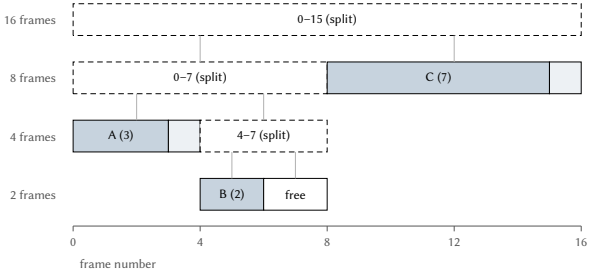


Figure 8.9: The blocks of the buddy allocator of Example 8.20 after step 3, with the number of frames requested in parentheses. Dashed blocks are split; the lightly shaded frames are lost to internal fragmentation.

Definition 8.21 (Slab allocator). A *slab allocator* keeps a cache for every common object type or size. Each cache consists of *slabs*, regions of contiguously allocated physical memory, divided into slots of exactly the size of the object. An allocation takes a free slot from the cache of the object's type, and a free returns the slot to the cache.

Since slots have exactly the size of the objects, the slab allocator avoids the rounding waste of the buddy allocator—only the remainder at the end of a slab is unused—and external fragmentation is not an issue: a slot that is freed fits the next object of the same type (Figure 8.10). The underlying slabs are obtained from the buddy allocator.

Each cache also keeps per-CPU lists of free slots, so that the common case of an allocation touches no shared state and takes no lock (Lesson 10).

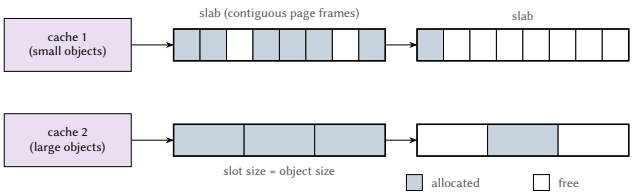


Figure 8.10: A slab allocator with a cache for small and a cache for large objects. Every slab is divided into slots of the object size.

Example 8.22. A kernel object occupies 1200 bytes. Allocated separately from a buddy allocator, each object would receive at least one page of 4 KiB and waste 2896 bytes. A slab of one page holds $\lfloor 4096/1200 \rfloor = 3$ such objects and leaves $4096 - 3600 = 496$ bytes unused, about 165 bytes per object.

A kernel structure gets its cache with `kmem_cache_create`; `task_struct` (the PCB of Lesson 2) has its own. `/proc/slabinfo` lists the caches and their object sizes.

Key point
Linux allocates page frames with a buddy allocator, whose power-of-two blocks are found and merged without searching but which rounds requests up, and kernel objects with a slab allocator, whose slots have the size of the object and are reused by the next object of the same type.

8.13 Demand paging

Source P3L2, demand paging; OSTEP ch. 21; Silberschatz et al. ch. 10.

Since virtual memory is far larger than physical memory, a virtual page is not always present in a page frame. The contents of page frames are saved to and restored from secondary storage.

Definition 8.23 (Swapping and demand paging). Moving the contents of pages between physical memory and secondary storage is called *swapping*. The pages are stored in a *swap partition*, a region of a disk, of flash memory, or even of the memory of a remote machine. With *demand paging*, a page is swapped into memory only when the process accesses it.

Caution Silberschatz et al. (§9.5) use *swapping* for whole processes and *paging* for single pages; the lecture and Linux say swap for pages.

When a process accesses a page that is not in memory, the following steps take place (Figure 8.11):

1. The process references a memory address, and the MMU finds the valid bit of its PTE cleared.
2. The MMU raises a page fault, which traps into the operating system.
3. The operating system determines that the access is legal and where the page is stored in the swap partition.
4. It reads the page from the swap partition into a free page frame, using an I/O operation.
5. It updates the PTE with the new PFN and sets the valid bit.
6. It resets the program counter to the faulting instruction and lets the process continue, so that the reference is executed again and now succeeds.

The page frame that the page occupies after it is swapped in is generally not the one it occupied before. If no page frame is free, the operating system must first free one (Section 8.14).

The 8 ms of the example below is a rotating disk. An NVMe SSD reads a page in tens of microseconds (typical vendor specifications), which makes a fault rate two orders of magnitude higher tolerable.

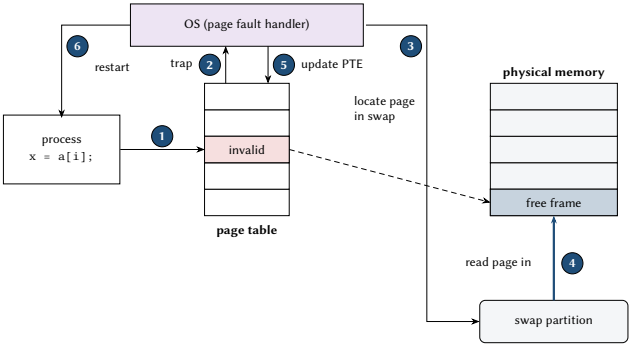


Figure 8.11: Demand paging: the steps taken when a process references a page that is in the swap partition.

Some pages must not be swapped. A *pinned page* is a page for which swapping is disabled, so that it stays in physical memory. Pages are pinned, for example, when a device accesses them directly by their physical address during direct memory access (Lesson 11).

Example 8.24. A memory access costs 100 ns, and handling a page fault that reads the page from disk costs 8 ms, i.e., $8 \cdot 10^6$ nanoseconds, 80 000 times as much. With a fraction p of accesses faulting, the average access costs $(1 - p) \cdot 100 + p \cdot 8 \cdot 10^6$ nanoseconds. For $p = 10^{-4}$ this is about 900 ns, nine times slower; to stay within 10 % of 100 ns, p must not exceed $1.25 \cdot 10^{-6}$, one fault in 800 000 accesses.

8.14 Page replacement

When physical memory runs short, the operating system must free page frames by swapping pages out. A page that is swapped out but needed again soon costs a page fault and an I/O operation, which is expensive, as the example of Section 8.13 showed. The operating system must therefore choose with care the pages that it swaps out. This choice is called *page replacement*, and it raises two questions: when pages should be swapped out, and which ones.

Source P3L2, freeing up physical memory; OSTEP ch. 22; Silberschatz et al. ch. 10; Bovet and Cesati ch. 17.

Pages are swapped out by a *page-out daemon*, a kernel thread that runs when memory usage exceeds a threshold, the *high watermark*. It preferably runs while CPU usage is below a threshold, the *low watermark*, so that it disturbs the applications as little as possible.

Caution Terminology differs: the lecture's high watermark is a memory-usage threshold, its low watermark a CPU-usage one; Linux's min/low/high watermarks all bound free memory (reclaim starts below low, stops above high).

The ideal victims are pages that will not be used in the future. The future is unknown, so the operating system predicts it from the history of accesses.

- The *least recently used* (LRU) policy swaps out the page whose last access lies furthest in the past, assuming that a page that has not been used recently will not be used soon. The access bit of the PTE tells the operating system whether a page has been referenced since the bit was last cleared.
- Pages that need not be written out are cheaper to replace. The dirty bit tells whether a page has been modified; the contents of a clean page that is already stored on disk can simply be discarded.
- Pages that must stay in memory, such as pinned pages and certain pages of the kernel, are never chosen.

Linux provides parameters that tune when reclaim starts and stops, categorizes pages into types, for instance reclaimable and swappable pages, and uses a variation of LRU that gives pages a second chance.

Linux's page-out daemon is the kernel thread `kswapd`; the watermarks are in `/proc/zoneinfo`, swap devices in `/proc/swaps`.

Definition 8.25 (Second chance algorithm). The *second chance algorithm* approximates LRU with the access bit. It scans the page frames in circular order. A page whose access bit is set is given a second chance: its bit is cleared, and the scan moves on. A page whose access bit is clear, i.e., one that has not been referenced since the previous scan cleared its bit, is swapped out.

Example 8.26. Five page frames hold pages A–E with access bits 1, 0, 1, 1, 0, and the scan starts at A. Table 8.4 traces two replacements. The second scan resumes after the frame that received F, so it examines only C, D and E: C and D are given a second chance, and E, whose bit is clear, is swapped out. A is not examined at all; because the process referenced it again after the first scan had cleared its bit, it is given a second chance when the scan next reaches it. C, whose bit was cleared in step 3 and which has not been referenced since, is then the victim.

Table 8.4: Trace of the second chance algorithm. Access bits are listed in scan order; a newly loaded page has its bit set.

Step	Event	Pages examined	Access bits after
0	—	—	A1 B0 C1 D1 E0
1	page fault, load F	A: clear; B: swap out	A0 F1 C1 D1 E0
2	process references A, D	—	A1 F1 C1 D1 E0
3	page fault, load G	C: clear; D: clear; E: swap out	A1 F1 C0 D0 G1

Key point
Demand paging brings a page in on the page fault that first needs it, and a page-out daemon frees frames before memory runs out. Which pages leave is decided with an approximation of LRU, such as the second chance algorithm, which reads and clears the access bit; the dirty bit tells whether a victim must be written out at all.

8.15 Copy-on-write

The MMU translates addresses, tracks accesses and enforces protection. These abilities are also useful for building other services and optimizations. One of them avoids unnecessary copies when a process is created.

Source P3L2, copy on write; OSTEP ch. 23; Silberschatz et al. ch. 10.

A process created with `fork` receives a copy of the entire address space of its parent (Lesson 2). Many of its pages, such as the code, are static and never change, and a child often calls `exec` soon after its creation, so most of the copying would be wasted.

Definition 8.27 (Copy-on-write). With *copy-on-write* (COW) the operating system does not copy the address space when it creates a process. It maps the virtual pages of the new process to the page frames of the original and marks these pages read-only in the page tables of both processes. A write to such a page causes a page fault, and only then does the operating system copy the page.

As long as both processes only read the shared pages, no memory is spent and no time is spent on copying. When one of them writes a page, the protection fault lets the operating system recognize the COW page; it copies this one page to a new page frame, maps the writing process's page to the copy with write permission, and restarts the write (Figure 8.12). The cost of a copy is paid only for pages that are actually modified.

The page tables of the private, written regions are still copied: 1 GiB of 4 KiB pages needs 2^{18} entries of 8 bytes, 2 MiB.

Caution The saving is not unconditional. Every first write traps into the kernel, on the order of a microsecond on current hardware (a typical figure), so a child that writes almost all of its pages is slower than one whose address space was copied eagerly.

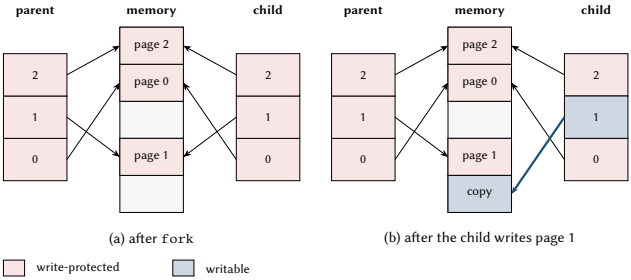


Figure 8.12: Copy-on-write. (a) After `fork`, parent and child map the same write-protected page frames. (b) The child's write to page 1 faults, and the operating system copies only that page.

Example 8.28. A process with an address space of 25 600 pages of 4 KiB (100 MiB) forks, and the child writes to 40 pages of its stack and data before it calls `exec`. Copy-on-write copies 40 pages instead of 25 600, i.e., 0.16 % of the address space.

8.16 Checkpointing

Source P3L2, failure management: checkpointing; not treated in OSTEP; Clark et al. [24] for live migration.

The same hardware support helps with failure management.

Definition 8.29 (Checkpointing). A failure and recovery management technique that periodically saves the state of a process is called *checkpointing*. After a failure, the process is restarted from its most recent checkpoint rather than from the beginning.

Failures cannot be excluded, but a checkpoint bounds the work lost and so shortens recovery. The simplest approach pauses the process and copies its entire state. The pause is long for a large address space, and the approach is expensive if it is repeated. The MMU enables a better approach.

The same write-protect-and-fault trick detects writes in distributed shared memory (Lesson 15).

- The operating system write-protects all pages of the process and copies the entire state once, while the process continues to run.
- From then on it records which pages the process modifies, through the write faults or the dirty bits, and later checkpoints copy only these dirtied pages. Such *incremental* checkpoints store the differences to the previous checkpoint.
- To restore the process, its state is rebuilt from the full copy and the subsequent differences; the differences can also be merged in the background.

Checkpointing is the basis of further services.

Debugging With *rewind-replay*, a program is restarted (rewound) from a checkpoint and re-executed. If the error does not reappear, successively older checkpoints are used until it is found.

Migration With *process migration* the checkpoint of a process is transferred to another machine, where the process continues. Migration serves disaster recovery and consolidation of processes onto fewer machines.

Clark et al. [24] migrated running Xen virtual machines with 210 ms of downtime for a web server and 60 ms for a game server.

To keep the pause of a migration short, the checkpoints are repeated in a fast loop: the operating system copies all pages while the process runs, then the pages dirtied during that copy, then those dirtied during the second copy, and so on, until so few pages remain that a

final pause-and-copy is acceptable—or until it is unavoidable because the process dirties pages as fast as they are copied.

This loop is called *pre-copy* migration. *Post-copy* does the opposite: it resumes the process at the destination at once and faults its pages across the network.

Example 8.30. A process of 50 000 pages is migrated over a connection that copies 25 000 pages per second, while the process dirties 2000 distinct pages per second. Pausing the process and copying everything takes 2 s. With repeated rounds, each of which copies the pages dirtied during the previous one, every round copies 8 % of the pages of the round before, and after three rounds the final pause lasts about 1 ms:

Round	Pages copied	Duration	Pages dirtied meanwhile
1 (process runs)	50 000	2 s	4000
2 (process runs)	4000	160 ms	320
3 (process runs)	320	12.8 ms	26
final (paused)	26	1.04 ms	—

Key point — Lesson summary

Virtual memory decouples address spaces from physical memory; the operating system allocates memory and arbitrates every access, using paging or segmentation and the MMU, which translates addresses and raises faults. A page table maps VPNs to PFNs (Equation (8.1)), and its entries carry valid, dirty, access and protection bits; a page fault hands an access that the MMU cannot complete to the operating system. Flat page tables are too large (Equation (8.2)), so they are multi-level or inverted and hashed, and the TLB keeps translation fast (Equation (8.3)). Large pages shrink page tables and raise TLB coverage at the cost of internal fragmentation. Allocators fight external fragmentation; Linux uses a buddy allocator for page frames and a slab allocator for kernel objects. Demand paging swaps pages in on a page fault, and a page-out daemon swaps pages out, choosing them with LRU approximations such as the second chance algorithm. The MMU also enables copy-on-write and incremental checkpointing, which in turn supports debugging and process migration.

Lesson 9

Inter-Process Communication

Lecture	P3L3
Textbook	Silberschatz et al. [2] ch. 3 (§§3.4–3.8); Kerrisk [11] ch. 43–54, 56–57
Keywords	inter-process communication, message passing, port, pipe, message queue, socket, shared memory, copy versus map, local procedure call, shared memory segment, System V IPC, POSIX shared memory, memory-mapped file, process-shared mutex

Lesson 2 introduced inter-process communication (IPC) as the way in which processes, each isolated in its own address space, exchange data, and it distinguished message passing from shared memory. This lesson examines the mechanisms that operating systems provide for both. Shared memory builds on the address translation of Lesson 8, which lets the operating system map the same physical memory into several address spaces; it keeps the kernel out of every exchange, but it leaves the synchronization and the communication protocol, which the kernel handles for message passing, to the processes.

9.1 Inter-process communication

Source P3L3, inter-process communication; Silberschatz et al. ch. 3.

IPC comprises the mechanisms that the operating system supports for the interaction of processes: *communication*, the transfer of data from one address space to another, and *coordination* of the processes' actions. IPC is necessary because the operating system isolates the address spaces of processes (Lesson 2): no process can read or write the memory of another, so every exchange of data goes through a mechanism that the operating system provides. The mechanisms fall into four groups.

Message passing Processes exchange messages through a channel that the kernel maintains, e.g. sockets, pipes and message queues.

Memory-based IPC The operating system makes memory accessible to several processes, which communicate by reading and writing it, e.g. shared memory and memory-mapped files.

Higher-level semantics Mechanisms that give the data transfer more structure: files that several processes read and write, and remote procedure calls (Lesson 13).

Synchronization primitives Constructs such as mutexes and semaphores with which processes coordinate (Lesson 10).

This lesson concentrates on message passing, on shared memory, and on the synchronization that shared memory requires.

9.2 Message-based IPC

Source P3L3, message-based IPC; Silberschatz et al. ch. 3.

In message-based IPC the processes create messages and send and receive them through a channel that the operating system creates and maintains, typically a buffer in kernel memory organized as a FIFO queue. The processes never access this buffer directly.

Definition 9.1 (Port). A *port* is the interface through which a process uses a message-passing channel: the process sends (writes) messages to a port and receives (reads) messages from a port, and the kernel moves the messages between the ports.

The kernel is involved both in establishing the communication and in every single IPC operation. A send is a system call that copies the message from the address space of the sender into the channel; a receive is a system call that copies it from the channel into the address space of the receiver. A request-response exchange between a client and a server therefore costs four user/kernel crossings and four data copies (Figure 9.1).

A null system call on x86-64 costs on the order of 100 ns (lmbench `lat_syscall`); the Meltdown mitigation (KPTI) can double it.

The advantage of message passing is its simplicity: the kernel performs the channel management and the synchronization—a receive on an empty channel, for instance, blocks until a message arrives. The disadvantage is overhead: the crossings and copies are paid for every message.

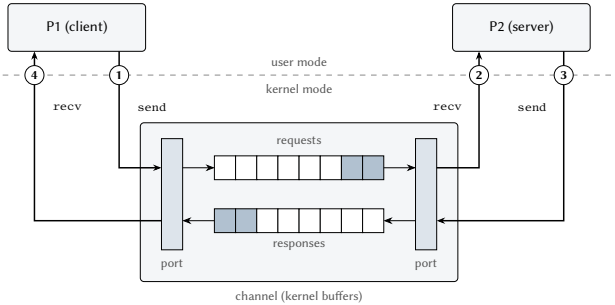


Figure 9.1: A request-response exchange over message-based IPC. Each numbered crossing of the user/kernel boundary is one system call and one copy between a process's address space and the kernel's channel.

9.3 Forms of message passing

Unix-like systems provide three common forms of message passing, which differ in what they transfer and in how much processing the kernel performs.

Source P3L3, forms of message passing; OSTEP ch. 5; Silberschatz et al. ch. 3; Kerrisk ch. 44, 46, 52, 56–57.

Definition 9.2 (Pipe). A *pipe* is a channel between two endpoints that carries a byte stream: bytes written into one end are read from the other end in the same order, without message boundaries.

The POSIX call `pipe(fd)` creates a pipe and stores one file descriptor for each end in the two-element array `fd`. A typical use connects the output of one process to the input of another, as the shell does for `ls | wc -l`.

A Linux pipe holds 16 pages, 64 KiB, by default; `fcntl` with `F_SETPIPE_SZ` raises this up to `pipe-max-size` (1 MiB); writes of at most `PIPE_BUF` = 4 kB are atomic (`pipe(7)`).

Such a pipe has no name: only the processes that hold its descriptors, the creator and the children that inherit them through `fork`, can use it. A *named pipe*, or FIFO, created with `mkfifo(path, mode)`, has a name in the file system, through which unrelated processes open it with `open` like a file; the data still pass through a kernel buffer and never reach the disk.

Definition 9.3 (Message queue). A *message queue* is a channel that carries discrete messages among processes: a sender submits a properly formatted message, and a receiver obtains complete messages.

Tip `ls -l /proc/self/fd | cat` shows the pipe that connects the two commands as file descriptor 1 of `ls`: `1 -> pipe:[…]`.

Because the kernel knows where messages begin and end, it can manage them, for example deliver messages in the order of the priority that the sender attaches to them (POSIX) or let a receiver select messages by type (System V). Unix-like systems offer a System V and a POSIX interface for message queues (Table 9.1). In System V, `msgget` creates or opens the queue associated with a key and returns its identifier; every message begins with a `long` type field, and `msgrcv` with type 0 takes the first message of the queue, with a positive type the first message of that type. In POSIX, `mq_open` opens a queue by name and returns a descriptor; `mq_receive` always returns the oldest message of the highest priority. As with shared memory, a queue persists until it is removed explicitly, by `msgctl` with `IPC_RMID` or by `mq_unlink`; `mq_close` only releases the descriptor.

Linux defaults: 8 kB per System V message, 16 kB per queue, ten messages per POSIX queue (`msgsnd(2)`, `mq_overview(7)`).

Table 9.1: The message queue calls of the two interfaces; `m` is the message buffer, `n` its length, `flg` the flags.

Operation	System V	POSIX
Create or open	`msgget(key, flg)`	`mq_open(name, flg, …)`
Send	`msgsnd(id, m, n, flg)`	`mq_send(q, m, n, prio)`
Receive	`msgrcv(id, m, n, type, flg)`	`mq_receive(q, m, n, &prio)`
Close	—	`mq_close(q)`
Destroy	`msgctl(id, IPC_RMID, NULL)`	`mq_unlink(name)`

Definition 9.4 (Socket). A *socket* is a communication endpoint created with the `socket()` system call, which allocates a kernel-level socket buffer and associates with it the kernel-level

processing that the communication needs, such as a TCP/IP protocol stack.

The calls `send()` and `recv()` pass data from a user buffer into the socket buffer and out of it; a *stream socket* (TCP) delivers a byte stream, a *datagram socket* (UDP) discrete messages. A connection between two stream sockets is set up asymmetrically. The server creates a socket with `socket()`, assigns it an address with `bind()`, marks it as accepting connections with `listen()`, and takes each incoming connection with `accept()`, which returns a new socket for that client. The client creates a socket and connects it to the server's address with `connect()`. Datagram sockets need no connection: `sendto()` and `recvfrom()` name the peer of each message.

The address family chosen in `socket()` decides the path of the data. With `AF_INET`, the channel connects the process through the TCP/IP protocol stack to a network device; even between processes on the same machine, through the loopback address `127.0.0.1`, the data traverse the whole stack, and only the network device is skipped. *Unix-domain sockets* (`AF_UNIX`), named by a path in the file system, connect processes on the same machine and bypass the network protocol stack entirely. Table 9.2 compares the three forms.

Table 9.2: The three forms of message passing.

	Pipe	Message queue	Socket
Unit of transfer	byte stream	message	byte stream or datagram
Kernel processing	buffering	boundaries, priorities	TCP/IP or Unix domain
Machines	same	same	same or different
Interface	`pipe()`	System V, POSIX	`socket()`

9.4 Shared-memory IPC

Source P3L3, shared memory IPC; Silberschatz et al. ch. 3.

In shared-memory IPC the processes read and write a region of memory that all of them can access. The operating system establishes this channel by mapping the same physical pages into the virtual address spaces of the processes: page-table entries of every participating process point to the same page frames (Figure 9.2). As with any paged mapping (Lesson 8), the virtual addresses at which the processes see the region need not be the same, and the physical frames need not be contiguous.

Caution A pointer that one process stores in the region is not valid in the other, since $VA_1 \neq VA_2$.

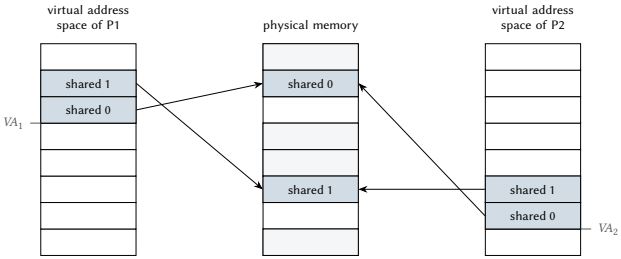


Figure 9.2: Shared memory. Two page frames are mapped into both address spaces; P1 sees the region at virtual address VA_1, P2 at a different address VA_2, and the frames are not adjacent in physical memory.

Once the mapping exists, the operating system is out of the way: reads and writes are ordinary memory accesses, and system calls are needed only for the setup. Data copies are potentially reduced, but not eliminated. Data that a process creates in the shared region are immediately visible to the others, whereas data that already exist elsewhere in its address space must still be copied into the region.

Mapping moves no data, only page-table entries (Lesson 8); one 2 MB huge page covers 512 base pages with a single entry (`SHM_HUGETLB`).

The price is that the operating system no longer performs the services it provides for message passing. The processes must synchronize their concurrent accesses explicitly (Section 9.8), and the communication protocol—the format of the data, how a receiver learns that data are available, how the space in the region is managed—is the programmer's responsibility (Section 9.10). Unix and Linux support shared memory through System V shared memory (Section 9.6), POSIX shared memory (Section 9.7) and memory-mapped files; Android adds its own facility, ashmem.[1]

[1] An ashmem region can be *unpinned*: under memory pressure the kernel may then reclaim its expendable contents, and a later pin reports whether they were purged. Android is moving these uses to `memfd_create`.

9.5 Copy versus map

Source P3L3, copy vs. map; Silberschatz et al. ch. 3.

Message passing and shared memory pursue the same goal, transferring data from one address space into another, but spend CPU time on different operations.

Copy (message passing) CPU time is spent on copying the data into the port and out of it again, with a system call for every send and receive—for every message.

Map (shared memory) CPU time is spent on mapping physical memory into the address spaces—once, when the region is set up—and on copying data into the region whenever they are not created there.

Mapping is relatively expensive. Set up once and used for many transfers, it pays off, as the example in Lesson 2 showed. For large data it can pay off even for a single transfer, because copying a large amount of data from one address space into another takes far longer than mapping it: $t_{\text{copy}} \gg t_{\text{map}}$. A simple cost model makes both statements precise. Let a system call cost t_{sys}, copying cost c per byte, and let n transfers of S bytes each go from one process to another. Message passing needs two system calls and two copies per transfer. Shared memory needs a one-time setup of cost t_{setup}; per transfer, it needs one copy, into the region, from which the receiver reads the data in place, and the time t_{sync} that the processes spend synchronizing their accesses and notifying each other (Section 9.8):

$$T_{\text{copy}}(n) = n\,(2\,t_{\text{sys}} + 2cS), \qquad T_{\text{map}}(n) = t_{\text{setup}} + n\,(cS + t_{\text{sync}}). \tag{9.1}$$

If $t_{\text{sync}} < 2\,t_{\text{sys}} + cS$, shared memory is cheaper for $n > n^* = t_{\text{setup}}/(2\,t_{\text{sys}} + cS - t_{\text{sync}})$; otherwise it never catches up. The synchronization cost is close to zero when an uncontended process-shared mutex protects the region and the receiver does not block, but a process that blocks and is woken up adds system calls. The setup cost consists of a fixed part and a part proportional to the number of pages mapped; as long as mapping a page costs less than copying its contents, n^* falls as S grows and drops below one for large data.

Tip Setting $n^* \le 1$ for the constants of Example 9.5 gives $S \ge 640$ kB: above that a single transfer already pays for the mapping.

Example 9.5. Let $t_{\text{sys}} = 1\,\mu\text{s}$, $c = 0.1\,\mu\text{s}$ per kilobyte, and let the setup cost 50 µs plus 0.1 µs per 4 kB page. A single transfer of $S = 40$ MB (10^4 pages) costs $2 + 2 \times 4000 = 8002\,\mu\text{s}$ by message passing, but, with $t_{\text{sync}} \approx 0$, $1050 + 4000 = 5050\,\mu\text{s}$ over shared memory, and only the 1050 µs of setup if the sender creates the data directly in the region; $n^* = 1050/4002 \approx 0.26$. A blocking wait and a wake-up, $t_{\text{sync}} = 2\,\mu\text{s}$, leave this unchanged ($n^* = 1050/4000 \approx 0.26$), whereas for $S = 4$ kB they raise n^* from $50.1/2.4 \approx 21$ to $50.1/0.4 \approx 125$.

Here 0.1 µs per kilobyte is 10 GB s^{-1}, the order of magnitude of one core's copy bandwidth to main memory.

Windows exploits this trade-off in its facility for communication between processes on the same machine, the *local procedure call* (LPC). Small messages are copied through a port; for larger data, a shared memory region is mapped into both processes, and the data are exchanged through it.

Current Windows versions implement this as ALPC (Silberschatz et al. ch. 3).

> **Key point**
> Message passing is simple—the kernel manages the channel and synchronizes the processes—but every message pays for system calls and two copies. Shared memory pays a one-time mapping cost and then bypasses the kernel, at the price of explicit synchronization and a programmer-defined protocol. Small messages are better copied; large or reused data better mapped.

9.6 System V shared memory

Source P3L3, SysV shared memory, SysV shared memory API; Kerrisk ch. 45, 48.

System V IPC, one of the oldest Unix IPC interfaces, comprises message queues, semaphore sets and shared memory, each object identified by a key; it organizes shared memory in segments.

Definition 9.6 (Shared memory segment). A *shared memory segment* is a region of physical memory, not necessarily made of contiguous pages, that the operating system manages as a system-wide object and that any number of processes can map into their address spaces.

Because segments are system-wide, the operating system limits both their number and their total size. A segment is identified by a *key*: every process that uses the same key accesses the same segment. Its life cycle consists of four operations (Figure 9.3).

Linux defaults in `/proc/sys/kernel`: `shmmni` 4096 segments; `shmmax` and `shmall` are effectively unlimited since Linux 3.16 (older kernels: `shmmax` 32 MiB) (`shmget(2)`, `sysvipc(7)`).

Create The operating system allocates the memory of the segment and associates it with the key.

Attach To *attach* a segment, the operating system maps it into the address space of the calling process, i.e., it establishes valid mappings from virtual addresses of the process to the

physical memory of the segment. Several processes may attach the same segment, each at its own virtual address.

Detach To *detach* it, the operating system invalidates these mappings: the page-table entries for the virtual address range of the segment are no longer valid. The segment itself remains.

Destroy Only an explicit request removes a segment. Until then it persists, even when no process has it attached, and it may be attached and detached many times.

Caution `ipcs -m` marks a segment removed while still attached with status `dest`; it disappears when `nattch` drops to 0.

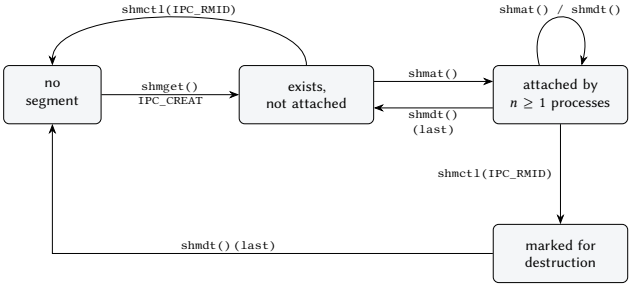


Figure 9.3: Life cycle of a System V shared memory segment. A segment outlives the processes that use it and is removed only by `IPC_RMID`; a segment removed while still attached is destroyed when the last process detaches.

A segment therefore behaves like a file rather than like the private memory of a process: a process that exits is detached from its segments, but the segments remain until they are destroyed or the system reboots.

Caution A segment that no process destroys occupies memory until reboot (Section 9.9).

9.6.1 The System V API

The interface consists of four calls and a helper for generating keys.

`shmget(key, size, flags)` opens the segment associated with `key`; if none exists and `flags` contains `IPC_CREAT`, it creates one of the given size (with `IPC_CREAT | IPC_EXCL` the call fails if the segment already exists). It returns the segment's identifier. The low nine bits of `flags` are the access mode, as for a file: `0600` lets only the owner read and write. A process that opens an existing segment passes size 0.

`size` is rounded up to whole pages, so the smallest segment is one page (4 kB on x86-64) (`shmget(2)`).

`ftok(pathname, proj_id)` generates a key from the identity (device and i-node number) of an existing file and the low 8 bits of a nonzero integer. As long as the file is not re-created, the same file and number yield the same key, so unrelated processes can find a segment by agreeing on a file name and a number.

Caution The key holds only the `proj_id` byte, the low 16 bits of the i-node number and the low 8 bits of the device number, so different files can collide on one key (`ftok(3)`).

`shmat(id, addr, flags)` attaches the segment and returns the virtual address at which it is mapped; with `addr` set to `NULL` the operating system chooses the address. The returned untyped pointer can be cast to any data structure placed in the segment.

`shmdt(addr)` detaches the segment mapped at `addr`.

`shmctl(id, cmd, buf)` passes a control command for the segment to the operating system; `IPC_RMID` destroys it.

Example 9.7. Process A creates a segment and writes a string into it; process B, which runs later, finds the segment through the same key, prints the string and destroys the segment (error checks omitted; `/tmp/app` must exist, as `ftok` returns −1 otherwise).

```
/* process A */
key_t key = ftok("/tmp/app", 'A');
int id = shmget(key, 4096, IPC_CREAT | 0600);
char *p = shmat(id, NULL, 0);
strcpy(p, "hello");                 /* visible to B  */
shmdt(p);                           /* segment stays */

/* process B */
key_t key = ftok("/tmp/app", 'A'); /* same key      */
int id = shmget(key, 0, 0);         /* open          */
char *q = shmat(id, NULL, 0);
printf("%s\n", q);
```

```
shmdt(q);
shmctl(id, IPC_RMID, NULL);          /* destroy       */
```

Between A's `shmdt` and B's `shmat` no process has the segment attached, yet its contents are preserved.

9.7 POSIX shared memory

POSIX standardizes a second shared memory interface, *POSIX shared memory*, which is not as widely supported as System V; Linux, for instance, has supported it since kernel 2.4. Instead of segments identified by keys it uses *shared memory objects* identified by a name and accessed like files; on Linux they are files in a memory-backed temporary file system (tmpfs). The processes access them through the general mechanism for mapping files into memory.

Source P3L3, POSIX shared memory API; Kerrisk ch. 49, 54.

Definition 9.8 (Memory-mapped file). A *memory-mapped file* is a file whose contents are mapped into the virtual address space of a process, so that the process accesses the file by reading and writing memory. Processes that map the same file as shared map the same physical pages and see each other's changes.

A memory-mapped file on an ordinary file system is itself an IPC mechanism whose data also persist in the file; POSIX shared memory applies the same mechanism to an object that exists only in memory. Its operations correspond to those of System V (Table 9.3). `shm_open(name, flags, mode)` creates or opens the shared memory object and returns a file descriptor for it, and `ftruncate` sets its size. `mmap` with the flag `MAP_SHARED` attaches the object and returns its virtual address; `munmap` detaches it. `close` releases the file descriptor without affecting an established mapping, and `shm_unlink(name)` removes the name; the object is destroyed once no process has it open or mapped.

The objects appear under `/dev/shm`, where Linux mounts this tmpfs; `df /dev/shm` shows its size limit, by default half the RAM (`shm_overview(7)`, `tmpfs(5)`).

Tip `shm_open` names start with one slash and contain no other; on glibc before 2.34, link with `-lrt` (`shm_open(3)`).

Caution With `MAP_PRIVATE` the mapping is copy-on-write instead: the first write gives the process a private page and nothing is shared (`mmap(2)`).

Table 9.3: Corresponding operations of the two shared memory interfaces.

Operation	System V	POSIX
Name the region	key from `ftok()`	object name (`/name`)
Create or open	`shmget()`	`shm_open()`, `ftruncate()`
Attach	`shmat()`	`mmap()`
Detach	`shmdt()`	`munmap()`
Destroy	`shmctl(IPC_RMID)`	`shm_unlink()`

9.8 Shared memory and synchronization

Processes that access a shared region concurrently are in the situation of the threads of Lesson 3, which share state within one address space: without synchronization their accesses form race conditions. Two kinds of mechanisms are available: those of a threading library, such as the Pthreads mutexes and condition variables, configured for use across processes; and synchronization based on IPC mechanisms that the operating system provides, such as message queues and semaphores. With either kind the processes must coordinate two things: the number of concurrent accesses to the shared region, which mutual exclusion limits, and the moment at which data become available for consumption, which one process must be able to wait for and the other to announce, as with a condition variable.

Source P3L3, shared memory and sync, PThreads sync for IPC, sync for other IPC; Kerrisk ch. 30, 46–47, 52–53.

9.8.1 Pthreads synchronization across processes

The Pthreads mutexes and condition variables of Lesson 4 can synchronize processes under two conditions. First, both attribute objects with which they are initialized, `pthread_mutexattr_t` and `pthread_condattr_t`, must have the *process-shared attribute* set to `PTHREAD_PROCESS_SHARED`; by default only the threads of the creating process may use them. Second, the mutex and the condition variable themselves must reside in the shared region, so that all processes operate on the same data structures.

Example 9.9. A process defines the layout of the region, which begins with the synchronization constructs:

```
typedef struct {
  pthread_mutex_t mutex;
  pthread_cond_t  cond;
  char            data[1024];
} shm_data_t;
```

It creates and attaches the segment:

```
key_t key = ftok("/tmp/app", 120);
int id = shmget(key, sizeof(shm_data_t),
                IPC_CREAT | IPC_EXCL | 0600);
shm_data_t *shm = shmat(id, NULL, 0);
```

and it initializes a process-shared mutex and condition variable inside the region:

```
pthread_mutexattr_t ma;
pthread_condattr_t  ca;
pthread_mutexattr_init(&ma);
pthread_mutexattr_setpshared(&ma,
                             PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&shm->mutex, &ma);
pthread_condattr_init(&ca);
pthread_condattr_setpshared(&ca,
                            PTHREAD_PROCESS_SHARED);
pthread_cond_init(&shm->cond, &ca);
pthread_mutexattr_destroy(&ma);
pthread_condattr_destroy(&ca);
```

Both attribute objects carry PTHREAD_PROCESS_SHARED; once the constructs are initialized, the attribute objects are no longer needed and are destroyed.[2] The flag IPC_EXCL makes shmget fail if the segment already exists, so exactly one process, the creator, initializes the synchronization constructs. The other processes open the segment without IPC_EXCL, but they must not use the constructs until the creator signals that the initialization is complete, e.g. with a message, a semaphore or an atomic "initialized" flag that it sets after the calls above; otherwise a process could lock shm->mutex before pthread_mutex_init has run. From then on, every process that attaches the segment locks shm->mutex and waits on shm->cond exactly as threads would.

[2] A process can die inside its critical section, and the mutex then stays locked for ever. A mutex whose attribute object sets PTHREAD_MUTEX_ROBUST hands the next locker EOWNERDEAD instead; that process repairs the shared state and calls pthread_mutex_consistent, or the mutex becomes unusable (ENOTRECOVERABLE).

9.8.2 Synchronization with other IPC mechanisms

Processes can also synchronize through the IPC facilities of the operating system. A message queue can provide mutual exclusion through a protocol of sends and receives (Figure 9.4). P1 writes data into the shared region and then sends a "ready" message; P2, blocked in a receive on the queue, reads the data when that message arrives and replies with an "ok" message, which P1 awaits before it accesses the region again. Because each process accesses the region only after receiving the other's message, the accesses never overlap.

As both directions use one queue, each process must receive only its peer's messages, e.g. by selecting them by message type (System V).

A semaphore (Lesson 10) is a synchronization construct of the operating system that holds an integer value. Used with the values 0 and 1 it acts like a mutex: a process that finds the value 0 blocks; a process that finds the value 1 decrements it to 0 and proceeds, and it increments the value again when it leaves the critical section. System V semaphores come in sets: semget(key, nsems, flg) creates or opens a set of nsems semaphores identified by a key, like a segment; semop(id, ops, nops) applies an array of increments and decrements to members of the set atomically, blocking while a decrement would make a value negative; and semctl sets the values (SETVAL) and destroys the set (IPC_RMID). The POSIX interface is simpler: sem_open creates or opens a named semaphore, sem_wait decrements it, blocking while its value is 0, and sem_post increments it (Lesson 10).

A POSIX named semaphore (sem_open) is process-shared by construction; an unnamed one needs pshared set and must lie in the region (sem_overview(7)).

> **Key point**
> Shared memory needs the same synchronization as shared state in a multithreaded process. Pthreads mutexes and condition variables work across processes when they are process-shared and reside in the shared region; message queues and semaphores provide the same coordination through the operating system.

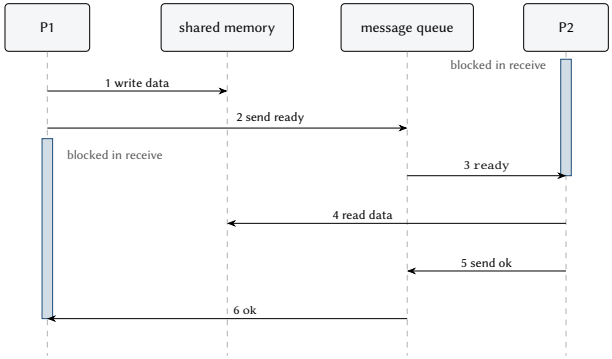


Figure 9.4: A message queue coordinates access to shared memory: P2 reads only after "ready", and P1 continues only after "ok".

9.9 IPC command-line tools

Because System V IPC objects outlive the processes that created them, Unix-like systems provide commands to inspect and remove them. The command `ipcs` lists all IPC facilities in the system—message queues, shared memory segments and semaphore sets—with their keys, identifiers, owners and permissions; `ipcs -m` restricts the list to shared memory. The command `ipcrm` removes a facility: `ipcrm -m` followed by a segment identifier destroys that segment, which is the usual way to reclaim a segment that a program failed to destroy.

Source P3L3, IPC command line tools; Kerrisk ch. 45.

Tip The `nattch` column counts the processes attached to a segment right now; a segment with `nattch` 0 that no program will open again is a leak. `ipcs -l` prints the kernel limits; `ipcrm -M` followed by a key removes a segment by key (`ipcs(1)`, `ipcrm(1)`).

9.10 Designing communication over shared memory

The operating system provides shared memory and then stays out of the way. The choice of synchronization mechanism, the format of the data and the protocol for passing them are left to the application. This flexibility makes the programmer responsible for two design decisions in particular.

Source P3L3, shared memory design considerations, how many segments, design considerations.

9.10.1 How many segments?

Processes that communicate over shared memory can organize it in two ways (Figure 9.5).

One large segment All communication uses a single segment. The application then needs a memory manager that allocates portions of the segment to individual exchanges and frees them afterwards.

Many small segments Each pair of communicating processes uses a segment of its own. Because creating segments is costly and their number is limited, a pool of segments is created in advance, and the identifiers of the free ones are kept in a queue. A process takes a segment from the pool and tells its peer the identifier through another mechanism, such as a message queue; after the exchange the segment returns to the pool.

Allocating a pool in advance and handing out its entries is the argument of the boss-workers thread pool of Lesson 3, applied to segments.

Caution A pool of segments is bounded by `shmmni`; running out gives `ENOSPC` from `shmget`, easy to mistake for a full disk (`shmget(2)`).

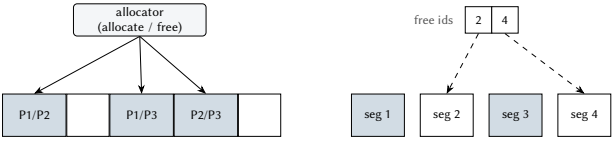


Figure 9.5: Organizing shared memory for several pairs of processes: (a) one large segment whose portions an allocator hands out; (b) a pool of small segments whose free identifiers are kept in a queue.

9.10.2 How large a segment?

The second decision is the size of a segment, and what to do with data that do not fit into one.

Segment size equal to the data size Every data item fits into one segment. This works well when the data sizes are known and fixed, but it limits the size of the data to the maximum segment size.

Segment smaller than the data Larger data are transferred in rounds, one portion at a time. The processes then need a protocol that tracks the progress of the transfer, typically a data structure at the start of the segment that holds the synchronization constructs and records how much data the segment contains and whether the transfer is complete.

Example 9.10. A segment starts with the following header; the rest of the segment holds 2 MB of payload.

```
typedef struct {
  pthread_mutex_t mutex;  /* process-shared          */
  pthread_cond_t  cond;   /* process-shared          */
  size_t nbytes;          /* bytes in buf; 0 = empty */
  int    done;            /* set with last portion   */
  char   buf[];           /* payload                 */
} xfer_t;
```

This is the producer/consumer bounded buffer of Lesson 4 with a single slot, the slot being the payload area of the segment.

Holding the mutex, the sender waits until `nbytes` is 0, copies the next portion into `buf`, sets `nbytes` (and `done` with the last portion), and signals. The receiver waits until `nbytes` is non-zero, copies the portion out, resets `nbytes` to 0, and signals. A transfer of 5 MB takes $\lceil 5/2 \rceil = 3$ rounds:

Step	Process	Action	`nbytes` (MB)	`done`
1	sender	writes bytes 0–2 MB	2	0
2	receiver	reads them	0	0
3	sender	writes bytes 2–4 MB	2	0
4	receiver	reads them	0	0
5	sender	writes bytes 4–5 MB	1	1
6	receiver	reads them, sees `done`	0	1

9.10.3 Message passing or shared memory?

The amount of data also decides between the two mechanisms of this lesson. Small data exchanged at a low rate favour message passing: the kernel provides the channel and the synchronization, and the few copies cost little. Large data, or a region reused for many exchanges, favour shared memory, whose one-time setup is then amortized. Where the break-even point lies depends on the cost of synchronization per exchange, t_{sync} in Equation (9.1): a protocol that blocks and wakes the peer for every exchange can cancel the advantage of shared memory for small data.

> **Key point — Lesson summary**
> IPC mechanisms move data across address spaces. Message passing—pipes, message queues and sockets—sends every message through a kernel channel via ports, at the cost of system calls and copies for every message. Shared memory maps the same physical pages into several address spaces, so that after a one-time setup the processes communicate without the kernel; whether copying or mapping is cheaper depends on the data size and on how often the mapping is reused (Equation (9.1)), a trade-off that Windows LPC exploits. System V segments (`shmget`, `shmat`, `shmdt`, `shmctl`) and POSIX shared memory (`shm_open`, `mmap`, `munmap`, `shm_unlink`) persist until explicitly destroyed; `ipcs` and `ipcrm` inspect and remove System V objects. Processes that share memory must synchronize, with process-shared Pthreads mutexes and condition variables placed in the region or with message queues and semaphores, and they must decide how many segments to use and how to transfer data larger than a segment.

Lesson 10

Synchronization Constructs

Lecture	P3L4
Textbook	OSTEP [1] ch. 28, 31; Silberschatz et al. [2] ch. 6–7
Keywords	spinlock, busy waiting, semaphore, reader-writer lock, monitor, barrier, wait-free synchronization, RCU, atomic instruction, test-and-set, cache coherence, write-invalidate, test-and-test-and-set, queueing lock

Lesson 3 introduced mutexes and condition variables, and Lesson 9 showed that processes communicating through shared memory need synchronization just as threads do. This lesson surveys further synchronization constructs—spinlocks, semaphores, reader-writer locks, monitors and several less common ones—that are either simpler or more expressive than mutexes and condition variables. It then turns to what every such construct ultimately rests on: atomic instructions provided by the hardware, and their interaction with the caches of a shared memory multiprocessor.

10.1 Beyond mutexes and condition variables

Mutexes and condition variables can express the synchronization policies of Lesson 3, but they have three limitations that motivate other constructs.

Source P3L4, more about synchronization; OSTEP ch. 28.

Error-prone use Correctness rests entirely on the programmer. Every access to shared state must lock the right mutex and release it on every path, and every change of state must signal or broadcast the right condition variable. Most pitfalls listed in Lesson 3, such as unlocking the wrong mutex or signalling the wrong condition variable, violate these rules, and the constructs themselves detect none of them.

Limited expressive power A mutex enforces exactly one policy: one thread at a time. A richer policy, such as admitting several readers or one writer, or giving waiting writers precedence, must be encoded with helper variables and explicit checks, like the proxy variable of the readers-writers solution in Lesson 3.

Dependence on low-level support A mutex is itself shared state. Its lock operation must find the mutex free and mark it as owned in one indivisible step; practical implementations obtain such a step from atomic instructions provided by the hardware (Section 10.7).

Semaphores, reader-writer locks, monitors and the constructs of Sections 10.3 to 10.6 address the first two limitations; the spinlock of Section 10.2 is the low-level building block beneath all of them, and the third limitation is the subject of the second half of the lesson.

10.2 Spinlocks

The simplest construct beyond the mutex is the spinlock.

Source P3L4, spinlocks; OSTEP ch. 28.

Definition 10.1 (Spinlock). A *spinlock* is a lock that provides mutual exclusion through lock and unlock operations, like a mutex, except that a thread which finds the lock held does not block. The thread keeps running and repeatedly checks the lock until the lock becomes free or the thread is preempted.

Checking a condition over and over in a loop, instead of giving up the CPU until the condition holds, is called *busy waiting*; a thread that busy-waits on a spinlock is said to *spin*. A spinlock is used exactly like a mutex:

```
spinlock_lock(s);
// critical section
spinlock_unlock(s);
```

The two constructs differ only in what a thread does while the lock is held by another thread, and they incur different costs (Figure 10.1). A thread that blocks on a mutex consumes no CPU time while it waits, but it pays for a context switch away from it and, once the mutex is released, for a wake-up and a context switch back. A spinning thread avoids both context switches, but it occupies its CPU for as long as the lock remains held.

POSIX: `pthread_spin_lock` and `pthread_spin_unlock`.

Spinning therefore pays off only when the lock is held for less time than it takes to block and resume a thread, and only when the owner is running on another CPU at the same time. On a

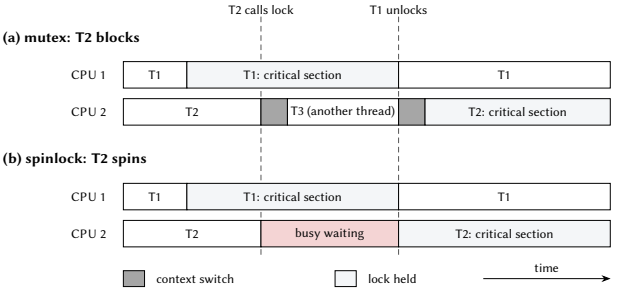


Figure 10.1: Thread T2 requests a lock held by T1 on another CPU. (a) On a mutex, T2 blocks, the CPU runs another thread, and T2 is switched back in after the unlock. (b) On a spinlock, T2 busy-waits and enters the critical section as soon as T1 unlocks.

Tip Spin for about what blocking would cost, then block: never worse than twice the best possible choice (Karlin et al., 1991).

Measured on a 2007 dual Xeon: a context switch costs 3.8 µs directly, plus the cache refill after it (Li et al., ExpCS 2007; a dated number).

single CPU the owner cannot release the lock while the spinning thread occupies the CPU, so spinning merely wastes the remainder of the timeslice. The adaptive mutexes of Lesson 5 combine both behaviours: they spin when the owner is running on another CPU and block otherwise.

Example 10.2. Suppose blocking a thread and later resuming it costs 8 µs in total. If the owner holds a lock for at most 0.5 µs, a waiting thread spins for at most 0.5 µs, one sixteenth of the cost of blocking. If instead the owner performs a 5 ms disk read while holding the lock, a spinning thread burns 5 ms of CPU time, $5000/8 = 625$ times the cost of blocking.

Because a spinlock needs nothing but a shared word and an atomic instruction, it is the basic construct from which more complex constructs are built. An operating system kernel, for example, protects the short critical sections that update the state of a mutex and its queue of blocked threads with a spinlock.

In Linux, `spin_lock` also disables preemption on the local CPU, so a lock holder is never descheduled while others spin (kernel locking documentation).

10.3 Semaphores

Source P3L4, semaphores, POSIX semaphores API; Dijkstra [25]; OSTEP ch. 31.

Semaphores, proposed by Dijkstra [25], are a common synchronization construct in operating system kernels. They generalize the mutex from one thread at a time to a bounded number of threads.

Definition 10.3 (Semaphore). A *semaphore* is a synchronization construct represented by an integer count, initialized with a non-negative integer, and two operations. *Wait*: if the count is positive, decrement it and proceed; if it is zero, block until it becomes positive. *Post*: increment the count and, if threads are blocked, wake one of them.

The count records how many more threads may pass. A thread decrements it with wait when it enters and increments it with post when it leaves, so the initial value is the maximum number of threads that can be inside at the same time. The two operations are also known as P and V, commonly explained as the Dutch *proberen* (to test) and *verhogen* (to increase).

Semaphores are distinguished by the range of the count. A *counting semaphore* may take any non-negative value and admits as many threads at once as its initial value. A semaphore whose count takes only the values 0 and 1 is a *binary semaphore*. Initialized to 1, it admits one thread at a time and behaves like a mutex.

Tip Read the count as permits: 1 is a mutex, n is n slots, 0 is an event a thread waits for.

Example 10.4. A counting semaphore initialized to 2 guards two identical devices, and three threads use them. Table 10.1 traces the count and the blocked threads.

Table 10.1: A counting semaphore with initial value 2 used by three threads. T3 blocks while the count is zero and proceeds after T1's post.

Step	Event	Count	Blocked
1	T1 waits: count > 0, decrements, proceeds	1	—
2	T2 waits: count > 0, decrements, proceeds	0	—
3	T3 waits: count = 0, blocks	0	T3
4	T1 posts: increments, wakes T3	1	—
5	T3 re-tests: count > 0, decrements, proceeds	0	—
6	T2 posts	1	—
7	T3 posts	2	—

10.3.1 POSIX semaphores

POSIX defines semaphores in `<semaphore.h>`:

```
#include <semaphore.h>

sem_t sem;
int sem_init(sem_t *sem, int pshared,
             unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
```

`sem_init` sets the initial count to `value`. The flag `pshared` states whether the semaphore is used only by the threads of one process (zero) or shared among processes (non-zero); in the latter case the `sem_t` itself must reside in memory that those processes share, such as a shared memory segment of Lesson 9. `sem_wait` and `sem_post` are the wait and post operations.

Since Linux 2.6 (glibc NPTL) POSIX semaphores are built on futexes: an uncontended `sem_wait` or `sem_post` is a single atomic instruction in user space.

Caution A semaphore has no owner: any thread may post, even one that never waited.

Example 10.5. A server allows at most four requests at a time to reach its back-end database:

```
#include <semaphore.h>

#define MAX_BACKEND 4
sem_t backend_slots;

void init_slots(void) {
  sem_init(&backend_slots, 0, MAX_BACKEND);
}

void handle_request(struct request *r) {
  sem_wait(&backend_slots);   /* take a slot */
  forward_to_backend(r);      /* at most 4 here */
  sem_post(&backend_slots);   /* free the slot */
}
```

A fifth thread that calls `handle_request` while four requests are in progress blocks in `sem_wait` until one of them posts.

Caution POSIX `sem_wait` can fail with `EINTR` when a signal arrives; callers must test the return value and retry.

10.4 Reader-writer locks

Lesson 3 built the readers-writers policy—any number of readers or a single writer—from a mutex, two condition variables and a proxy variable. Because the policy is so common, operating systems and thread libraries offer it as a construct of its own.

Source P3L4, reader/writer locks, using reader/writer locks; OSTEP ch. 31.

Definition 10.6 (Reader-writer lock). A *reader-writer lock* is a lock with two modes: *read* (shared) mode, which any number of threads may hold at the same time, and *write* (exclusive) mode, which one thread may hold only while no other thread holds the lock in either mode.

A thread states in which mode it needs the lock, and the construct enforces the policy. In the Linux kernel the type is `rwlock_t`:

Linux's `rwlock_t` spins; the blocking variant is `struct rw_semaphore`.

```
#include <linux/spinlock.h>

rwlock_t m;

read_lock(&m);
/* critical section: read shared state */
```

```
read_unlock(&m);

write_lock(&m);
/* critical section: read and modify */
write_unlock(&m);
```

POSIX threads, Java and the .NET framework of Windows provide the same construct; in POSIX threads it reads:

```
pthread_rwlock_t m;
pthread_rwlock_rdlock(&m);    /* acquire in read mode */
pthread_rwlock_wrlock(&m);    /* acquire in write mode */
pthread_rwlock_unlock(&m);    /* release either mode */
```

Tip Use a reader-writer lock only when readers are many and critical sections long enough to pay for the extra bookkeeping; for short sections a plain mutex is often faster.

Implementations agree on the basic policy but differ in details that affect program behaviour:

- **Recursive read locking.** When a thread acquires the read lock twice, implementations differ in whether one `read_unlock` releases both acquisitions or only one of them.
- **Upgrading and downgrading.** Some implementations let a reader convert its lock into a write lock, or a writer convert its lock into a read lock, without releasing it; they differ in whether this is allowed and in the priority such a request has over threads already waiting.
- **Interaction with scheduling.** An implementation may block an arriving reader while a writer, possibly of higher priority, is waiting, even though the lock is currently held only in read mode.

Example 10.7. Readers arrive every 3 ms and each holds the lock in read mode for 5 ms. Every reader arrives before its predecessor leaves, so the lock is never free. An implementation that always admits a reader while the lock is held in read mode therefore starves a waiting writer indefinitely. An implementation that blocks new readers once a writer waits admits the writer at most 5 ms after it starts waiting, as soon as the readers already inside have left.

10.5 Monitors

Source P3L4, monitors; Hoare [26]; Lampson and Redell [10]; Silberschatz et al. ch. 6.

The constructs so far still leave it to the programmer to lock and unlock at the right places. Monitors move this responsibility into the construct.

Definition 10.8 (Monitor). A *monitor* is a high-level synchronization construct that encapsulates a shared resource. It specifies the resource, the procedures through which alone the resource may be accessed—each an *entry procedure*—and the condition variables on which threads inside the monitor may wait.

Mutual exclusion is implicit (Figure 10.2). When a thread calls an entry procedure, the monitor's lock is acquired automatically, and threads that find the monitor occupied wait on its entry queue; when the procedure returns, the lock is released automatically. A thread that waits on a condition variable inside the monitor releases the lock, and when it is woken it re-acquires the lock before it continues, again without any explicit lock operation. Conceptually the monitor performs the locking and the checks on entry, and the unlocking, the checks and the signalling on exit. The signalling itself is nevertheless explicit in the classical designs—Hoare's monitors offer wait and signal; Mesa replaces signal with notify, a hint, and adds broadcast (Lampson and Redell [10])—and so are the conditions on which threads wait.

Monitor designs with automatic signalling go further: they re-evaluate the waiting conditions whenever a thread leaves the monitor.

Example 10.9. A bounded buffer of capacity `N` as a monitor, in C-like pseudocode:

```
monitor BoundedBuffer {
  item buf[N];
  int count = 0, in = 0, out = 0;
  condition not_full, not_empty;

  entry void put(item x) {
    while (count == N)
      wait(not_full);
    buf[in] = x;  in = (in + 1) % N;  count++;
    signal(not_empty);
  }

  entry item get(void) {
    while (count == 0)
```

```
      wait(not_empty);
    item x = buf[out];  out = (out + 1) % N;
    count--;
    signal(not_full);
    return x;
  }
}
```

No lock operation appears in the code, so none can be forgotten or applied to the wrong lock.

Caution `while`, not `if`: a Mesa signal only wakes the thread; another may take the item before it runs.

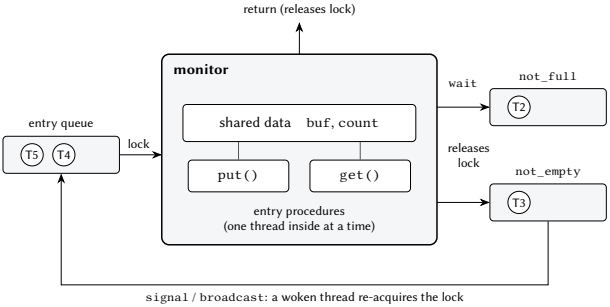


Figure 10.2: A monitor. Threads enter only through entry procedures, which acquire the monitor's lock implicitly; a thread that waits on a condition variable releases the lock and, with the Mesa semantics shown here, must re-acquire it after being woken.

Monitors were part of the Mesa language developed at Xerox PARC [10]. Java supports them today: every object has an internal lock, and a method declared `synchronized` is an entry procedure whose locking and unlocking the language runtime performs. Notifications, however, remain explicit calls to `notify` and `notifyAll`.

Mesa and Java: a woken thread re-tests its condition in a `while` loop.

In HotSpot an uncontended `synchronized` block is one compare-and-swap on the object header; contention inflates it.

The term *monitor* also denotes a programming style. Structuring every critical section with explicit enter and exit code built from a mutex and condition variables, as in Lesson 3, applies the discipline of a monitor without support from the language.

Same wait-in-a-loop discipline as the condition variables of Lesson 3; the monitor only hides the mutex.

10.6 Other synchronization constructs

Several further constructs aim at making synchronization policies easier to state, or at avoiding waiting altogether.

Source P3L4, more synchronization constructs.

Serializers A *serializer* [27] guards a resource like a monitor, but it makes priorities easy to define and hides explicit signalling. A thread waits in a queue together with the condition under which it may proceed; the serializer resumes it automatically when the condition holds and the thread is at the head of its queue, so neither condition variables nor signal calls appear in the program, and the order of the queues expresses priority.

Path expressions A *path expression* [28] states, as a regular expression over the names of the operations on a resource, the orders in which the operations may execute; the run-time system then interleaves the operations only in ways the expression allows. The policy "many reads or a single write" is written `path {read}, write end`: the braces allow any number of simultaneous reads, the comma selects one of the alternatives, and the path repeats indefinitely.

Barriers A *barrier* is the reverse of a semaphore. A semaphore lets up to n threads proceed and blocks those that come after them; a barrier blocks threads until n of them have arrived and then lets all of them proceed together (Figure 10.3). POSIX threads provide `pthread_barrier_t` with `pthread_barrier_wait`.

Rendezvous points A *rendezvous point* is a point in the execution of two or more specific threads at which each waits until the others have reached their corresponding points; the threads then continue, typically after exchanging data.

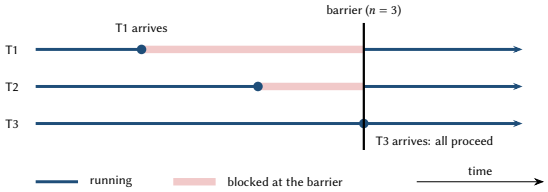


Figure 10.3: A barrier for three threads. T1 and T2 block when they reach the barrier; the arrival of T3 releases all three.

10.6.1 Wait-free synchronization and RCU

All the constructs above make threads wait. An optimistic alternative lets threads proceed without locks, betting that concurrent operations will not conflict, and designs the data structure so that concurrent readers remain safe while it is being modified.

Definition 10.10 (Wait-free synchronization). Synchronization in which every thread completes each of its operations in a finite number of its own steps, regardless of how fast or slow the other threads are, is called *wait-free synchronization* [29]. No thread ever waits for another.

The *read-copy-update* mechanism [30], now part of the Linux kernel, applies this idea to data that is read far more often than it is written. Readers take no lock at all. A writer copies the current version, modifies the copy and publishes it by atomically replacing the pointer through which readers find the data, so readers see either the old or the new version, never a half-modified one; the old version is freed once the readers that may still hold a reference to it have finished. Readers therefore never wait, while writers coordinate among themselves with an ordinary lock.

Table 10.2 compares the constructs of this lesson with the mutex. Each of them, whether it makes threads wait or not, depends on updating a memory location atomically—setting a lock word, adjusting a count or replacing a pointer—and this requires support from the hardware.

Table 10.2: Synchronization constructs compared.

Construct	Admitted at once	Waiting thread	Locking
Mutex	one	blocks	explicit
Spinlock	one	spins	explicit
Counting semaphore	up to the initial count	blocks	explicit
Reader-writer lock	any readers or one writer	blocks or spins	explicit
Monitor	one	blocks	implicit
Barrier	n, once n have arrived	blocks	explicit
RCU	any readers; one writer	readers never wait	writers only

10.6.2 Synchronization in the Linux kernel

Source Silberschatz et al. ch. 7; Linux kernel locking documentation.

The Linux kernel offers most of the constructs of this lesson under its own names.

- `spinlock_t` with `spin_lock` and `spin_unlock`: a spinlock.
- `rwlock_t`: the spinning reader-writer lock of Section 10.4.
- `struct semaphore` with down and up, and `struct rw_semaphore`: a counting semaphore and a reader-writer lock that block.
- `struct mutex` with `mutex_lock` and `mutex_unlock`: a blocking lock with an owner.
- RCU, as described above.

Beneath them, the type `atomic_t` provides atomic integer operations such as `atomic_inc` and `atomic_cmpxchg`, built on the atomic instructions of Section 10.7. A spinning lock may be held only briefly, and its holder must not sleep. A blocking lock may be held across operations that sleep, such as I/O, but it can be taken only where sleeping is allowed, not in an interrupt handler. Most critical sections in the kernel are a few instructions long, far shorter than a context switch, so the kernel mostly spins (Section 10.2); because `spin_lock` disables preemption, a holder runs until it releases the lock, and the spinning stays short.

Key point
Spinlocks trade CPU time for context switches; semaphores admit a bounded number of threads; reader-writer locks package the readers-writers policy; monitors make locking implicit. Serializers, path expressions, barriers and rendezvous points express further policies, and wait-free schemes such as RCU avoid waiting altogether.

10.7 Hardware support for synchronization

Source P3L4, sync building block: spinlock, need for hardware support, atomic instructions; OSTEP ch. 28; Silberschatz et al. ch. 6.

We first attempt a spinlock with ordinary instructions.

```
#define FREE 0
#define BUSY 1

void naive_lock(int *lock) {
  while (*lock == BUSY)
    ;                 /* spin */
  *lock = BUSY;       /* separate from the test */
}
```

The test and the update are separate instructions, and between them another thread can observe the same free lock. Table 10.3 shows two threads on different CPUs that both enter the critical section. Protecting the two steps with another lock only moves the problem into that lock. A lock of this form is correct only if checking it and setting it happen as one indivisible step, and practical implementations obtain such a step from the hardware.

Caution A compiler may also hoist the plain load out of the loop; real code uses `volatile` or an atomic load.

Software algorithms such as Peterson's do achieve mutual exclusion with ordinary reads and writes.

Table 10.3: Two threads executing `naive_lock` on different CPUs. Both read the lock before either writes it, and both enter the critical section.

Step	T1 on CPU 1	T2 on CPU 2	`*lock`
1	reads FREE, leaves loop		FREE
2		reads FREE, leaves loop	FREE
3	writes BUSY, enters		BUSY
4		writes BUSY, enters	BUSY

Definition 10.11 (Atomic instruction). An *atomic instruction* is a processor instruction that performs several operations, typically reading a memory location and writing it, as one indivisible step. The hardware guarantees atomicity (the operations take effect together), mutual exclusion (no other atomic instruction on the same memory location interleaves with it), and that of several concurrent executions on the same location all but one are made to wait.

An atomic instruction is thus a critical section protected by the hardware. Common examples are:

`test_and_set` The *test-and-set* instruction returns the current value of a lock word and sets the word to BUSY.

`read_and_increment` returns the value of a memory location and increments it.

`compare_and_swap` The *compare-and-swap* instruction compares a memory location with an expected value and, only if they are equal, writes a new value; it reports whether the write took place.

Further variants, such as `read_and_decrement` and `read_and_clear`, and in general `fetch_and_op` for an operation *op*, follow the same pattern; which of them exist differs per architecture.

Caution Compare-and-swap tests values, not history: a change from A to B and back to A passes (the ABA problem).

Written as C functions that the processor executes atomically, the first and the last are:

```
/* each function executes as one atomic step */
int test_and_set(int *lock) {
  int old = *lock;
  *lock = BUSY;
  return old;
}

int compare_and_swap(int *p, int expected,
                     int new_val) {
  if (*p != expected)
    return 0;             /* value has changed */
```

```
  *p = new_val;
  return 1;
}
```

With `test_and_set` the spinlock becomes correct:

```
void spinlock_lock(int *lock) {
  while (test_and_set(lock) == BUSY)
    ;   /* spin */
}

void spinlock_unlock(int *lock) {
  *lock = FREE;
}
```

If the lock is free, `test_and_set` returns FREE and has already marked the lock busy, so the thread leaves the loop as the owner. If the lock is busy, it returns BUSY after writing BUSY over BUSY, which changes nothing, and the thread tries again. Of several threads that execute `test_and_set` on a free lock at the same time, exactly one obtains FREE.

On x86, `xchg` with a memory operand is implicitly locked, so no `lock` prefix is needed (Intel SDM vol. 2).

Example 10.12. Compare-and-swap updates shared data without any lock. A counter is incremented by re-reading it until the swap succeeds:

```
void increment(int *counter) {
  int old;
  do {
    old = *counter;
  } while (!compare_and_swap(counter, old, old + 1));
}
```

Caution The loop never blocks, but a thread may have to retry indefinitely, so it is not wait-free.

The swap fails exactly when another thread changed the counter between the read and the swap; the loop then re-reads and retries, so no increment is lost.

10.7.1 Portability

Atomic instructions are specific to the processor architecture: which ones exist and how efficient they are differ between platforms. On x86, for example, `xchg` with a memory operand serves as test-and-set and `lock cmpxchg` as compare-and-swap. Software that implements synchronization constructs, such as a kernel or a thread library, must therefore be ported to every architecture it runs on. It is also specialized for each platform: conditional compilation selects, among several implementations of the same construct, the one built on the most efficient instructions the platform offers.

10.8 Shared memory multiprocessors and caches

Source P3L4, shared memory multiprocessors, cache coherence, cache coherence and atomic instructions; Silberschatz et al. ch. 1.

Synchronization constructs matter most on machines with several CPUs, where threads really do run at the same time. A shared memory multiprocessor (Lesson 7) has several CPUs and memory that all of them can access; such systems are also called symmetric multiprocessors (SMP). Figure 10.4 shows two configurations. In a bus-based configuration, typical of older systems, the CPUs and a single memory component are attached to one shared bus, which carries only one memory access at a time. In an interconnect-based configuration, common today, the CPUs and possibly several memory components are connected through an interconnect on which several memory accesses can be in progress at the same time.

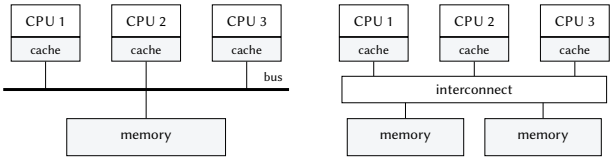


Figure 10.4: Shared memory multiprocessor configurations. In both, every CPU has its own cache and all CPUs can access all memory.

Every CPU has its own cache. Memory is slower to reach than the cache, both because it is physically farther away and because the CPUs contend for the bus or interconnect that leads to it; the caches hide this latency.

Skylake (2015): L1 4 cycles, L2 12, L3 about 40, DRAM about 80 ns. Intel optimization manual.

When several CPUs work on the same data, however, copies of one memory location may reside in several caches at once. If one CPU writes the location, the copies held by the other CPUs become stale, and a thread that read such a copy would see an outdated value (Figure 10.5). The property that every CPU observes the most recent value of a memory location, even though copies of it may be cached on several CPUs, is called *cache coherence*.

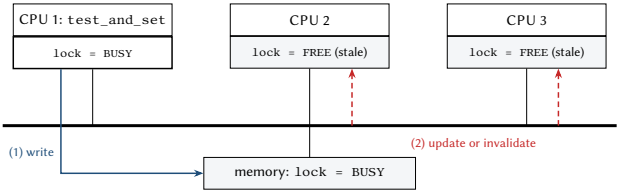


Figure 10.5: CPU 1 sets `lock` to BUSY. The copies in the caches of CPU 2 and CPU 3 become stale; on a cache-coherent platform the hardware updates or invalidates them.

Platforms differ in who is responsible for cache coherence. On a non-cache-coherent (NCC) platform the hardware does not propagate a write to the copies in other caches, and software must deal with stale copies. On a cache-coherent (CC) platform the hardware keeps the caches coherent, following one of two strategies.

Write-invalidate Under the *write-invalidate* strategy a write invalidates the copies of the location held by the other caches. Only the invalidation travels over the bus or interconnect, so the bandwidth per write is low and further writes by the same CPU cost nothing; the next read on another CPU, however, misses its cache and has to fetch the location again.

Write-update Under the *write-update* strategy a write carries the new value to every cache that holds a copy. Threads on the other CPUs read the new value from their own cache immediately, at the price of more traffic per write.

Write-invalidate is the more common of the two. Either way, the cost of the coherence mechanism is incurred by software that writes shared locations.

10.8.1 Atomic instructions on multiprocessors

On a multiprocessor an atomic instruction must be atomic across all CPUs: its execution on one CPU must exclude executions on the same location on every other CPU. In the model used here it therefore does not work on a private cached copy; it bypasses the cache, is applied to the memory location itself, and leaves every cache coherent. An atomic instruction on a multiprocessor is consequently more expensive than the same instruction on a single-CPU system, and than an ordinary instruction on the same data: it reaches memory, it contends with the other CPUs for the bus or interconnect, and it sets off the coherence mechanism.

Real processors instead take exclusive ownership of the cache line, at a cost that is high only when the line must be fetched from another cache.

Spinlocks make this worse. `test_and_set` always writes, even when it only replaces BUSY with BUSY, and every write starts the cache coherence mechanism. A spinning thread executes `test_and_set` continuously, so a spinlock that several threads wait for generates a steady stream of coherence updates on every CPU that holds a copy of the lock word.

> **Key point**
> Every synchronization construct rests on atomic instructions such as test-and-set and compare-and-swap. On a shared memory multiprocessor they must be atomic across all CPUs, and the hardware keeps the caches coherent by invalidating or by updating the other copies, which makes atomic instructions, and especially repeated writes to a contended lock word, expensive.

10.9 Spinlock performance

Source P3L4, spinlock performance metrics, test-and-set spinlock, spinlock delay alternatives, queueing lock, spinlock performance comparisons; Anderson [31].

The cost of atomic instructions on a multiprocessor affects how a spinlock should be written, a question studied by Anderson [31]. Three metrics characterize a spinlock implementation:

Latency the time a thread needs to acquire a lock that is free. Ideally the thread executes the atomic instruction immediately.

Delay the time between the release of the lock and its acquisition by a spinning thread. Ideally a spinning thread notices the release immediately, which requires it to test the lock continuously.

Contention the overhead caused by *lock contention*, the situation in which several threads try to acquire the same lock at the same time; above all, the cache coherence traffic caused by their atomic instructions. Ideally a spinning thread writes nothing while the lock is held.

No implementation is ideal in all three metrics: the first two ask a waiting thread to touch the lock as often as possible, the third asks it to touch the lock as rarely as possible. The `test_and_set` loop of Section 10.7 is ideal in latency and delay but poor in contention, because it writes on every iteration.

Tip Latency = free lock, delay = release to acquire, contention = traffic while waiting.

10.9.1 Spinning on read

The *test-and-test-and-set* technique first reads the lock with an ordinary instruction and executes `test_and_set` only when the lock appears free:

```
void spinlock_lock(int *lock) {
  while (*lock == BUSY || test_and_set(lock) == BUSY)
    ;   /* spin */
}
```

The short-circuit `||` evaluates `test_and_set` only when the read found FREE. The read alone does not decide anything—another thread may acquire the lock between the read and the `test_and_set`—but in that case the atomic `test_and_set` returns BUSY and the loop continues, so the lock remains correct. The price is one extra read before every `test_and_set`, which slightly increases both latency and delay.

What the technique does for contention depends on the platform:

NCC Neither the read nor the atomic instruction can be served from a cache, so every iteration of every spinning thread reaches memory and contention is no better than with the plain loop.

Software must then handle stale copies, as the distributed shared memory of Lesson 15 does.

CC, write-update While the lock is held, the spinning threads read their cached copies and generate no traffic at all. The release is propagated to all caches, so every waiting thread sees FREE at the same moment and they issue their `test_and_set` instructions together.

CC, write-invalidate Contention is improved only during the hold. At each release every waiter misses, reads, and issues a `test_and_set`, and every failing `test_and_set` invalidates all copies again; with short critical sections these bursts make it nearly as bad as the plain loop.

All waiters reacting to one release is a *thundering herd*, the effect FUTEX_CMP_REQUEUE avoids in Lesson 3.

Spinning on read removes the traffic of the waiting period, not the traffic of the release. When critical sections are short and many threads wait, the releases follow one another so closely that these bursts dominate. Under heavy contention on write-invalidate hardware the technique is therefore the worst of the alternatives Anderson compared against the queue and delay locks; plain `test_and_set` is worse still.

Example 10.13. A light-load case, in which spinning on read is at its best. On four CPUs a thread on CPU 1 holds a lock for 1 ms while threads on CPUs 2–4 spin. With the plain `test_and_set` loop, one iteration taking 50 ns, each spinning thread performs 1 ms/50 ns = 20 000 iterations, so the hold costs $3 \times 20\,000 = 60\,000$ writes to the lock word, each of which invalidates or updates the copies in the other caches. With test-and-test-and-set the hold costs no write at all: the iterations are reads, and there are more of them, since a read is cheaper. The release then costs about three `test_and_set` instructions, one per waiting thread, of which one succeeds. Under heavy load, with a critical section of a few hundred nanoseconds, that burst and the cache misses following it would recur at every release.

Uncontended, a `lock`-prefixed operation costs about 20 cycles on Skylake-class cores. Agner Fog, instruction tables.

10.9.2 Delay-based spinlocks

The bursts arise because all waiting threads react to a release at the same moment. Making them wait a moment before they touch the lock again spreads the reactions out. The delay can be placed after the lock has been seen to become free:

```
void spinlock_lock(int *lock) {
  while (*lock == BUSY || test_and_set(lock) == BUSY) {
    while (*lock == BUSY)
      ;         /* spin on read while it is held */
    delay();    /* it became free: wait a moment */
  }
}
```

A thread that fails to acquire the lock spins on read as before, and once the lock becomes free it waits before it tries the atomic instruction, so the waiting threads attempt their `test_and_set` at different times and each release is answered by fewer of them. Latency is that of spinning on read, since a thread that meets a free lock never enters the body of the loop. Contention improves markedly, but delay becomes worse: a thread may sit in its delay while the lock is already free.

The delay can instead be placed on every iteration:

```
void spinlock_lock(int *lock) {
  while (*lock == BUSY || test_and_set(lock) == BUSY)
    delay();
}
```

A waiting thread now references the lock once per delay period instead of continuously. This reduces the coherence traffic as well, and unlike spinning on read it helps on an NCC platform too, where every reference reaches memory. The price is again delay: a release may go unnoticed for as long as one delay period.

Both variants need a value for the delay. A *static* delay gives each thread a fixed value derived from something that distinguishes it, its CPU identifier for instance, so that the attempts are spread out by construction; it is simple, but it delays a thread even when no other thread is waiting. A *dynamic* delay is drawn at random from a range that grows with the contention a thread perceives, estimated for example from the number of `test_and_set` attempts that have failed.

Caution A failed attempt may mean a long critical section rather than many waiting threads.

10.9.3 Queueing locks

Delays spread the reactions to a release out in time. A queueing lock avoids the bursts altogether by letting exactly one thread react.

Definition 10.14 (Queueing lock). A *queueing lock* gives every thread that requests the lock a place in a queue and a flag of its own. A waiting thread spins on its own flag; the thread that releases the lock sets the flag of its successor in the queue, which thereby becomes the owner.

The queue is an array of flags, one per thread, each either `HAS_LOCK` or `MUST_WAIT`, together with the index of the next free place, which an arriving thread obtains with `read_and_increment`:

```
/* N is a power of two, so % N survives wraparound */
int flags[N];    /* flags[0] = HAS_LOCK, rest wait */
unsigned queuelast = 0;

unsigned queue_lock(void) {
  unsigned me = read_and_increment(&queuelast) % N;
  while (flags[me] == MUST_WAIT)
    ;                     /* spin on a private flag */
  flags[me] = MUST_WAIT;  /* reset for the next round */
  return me;
}

void queue_unlock(unsigned me) {
  flags[(me + 1) % N] = HAS_LOCK;
}
```

A release writes a single flag, so exactly one waiting thread is woken and only one cached copy is invalidated or updated. Against the three metrics the queueing lock has

- worse *latency*: `read_and_increment` is a more complex atomic instruction than `test_and_set`, and every acquisition executes it and then indexes the array;
- good *delay*: the owner hands the lock directly to its successor, which does not have to detect the release itself;
- much better *contention*: a release generates traffic for one flag only, provided the flags of

A cache line is 64 B on x86-64, so each flag must be padded to that size (false sharing, Lesson 15).

different threads lie in different cache lines.
In exchange the lock occupies an array of N entries instead of a single word, where N is the number of threads that may request it, and `read_and_increment` is not available on every platform.

10.9.4 Comparison

The Sequent Symmetry put 20 Intel 80386 CPUs on one shared bus, which all coherence traffic had to cross.

Table 10.4 collects the assessments. Anderson measured the implementations on a Sequent Symmetry with 20 CPUs and write-invalidate caches [31]. Under high load—many CPUs competing for the same lock—the queueing lock performed best and test-and-test-and-set worst of the alternatives compared, with the delay-based locks in between; plain `test_and_set`, measured separately, was worse still. Among the delay-based locks, a static delay did slightly better than a dynamic one at high load, and a delay after every reference slightly better than a delay after release, which Anderson attributes to the invalidation-based coherence of the Symmetry. Under light load the order was reversed: test-and-test-and-set performed best, because its latency and delay are the smallest, and the queueing lock had the highest latency, because every acquisition pays for its bookkeeping. On the Symmetry that bookkeeping included a lock: the machine had no atomic `read_and_increment`, so the queueing lock obtained its sequence number through an explicit lock with backoff, which is part of the measured latency. Static delays, finally, are inappropriate when few threads spin, so at light load the dynamic delay is the better choice. No implementation is best under all conditions; the choice depends on the expected contention and on the instructions the platform provides.[1]

[1] Anderson's queueing lock needs an array of N flags per lock. The MCS lock (Mellor-Crummey and Scott, 1991) replaces it with a list of per-thread nodes, so the lock itself is one word. Linux adopted a FIFO ticket lock in 2.6.25 (2008), after unfair hand-offs were found to starve threads on an 8-core Opteron (LWN, "Ticket spinlocks", 2008), and the MCS-based `qspinlock` in 4.2 (2015).

Table 10.4: Spinlock implementations against the three metrics. The entries are relative to each other; the measured ranking depends on the load.

Implementation	Latency	Delay	Contention
`test_and_set`	good	good	worst
Test-and-test-and-set	fair	fair	poor on write-invalidate
Delay after release	fair	poor	better
Delay on every reference	fair	poor	better, also on NCC
Queueing lock	poor	good	best

Key point — Lesson summary

Mutexes and condition variables are error-prone, limited in expressive power and dependent on hardware support. Spinlocks busy-wait instead of blocking and pay off only for short critical sections on multiple CPUs; semaphores admit up to a given number of threads, and a binary semaphore acts as a mutex; reader-writer locks provide shared and exclusive modes; monitors acquire and release their lock implicitly in entry procedures. Serializers, path expressions, barriers, rendezvous points and wait-free schemes such as RCU complete the picture. All of them depend on atomic instructions such as test-and-set and compare-and-swap, which on a shared memory multiprocessor must also keep the CPUs' caches coherent. Testing the lock before test-and-set helps at light load, but under heavy load on write-invalidate platforms it is the worst of the alternatives Anderson compared against the queue and delay locks, and plain test-and-set is worse still; delaying the waiting threads, and above all queueing them on private flags, keeps the coherence traffic of a contended lock under control.

I/O Management

Lecture	P3L5
Textbook	OSTEP [1] ch. 36–37, 39–42; Silberschatz et al. [2] ch. 11–15; Kerrisk [11] ch. 13–14
Keywords	device register, device driver, block device, character device, memory-mapped I/O, polling, programmed I/O, DMA, operating system bypass, virtual file system, inode, ext2, buffer cache, journaling

Besides the CPU and memory, whose management Lessons 7 and 8 described, an operating system manages the devices through which a computer receives input, produces output and stores data persistently. This lesson examines how the operating system represents and controls such devices: the interface a device presents, the drivers that encapsulate device-specific knowledge, and the ways in which the CPU and a device exchange commands, data and notifications. It then follows a file access down the block device stack and studies how file systems—the virtual file system and ext2 in particular—organize data on disk and reduce the cost of accessing it.

11.1 I/O devices

Source P3L5, I/O devices, basic I/O device features; OSTEP ch. 36; Silberschatz et al. ch. 12.

Input/output (I/O) devices connect a computer to its environment. Examples are keyboards, mice, microphones, speakers, displays, network interface cards (NICs) and disks. They differ widely in function—some only deliver input to the computer, some only accept output from it, many do both—and in speed, from a device that produces a few characters per second to a network interface that transfers gigabits per second. The I/O management of an operating system follows three principles:

- it defines *protocols*, standard interfaces through which devices are accessed;
- it provides *dedicated handlers*, the device drivers and interrupt handlers, for the device-specific work;
- it *decouples* the details of individual devices from the rest of the system, so that applications use abstractions such as files rather than device operations.

Definition 11.1 (Device register). A *device register* is a register of a device that the CPU reads or writes to interact with the device. Devices typically provide *command* registers, which the CPU writes to tell the device what to do; *data* registers, through which data are transferred to and from the device; and *status* registers, which the CPU reads to learn what the device is doing.

Caution A device register is not a CPU register: it belongs to the device and is named by an address, not by a register name in an instruction.

The registers form the interface of the device (Figure 11.1). Behind it lie the device internals: a microcontroller, which is the device's own CPU, on-device memory, and other device-specific chips, such as the analog-to-digital converters of a microphone. Only the interface is visible to the operating system; how the device uses its internals to carry out a command is hidden from it.

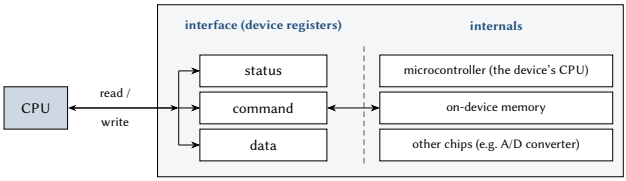


Figure 11.1: A basic I/O device. The CPU interacts with the device only through its command, status and data registers; the internals that carry out the commands are device-specific.

11.2 CPU–device interconnect

Source P3L5, CPU device interconnect; OSTEP ch. 36; Silberschatz et al. ch. 12.

Devices are attached to the CPU and memory through controllers on an interconnect. The interconnect limits how fast data can move between a device and the rest of the system and

how many devices can be attached. A standard family of such interconnects is the *Peripheral Component Interconnect* (PCI). Its original form is a shared parallel bus, which PCI-X (PCI eXtended) made faster. Current platforms use PCI Express (PCIe), which remains compatible with the software interface of PCI but offers more bandwidth, faster and lower-latency access, and connections for more devices.

Although commonly called a bus, PCIe connects each device through a point-to-point serial link and switches. A PCIe 4.0 lane carries about 2 GB s^{-1} each way, a ×16 slot about 32 GB s^{-1}; PCIe 5.0 doubles it to about 4 GB s^{-1} per lane (PCI-SIG).

Not every device is connected to PCIe directly (Figure 11.2). A SCSI controller attached to PCIe connects disks on its own SCSI bus, and an expansion bus interface connects slower peripherals such as keyboards on an expansion (peripheral) bus. Bridging controllers handle the differences between the buses, and the bridge that connects the CPU to memory also connects both to PCIe. An access from the CPU to a device may therefore cross several interconnects.

Tip `lspci -t` prints the PCI topology as a tree of bridges and devices; `lsusb -t` does the same for USB.

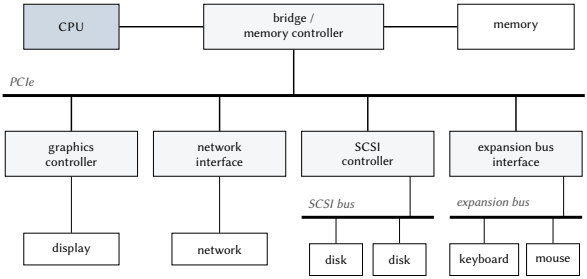


Figure 11.2: A typical platform interconnect. Device controllers are attached to PCIe; disks and keyboards are attached to further buses behind their own controllers. PCIe is drawn as a bus for simplicity; each controller is in fact attached through its own point-to-point link.

11.3 Device drivers

Source P3L5, device drivers; OSTEP ch. 36; Silberschatz et al. ch. 12.

The operating system does not itself contain the knowledge of how to operate each device.

Definition 11.2 (Device driver). A *device driver* is a device-specific software component of the operating system that is responsible for all access to, management of and control of the devices of one type.

There is one driver per device type, and drivers are typically provided by the manufacturers of the devices; in a modular operating system they are loaded as kernel modules (Lesson 1). The operating system in turn standardizes the interfaces that drivers implement, the *device driver framework*. This standardization yields two properties:

Device independence the operating system does not have to be specialized for particular devices, since it uses every device of a type through the same interface.

Device diversity the operating system can support arbitrarily different devices, including new ones: a device can be used as soon as a driver implements the interface for it.

11.4 Types of devices

Source P3L5, types of devices; Silberschatz et al. ch. 12; Kerrisk ch. 14.

Operating systems group devices into classes. The members of a class are used in the same way, and each class has its own driver interface. Three classes are distinguished:

- A *block device*, such as a disk, stores data in fixed-size blocks, and every individual block can be read or written directly by its number.
- A *character device*, such as a keyboard, delivers or accepts a sequential stream of characters; its basic operations get or put one character.
- A *network device*, such as an Ethernet interface, lies between the two: it delivers a stream of data, but in chunks of varying size rather than in fixed-size blocks or single characters.

11.4.1 Devices as files

Operating systems represent devices in the file system.

Definition 11.3 (Device file). A *device file*, also called a *special file*, is a file that represents a device. The operations that a process applies to it—open, read, write, close—are carried out on

the device by the device's driver.

In UNIX-like systems device files reside in the directory `/dev`, which is managed by a special file system: historically devfs (removed in Linux 2.6.18); on current Linux devtmpfs, an instance of tmpfs (the home of POSIX shared memory in Lesson 9) that the kernel populates and udev manages. A long listing with `ls -l` marks a block device file with the type `b` and a character device file with `c`. Because a device looks like a file, programs that know nothing about devices can use it.

On Linux, network devices are the exception: they have no device files and are used through sockets.

Major and minor numbers in `ls -l /dev` identify the driver and the device instance (Kerrisk ch. 14).

Example 11.4. The command

```
dd if=/dev/sda of=head.img bs=512 count=1
```

copies the first 512 bytes of the first disk into an ordinary file. The program `dd` uses only open, read and write; that its input is a block device, and the disk protocol that retrieves the block, are handled by the operating system and the disk's driver.

Not every device file corresponds to hardware.

Definition 11.5 (Pseudo device). A *pseudo device* is a device, represented by a device file like any other, that corresponds to no physical hardware; its driver implements the device's functionality entirely in software.

For example, reading `/dev/zero` returns an endless stream of zero bytes, which is convenient for creating a file of a given size filled with zeros, and `/dev/null` discards everything written to it and returns end-of-file on every read.

> **Key point**
> A device is operated through its command, data and status registers, by a device driver that hides the device behind an interface the operating system standardizes for the whole class of devices. The classes are block, character and network devices, and the first two are represented as device files, so that ordinary file operations reach them.

11.5 CPU–device interactions

To use a device, the CPU must be able to address its registers, and the device must be able to tell the CPU when it needs attention. There are two ways to address the registers.

Source P3L5, CPU device interactions; OSTEP ch. 36; Silberschatz et al. ch. 12.

Definition 11.6 (Memory-mapped I/O). With *memory-mapped I/O* the device registers appear to the CPU as memory locations at specific physical addresses. A part of the physical address space is dedicated to device interactions, and the CPU accesses a register with an ordinary load or store instruction on its address, which the platform directs to the device instead of to memory.

Caution A memory-mapped register is not ordinary memory: it is normally mapped uncached (write-combining for prefetchable regions such as frame buffers), and drivers access it with `read1` and `write1`, not plain pointer dereferences.

Memory-mapped I/O must not be confused with a memory-mapped file (Lesson 9), which maps the contents of a file into the virtual address space of a process. For a PCI device, the base address registers (BARs) in its configuration space set the size and location of its region; they are configured during boot.

The second way uses dedicated CPU instructions, such as `in` and `out` on x86. An *I/O port* is an address, separate from the memory addresses, that designates a device register; the instruction specifies the target port and the value to be written to the register, or receives the value read from it.

The x86 port space is only 64 KiB (16-bit port numbers), one reason why memory-mapped I/O carries most of today's device traffic.

11.5.1 Interrupts and polling

In the other direction, a device must inform the CPU that it has completed a request or that data have arrived. It can raise an interrupt (Lesson 5), or the operating system can find out by polling.

Definition 11.7 (Polling). The technique in which the operating system reads the status register of a device, at times of its own choosing, to find out whether the device requires attention, instead of waiting for the device to raise an interrupt, is called *polling*.

Table 11.1 compares the two. An interrupt reaches the CPU as soon as the device generates it, but every interrupt incurs the handling overhead: executing the interrupt handler, setting and

resetting interrupt masks, and indirect effects such as the eviction of the interrupted thread's data from the CPU caches. Polling happens when it is convenient for the operating system, but if it happens rarely, events wait until the next poll; if it happens often, most polls find nothing and waste CPU time. Which method is preferable depends on the type of device and on the objectives, such as how frequent the events are and how quickly they must be handled.[1]

[1] Linux network drivers switch between the two with NAPI: on the first packet the driver disables the interface's receive interrupts and the kernel polls the device until its backlog is drained, then re-enables them. A lightly loaded interface is thus interrupt-driven and a busy one polled.

Table 11.1: Interrupts and polling.

	Interrupt	Polling
Initiated by	the device	the operating system
Event noticed	as soon as it is generated	at the next poll
Cost	handling overhead per interrupt, cache pollution	one status read per poll, including polls that find nothing
Drawback	overhead when events are frequent	delay if polls are rare, CPU overhead if they are frequent

Example 11.8. Suppose that taking an interrupt costs 4 µs of CPU time beyond the processing of the event itself, and that one poll, which collects all events pending since the previous poll, costs 0.5 µs. A device that reports 50 events per second costs 200 µs of CPU time per second with interrupts; polling every 1 ms would cost 500 µs per second and delay each event by up to 1 ms. A network interface that receives 100 000 packets per second, however, costs 400 ms per second—40 % of a CPU—with one interrupt per packet, while polling every 1 ms still costs only 500 µs per second.

11.6 Device access: programmed I/O and DMA

Source P3L5, device access PIO, device access DMA; OSTEP ch. 36.

The CPU and a device can exchange data in two ways, which differ in whether the data pass through the CPU. Both ways use the device registers of Section 11.1 to issue commands.

Definition 11.9 (Programmed I/O). With *programmed I/O* (PIO) the CPU performs every step of the interaction through the device registers: it writes the command register to request an operation, and it moves the data by writing or reading the data register, one register-sized piece at a time.

PIO requires no hardware support beyond the registers. To send a network packet with PIO, the CPU writes a command requesting transmission into the command register, copies the first part of the packet into the data register, and repeats the copy until the whole packet has been transferred (Figure 11.3a). The number of CPU accesses grows with the size of the data.

Definition 11.10 (Direct memory access). With *direct memory access* (DMA) the CPU still issues commands through the command register, but the data do not pass through the CPU. The CPU places the data in a buffer in main memory and passes the address and size of the buffer to the device; a DMA controller then copies the data between memory and the device on its own.

To send the packet with DMA, the CPU writes the command, places the packet in a memory buffer, and configures the DMA controller with the buffer's address and size (Figure 11.3b). The number of CPU accesses no longer depends on the size of the data.

Example 11.11. A network interface with an 8-byte data register sends a 9000-byte frame. With PIO the CPU performs 1 access for the command and $9000/8 = 1125$ accesses for the data, 1126 accesses in total. With DMA it performs 2: one for the command and one for the DMA configuration.

Tip DMA pays off when a transfer spans many register widths; for transfers of a few bytes PIO is cheaper.

A raw physical address also gives the device the run of memory; platforms therefore interpose an IOMMU (Intel VT-d, AMD-Vi), and the driver maps the buffer for DMA instead of handing over the address.

DMA needs fewer steps, but configuring DMA is a more complex operation than a single register write. For small transfers, a few register writes with PIO can therefore cost less than one DMA configuration: PIO is cheaper whenever the data accesses it needs cost less than the configuration of DMA. A further requirement arises because the device accesses the buffer by its physical address: the buffer must remain in physical memory until the transfer completes. Its pages are therefore pinned (Lesson 8), that is, excluded from swapping; otherwise the device could read or overwrite whatever page had replaced the buffer.

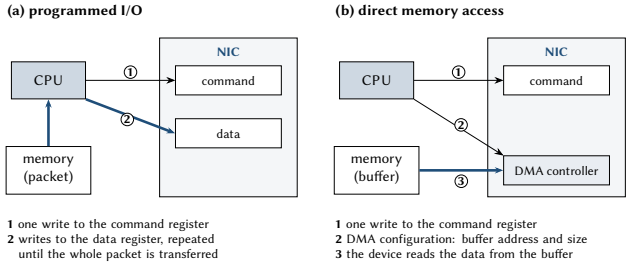


Figure 11.3: Sending a packet. (a) With programmed I/O the CPU copies the packet into the data register piece by piece. (b) With DMA the CPU only writes the command and configures DMA; the device reads the packet from memory.

Key point
The CPU reaches device registers through memory-mapped I/O or I/O ports; a device reaches the CPU through interrupts, or the operating system polls it. Data move either through the CPU register by register (PIO), or directly between a memory buffer and the device (DMA), which saves CPU accesses at the price of a more complex configuration and pinned memory.

11.7 Typical device access

An application does not normally interact with a device by itself; its request passes through several layers of the operating system. The typical path of a device access is as follows (Figure 11.4a):

1. The process performs a system call that requests an operation, for example to send data over a socket or to read from a file.
2. The kernel executes the in-kernel stack associated with the device: the TCP/IP stack forms a packet, or the file system determines which block holds the requested data.
3. The kernel invokes the appropriate device driver, here the network driver or the block device driver.
4. The driver configures the request on the device, with PIO register writes or a DMA configuration.
5. The device performs the request: it transmits the packet or reads the blocks from the disk.

Results and completion notifications travel back the same way: the device raises an interrupt or is polled, the driver passes the data up the stack, and the system call returns to the process.

Source P3L5, typical device access, OS bypass; OSTEP ch. 36.

11.7.1 Operating system bypass

Going through the kernel is not strictly necessary.

Definition 11.12 (Operating system bypass). A method of device access in which the registers and memory of a device are made directly accessible to a user process is called *operating system bypass*. The operating system configures the access and then stays out of the way, so that the process performs device operations without executing kernel code (Figure 11.4b).

The code that would otherwise run in the kernel driver is then provided by a *user-level driver*, a library linked into the application.[2] The operating system retains coarse-grained control: it can, for example, enable or disable the device, or grant additional processes access to it. Bypass relies on features of the device:

- The device must have sufficient registers, so that some of them can be mapped into a user process while those needed for control remain accessible only to the kernel.
- If several processes use the device, the device must be able to *demultiplex*, that is, determine to which process data belong. In a typical device access the kernel does this: its network stack, for example, inspects the headers of an arriving packet to find the receiving process. With bypass the device itself must do so.

[2] Kernel bypass is routine in high-performance networking: InfiniBand and RoCE adapters expose queue pairs to a process through the verbs API, and DPDK drives Ethernet adapters entirely from user space, the kernel only setting them up. Lesson 15 uses such adapters for remote direct memory access.

PCIe devices can do this with SR-IOV; see Lesson 12.

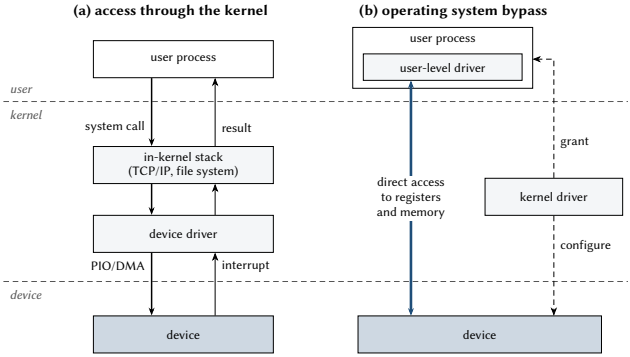


Figure 11.4: (a) A typical device access passes through the system call, the in-kernel stack and the device driver. (b) With operating system bypass the process accesses the device through a user-level driver; the kernel only configures and controls the access.

11.8 Synchronous and asynchronous access

Source P3L5, sync vs. async access; Silberschatz et al. ch. 12.

While a device carries out a request, the thread that issued it can either wait or continue.

Definition 11.13 (Synchronous I/O). With *synchronous I/O* a thread that issues an I/O request is blocked until the request completes. The operating system places the thread on the wait queue associated with the device, the I/O queue of Lesson 2, and makes it runnable again when the device reports completion.

With *asynchronous I/O* (Lesson 6) the call returns at once and the thread continues executing (Figure 11.5). It learns the outcome later in one of two ways: either the thread itself checks whether the operation has completed and then retrieves the results, or the operating system notifies it that the operation has completed and that the results are available. When the kernel does not support asynchronous access to a device, it can be emulated in user space with helper threads or processes that issue the blocking calls on behalf of the main thread, as the Flash web server of Lesson 6 does.

glibc implements POSIX `aio_read` with exactly this helper-thread emulation; `io_uring` is the Linux kernel interface that makes the emulation unnecessary (`io_uring(7)`).

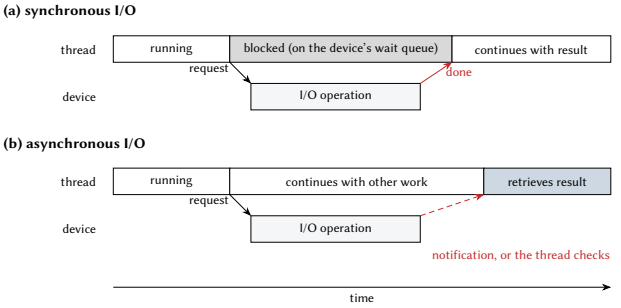


Figure 11.5: (a) With synchronous I/O the thread blocks until the device completes the operation. (b) With asynchronous I/O the thread continues and retrieves the result after being notified or after checking.

Key point
A device access normally passes from the system call through the in-kernel stack and the driver to the device, and the result returns the same way. Operating system bypass removes the kernel from this path when the device can protect and demultiplex its own resources. Independently of the path, the requesting thread either blocks (synchronous I/O) or continues and collects the result later (asynchronous I/O).

11.9 Block device stack

Source P3L5, block device stack; OSTEP ch. 36, 39.

Processes rarely address a block device by block numbers. They use files, the logical storage units, through the POSIX API with operations such as `open`, `read`, `write` and `close`. Beneath the API the kernel is organized in layers, each of which uses the interface of the layer below it (Figure 11.6).

Definition 11.14 (File system). A *file system* is the kernel component that organizes the blocks of a storage device into files and directories: it determines where the data of each file reside, and how a file is found and accessed.

The operating system specifies the interface that every file system implements. Below the file systems lies a layer that hides the differences between block devices.

Definition 11.15 (Generic block layer). The *generic block layer* is the kernel layer that provides a single, operating-system-standardized interface for reading and writing blocks on any block device, and that translates each request into the interface of the driver of the particular device.

The same file system therefore works on every kind of block device: the generic block layer adapts each request to the driver of the device, and the driver speaks the protocol of its device, such as SCSI or ATA, towards the hardware.

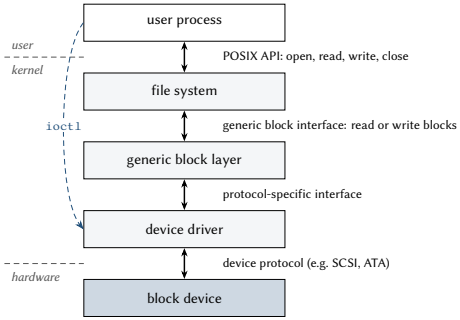


Figure 11.6: The block device stack. Each layer uses the interface shown below it; `ioctl` carries device-specific requests to the driver.

Some operations on a device fit neither `read` nor `write`. Linux and other UNIX-like systems provide the system call `ioctl` ("I/O control") through which a process accesses and manipulates device-specific information. It takes a file descriptor (Section 11.11) of an open device file, a request code defined for the device class or driver, and, depending on the request, a pointer to request-specific data.

Caution `ioctl` request codes and argument layouts are driver-specific, so portable programs avoid it (`ioctl(2)`).

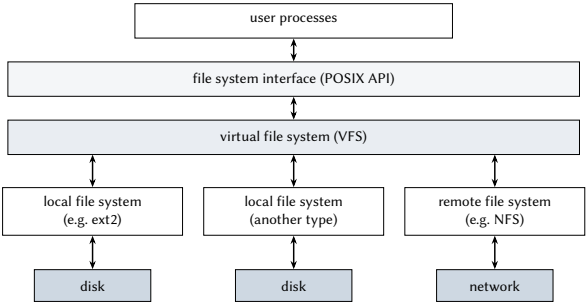


Figure 11.7: The virtual file system maps the operations of the file system interface to the concrete local and remote file systems.

Example 11.16. A program opens the tray of the first optical drive:

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/cdrom.h>

int main(void)
{
  int fd = open("/dev/sr0", O_RDONLY | O_NONBLOCK);
  if (fd >= 0) {
    ioctl(fd, CDROMEJECT);  /* no read/write equivalent */
    close(fd);
  }
  return 0;
}
```

The flag `O_NONBLOCK` allows the device file to be opened even when no disc is inserted.

11.10 The virtual file system

Source P3L5, virtual file system; OSTEP ch. 39; Silberschatz et al. ch. 15; Kerrisk ch. 14.

A single file system on a single local device is not enough for most systems. The files may reside on more than one device; different devices may be served best by different file system implementations; and files may not reside on a local device at all, but on a remote server that is accessed over the network (Lesson 14).

Definition 11.17 (Virtual file system). A *virtual file system* (VFS) is a kernel layer that presents one file system interface to applications, and that maps each operation to the concrete file system in which the file resides.

The VFS hides all details of the underlying file systems from applications (Figure 11.7). Applications use the same API regardless of where a file resides, and the VFS forwards each operation to the file system concerned, such as a local ext2 file system on a disk or a remote file system such as NFS.

11.11 VFS abstractions

Source P3L5, virtual file system abstractions, VFS on disk; OSTEP ch. 39–40; Silberschatz et al. ch. 13, 15; Kerrisk ch. 14.

Every concrete file system implements a small set of abstractions defined by the VFS. The *file* is the element on which the VFS operates. A process refers to an open file through a file descriptor (Figure 11.8).

Definition 11.18 (File descriptor). A *file descriptor* is the operating system's representation of an open file for a process: a small integer that the process obtains from `open` and passes to the operations on the file, such as `read`, `write`, `sendfile`, locking and `close`.

Descriptors 0, 1 and 2 are the standard input, output and error of a process; the usual default soft limit on open files is 1024 (`ulimit -n`; `RLIMIT_NOFILE` in `getrlimit(2)`).

The properties of the file itself, and the locations of its data blocks, are recorded in its inode.

Definition 11.19 (Inode). An *inode* is the persistent representation of a file: an index that

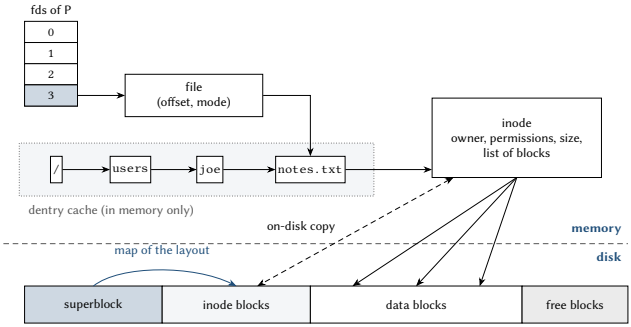


Figure 11.8: VFS abstractions. File descriptor 3 of process P refers to an open file, whose dentry leads to the inode; the dentries of the path exist only in memory. On disk, the superblock, with the structures it locates, maps the blocks that hold inodes, data and free space.

lists all the data blocks of the file, together with metadata such as the device, the permissions and the size of the file.

Caution Several names can share one inode (a hard link); removing a name only decrements the link count, and the file is freed when the count reaches zero and no descriptor is open.

A directory is a file, too; its data consist of information about the files it contains and their inodes. File names therefore appear in directories, not in inodes. The components of a path are represented by further objects.

Definition 11.20 (Directory entry). A *directory entry* (dentry) is the VFS object that corresponds to a single component of a path.

Looking up the path `/users/joe` creates a dentry for each of `/`, `users` and `joe`. The VFS keeps dentries in the *dentry cache*, so that a later lookup of a path with the same prefix does not have to read the corresponding directories again. Dentries are soft state: they exist only in memory and have no on-disk representation.

Definition 11.21 (Superblock). A *superblock* contains file-system-specific information about the layout of a file system.

Example 11.22. A process opens `/users/joe/notes.txt`. Starting from the dentry of `/`, the VFS reads each directory on the path to find the next component and its inode, creating the dentries of `users`, `joe` and `notes.txt` on the way. If the process then opens `/users/joe/todo.txt`, the dentries of `/`, `users` and `joe` are found in the dentry cache, and only the last component must be looked up in the directory `joe`.

11.11.1 VFS on disk

On disk, the abstractions are placed as follows (Figure 11.8). The data of a file are kept in data blocks. The inode, which tracks the blocks of the file, also resides on disk, in some block. The superblock, together with the structures it locates, such as the bitmaps of ext2 (Section 11.12), provides the overall map of the disk blocks: which blocks hold inodes, which hold data, and which are free.

11.12 The second extended file system

Linux has long used a family of file systems whose on-disk structures correspond closely to the abstractions of the VFS. The second extended file system, *ext2*, was for many years the default file system of Linux and is the basis of its successors ext3 and ext4. An ext2 partition begins with 1024 B reserved for booting—all of block 0 when blocks are 1 KiB—and the rest is divided into groups of blocks (Figure 11.9).

Source P3L5, ext2; OSTEP ch. 40–41; Silberschatz et al. ch. 14.

Definition 11.23 (Block group). A *block group* is one of the equal-sized (except possibly the last) groups of consecutive blocks into which ext2 divides a disk partition. Each block group holds its own inodes and data blocks together with the metadata that describe them.

Each block group contains the following structures; the first two are present only in some

A group is not a region of the disk geometry: it holds as many blocks as one block of bitmap can track—with 4 KiB blocks, $8 \times 4096 = 32\,768$ blocks, that is 128 MiB.

groups, as explained below:

a copy of the superblock recording file-system-wide information: the numbers of inodes and of disk blocks, the numbers of free inodes and blocks, and the size of a block;

a copy of the group descriptor table with one descriptor per group, recording the locations of the bitmaps and the inode table of that group and its numbers of free blocks, free inodes and directories;

a block bitmap and an inode bitmap tracking which blocks and which inodes of the group are free;

the inode table holding a consecutive range of the inodes of the file system, which are numbered from 1 to the maximum number, one per file;

the data blocks holding the file data.

The superblock and the group descriptor table describe the file system as a whole; the copies in block group 0 are the ones normally used, and the other copies serve as backups. With the `sparse_super` feature, which `mke2fs` enables by default, only groups 0 and 1 and the groups whose number is a power of 3, 5 or 7 hold such copies (`ext4(5)`); the other groups begin directly with their bitmaps. Keeping the inodes and the data blocks of a file in the same group places related blocks close together on the disk and reduces the distance the disk head must travel between them.

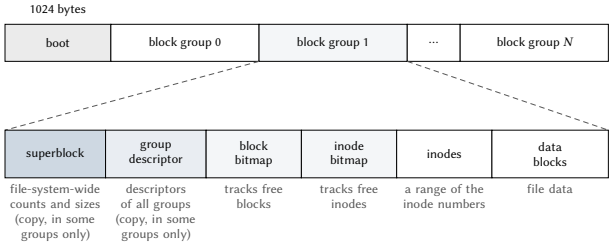


Figure 11.9: Layout of an ext2 partition. After the space reserved for booting, the partition is divided into block groups, each with its own bitmaps, inodes and data blocks. Groups 0 and 1 and the groups numbered by a power of 3, 5 or 7 also hold a copy of the superblock and of the group descriptor table (`sparse_super`).

11.13 Inodes

Source P3L5, inodes, inodes with indirect pointers; OSTEP ch. 40; Silberschatz et al. ch. 14; Kerrisk ch. 14.

A file is identified by its inode, and the inode is the index of all disk blocks of the file: it lists the blocks, in order, together with the metadata. In its simplest form the inode contains the block numbers of all blocks of the file. Sequential and random access are then equally easy: with block size B, the byte at offset o lies in the block whose number is entry $\lfloor o/B \rfloor$ of the list. The size of a file, however, is limited by the number of entries that fit into the inode.

Example 11.24. An inode of 512 bytes that uses 64 bytes for metadata has room for $448/4 = 112$ block numbers of 4 bytes each. With 4 KiB blocks, no file can be larger than $112 \times 4\,\text{KiB} = 448\,\text{KiB}$.

11.13.1 Indirect pointers

To support large files with small inodes, an inode holds pointers of several kinds (Figure 11.10).

Direct pointers A *direct pointer* points to a data block.

Indirect pointers An *indirect pointer* points to a block that contains pointers to data blocks.

Double indirect pointers A *double indirect pointer* points to a block of pointers to blocks of pointers to data blocks.

A *triple indirect pointer* adds a further level. The inodes of ext2 and of many UNIX file systems contain 12 direct pointers and one pointer of each indirect kind.

Real ext2 numbers: a 256 B inode (128 B on very small file systems), one per 16 KiB of partition, and 4-byte block numbers (`mke2fs.conf(5)`, `mke2fs(8)`).

With block size B and pointer size p, a block holds $P = B/p$ pointers. An inode with n_d direct pointers and one pointer of each of the three indirect kinds can address files of up to

$$S_{\max} = \left(n_d + P + P^2 + P^3\right) \times B. \tag{11.1}$$

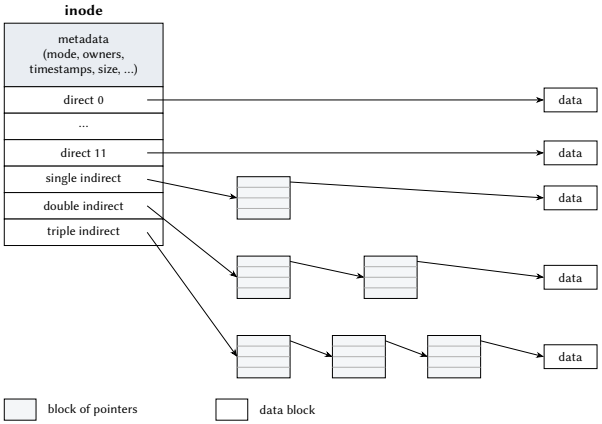


Figure 11.10: An inode with 12 direct pointers and single, double and triple indirect pointers. Each level of indirection multiplies the number of reachable data blocks by the number of pointers per block.

Example 11.25. With 4 KiB blocks and the 4-byte block numbers of ext2, a block holds $P = 4096/4 = 1024$ pointers. The 12 direct pointers address $12 \times 4\,\text{KiB} = 48\,\text{KiB}$; the indirect pointer $1024 \times 4\,\text{KiB} = 4\,\text{MiB}$; the double indirect pointer $1024^2 \times 4\,\text{KiB} = 4\,\text{GiB}$; and the triple indirect pointer $1024^3 \times 4\,\text{KiB} = 4\,\text{TiB}$. Equation (11.1) gives about 4.004 TiB; with 1 KiB blocks, $P = 256$ and the bound is about 16.06 GiB. With 4 KiB blocks, real ext2 files stop at 2 TiB, however, because the inode counts a file's blocks in
a 32-bit field of 512-byte sectors ($2^{32} \times 512\,\text{B}$).

The benefit of indirect pointers is that a small inode supports very large files. The cost is slower file access, because every level of indirection adds a disk access to reach a data block. Reading a block through a direct pointer requires two disk accesses, one for the inode and one for the data block; reading a block through a double indirect pointer requires up to four.

Example 11.26. With the parameters of the previous example, a process reads the byte at offset 600 MiB of a file. It lies in block $600 \times 1024/4 = 153\,600$ of the file. Blocks 0–11 are reached directly, blocks 12–1035 through the indirect pointer, and blocks 1036–1 049 611 through the double indirect pointer, so the read requires the inode, two blocks of pointers and the data block: four disk accesses if none of them is cached.

Key point
The VFS operates on files through file descriptors, dentries, inodes and superblocks. On disk, a file is its inode plus the data blocks the inode indexes; direct and indirect pointers let a fixed-size inode index very large files at the price of extra disk accesses per level of indirection.

Tip To see which pointer reaches block i of a file, compare i with the running sums n_d, $n_d + P$ and $n_d + P + P^2$: the first one it falls below names the pointer class.

11.14 Disk access optimizations

Accessing a file costs disk accesses, and on a hard disk every access that is not sequential costs time to move the head. File systems use four techniques to reduce the overhead of file access (Table 11.2).

Source P3L5, disk access optimizations; OSTEP ch. 37, 40, 42; Silberschatz et al. ch. 11, 14; Kerrisk ch. 13–14.

The gap these techniques attack: a 7200 rpm disk reads 150 MB s^{-1} to 280 MB s^{-1} sequentially (vendor datasheets), but random 4 KiB accesses, at roughly 100 per second, amount to a few hundred kilobytes per second (OSTEP ch. 37).

The cache has no fixed size: it keeps whatever memory the replacement policy of Lesson 8 leaves to it.

11.14.1 Caching and buffering

Definition 11.27 (Buffer cache). A *buffer cache* is a cache of file data in main memory. Reads are served from the cache when possible and writes update the cache; modified data are written to disk periodically, or when a process requests it with `fsync`.

The kernel keeps file data in the *page cache*, so repeated reads and delayed writes avoid the

Table 11.2: Techniques that reduce file access overhead.

Technique	Effect
Caching and buffering	fewer disk accesses
I/O scheduling	less head movement; more sequential, less random access
Prefetching	more cache hits, by exploiting locality
Journaling	less random access for writes, without keeping updates only in memory

device; Linux unifies the buffer cache and the page cache.

A process that must make an update durable follows the write with `fsync`:

```
write(fd, rec, sizeof *rec);  /* updates the cache */
fsync(fd);                    /* flushes it to disk */
```

Caution A successful `write` does not mean the data are on disk; a crash loses data still in the cache.

`fsync` returns only after the data of the file have been written to the device.

11.14.2 I/O scheduling

Ordering the pending disk requests so as to reduce the movement of the disk head is called *I/O scheduling*: requests are served in an order that maximizes sequential access and minimizes random access. Sorting the requests by block address and serving them in one sweep of the head is the *elevator algorithm*, which the Linux I/O schedulers refine.

Example 11.28. The head is positioned at block 10, and writes to blocks 90, 12, 91, 13 and 50 arrive in this order. Assuming that block numbers reflect positions on the disk, serving the requests in arrival order moves the head across $80 + 78 + 79 + 78 + 37 = 352$ blocks. Serving them in the order 12, 13, 50, 90, 91 moves it across only $2 + 1 + 37 + 40 + 1 = 81$ blocks, and the pairs 12, 13 and 90, 91 become sequential writes.

The elevator algorithm is also called SCAN. Linux ships `mq-deadline`, BFQ and Kyber, and usually runs NVMe with none.

11.14.3 Prefetching

Reading more blocks than a request asks for is called *prefetching* or read-ahead: when a block is read, the blocks likely to be needed next are read with it. Prefetching exploits locality—a file that is being read sequentially will soon need the following blocks—and increases the hit rate of the buffer cache. If a process reads block 40 of a file and the file system also reads blocks 41–43 in the same disk operation, the next three reads are served from memory without further head movement.

Linux reads 128 KiB ahead by default (`VM_READAHEAD_PAGES`); the window is per block device, in `/sys/block/*/queue/read_ahead_kb` (sysfs-block documentation).

11.14.4 Journaling

Writing every update to its proper location immediately causes random access, whereas keeping updates only in the buffer cache risks losing them. The technique of *journaling*, or logging, avoids both problems. The file system records each update in a log, the *journal*, which it writes sequentially to a dedicated area of the disk, and it periodically applies the logged updates to their proper disk locations (Figure 11.11). Because the journal is on disk, an update recorded in it survives a crash and can be applied afterwards. The ext3 and ext4 file systems add such a journal to the design of ext2. Their journaling layer (JBD in ext3, JBD2 in ext4) logs the complete new image of every modified block, not a description of the change. In the default mode, `data=ordered`, only metadata blocks are logged, and they are committed only after the file data have been written to their final locations; the mode `data=journal` logs the data blocks as well.

Caution Journaling saves no writes—every logged block is written twice—but replaces scattered writes with one sequential write plus a later, batched update.

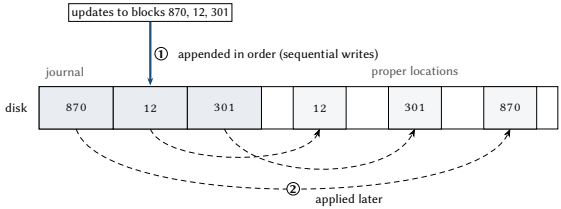


Figure 11.11: Journaling. Updates to scattered blocks are first appended in order to the journal and later applied to their proper locations.

Key point — Lesson summary

An I/O device presents command, data and status registers, reached through interconnects such as PCIe and operated by a device driver behind an interface standardized by the operating system. Devices are block, character or network devices and are represented as device files. The CPU addresses registers through memory-mapped I/O or I/O ports, learns of device events through interrupts or polling, and transfers data with programmed I/O or DMA. A device access normally passes through the kernel, but operating system bypass can remove the kernel from the data path; the requesting thread either blocks or continues asynchronously. Files reach block devices through the file system, the generic block layer and the driver. The virtual file system unifies file systems through file descriptors, dentries, inodes and superblocks; ext2 organizes a partition into block groups, and its inodes use direct and indirect pointers. Caching, I/O scheduling, prefetching and journaling reduce the cost of disk access.

Lesson 12

Virtualization

Lecture	P3L6
Textbook	OSTEP [1] app. B; Silberschatz et al. [2] ch. 18
Keywords	virtualization, virtual machine, hypervisor, bare-metal hypervisor, hosted hypervisor, protection ring, trap-and-emulate, binary translation, paravirtualization, hypercall, shadow page table, split-device driver model, hardware-assisted virtualization, extended page table

The preceding lessons showed how an operating system manages the CPUs, the memory and the devices of a machine on behalf of the processes that run on it. Virtualization moves this role one level down: a thin layer of software manages the hardware on behalf of several complete operating systems, each of which behaves as if it owned the machine. This lesson defines virtualization, explains why it has become widespread, and compares the two ways in which virtualization software is organized. It then examines how the CPU, memory and devices are virtualized, first with software techniques alone and then with the support that current processors provide. The protection boundary of Lesson 1, the page tables of Lesson 8 and the device drivers of Lesson 11 reappear throughout, now with one more layer of software beneath the operating system.

12.1 What is virtualization?

Source P3L6, what is virtualization, defining virtualization; Popek and Goldberg [32]; OSTEP app. B; Silberschatz et al. ch. 18.

Virtualization is an old idea. It originated at IBM in the 1960s, when computing was done on a small number of large and expensive mainframes, and the owner of such a machine wanted to use it for workloads that had been developed for different operating systems.

Definition 12.1 (Virtualization). Allowing several operating systems, each together with its applications, to execute concurrently on the same physical machine is called *virtualization*.

None of these operating systems is in control of the hardware any more. Each runs in an environment of its own and is given the illusion that it owns the hardware resources—CPUs, memory and devices—although the resources it sees are *virtual resources* that stand in for a share of the physical ones (Figure 12.1).

Definition 12.2 (Virtual machine). A *virtual machine* (VM) is an operating system together with its applications and the virtual resources on which they run.

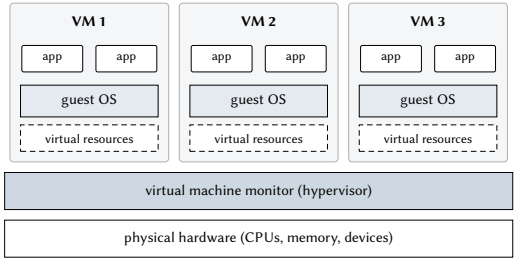


Figure 12.1: Virtualization. Each VM consists of a guest OS, its applications and the virtual resources presented to it; the virtual machine monitor manages the physical hardware beneath all VMs.

Definition 12.3 (Virtual machine monitor). The *virtual machine monitor* (VMM), also called the *hypervisor*, is the virtualization layer beneath the VMs. It manages the physical hardware, provides the virtual resources to the VMs, coordinates their access to the physical resources, and isolates them from one another.

The operating system that runs inside a VM is called a *guest operating system* (guest OS), and a VM as a whole is also called a *guest domain*.

Popek and Goldberg [32] gave the classic definition: a virtual machine is "an efficient, isolated duplicate of the real machine". The VMM that supports it must provide three properties.

Popek and Goldberg name the three properties equivalence, efficiency and resource control.

Fidelity Programs, operating systems included, run in an environment essentially identical to the original machine and behave as they would on it, apart from timing and the amount of available resources. This *fidelity* is what lets an unmodified operating system run in a VM; paravirtualization (Section 12.8) deliberately relaxes it.

Performance Programs in a VM show at worst a minor decrease in speed. Most instructions must therefore execute directly on the hardware rather than be interpreted by the VMM.

Safety and isolation The VMM is in complete control of the system resources. A VM cannot access resources that the VMM has not allocated to it, and it cannot interfere with other VMs or with the VMM itself.

Note 12.4. The definition excludes technologies that are also called virtual machines. A high-level language virtual machine executes programs compiled for an abstract instruction set, not for the real machine, so it is not a duplicate of the hardware beneath it. An emulator that implements a different hardware platform in software duplicates that platform, not the real machine, and it executes every instruction in software, by interpretation or translation, none of them directly on the processor, so it does not meet the performance requirement.

12.2 Benefits of virtualization

Running several operating systems on one physical machine has a number of benefits.

Source P3L6, benefits of virtualization; Rosenblum and Garfinkel [33]; Silberschatz et al. ch. 18.

Consolidation The workloads of several physical machines, each with its own operating system and applications, can run as VMs on fewer machines. This *consolidation* decreases cost—fewer machines to buy, house, power and cool—and improves manageability, because fewer machines need administration.

Migration A VM is decoupled from the physical machine, so its complete state, operating system and applications included, can be moved from one machine to another, for example before a machine is taken down for maintenance. Migration improves availability and reliability.

Security Bugs and malicious behaviour are contained within the VM in which they occur and do not affect the other VMs.

Debugging New operating systems and new operating system features can be brought up and tested quickly in a VM, without a dedicated machine; a crash takes down only that VM.

Support for legacy operating systems An operating system that no longer runs on current hardware, together with the applications that depend on it, can continue to run in a VM next to current systems.

Although IBM mainframes supported virtualization from the 1960s on, it remained confined to them until the late 1990s [33].[1] Mainframes were expensive and few. Once commodity hardware had become cheap and its operating systems supported multitasking, it was simpler to buy another machine for another application than to share one machine among several operating systems, and interest in VMMs faded. This practice eventually created the problems that revived them: datacenters filled with machines that each ran a single application and were underutilized, so that the space, power and administration they consumed grew out of proportion to the work they did. Consolidating the applications onto fewer machines addresses these costs at once, but it requires virtualizing commodity x86 hardware, which, as Section 12.6 shows, had not been designed for it.

[1] The first such system was CP-40 with CMS, run by IBM's Cambridge Scientific Center from January 1967; CP-67 followed on the System/360 Model 67, whose hardware translated addresses. The line continued as VM/370, announced in 1972, and survives as z/VM.

Example 12.5. A company runs 24 servers, each dedicated to one application and averaging 8 % CPU utilization. Their combined load corresponds to $24 \times 0.08 = 1.92$ fully used servers. If each physical machine is loaded to at most 70 %, leaving headroom for load peaks and for the overhead of virtualization, and memory and I/O capacity suffice, the 24 workloads fit as VMs on $\lceil 1.92/0.7 \rceil = 3$ machines, each at about 64 % utilization. The number of machines to power, cool and administer drops by a factor of eight.

Low utilization is the norm: Barroso and Hölzle report that datacenter servers spend most of their time between 10 % and 50 % CPU utilization [34].

12.3 Virtualization models

Virtualization software is organized in one of two ways, which differ in what runs directly on the hardware and, in particular, in who owns the device drivers.

Source P3L6, virtualization models: bare-metal, hosted; Barham et al. [35]; Kivity et al. [36]; Silberschatz et al. ch. 18.

12.3.1 Bare-metal hypervisors

Definition 12.6 (Bare-metal hypervisor). A *bare-metal hypervisor*, also called a hypervisor-based or *type 1* VMM, runs directly on the hardware. It manages all hardware resources and supports the execution of the VMs.

Such a hypervisor must also control the devices. Devices are diverse, and supporting all of them would put a large body of device drivers into the hypervisor. Bare-metal designs therefore often run a privileged *service VM* next to the guest VMs (Figure 12.2a). The service VM runs a standard operating system with the device drivers, has direct access to the devices, and performs configuration and management tasks, so that the hypervisor itself stays small.

Caution The type 1 and type 2 labels do not split cleanly. KVM is a module of Linux, so its hypervisor does run on the hardware; it counts as type 2 because a complete host OS remains around it.

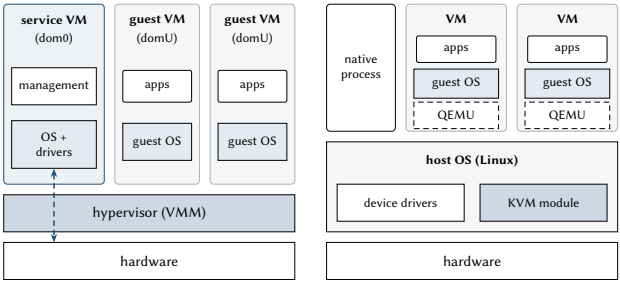


Figure 12.2: Virtualization models. (a) A bare-metal hypervisor with a privileged service VM that owns the device drivers, as in Xen. (b) A hosted hypervisor: a module of the host OS, as KVM in Linux; each VM is a host process in which QEMU emulates the devices.

The hypervisor *Xen* [35], available as open source and as the commercial XenServer (named Citrix Hypervisor from 2019 to 2023), follows this design. Its VMs are called *domains*. The privileged service VM is *dom0*, which runs Linux with the device drivers and the management tools; the guest VMs are unprivileged domains, *domU*, and the Xen hypervisor itself contains no drivers for I/O devices such as disks and network cards. VMware ESX, today ESXi, takes the other route and places the drivers in the hypervisor. This is practical because ESX targets servers from a limited number of vendors, who supply drivers for it. ESX is administered through remote management APIs, which replaced an earlier Linux-based control core.

In the first Xen version, the one described by Barham et al. [35], these drivers still resided in the hypervisor.

12.3.2 Hosted hypervisors

Definition 12.7 (Hosted hypervisor). A *hosted hypervisor*, or *type 2* VMM, runs on top of a conventional operating system, the *host operating system* (host OS), which owns all the hardware. A special VMM module in the host OS provides the hardware interfaces to the VMs and handles the switches between VMs.

A hosted hypervisor reuses what the host OS already provides. Device drivers, scheduling, memory management and other services need not be re-implemented, and ordinary applications continue to run natively on the host next to the VMs (Figure 12.2b).

KVM, the *Kernel-based Virtual Machine* [36], is the hosted hypervisor of Linux. A loadable kernel module turns Linux into a hypervisor. Each VM is an ordinary Linux process, whose virtual CPUs are threads of that process, so the Linux scheduler and memory manager serve VMs as they serve other processes. The devices of a VM are emulated within its process by *QEMU*, a hardware emulator running at user level, while the instructions of the guest execute directly on the CPU with the hardware support described in Section 12.11. Because KVM is part of Linux, it benefits from the Linux open-source community: new features, bug fixes and drivers for new devices become available to it without separate effort.

Tip On a Linux host `ps` lists each KVM guest as one `qemu-system-*` process; its vCPUs are threads of that process.

> **Key point**
> A bare-metal hypervisor runs directly on the hardware and typically leaves device drivers to a privileged service VM, such as dom0 in Xen. A hosted hypervisor is a module of a host OS, such as KVM in Linux, and reuses the drivers and services of that OS.

12.4 Hardware protection levels

A guest OS is written to run in kernel mode and to execute privileged instructions, yet under virtualization only the hypervisor may control the hardware. The two modes of Lesson 1 cannot separate three kinds of software—applications, guest OS and hypervisor—but commodity processors offer more than two privilege levels.

Source P3L6, hardware protection levels; Silberschatz et al. ch. 18.

Definition 12.8 (Protection ring). A *protection ring* is one of a hierarchy of privilege levels enforced by the processor; software in a ring may execute only the instructions and access only the resources permitted at its level. x86 processors provide four rings, from ring 0, the most privileged, to ring 3, the least privileged.

Without virtualization, the OS kernel runs in ring 0 and applications in ring 3, and rings 1 and 2 are unused (Figure 12.3a). With virtualization, the hypervisor takes ring 0, the guest OS moves to ring 1, and applications remain in ring 3 (Figure 12.3b). When the guest OS attempts a privileged operation from ring 1, the processor traps to the hypervisor in ring 0.

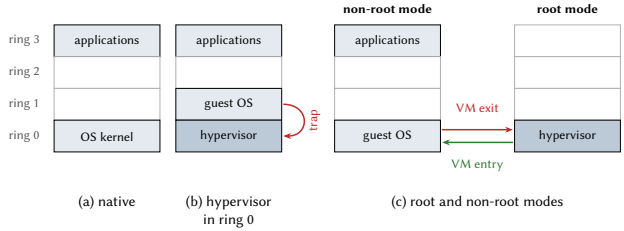


Figure 12.3: Use of the x86 protection rings. (a) Native execution. (b) Virtualization without hardware support: the guest OS is deprivileged to ring 1, and its privileged operations trap to the hypervisor. (c) Processors with virtualization support: the guest OS runs in ring 0 of non-root mode, and a VM exit transfers control to the hypervisor in root mode.

Processors with virtualization support, including current x86-64 processors, go further: they provide two modes of operation, each with its own four rings (Figure 12.3c).

Definition 12.9 (Root mode and non-root mode). In *root mode* the processor runs the hypervisor, in ring 0, with full control of the hardware. In *non-root mode* it runs the VMs, with the guest OS in ring 0 and applications in ring 3, and privileged operations are restricted. For a hosted hypervisor, the host OS and its processes also run in root mode.

The guest OS thus runs at the privilege level for which it was written. Operations that the hypervisor has chosen to intercept, selected per VM in the control structure (Section 12.11), cause a trap called a *VM exit*, which switches the processor to root mode and passes control to the hypervisor. When the hypervisor has handled the operation, a *VM entry* switches the processor back to non-root mode and resumes the VM.

12.5 Processor virtualization

To meet the performance requirement of Section 12.1, the hypervisor does not interpret the instructions of a VM but lets them execute directly on the hardware. The question is what must happen when such an instruction is privileged.

Source P3L6, processor virtualization; Popek and Goldberg [32]; OSTEP app. B.

Definition 12.10 (Trap-and-emulate). In *trap-and-emulate* virtualization, the instructions of a guest execute directly on the CPU, with the guest OS deprivileged. Non-privileged instructions run at hardware speed. A privileged instruction traps to the hypervisor, which determines what to do: if the operation is illegal for the VM, the hypervisor terminates the VM; if it is legal, the hypervisor emulates the behaviour that the guest OS expects from the hardware and resumes the guest.

Figure 12.4 summarizes the decisions. The emulation acts on the virtual state of the VM, not on the physical hardware. When a guest OS disables interrupts, for example, the hypervisor records that interrupts are disabled for this VM and holds back the interrupts destined for it, while physical interrupts remain enabled so that the hypervisor and the other VMs continue

to receive theirs. Every trap costs a transition into the hypervisor, so the overhead of trap-and-emulate grows with the number of privileged operations that the guest performs.

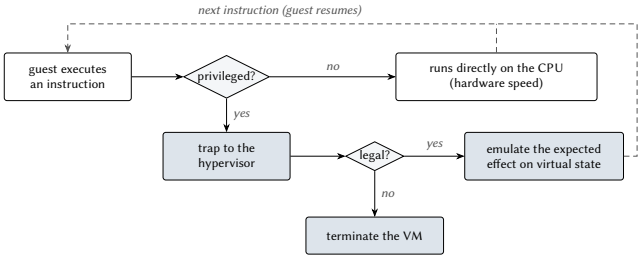


Figure 12.4: Trap-and-emulate. Only privileged instructions involve the hypervisor; legal ones are emulated on the virtual state of the VM.

Note 12.11. Trap-and-emulate is correct only if every instruction whose effect the hypervisor must control actually traps. Popek and Goldberg [32] call such instructions *sensitive*: they change or reveal the configuration of the machine, or behave differently depending on the privilege level. An architecture can be virtualized in this way if every sensitive instruction is privileged, that is, traps when executed outside the most privileged level.

12.6 x86 virtualization before hardware support

Source P3L6, x86 virtualization in the past; Robin and Irvine [37].

Before 2005, x86 processors had four rings but no root and non-root modes, so a hypervisor ran in ring 0 and deprivileged the guest OS to ring 1. The architecture did not satisfy the condition of trap-and-emulate: Robin and Irvine [37] identified 17 sensitive instructions that do not trap when executed in a less privileged ring. Executed in ring 1, they fail silently or reveal machine state that the hypervisor intends to hide, and trap-and-emulate alone cannot virtualize the processor correctly.

The best-known examples are PUSHF and POPF, which save the flags register on the stack and restore it. The flags register contains the interrupt-enable flag IF, and kernels commonly re-enable interrupts at the end of a section that ran with interrupts disabled by restoring the saved flags with POPF. In ring 0, POPF restores IF. In ring 1 the processor ignores the IF bit of the restored value, without a trap. The hypervisor is not invoked, so it does not learn that the guest wanted to change the interrupt state and cannot emulate the change; the guest OS receives no error and assumes that the change succeeded. Table 12.1 traces the consequence.

The hypervisor sets the I/O privilege level to 0, so POPF in ring 1 leaves IF unchanged, while CLI traps there.

Caution 32-bit software VMMs guard their own memory with segment limits. AMD64 long mode drops those checks, and paging does not distinguish ring 1 from ring 0, so 64-bit paravirtualized Xen guests run their kernel in ring 3, and VMware's binary translation needed either the segment limits that later AMD processors restored or VT-x.

Table 12.1: A guest kernel in ring 1 saves the flags, disables interrupts and later restores the flags. CLI traps and is emulated; POPF does not trap. *Virtual IF* is the interrupt flag that the hypervisor records for the VM; physical interrupts stay enabled throughout.

Step	Guest kernel executes	Traps	Virtual IF	Guest assumes
1	PUSHF (saves IF = 1)	no	1	enabled
2	CLI	yes	0	disabled
3	critical section	no	0	disabled
4	POPF (restores IF = 1)	no	0	enabled
5	waits for a device interrupt	no	0	enabled

From step 4 on, the hypervisor considers interrupts disabled for the VM and withholds its interrupts, while the guest OS waits for an interrupt that it believes to be enabled: the guest hangs. Two remedies were developed in software, one that leaves the guest OS unmodified and one that modifies it.

12.7 Binary translation

Source P3L6, binary translation; Bugnion et al. [38]; Silberschatz et al. ch. 18.

The first remedy rewrites the binary code of the guest so that it never executes the problematic instructions. The approach was pioneered by Mendel Rosenblum's group at Stanford University and commercialized by VMware [38]. Its goal is to run the guest OS without any modification.

Definition 12.12 (Full virtualization). Virtualization in which the guest operating system runs unmodified, as it would on the physical machine, is called *full virtualization*.

Definition 12.13 (Binary translation). In dynamic *binary translation* the hypervisor rewrites guest code at run time, just before it executes. Problematic instructions are translated into an alternative instruction sequence with the intended effect; all other instructions are kept as they are and run at hardware speed.

In VMware's design only guest kernel code is translated; applications in ring 3 run directly [38].

The translation proceeds block by block (Figure 12.5).

1. Inspect the block of code that is about to be executed, for example the instructions up to the next branch.
2. If the block contains problematic instructions, translate them into a sequence that emulates the desired behaviour, for example a call to an emulation routine of the hypervisor. The translation may even avoid a trap where the original instruction would have trapped.
3. Otherwise take the block over unchanged, with only its control transfers adjusted, so that it runs at hardware speed while execution stays under the control of the translator.

The resulting blocks are kept in a *translation cache*, so the cost of translating a block is paid once and amortized over its later executions.[2] The translation must be dynamic because which code executes, and where indirect branches lead, is known only at run time.

[2] Binary translation did not become obsolete when hardware support arrived: on the first VT-x processors the software VMM still won on most workloads, where exits were expensive and page tables had to be shadowed either way [39].

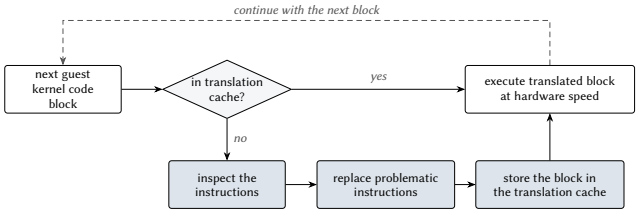


Figure 12.5: Dynamic binary translation with a translation cache. A block is inspected and translated only the first time it is about to run.

Example 12.14. A block of guest kernel code executes natively in 5 µs. Translating it takes 2 ms, and because one instruction is replaced by a call into the hypervisor, the translated block takes 6 µs. If the block runs 20 000 times, translating it once and caching the result costs $2\,\text{ms} + 20\,000 \times 6\,\mu\text{s} = 122\,\text{ms}$, against 100 ms natively, an overhead of 22 %. Translating it anew before every execution would cost $20\,000 \times (2\,\text{ms} + 6\,\mu\text{s}) \approx 40.1\,\text{s}$, about 400 times the native time.

12.8 Paravirtualization

The second remedy gives up on unmodified guests in exchange for performance. The guest OS is ported to the hypervisor, much as an operating system is ported to a new hardware platform.

Source P3L6, paravirtualization; Barham et al. [35].

Definition 12.15 (Paravirtualization). In *paravirtualization* the guest operating system is modified so that it knows it runs in a VM. Instead of executing privileged operations and relying on traps, it requests them from the hypervisor explicitly.

An explicit request of a guest OS to the hypervisor is a *hypercall*. It plays the role for a guest OS that a system call plays for an application (Lesson 1): the guest places the context information—the arguments—in a well-defined location, specifies which hypercall it requests, and traps into the hypervisor, which performs the operation and returns control to the guest. The hypervisor therefore does not have to infer the intended operation from individual trapped instructions, and one hypercall can carry several operations, as Section 12.9 shows. Only the guest OS is modified; applications run unchanged, because the interface that the guest OS offers them stays the same [35].

Xen, developed at the University of Cambridge and later commercialized by XenSource and Citrix, is the best-known paravirtualizing hypervisor. Table 12.4 in Section 12.11 compares paravirtualization with the other approaches to processor virtualization.

Key point
Trap-and-emulate requires every sensitive instruction to trap, which x86 processors before 2005 did not guarantee. Binary translation rewrites the problematic instructions at run time and keeps the guest OS unmodified; paravirtualization modifies the guest OS to issue hypercalls and gains performance.

12.9 Memory virtualization

12.9.1 Full virtualization

Source P3L6, memory virtualization: full, paravirtualized; OSTEP app. B; Barham et al. [35].

A guest OS expects what an operating system finds on a physical machine: physical memory that is contiguous and starts at address 0, which it manages with page tables as described in Lesson 8. The hypervisor must give every VM this illusion while dividing the memory of the machine among the VMs, and it should still let the hardware MMU and TLB translate addresses. Three kinds of addresses are therefore distinguished:

Virtual addresses are used by the software in a VM, as in Lesson 2.

Physical addresses are the addresses that the guest OS believes to be those of physical memory.

Machine addresses are the actual addresses of the memory of the machine. A *machine address* is known only to the hypervisor under full virtualization.

Page numbers make the same distinction between virtual page numbers, physical frame numbers and machine frame numbers. The guest OS maintains page tables that map virtual to physical addresses, and the hypervisor maintains a mapping from physical to machine addresses (Figure 12.6).

Tip Virtual → physical belongs to the guest, physical → machine to the hypervisor; the shadow page table is their composition.

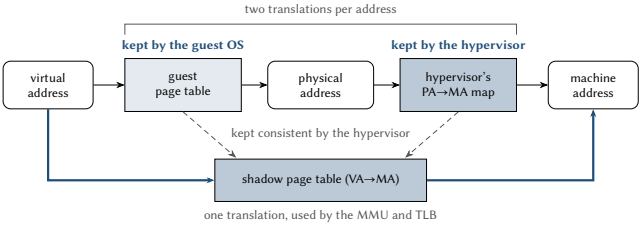


Figure 12.6: Memory virtualization. Translating through the guest page table and then through the hypervisor's mapping takes two steps; the shadow page table combines them into one mapping that the MMU uses directly. Hardware with extended page tables performs the two-step translation itself (Section 12.11).

A first option translates every address in these two steps. The MMU, however, walks only one page table, and the TLB caches one translation per address. Performing the second translation in software on every memory reference is far too expensive.

Definition 12.16 (Shadow page table). A *shadow page table* is a page table maintained by the hypervisor that maps the virtual addresses of a guest process directly to machine addresses. The hypervisor installs it in the MMU in place of the guest's page table, so that the hardware translates each address in one step and the TLB caches the result.

The guest OS continues to create and change its own page tables, which the hardware no longer uses, and the hypervisor must keep the shadow page tables consistent with them.

- It write-protects the memory that holds the guest page tables. When the guest OS adds or changes a mapping, the write traps; the hypervisor performs the write and updates the shadow page table with the corresponding machine address.
- When the guest OS switches between its processes, it loads another page table into the page table base register, a privileged operation that traps. The hypervisor then installs the shadow page table of the new process, or invalidates the current shadow entries and rebuilds them as they are used.

These traps make page table updates and context switches in a guest far more costly than on a physical machine.

Write-protecting a page in order to learn about a write is the mechanism of copy-on-write in Lesson 8, used here to observe the guest rather than to copy a page.

The page table base register of Lesson 8 is CR3 on x86, so under shadow paging every guest write to CR3 traps.

Example 12.17. A guest process uses virtual pages 2 and 3. The guest page table maps them

to physical frames 5 and 9, which the hypervisor has backed with machine frames 41 and 17, so the shadow page table maps page 2 to machine frame 41 and page 3 to machine frame 17. The guest OS now maps virtual page 4 to physical frame 12. Its write to the protected page table traps; the hypervisor finds that frame 12 is backed by machine frame 26, performs the write, adds the entry 4 → 26 to the shadow page table and resumes the guest. Table 12.2 shows the resulting mappings.

Table 12.2: Mappings of the guest process of the example. The last shadow entry is added by the hypervisor when the guest's page table update traps.

Virtual page	Guest page table: physical frame	Hypervisor: machine frame	Shadow page table: machine frame
2	5	41	41
3	9	17	17
4	12	26	26

12.9.2 Paravirtualization

With paravirtualization the guest OS knows that it runs in a VM, and memory virtualization becomes a cooperation between guest and hypervisor. The strict requirement that physical memory be contiguous and start at address 0 disappears. In Xen the guest OS knows which machine frames it has been given, and its page tables map virtual addresses directly to machine addresses [35]. The guest registers its page tables with the hypervisor explicitly, and from then on it has only read access to them. To change a page table, the guest issues a hypercall, and the hypervisor validates the change—for example, that the new mapping refers to a machine frame that belongs to this VM—before applying it. Because every hypercall costs a transition into the hypervisor, the guest *batches* page table updates so that many of them share one hypercall. Other optimizations follow the same idea of reducing transitions: Xen, for instance, occupies the top 64 MB of every guest address space, so that entering the hypervisor requires no change of page table and no TLB flush [35].

Caution The names differ between systems: what Xen and this lesson call physical and machine addresses, Intel calls guest-physical and host-physical, and VMware guest physical and host physical.

Example 12.18. A paravirtualized Linux guest on Xen changes three page table entries with a single hypercall. Each request gives the machine address of a page table entry and the new contents of that entry:

```
struct mmu_update req[3];
int done;

for (int i = 0; i < 3; i++) {
  /* machine address of the entry to change */
  req[i].ptr = pte_maddr[i] | MMU_NORMAL_PT_UPDATE;
  /* new entry: machine frame number and flags */
  req[i].val = (mfn[i] << PAGE_SHIFT) | flags;
}
/* one transition into Xen for all three updates */
HYPERVISOR_mmu_update(req, 3, &done, DOMID_SELF);
```

Xen checks every request before applying it and reports in done how many updates succeeded. Issued one by one, the same updates would enter the hypervisor three times.

On newer platforms, the hardware support of Section 12.11 eliminates or greatly reduces the overheads of memory virtualization.

12.10 Device virtualization

CPUs and memory show little diversity: the instruction set architecture standardizes their interface, so a few techniques suffice for all processors of a family. Devices are highly diverse, and there is no standard specification of their interface and behaviour; as Lesson 11 showed, each type of device needs its own driver. Three models for virtualizing devices have emerged. They differ in where the driver that controls the physical device resides and in how the device accesses of a VM reach it (Figure 12.7).

Source P3L6, device virtualization; passthrough, hypervisor-direct and split-device driver models; Silberschatz et al. ch. 18.

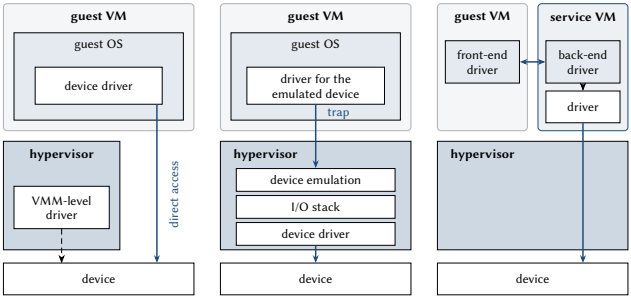


Figure 12.7: Device virtualization models. (a) The VM's own driver accesses the device directly. (b) The hypervisor intercepts device accesses and emulates them with its own I/O stack and driver. (c) A front-end driver in the guest passes requests to a back-end driver in the service VM, which drives the device.

12.10.1 Passthrough model

Definition 12.19 (Passthrough model). In the *passthrough model* a VMM-level driver configures the access permissions of a device so that a VM accesses the device directly, with its own driver, bypassing the hypervisor.

The VM obtains exclusive access to the device, and its device operations follow the same path as on a physical machine. This *VMM bypass*, the counterpart of the operating system bypass of Lesson 11, gives the best performance. The model has three drawbacks.

- Sharing the device is difficult: it must be reassigned from one VM to another.
- The machine must have exactly the type of device that the driver in the VM expects, since that driver operates the hardware directly.
- Migration is tricky, because part of the state of the VM resides in the device, and the destination machine must provide an identical device into which that state can be transferred.

SR-IOV (PCI-SIG, 2007 [40]) softens this: one card presents a physical function and a number of virtual functions—up to 65 535 by the specification, in practice up to a few hundred (256 on Intel's E810)—each of which can be assigned to a VM.

As with the operating system bypass of Lesson 11, a directly assigned network card runs at line rate, but the VM that uses it is hard to live-migrate.

12.10.2 Hypervisor-direct model

Definition 12.20 (Hypervisor-direct model). In the *hypervisor-direct model* the hypervisor intercepts every device access of a VM and emulates the device operation: it translates the operation into a generic I/O operation, passes it through the I/O stack that resides in the hypervisor, and invokes the hypervisor's own driver for the physical device.

The emulated devices are deliberately old and common: QEMU's PC machine presents an Intel e1000 network card and an IDE disk controller, for which every guest OS already has a driver.

The key benefit is that the VM is decoupled from the physical device. The VM sees an emulated device, which can be the same on every machine, independently of the physical device; the device can be shared among VMs; migration becomes simpler; and the hypervisor deals with the specifics of each device. VMware ESX uses this model. The costs are the latency that interception and emulation add to every device operation, and the complexity of the device driver ecosystem inside the hypervisor, which must contain, and remain robust with, drivers for all supported devices.

12.10.3 Split-device driver model

Definition 12.21 (Split-device driver model). In the *split-device driver model* device access is controlled by two cooperating drivers: a *front-end driver* in the guest VM, which presents the device interface to the guest OS, and a *back-end driver* in the service VM or the host OS, which accesses the physical device through the regular driver.

Linux's virtio (Russell, 2008 [41]; an OASIS specification since 2016, virtio 1.0) is the common front-end/back-end interface; its *virtqueues* are the shared memory of Lesson 9.

The front-end driver does not manipulate device registers that would have to be trapped and emulated. It passes complete requests, such as a packet to send, to the back-end driver, in Xen through descriptor rings in memory shared between the two VMs. This eliminates the emulation overhead of the hypervisor-direct model, and because all requests for a device pass through the back-end driver, the service VM can manage how the device is shared among the

VMs. The price is a modified guest: its drivers must be replaced by front-end drivers written for the hypervisor, so the model applies to paravirtualized guests and to guests in which such drivers have been installed. Table 12.3 summarizes the three models.

Table 12.3: Device virtualization models compared.

Model	Physical device driven by	Driver in the guest	Sharing and migration	Cost per device operation
Passthrough	guest VM	native driver	difficult	none beyond native
Hypervisor-direct	hypervisor	driver for the emulated device	supported	interception and emulation
Split-device driver	service VM or host OS	modified front-end	supported, managed by back-end	request passed to back-end

Key point
Devices lack a standard interface, so their virtualization trades performance against flexibility: passthrough is fastest but ties a VM to a device, hypervisor-direct decouples the VM at the cost of emulation, and the split-device driver model avoids emulation by modifying the guest's drivers.

12.11 Hardware support for virtualization

Around 2005, AMD and Intel added support for virtualization to their processors: AMD with the technology code-named Pacifica, now AMD-V, and Intel with Vanderpool, now Intel VT.

Source P3L6, hardware virtualization, x86 VT revolution; Uhlig et al. [42]; Rosenblum and Garfinkel [33].

Definition 12.22 (Hardware-assisted virtualization). Virtualization in which the processor itself provides the mechanisms needed to virtualize it, so that unmodified guest operating systems run under trap-and-emulate without binary translation or paravirtualization, is called *hardware-assisted virtualization*.

The extensions address the problems of the preceding sections.

Closing the holes in the x86 ISA In non-root mode the guest OS runs in ring 0, where instructions such as POPF behave as it expects, and every operation that the hypervisor must control, such as sensitive instructions, access to control registers, I/O and interrupts, can be made to cause a VM exit. The architecture thereby becomes virtualizable by trap-and-emulate.

Root and non-root modes The modes of Section 12.4, called host and guest mode by AMD, let the guest OS run in ring 0 while the hypervisor keeps control.

VM control structure A data structure per VM, more precisely per virtual CPU, that the hardware reads and updates. This *VM control structure* (the VMCS of Intel, the VMCB of AMD) holds the state of the virtual CPU, saved on a VM exit and restored on a VM entry, and it specifies which instructions and events cause VM exits, so the hypervisor decides for each VM what to trap.

Extended page tables The MMU performs the two-step translation of Figure 12.6 itself. The guest page table maps virtual to physical addresses as before, and an *extended page table* (EPT), maintained by the hypervisor, maps physical to machine addresses. On a TLB miss the MMU walks both tables, and the TLB caches the resulting translation from virtual to machine address. The guest OS changes and switches its page tables without VM exits, and shadow page tables become unnecessary. AMD's equivalent is called nested page tables.

Tagged TLB A *tagged TLB* marks each entry with an identifier of the address space (AMD's ASID) or of the virtual processor (Intel's VPID). Entries of different VMs and of the hypervisor coexist in the TLB, so neither a VM exit or entry nor a switch between VMs requires flushing it.

Multiqueue devices and interrupt routing Devices such as network cards provide several logical interfaces, or queues, each of which can be assigned to a different VM, and interrupts are routed to the core on which the VM that owns the queue runs.

Security and management support Further extensions protect the hypervisor and support the management of VMs.

New instructions enter non-root mode, read and write the VM control structure and issue hypercalls. Hosted hypervisors such as KVM rely on them.

Caution "Root/non-root" is Intel's name and host/guest mode AMD's; ARM (exception level EL2) and RISC-V (the H extension) provide a comparable split.

Tip A walk through g guest levels under n extended levels costs $(g+1)(n+1)-1$ references: 24 for $g = n = 4$, against 4 natively [43].

On Intel: VMLAUNCH and VMRESUME for VM entries, VMREAD and VMWRITE for the control structure, VMCALL for hypercalls.

VT-x shipped in November 2005 with the Pentium 4 662 and 672, AMD-V in May 2006, already with ASID-tagged TLBs. Nested page tables came with Barcelona (2007), extended page tables and the 16-bit VPID tag with Nehalem (2008).

Intel extended its support in stages, from the processor outwards. *VT-x* covers the processor: root and non-root modes, VM exits and entries, the VM control structure, and, in later processors, extended page tables and tagged TLB entries.

VT-d (directed I/O) adds to the chipset an I/O memory management unit that translates the addresses used in DMA (Lesson 11) and remaps interrupts, so that a device assigned to a VM in the passthrough model cannot access the memory of other VMs. *VT-c* (connectivity) brings virtualization support to network devices, with multiple queues per device and devices that present themselves as several virtual devices, each assignable to a VM.

Table 12.4: Approaches to processor virtualization on x86.

Approach	Guest OS	Problematic instructions	Main overhead	Example
Trap-and-emulate alone, before 2005	unmodified	some fail silently: incorrect	traps	—
Binary translation	unmodified	rewritten at run time	translation, emulation	VMware
Paravirtualization	modified	replaced by hypercalls	hypercalls	Xen
Hardware-assisted	unmodified	cause VM exits	VM exits	KVM

Key point — Lesson summary

Virtualization runs several operating systems concurrently on one machine; a hypervisor gives each VM an efficient, isolated duplicate of the machine. Bare-metal hypervisors run on the hardware, usually with a service VM that owns the drivers; hosted hypervisors extend a host OS. The CPU is virtualized by trap-and-emulate, which x86 processors before 2005 did not fully support, hence binary translation for unmodified guests and paravirtualization with hypercalls for modified ones. Memory virtualization adds machine addresses beneath physical addresses, handled by shadow page tables or by validated, batched page table updates. Devices are virtualized by the passthrough, hypervisor-direct or split-device driver model. Hardware-assisted virtualization, with root and non-root modes, VM exits, extended page tables and tagged TLBs, removes most of that overhead.

Remote Procedure Calls

Lecture	P4L1
Textbook	OSTEP [1] ch. 48; Silberschatz et al. [2] ch. 3; Birrell and Nelson [44]; RFC 5531 [45], RFC 4506 [46], RFC 1833 [47]
Keywords	remote procedure call, stub, RPC runtime, interface definition language, marshalling, unmarshalling, binding, registry, partial failure, Sun RPC, XDR, rpcgen, rpcbind, Java RMI

The preceding lessons considered a single machine, managed by one operating system or, in Lesson 12, by a hypervisor beneath several. This lesson opens the part on distributed systems, in which cooperating processes run on different machines connected by a network. Across machines, the processes can only exchange messages, typically through the sockets of Lesson 9, and every application that uses them repeats the same low-level communication code. Remote procedure calls hide this code behind the interface of an ordinary procedure call. The lesson presents the structure of RPC systems and the design decisions that each of them must make—interface definition, marshalling, binding, pointer arguments and partial failures—and then examines how Sun RPC and Java RMI make them.

Light in fibre covers about 200 km ms^{-1}, so a call to a machine 5000 km away cannot return in under 50 ms, however fast the two machines are.

13.1 Why remote procedure calls?

Two client-server applications illustrate the problem. In the first, a client requests a file from a server, which returns the contents of the file. In the second, a client sends an image together with the name and the parameters of an image-processing algorithm, and the server returns the processed image. The services differ, yet the communication code of the two applications is largely the same. Each of them must

- create and initialize the sockets through which client and server communicate;
- allocate buffers for the messages and populate them;
- include protocol information that tells the other side how to interpret a message, such as the operation requested (retrieve a file, apply an algorithm) and the size of the data that follow;
- copy the data into the buffers: a file name and the contents of the file in the first application, the algorithm parameters and the image in the second;

and the receiving side performs the inverse steps. Only the operations and the types of the data exchanged are specific to the application.

Source P4L1, why RPC, benefits of RPC; Birrell and Nelson [44]; OSTEP ch. 48.

Definition 13.1 (Remote procedure call). A *remote procedure call* (RPC) is an inter-process communication mechanism with which a process invokes a procedure that executes in another address space, usually on another machine, in the same way as it invokes a local procedure. The RPC system performs the communication that transfers the arguments to the procedure and the results back to the caller.

RPC is intended to simplify the development of interactions that cross address spaces and machines. Its benefits are the following.

Higher-level interface Data movement and communication are expressed as calls of procedures with typed arguments and results, rather than as byte buffers written to and read from sockets.

Error handling The RPC system detects communication errors and reports them to the application in a uniform way.

Hiding complexity The details of the interaction between machines, from connection management to differences in how the machines represent data, are handled by the RPC system and by code that it generates.

Birrell and Nelson [44] described the design and implementation of an RPC facility for the Cedar programming environment at Xerox PARC in 1984. The structure they chose, with generated stubs and a runtime library on both sides (Section 13.3), underlies later RPC systems, including the two examined in this lesson.[1]

[1] Their measurements, between two Dorado workstations on a 2.94 Mbit s^{-1} Ethernet, give the scale of the overhead: a call with no arguments and no results took 1097 µs in the median, of which 131 µs was transmission, against 9 µs for the same call made locally [44].

13.2 RPC requirements

The goal of making a remote operation look like a procedure call leads to the following requirements for an RPC system.

Client-server interactions RPC targets the client-server model: a server offers a set of procedures, and clients call them.

Source P4L1, RPC requirements; OSTEP ch. 48.

Procedure call interface An RPC has *synchronous call semantics*. The calling process, or in a multithreaded process the calling thread, blocks when it makes the call and continues only when the procedure has completed and its result has been returned, exactly as for a local call.

Type checking Because arguments and results have declared types, the RPC system can check them and report an error when a call does not match the procedure. The types also tell the receiver how to interpret the bytes of a message, which a socket delivers as an uninterpreted sequence.

Cross-machine conversion Client and server may run on machines that represent data differently, and the RPC system must convert between the representations.

Sun RPC runs over UDP and TCP [45]; NFS began on UDP, and NFSv4 requires a transport with congestion control, such as TCP [48].

Higher-level protocol RPC is a protocol above the transport layer. It can incorporate access control, authenticating clients and authorizing their calls, and mechanisms for fault tolerance, and it should be able to use different transport protocols, such as TCP and UDP.

The most common difference in representation is the order in which the bytes of a multi-byte value are stored. A machine is *big-endian* if it stores the most significant byte of an integer at the lowest address, and *little-endian* if it stores the least significant byte there. The x86 processors are little-endian; many network protocols, and the encoding of Section 13.13, use big-endian order.

Example 13.2. The 32-bit integer $300 = \texttt{0x0000012C}$ occupies the bytes `00 00 01 2C`, in order of increasing address, on a big-endian machine and `2C 01 00 00` on a little-endian machine. If a little-endian client copied the bytes of this integer into a message unchanged, a big-endian server would read them as $\texttt{0x2C010000} = 738\,263\,040$.

13.3 Structure of an RPC system

In a local procedure call, the caller places the arguments on the stack or in registers and jumps to the procedure, which computes the result and returns. In an RPC, caller and procedure reside in different address spaces, so neither the arguments nor the code of the procedure are accessible to the other side. The call therefore jumps to a local procedure that has the same interface as the remote one but performs the communication instead (Figure 13.1).

Source P4L1, structure of RPC, RPC steps; Birrell and Nelson [44]; OSTEP ch. 48.

Definition 13.3 (Stub). A *stub* is a procedure, generated by the RPC system, that converts between a procedure call and messages. The *client stub*, linked into the client, has the interface of the remote procedure: it packs the arguments into a message and unpacks the result from the reply. The *server stub*, linked into the server, unpacks the arguments, calls the actual implementation of the procedure and packs its result into the reply.

Same idea as the system-call wrappers of Lesson 1: a local procedure whose body crosses a boundary.

Definition 13.4 (RPC runtime). The *RPC runtime* is the library, present on both sides, that transmits the messages between client and server: it manages the connections, sends and receives the messages, retransmits lost ones, and passes each received message to the right stub.

Neither caller nor procedure contains any communication code; each sees an ordinary local call.

Birrell and Nelson's system had the same parts, with a stub generator called Lupine [44].

13.3.1 Steps of an RPC

An RPC proceeds in the following steps, numbered as in Figure 13.1.

−1 Register The server registers the procedures it offers, their argument types and its location, so that clients can find it (Section 13.6).

0 Bind The client finds a server that provides the desired procedures and binds to it, for example by opening a connection.

1 Call The client calls the procedure. Control passes to the client stub, and the caller blocks.

2 Marshal The client stub marshals the arguments: it serializes them into a buffer (Section 13.5).

Tip In a trace, steps 2–5 and their counterparts on the way back are pure overhead: the cost that stubs and runtimes add to the local call of step 6.

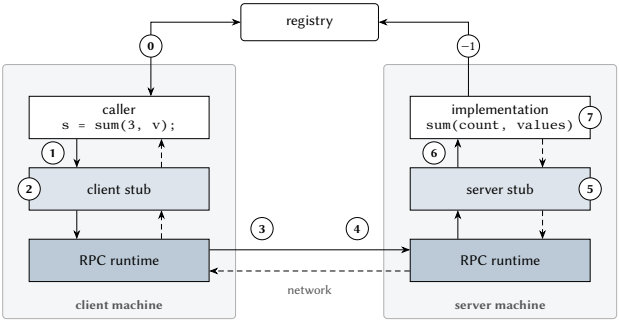


Figure 13.1: Structure of an RPC system and the steps of a call. Solid arrows: the call; dashed arrows: the return of the result. Steps −1 and 0 take place before the first call.

3 Send The RPC runtime of the client sends the message to the server, with the transport protocol agreed on during binding.
4 Receive The RPC runtime of the server receives the message and passes it to the server stub. Access control checks are performed at this point.
5 Unmarshal The server stub unmarshals the arguments: it extracts them from the buffer and creates the corresponding data structures.
6 Actual call The server stub calls the local implementation of the procedure.
7 Result The procedure computes the result, which returns to the client along the reverse path: the server stub marshals it, the runtimes transfer the reply, the client stub unmarshals it and returns it to the caller, which resumes.
Steps −1 and 0 are performed once, before the first call; steps 1–7 are repeated for every call.

Caution Sun RPC's default credentials, AUTH_SYS, are no more than the user and group identifiers that the client claims; real authentication needs RPCSEC_GSS [49].

> **Key point**
> An RPC looks like a local procedure call to both the caller and the procedure. The stubs convert between calls and messages, and the RPC runtime moves the messages between the machines.

13.4 Interface definition language

Client and server must agree on what the server can do and on the arguments and results of each of its procedures. The client needs this information to decide whether to bind to a server, and the RPC system needs it in a precise, machine-readable form to generate the stubs automatically.

Source P4L1, interface definition language, specifying an IDL; OSTEP ch. 48.

Definition 13.5 (Interface definition language). An *interface definition language* (IDL) is a language for describing the interface that a server exports: the names of its procedures, the types of their arguments and results, and a version number.

An RPC system can use one of two kinds of IDL.
Language-agnostic IDL A language of its own, independent of the programming languages in which clients and servers are written, from which stubs are generated for each supported language. Sun RPC uses XDR (Section 13.9).
Language-specific IDL The interface is written in the programming language of the application itself, so programmers need not learn another language. Java RMI uses Java (Section 13.14).
In either case the IDL describes only the interface of the server, not its implementation.

The version number supports the evolution of a service. When a service is upgraded, the servers are often not all upgraded at the same time, and a server may export the old and the new version of an interface side by side. By stating the version it expects, a client binds only to servers that implement that version.

13.5 Marshalling and unmarshalling

Source P4L1, marshalling, unmarshalling; OSTEP ch. 48.

The arguments of a call reside in the memory of the caller: values on the stack and in registers, and data elsewhere in the address space to which pointers refer. A message, in contrast, is a contiguous sequence of bytes.

Definition 13.6 (Marshalling). The conversion of the arguments or the result of a call into a contiguous buffer, in an encoding agreed between client and server, so that they can be sent as a message and correctly interpreted by the receiver, is called *marshalling* or *serialization*.

Definition 13.7 (Unmarshalling). The inverse conversion is called *unmarshalling*: parsing a received buffer according to the types given by the interface, extracting the values, and creating the corresponding data structures in the address space of the receiver, allocating and initializing memory as needed.

Caution Here marshalling and serialization are synonyms; where distinguished, serialization is the byte layout alone and marshalling adds what the receiver needs to rebuild the value, in Java the code base of a class.

For data of fixed size, the types in the interface tell the receiver where one value ends and the next begins. Data of variable size, such as arrays and strings, need additional information: either their length is encoded before their elements, or their end is marked by a special value, as the NUL character terminates a C string.

Caution A C string is NUL-terminated; an XDR string is sent as its length followed by the bytes, with no terminator (Example 13.16).

Example 13.8. The interface of a server declares a procedure `int sum(int count, int *values)` and states that `values` points to `count` integers. The client calls `sum(3, v)`, where `v` is an array at address `0x5a10` that holds 7, −2 and 300. With 4 B per integer, the client stub builds the 16 B buffer of Figure 13.2: the count followed by the three elements. The address `0x5a10` is not sent. The server stub reads the count, allocates memory for three integers, here at `0x9c40`, copies the elements into it, and calls the implementation with `count` = 3 and `values` = `0x9c40`.

The data are copied at least twice on the way, into the send buffer and out of the receive buffer; the shared memory of Lesson 9 exists to avoid exactly these copies.

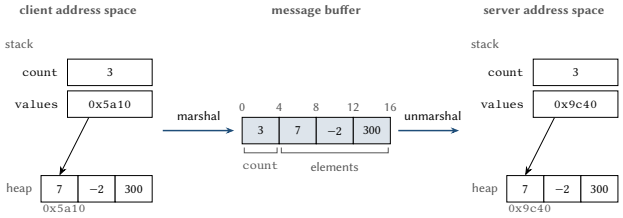


Figure 13.2: Marshalling and unmarshalling the arguments of `sum(3, v)`. The buffer contains the values of the arguments, including the array to which the pointer refers, but not the pointer itself; the numbers above it are byte offsets.

The message that the runtime sends also identifies the procedure that is being called; Section 13.13 shows the header that Sun RPC places in front of the encoded arguments.

Marshalling and unmarshalling code is not written by hand. The compiler of the IDL generates it from the interface, together with the stubs, as Section 13.10 shows for Sun RPC.

13.6 Binding and registries

Source P4L1, binding and registry; Birrell and Nelson [44].

Before a client can call the procedures of a server, it must determine which server to use and how to reach it.

Definition 13.9 (Binding). In *binding* a client determines which server it will communicate with, identified by the name and the version of the service, and how it will connect to that server, given by the network address, the transport protocol and the port, and it establishes the communication.

Definition 13.10 (Registry). A *registry* is a database of available services. Servers register their services in it, and clients search it by service name to find a server that offers the service and the details needed to contact that server.

Birrell and Nelson used Grapevine, a distributed database, for binding [44].

A registry can be organized in two ways.

Distributed registry A registry service, possibly replicated, with which any RPC service can register wherever it runs. Clients find services without knowing their location.

Machine-specific registry Every machine runs a registry for the services on that machine. A client must already know the address of the machine, and the registry provides the port number at which the service can be reached.

A registry also needs a naming protocol. The simplest requires the service name of the client to match the registered name exactly; a more elaborate one could also consider synonyms, so that a client looking for a service named `sum` would also find one registered as `add` or `summation`.

13.7 Pointers as arguments

A local procedure such as `sum` of Example 13.8 can receive a pointer and dereference it, because caller and procedure share an address space. In an RPC, the pointer refers to the address space of the client. On the server the same address is meaningless: it may be unmapped or hold unrelated data. An RPC system solves the problem in one of two ways.

Source P4L1, pointers in RPCs; OSTEP ch. 48.

Disallow pointers Remote interfaces may not contain pointer arguments.

Serialize the pointed-to data The client stub copies the data structure to which the pointer refers into the send buffer, and the server stub recreates it in the address space of the server and passes a pointer to this copy, as in Figure 13.2. For linked data structures, the stubs follow the pointers recursively.

With serialized pointers the server operates on a copy: changes that it makes to the pointed-to data are not visible to the client unless the interface returns the data as part of the result.

13.8 Partial failures

A local procedure call either returns or fails together with the process that made it: caller and procedure share the fate of a single process on a single machine. A distributed system can fail in parts.

Source P4L1, handling partial failures, RPC design choice summary; OSTEP ch. 48.

Definition 13.11 (Partial failure). A *partial failure* is a failure of some components of a distributed system, such as a machine, a process, a network link or an individual message, while the other components continue to operate.

When an RPC does not return, the cause may be that the server machine is down, that the server process has crashed or is overloaded, that the network is down, or that the request or the reply has been lost. The client observes the same thing in all these cases—no reply—and cannot tell which component has failed (Figure 13.3).

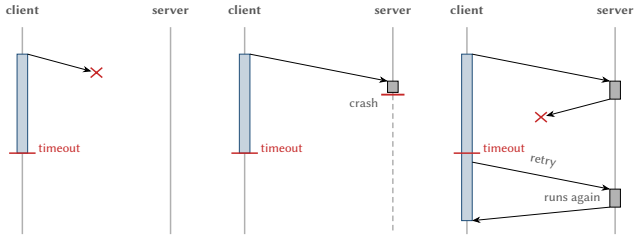


Figure 13.3: Three partial failures that the client cannot distinguish. The shaded bar marks the time during which the client is blocked, grey boxes the execution of the procedure, and a cross a lost message. (a) The request is lost; (b) the server crashes while executing the call; (c) the reply is lost, and the retry executes the procedure a second time.

An RPC system can bound the waiting with a *timeout* and retransmit the request when it expires, but timeouts and retries give no guarantees.[2] Retries cannot succeed while the server is down, and if the reply was lost, a retry executes the procedure again, as in Figure 13.3c. Repeated execution is harmless only for *idempotent* procedures, whose repeated execution has the same effect as a single one.

Example 13.12. A procedure that appends a record to a file is not idempotent: if the reply to a call that appends a 200 B record is lost, the retry appends the record a second time and

[2] Retrying until a reply arrives gives *at-least-once* semantics and giving up after the first attempt *at-most-once*; neither is exactly-once. A server comes close by keeping a cache of recent request identifiers together with their replies and answering a retransmission from the cache instead of executing the procedure again—NFS servers call it the duplicate request cache.

the file grows by 400 B. A procedure that writes 200 B at a given offset, such as offset 4096, is idempotent: executing it twice leaves the same contents.

RPC systems therefore provide a special error notification—an error code, a signal or an exception—that serves as a catch-all for the ways in which a call can fail partially. The caller must be prepared to receive it on every call, although it does not say which component has failed or whether the procedure has executed.

Birrell and Nelson: exactly-once execution if the call returns, otherwise an exception [44].

Key point — RPC design choices
Every RPC system must decide
- how to find the server: binding, through a registry;
- how to talk to the server and package the data: an IDL and marshalling;
- how to treat pointer arguments: disallow them or serialize the pointed-to data;
- how to handle partial failures: with special error notifications.

13.9 Sun RPC

Sun Microsystems developed *Sun RPC* in the 1980s for Unix systems; its best-known application is the Network File System (Lesson 14). It is now available on most platforms and has been standardized as ONC RPC (Open Network Computing RPC) [45]. Sun RPC makes the four design choices summarized at the end of Section 13.8 as follows.

Source P4L1, what is SunRPC, SunRPC overview, SunRPC XDR example; RFC 5531 [45]; RFC 4506 [46].

Binding A registry daemon runs on every machine that offers RPC services (Section 13.11).
IDL XDR serves both for specifying interfaces and for encoding data on the wire.
Pointers Pointers are allowed, and the pointed-to data are serialized.
Failures The runtime retries calls, using timeouts, and returns as much information as possible about the cause of a failure.

Definition 13.13 (External data representation). The language-agnostic IDL of Sun RPC, together with the standard binary encoding of the data that it describes, is called *external data representation* (XDR) [46].

Client and server interact through procedure calls. The interface of a service is specified in XDR in a `.x` file, from which `rpcgen` generates stubs for a specific language, C in the following (Section 13.10). A server registers with the registry daemon of its machine, which records the number of the service, its version, the transport protocols and the port. Binding creates a *client handle*, which the client passes to every call and in which the runtime keeps the state of the client's RPCs. Client and server may run on the same machine or on different machines. Current implementations descend from TI-RPC (transport-independent RPC), whose library does not depend on a particular transport protocol.

13.9.1 Interface specification

The following XDR specification, `sum.x`, describes a service that returns the sum of up to 64 integers.

```
const MAX_VALUES = 64;

struct sum_in {
  int values<MAX_VALUES>;    /* variable-length array */
};
struct sum_out {
  int total;
};

program SUM_PROG {                      /* service */
  version SUM_VERS {
    sum_out SUM_PROC(sum_in) = 1;       /* procedure 1 */
  } = 1;                                /* version 1 */
} = 0x20000123;                         /* program number */
```

The data types `sum_in` and `sum_out` describe the argument and the result. By default `rpcgen` allows a single argument and a single result (several arguments with `-N`), so several values are grouped in a structure. The service itself is a *program* that contains one or more versions, each of which contains procedures; this part of the `.x` language is the RPC language,

an extension of XDR defined together with the protocol [45]. Program, version and procedure are identified by numbers, and a request names all three. The 32-bit program number identifies the service uniquely. A client that sets `values` to the three integers 7, −2 and 300 and calls SUM_PROC receives a `sum_out` with `total` = 305.

Numbers 0x00000001–0x1fffffff are assigned by IANA—NFS is 100003—and 0x20000000–0x3fffffff are left to local administrators, which is where 0x20000123 comes from [45].

13.10 Compiling an interface with rpcgen

The stubs of Sun RPC are generated by *rpcgen*, the compiler that translates an XDR specification into C code. The command `rpcgen -C sum.x`, where `-C` selects ANSI C (the default in current Linux `rpcgen`), produces four files (Figure 13.4).

Source P4L1, compiling XDR, summarizing XDR compilation; OSTEP ch. 48.

`sum.h` The C declarations of the data types and of the stub procedures, and constants such as SUM_PROG, SUM_VERS and SUM_PROC. All other files include it.

`sum_clnt.c` The client stub `sum_proc_1`, named after the procedure and the version. It marshals the argument, sends the request through the runtime, waits for the reply and unmarshals the result.

`sum_svc.c` The server stub and a skeleton of the server program. Its `main` function creates the transports, registers the program and version with the registry daemon, and enters a loop that waits for requests. For each request, the dispatch procedure `sum_prog_1` determines the procedure number, unmarshals the argument, calls `sum_proc_1_svc`, and marshals and sends the result.

`sum_xdr.c` The XDR routines that marshal and unmarshal `sum_in` and `sum_out`, used by both sides (Section 13.12).

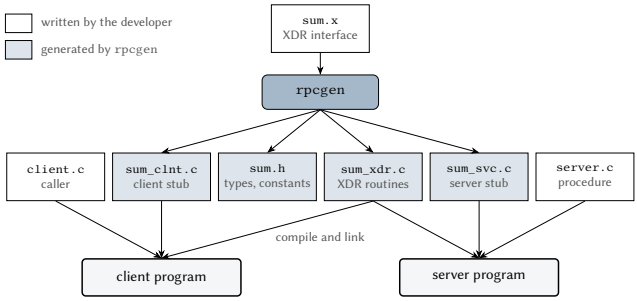


Figure 13.4: Building a Sun RPC client and server. The developer writes the interface, the client program and the implementation of the procedure; `rpcgen` generates the rest. `sum.h` is included by all C files.

The developer writes two pieces of code. On the server side it is the implementation of the procedure, `sum_proc_1_svc`, which follows; on the client side it is the program that includes `sum.h` and calls `sum_proc_1` (Section 13.11). Each is compiled and linked with the generated files. The RPC runtime library does the rest: the interaction with the operating system, such as the use of sockets, and the management of the communication.

Tip The generated names follow a rule: the procedure name in lower case, then the version number—`sum_proc_1`—with `_svc` added for the procedure the developer writes.

```
#include "sum.h"

sum_out *
sum_proc_1_svc(sum_in *in, struct svc_req *req)
{
  static sum_out out;       /* must outlive the call */
  u_int i;

  out.total = 0;
  for (i = 0; i < in->values.values_len; i++)
    out.total += in->values.values_val[i];
  return &out;              /* marshalled by the stub */
}
```

The result is returned by pointer, and it must remain valid after the procedure has returned, until the server stub has marshalled it, so it is kept in a static variable. The generated client stub likewise returns a pointer to a static variable, which the next call overwrites. This makes the

code generated by `rpcgen -C` unsafe for multithreaded clients and servers. With the option `-M`, `rpcgen` generates thread-safe code, in which the caller of a stub supplies the memory for the result. The option does not make the server multithreaded: the generated `main` still handles one request at a time. On Solaris, the additional option `-A` lets the runtime of the server create threads for requests automatically.

13.11 Registry and binding in Sun RPC

Source P4L1, SunRPC registry and binding; RFC 1833 [47].

The registry of Sun RPC is machine-specific: each machine that offers RPC services runs a registry daemon of its own, which clients contact at a well-known port.

Definition 13.14 (rpcbind). The registry daemon of Sun RPC is *rpcbind*. It listens on port 111 for both TCP and UDP and maps a program number, version number and transport protocol to the address at which the corresponding server listens.

The daemon `rpcbind` succeeded the older *portmapper* daemon, whose name still appears in the output below. Because port 111 is a privileged port, the daemon must be started with superuser privileges. When a server starts, the generated `main` registers each version of the program for each transport (Figure 13.5, step 1). The registrations of a machine can be listed with `rpcinfo -p`, of which an excerpt follows:

`rpcinfo -p host` queries a remote `rpcbind`; the Linux package is `rpcbind`, formerly `portmap`.

```
$ rpcinfo -p
   program vers proto   port  service
    100000    4   tcp    111  portmapper
    100000    2   udp    111  portmapper
    100003    3   tcp   2049  nfs
 536871203    1   udp  39871
 536871203    1   tcp  40123
```

Each line gives the program number, the version, the transport protocol and the port. The daemon itself appears as program `100000` on port 111, and the NFS server as program `100003`. The last two lines are the `sum` server, which the generated `main` registers for UDP and TCP; its program number `0x20000123` is printed in decimal.

NFS also listens on the registered port 2049, and NFSv4.1 clients should not use the RPC binding protocols [50].

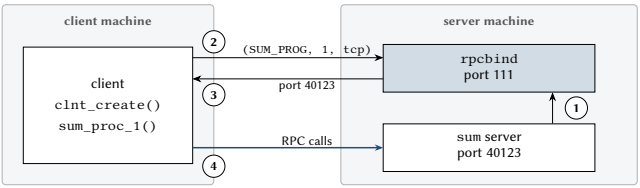


Figure 13.5: Registration and binding in Sun RPC. (1) The server registers its program, version, transport and port with `rpcbind`; (2, 3) `clnt_create` asks `rpcbind` on the server machine for the port; (4) the calls go directly to the server.

A client binds with `clnt_create`, whose arguments are the name of the server machine, the program number, the version number and the transport protocol. `clnt_create` asks `rpcbind` on that machine for the port of the server (steps 2 and 3) and returns a pointer to a client handle of type `CLIENT`, which the client passes to every stub. Besides the address of the server, the handle holds the authentication information and the status of the most recent call, from which error messages are derived. The complete client follows.

The server's own port is ephemeral and differs after every restart, so a firewall cannot open it in advance; `rpcbind` itself answers any query, and exposed daemons have been used in UDP reflection attacks [51].

```
#include <stdio.h>
#include <stdlib.h>
#include "sum.h"

int main(int argc, char *argv[])
{
  int v[3] = { 7, -2, 300 };
  sum_in in = { { 3, v } };   /* values_len, values_val */
  sum_out *out;
  CLIENT *clnt;

  if (argc != 2) {
```

```
    fprintf(stderr, "usage: %s host\n", argv[0]);
    exit(1);
  }
  /* bind: ask rpcbind on host argv[1] for the port */
  clnt = clnt_create(argv[1], SUM_PROG, SUM_VERS, "tcp");
  if (clnt == NULL) {
    clnt_pcreateerror(argv[1]);
    exit(1);
  }
  out = sum_proc_1(&in, clnt);          /* the RPC */
  if (out == NULL) {                    /* RPC failed */
    clnt_perror(clnt, argv[1]);
    exit(1);
  }
  printf("total = %d\n", out->total);
  clnt_destroy(clnt);
  return 0;
}
```

A stub that returns `NULL` reports a failed call. The runtime has recorded the reason in the handle—for example that the server machine could not be reached or that the call timed out—and `clnt_perror` prints it. A call that receives no reply times out; over UDP the runtime retransmits the request while it waits.[3]

[3] By default an RPC message over UDP is limited to 8800 B (`UDPMSGSIZE`); libtirpc retransmits every 15 s, so with the 25 s timeout of `rpcgen` stubs a request is resent once. glibc dropped its Sun RPC in version 2.32; Linux uses libtirpc.

13.12 XDR data types and routines

XDR provides the basic types familiar from C, such as integers, floating-point numbers, enumerations, Booleans and structures. It adds the following.
`const` a symbolic constant, translated into a `#define`;
`hyper` a 64-bit integer;
`quadruple` a 128-bit floating-point number;
`opaque` uninterpreted binary data, represented in C as `char` bytes.
Arrays and strings are declared as in Table 13.1. A variable-length array, whose maximum length is given in angle brackets, becomes a C structure with a length field and a pointer to the elements. The size of this structure does not depend on the maximum length; the elements reside in separately allocated memory. A string becomes a pointer to a NUL-terminated C string in memory, but it is transmitted as a length followed by the characters (Section 13.13).

Source P4L1, XDR data types, XDR routines; RFC 4506 [46].

The maximum is enforced: the generated routine refuses a longer array, so the call fails instead of overrunning the receiver.

Table 13.1: XDR arrays and strings and their C representation generated by `rpcgen`.

Kind	XDR declaration	C representation
Fixed-length array	`int data[8];`	`int data[8];`
Variable-length array	`int data<8>;`	`struct { u_int data_len; int *data_val; } data;`
String	`string name<32>;`	`char *name;`
Fixed-length opaque	`opaque id[16];`	`char id[16];`
Variable-length opaque	`opaque id<16>;`	`struct { u_int id_len; char *id_val; } id;`

Pointers are declared with an asterisk, as in C. Such *optional data* is encoded as a Boolean that states whether the pointer is non-null, followed by the pointed-to data if it is, so that a linked list is serialized element by element [46].

Caution A structure with a cycle defeats this encoding: the stubs would follow the pointers for ever. Java's object serialization instead gives every object a handle and writes a back-reference the second time.

13.12.1 XDR routines

The file `sum_xdr.c` contains one XDR routine for every data type of the interface. The routine for `sum_in`, essentially as generated by `rpcgen`, is the following.

```
bool_t
xdr_sum_in(XDR *xdrs, sum_in *objp)
{
  if (!xdr_array(xdrs,
      (char **)&objp->values.values_val,
      (u_int *)&objp->values.values_len, MAX_VALUES,
      sizeof(int), (xdrproc_t)xdr_int))
    return FALSE;
```

```
    return TRUE;
}
```

The same routine serves for marshalling, unmarshalling and freeing, and the field `x_op` of the XDR stream `xdrs` selects the direction. When it encodes, the routine writes the length and the elements into the buffer; when it decodes, it reads them and allocates the array if `values_val` is null; when it frees, it releases this memory. Memory allocated while unmarshalling must eventually be released. On the client, `xdr_free` applies a routine in the freeing direction to a result that is no longer needed. In servers generated with `-M`, the server stub calls a procedure written by the developer, `sum_prog_1_freeresult`, after the result has been returned to the client; it typically calls `xdr_free`.

13.13 XDR encoding

Source P4L1, encoding, XDR encoding; RFC 5531 [45]; RFC 4506 [46].

Over a byte stream a message is one record of one or more fragments, and each fragment is preceded by a 4 B header: a last-fragment flag in the top bit and a 31-bit length [45].

An RPC message on the wire consists of three parts.

Transport header the headers of the transport protocol, such as TCP or UDP, and of the protocols below it.

RPC header the identification of the call: a request identifier (the *xid*), with which the client matches replies to requests, the program, version and procedure numbers, and authentication data.

Data the arguments or the result, encoded into a byte stream according to their data types.

XDR thus comprises both the IDL and the encoding, the binary representation of the data on the wire. Its encoding rules are few.

Tip An item of n bytes occupies $4\lceil n/4 \rceil$ on the wire, and a variable-length array or string adds a 4 B length field.

- Every item is encoded in a multiple of 4 B; shorter data are padded with zero bytes.
- Multi-byte values are transmitted in big-endian order.
- Integers are represented in two's complement.
- Floating-point numbers are represented in the IEEE 754 format.

Each machine converts between its native representation and XDR when it marshals and unmarshals, so neither side needs to know the representation of the other.

Example 13.15. The client of Section 13.11 calls `SUM_PROC` with the values 7, −2 and 300. XDR encodes this variable-length array in the 16 B of Figure 13.2, the length followed by the three elements:

```
00 00 00 03   00 00 00 07   ff ff ff fe   00 00 01 2c
```

The element −2 appears in two's complement as `ff ff ff fe` and 300 as `00 00 01 2c`, whatever the byte order of the client. In front of these bytes the RPC header carries the request identifier and the program, version and procedure numbers of the call.

Strings show the difference between the representation in memory and on the wire.

Example 13.16. The string `"kernel"`, declared as `string name<32>`, takes 7 B in the memory of client and server: six characters and the terminating NUL. XDR encodes it in 12 B: the length 6 in 4 B, the six characters, and two bytes of padding to reach a multiple of four, without a terminator:

```
00 00 00 06   6b 65 72 6e   65 6c 00 00
```

> **Key point**
> XDR is both an IDL and an encoding. Every item is encoded in multiples of 4 B, in big-endian order, with two's complement integers and IEEE floating point; variable-length data carry their length. A message adds an RPC header, which identifies the request, program, version and procedure, to the transport header.

13.14 Java RMI

Source P4L1, Java RMI; Wollrath et al. [52].

Java offers a counterpart to RPC that matches its object-oriented semantics.

Definition 13.17 (Remote method invocation). The invocation of a method of an object that resides in another address space, with the syntax and semantics of a local method invocation, is called *remote method invocation* (RMI). In Java RMI the address spaces are those of Java virtual machines (JVMs), on the same machine or on different machines.

The IDL of Java RMI is Java itself, a language-specific IDL. A remote interface is a Java interface that extends `java.rmi.Remote`, and each of its methods declares the exception `RemoteException`, the error notification through which partial failures are reported. The following remote interface declares the `sum` service, and the client code beneath it calls the service.

```
public interface Summation extends java.rmi.Remote {
  int sum(int[] values) throws java.rmi.RemoteException;
}

/* client */
Registry reg = LocateRegistry.getRegistry("server");
Summation s = (Summation) reg.lookup("Summation");
int total = s.sum(new int[] { 7, -2, 300 });
```

The RMI registry is itself a remote object; `LocateRegistry.getRegistry` returns a stub for the one at TCP port 1099 unless another port is given.

The server creates an object that implements the interface and binds it under a name in the *RMI registry*; a client looks the name up and obtains a stub for the remote object, whose methods it then invokes (Figure 13.6). Arguments that are ordinary objects, such as the array of integers above, are passed by value: they are serialized together with the objects they refer to. Remote objects are passed by reference, as stubs.

Caution The copy is made on every invocation, and changes that the server makes to it are invisible to the client—the trap of the serialized pointers of Section 13.7.

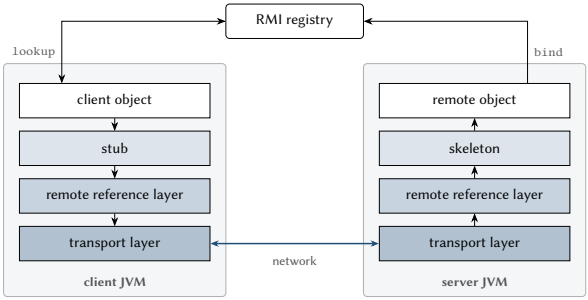


Figure 13.6: Java RMI. Invocations pass from the client through the stub, the remote reference layer and the transport layer to the server, where the skeleton delivers them to the remote object.

Table 13.2: Design choices of Sun RPC and Java RMI.

Choice	Sun RPC	Java RMI
Binding	per-machine registry daemon, `rpcbind`	RMI registry maps names to remote objects
IDL	XDR, language-agnostic, compiled by `rpcgen`	Java remote interfaces, language-specific
Encoding	XDR encoding	Java object serialization
Pointers and references	allowed; pointed-to data serialized	objects copied; remote objects passed as stubs
Partial failures	timeouts and retries; error status in the client handle	`RemoteException`

Java RMI has three layers on each side. The topmost is the stub on the client and, on the server, the *skeleton*, which unmarshals an invocation and dispatches it to the remote object. The RMI runtime provides the two layers beneath them.

Remote reference layer It determines the invocation semantics: *unicast*, to a single object, or *broadcast*, to a group of objects such as replicas, returning the first response or returning only if all responses match.

Transport layer It sets up and manages the connections and transfers the data. The architecture allows different transports, such as TCP, UDP or shared memory.

Table 13.2 compares the design choices of Sun RPC and Java RMI.

Since JDK 1.2 skeletons are unnecessary: the server runtime dispatches by reflection. Since J2SE 5.0 stubs are generated dynamically, and `rmic` was removed in JDK 15.

Contrast with Sun RPC, which copies pointed-to data: RMI copies serializable arguments and passes remote objects by reference. gRPC makes the same four choices differently: Protocol Buffers as IDL and encoding, DNS and a load balancer in place of a registry, and a status code returned by every call.

Key point — Lesson summary
RPC lets a process call a procedure in another address space, usually on another machine, with the synchronous semantics of a local call. Stubs generated from an IDL marshal and unmarshal arguments and results, and the RPC runtime transfers the messages. Every RPC system must decide how clients bind to servers through a registry, how interfaces are described and data encoded, how pointer arguments are treated, and how partial failures, which a client cannot diagnose, are reported. Sun RPC uses XDR as IDL and encoding, `rpcgen` to generate stubs, and a per-machine `rpcbind` registry; Java RMI uses Java as IDL and adds a remote reference layer that defines invocation semantics.

Distributed File Systems

Lecture	P4L2
Textbook	OSTEP [1] ch. 49–50; Silberschatz et al. [2] ch. 15, 19; Nelson et al. [53]
Keywords	distributed file system, replication, partitioning, upload/download model, stateless server, stateful server, UNIX semantics, session semantics, lease, NFS, file handle, Sprite, delayed write, concurrent write sharing

The virtual file system of Lesson 11 hides from applications where the files they use reside, and the remote procedure calls of Lesson 13 allow a process to have operations performed on another machine. A distributed file system combines the two: it stores files on machines connected by a network and serves them to clients that use the ordinary file system interface. This lesson examines the design space of such systems—how the work is divided between clients and servers, what state a server keeps, how cached copies are kept consistent and what a client observes when files are shared—and then studies two concrete designs: NFS, and Sprite, whose design was derived from measurements of how files are used.

14.1 Distributed file systems

Source P4L2, distributed file systems, DFS models; Silberschatz et al. ch. 19.

Operating systems hide the details of storage behind the file system interface. Beneath it, the virtual file system passes each operation to the file system in which the file resides, and that file system may keep its files on a local device or on other machines of a network.

Definition 14.1 (Distributed file system). A *distributed file system* (DFS) is a file system whose files are stored on, and served by, machines connected by a network, and which processes access through the ordinary file system interface whether or not a file resides on their own machine.

Three properties characterize a DFS:

- It is accessed through a well-defined interface. Applications use the same system calls as for local files, and the VFS passes each operation on a remote file to the DFS.
- It must maintain a consistent state. When several clients use the same files, the DFS must track which client accesses which file, propagate updates and keep cached copies coherent.
- It may mix several distribution models. Files can be replicated or partitioned among several servers, and the roles of client and server need not reside on different machines.

The distribution models are shown in Figure 14.1.

Client and server on different machines A single file server stores all files, and clients on other machines access them over the network.

Distributed file server The file service itself runs on several machines. In a *replicated* file server every machine holds all files; in a *partitioned* file server each machine holds a part of the files. Section 14.7 compares the two.

Peers Files are stored on and served from all machines.

In the last model the distinction between clients and servers is blurred. Such an organization is called *peer-to-peer*: every machine is a peer that both uses and provides the file service. Each peer stores a portion of the files and carries part of the load by serving requests, typically those for the files stored on it.

14.2 Remote file service

Source P4L2, remote file service: extremes, remote file service: a compromise; OSTEP ch. 49–50; Silberschatz et al. ch. 19.

Even with a single server, the work of accessing a remote file can be divided between client and server in very different ways. Two extremes bound the design space (Figure 14.2).

Definition 14.2 (Upload/download model). In the *upload/download model* a client that accesses a file downloads the entire file from the server, performs all operations on its local copy, and uploads the entire file to the server when it is done.

File transfer with FTP and version control systems such as Subversion work in this way. Reads and writes are performed locally at the client and are therefore fast. But the entire file is

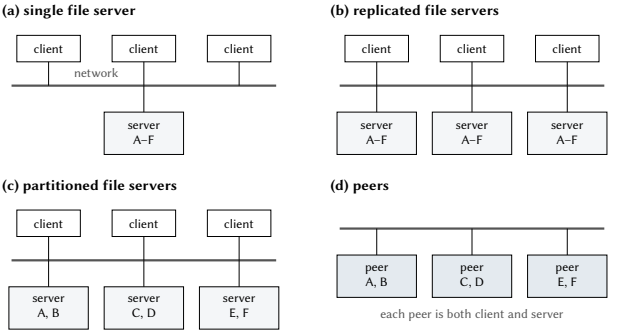


Figure 14.1: DFS models. (a) A single server stores all files. (b) Every server of a replicated file server holds all files. (c) Each server of a partitioned file server holds some of the files. (d) All machines are peers that store files and serve them.

transferred even when the client needs only a small part of it, and the server gives up control: once a file has been downloaded, the server knows neither what the client does with it nor when the modified file will be uploaded.

Definition 14.3 (Remote access model). In the *remote access model*, also called true remote file access, the file remains on the server and every operation on it—every read and every write—is sent to the server. Nothing is done locally at the client.

Because every access passes through the server, file access is centralized, and consistency is as easy to reason about as for a local file system shared by several processes. But every file operation pays for a network round trip and for processing at the server, and since the server executes every operation of every client, its capacity limits the number of clients the DFS can support.

Round trip within one datacenter about 0.5 ms, random disk seek about 10 ms (Dean, "Numbers everyone should know", 2009); the 1 ms assumed in Example 14.4 is a LAN.

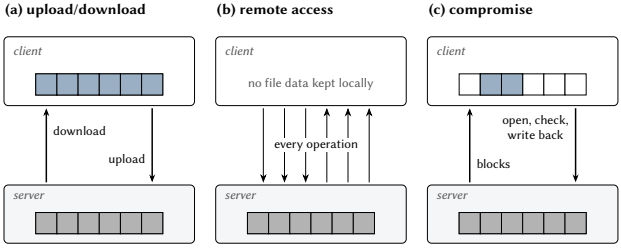


Figure 14.2: Remote file service. (a) The client downloads the whole file and uploads it when done. (b) The file stays at the server, which performs every operation. (c) The client caches some blocks but interacts with the server frequently.

Example 14.4. A client works with a 1 GiB file over a network that transfers 100 MiB/s; every request costs a round trip of 1 ms. If the client reads 50 blocks of 4 KiB, the remote access model needs 50 requests, about 50 ms, whereas downloading the file takes $1024/100 \approx 10.2$ s. If the client instead reads 200 000 blocks, the round trips of the remote access model alone take 200 s, while the download still takes 10.2 s and every read after it is local.

Caution Network rates are quoted in bits per second: 1 Gbit s^{-1} carries at most 125 MB s^{-1}, so the rate assumed here is a well-filled gigabit link.

Tip Downloading wins once the client reads more than $T_{\text{dl}}/t_{\text{RTT}}$ blocks, where $T_{\text{dl}} = S/r$ is the download time for a file of size S at rate r (about 10 000 here).

14.2.1 A compromise

Practical DFSs lie between the extremes and combine two principles:

1. Clients may keep parts of files, individual blocks, locally. Operations on local blocks have low latency, and the requests they save reduce the load on the server, so that a server can support more clients.
2. Clients must interact with the server, and do so frequently: when they open a file, to check whether their local blocks are still current, and to send their modifications back.

Linux NFS transfers data in units of `rsize`/`wsize`, negotiated up to 1 MiB (`nfs(5)`); a cached "block" is far larger than a disk sector.

The server thereby gains insight into what the clients are doing and control over which accesses it permits, which makes consistency easier to maintain.

The price is a more complex server, and file sharing semantics that differ from those of a local file system: a client that works on local copies of blocks cannot observe every change made by another client at the moment it happens (Section 14.5).

14.3 Stateless and stateful file servers

Source P4L2, stateless vs. stateful file server; OSTEP ch. 49; Silberschatz et al. ch. 19.

The division of work between client and server determines what the server must remember about its clients. Conversely, the state that a server keeps determines which of the models of Section 14.2 it can support.

Definition 14.5 (Stateless server). A *stateless server* keeps no information about its clients between requests: neither which files are open, nor which data a client has cached, nor where in a file a client is reading. Every request is self-contained and carries all information needed to serve it.

Definition 14.6 (Stateful server). A *stateful server* maintains information about its clients: which clients access which files, in which mode, and which data they have cached or locked.

Table 14.1: Stateless and stateful file servers.

	Stateless server	Stateful server
Suited to	upload/download and remote access models	the compromise model
Requests	self-contained; more bits per request	may refer to state kept by the server
Supports	neither consistent caching nor locking	locking, caching, incremental operations
Server resources	none for client state	memory and CPU time for state and consistency
After a failure	restart and continue	state must be checkpointed and recovered

A stateless server suffices for the two extreme models but not for the compromise, because keeping the blocks cached by clients consistent requires knowing which clients have cached which files. Its advantages are that it spends no memory or CPU time on client state, and that it recovers from a failure simply by restarting: no state has been lost, and a client whose request went unanswered sends it again. Resending is safe as long as the requests are idempotent (Lesson 13), as a read or a write at an explicit offset is. Its disadvantages are that it cannot support caching with consistency management, nor locking, and that every request must describe the operation completely—which file, which offset, how many bytes—so that more bits are transferred per request. Its clients may still cache data and ask the server, when they open a file or periodically, whether their copy is current (client-driven validation, Section 14.4, as in NFS), but the server can neither notify them of a change nor guarantee that their copies are coherent. NFSv3 therefore offers only client-driven validation and weaker consistency (Section 14.9).

Caution Not every NFSv3 operation is idempotent: a repeated REMOVE or exclusive CREATE returns an error, so servers keep a duplicate request cache for retransmissions (Lesson 13).

Caution Stateless refers to state about *clients*. The server still holds the files and their metadata; what it forgets between requests is who is reading what.

A stateful server is required for the compromise model. Because it knows what its clients do, it can support locking, client caching with consistency guarantees, and incremental operations such as reading the next block of a file. The price is twofold. When the server fails, its state is lost unless the server checkpoints it (Lesson 8) and a recovery mechanism restores it; and maintaining the state and enforcing consistency costs server resources, how much depending on the caching mechanism and the consistency protocol. Table 14.1 summarizes the comparison.

Example 14.7. A client reads a 12 KiB file in three requests of 4 KiB, and the server restarts after the second (Table 14.2). The stateless server receives the file and the offset with every read; with a file handle of 32 bytes (the fixed size in NFSv2), a 4-byte offset (also NFSv2's) and a 4-byte count, each request carries 40 bytes of arguments. The stateful server records the open file and its offset, so that a read needs only a 4-byte descriptor and the count, 8 bytes. After the restart, however, the stateful server no longer knows descriptor 5, and the third read fails unless the state is recovered, whereas the stateless server serves it as if nothing had happened.

Table 14.2: Reading a file from a stateless and a stateful server that restarts after the second read.

Step	Stateless server	Stateful server
open	`lookup(dir,"data")` returns handle h	`open("data")` returns 5; offset 0 recorded
read 1	`read(`h`,0,4096)`	`read(5,4096)`; offset 4096
read 2	`read(`h`,4096,4096)`	`read(5,4096)`; offset 8192
restart	nothing lost	open files and offsets lost
read 3	`read(`h`,8192,4096)` served	`read(5,4096)` fails

Key point
Practical DFSs let clients cache blocks while interacting frequently with the server, a compromise between the upload/download and the remote access models. The compromise requires a stateful server, which supports caching, locking and incremental operations at the cost of server resources and of recovery after failures. A stateless server is simpler and survives failures by restarting, but cannot guarantee that client caches are consistent.

14.4 Caching state in a DFS

Source P4L2, caching state in a DFS; OSTEP ch. 49–50; Silberschatz et al. ch. 19.

In the compromise model clients maintain a portion of the state of a file locally, typically some of its blocks, and perform operations such as open, read and write on this cached state. A client holds these blocks in its own memory, like the buffer cache of Lesson 11: the fastest place, but one that is limited in size and lost when the client crashes. On a local disk or SSD a client can cache more data and keep them across a crash, at the price of being slower than memory, though still faster than the network. The server caches blocks as well, in its own buffer cache, which saves it disk accesses; how much it saves depends on the load, since the requests of many clients for different files interleave there and displace one another's data. Once the same file is cached at several places, the copies can diverge. Suppose that clients 1 and 2 both cache file F and that client 2 modifies its copy into F' (Figure 14.3). The modification must reach the server, and client 1 must learn that its copy of F is stale.

The clients of NFS (Section 14.8) and of Sprite (Section 14.10) cache blocks in memory.

Caution 'Faster than the network' assumes a slow network: a random block of a 7200 rpm disk costs about 13 ms (OSTEP ch. 37), more than a round trip inside a datacenter.

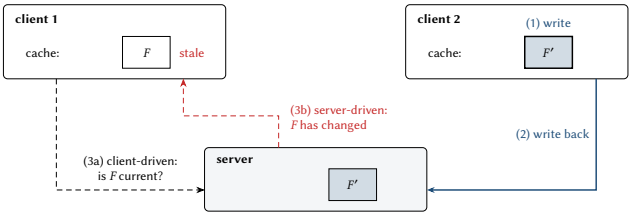


Figure 14.3: Client 2 modifies its cached copy of F and writes it back. Client 1 learns that its copy is stale either by asking the server (client-driven) or by being notified (server-driven).

The problem is that of cache coherence on shared memory multiprocessors (Lesson 10), where a write by one CPU makes the copies of a memory location in the caches of the other CPUs stale and, on a cache-coherent platform, the hardware updates or invalidates them. In a DFS the coherence mechanism is implemented in software, and it is either

client-driven the client asks the server, when it opens a file or periodically, whether its cached data are still current; or

server-driven the server, which knows which clients cache a file, notifies them when the file changes.[1]

Client-driven checks keep the server simple but cost a request each and leave a client with stale data until its next check; server-driven notification requires a stateful server. When and how often either mechanism acts is determined by the file sharing semantics that the DFS provides (Section 14.5).

Server-driven notification can invalidate the cached copies or update them, the choice between write-invalidate and write-update that distributed shared memory faces in Lesson 15.

[1] The Andrew File System, built at Carnegie Mellon at the same time as Sprite, carries the server-driven design further: a client caches whole files on its local disk, and the server holds a *callback* promise to notify it when the file is modified, so that an open costs nothing while the promise stands. OSTEP ch. 50 describes it.

14.5 File sharing semantics

On a single machine, when process A writes to a file that process B also has open, B's next read returns the new data: both processes access the file through the same buffer cache, so every write is visible immediately. In a DFS whose clients cache data, a write by client A first changes only A's cache. It reaches the server when A sends it there, and client B sees it only after B has obtained it from the server. The guarantees about when one client observes the writes of another form the *file sharing semantics* of a DFS.

Source P4L2, file sharing semantics on a DFS; Silberschatz et al. ch. 15; OSTEP ch. 49–50.

Definition 14.8 (UNIX semantics). Under *UNIX semantics* every write to a file is visible immediately to every subsequent read, on any machine.

UNIX semantics is what applications expect from a local file system. In a DFS it requires every write to be propagated before any other client reads the file, which makes it expensive to provide.

Definition 14.9 (Session semantics). Under *session semantics* a client obtains the current version of a file from the server when it opens the file, and writes its changes back to the server when it closes the file. The period between an open and the corresponding close is a *session*; changes made by other clients during a session are not visible in it.

Session semantics is easy to reason about, but it may be insufficient. Clients that share a file while both have it open do not see each other's changes, and the longer the sessions last, the longer the copies of different clients remain inconsistent.

With *periodic updates* a client writes its changes back to the server periodically, and cached data are invalidated or revalidated periodically. The period bounds the time during which clients can observe inconsistent data, and the DFS may offer explicit operations such as `flush` or `sync` with which an application forces its changes to the server at once. A client's right to use its cached data is thus limited in time: the client holds a lease on the data.

Tip On POSIX, `fsync` forces a file's dirty data to stable storage; on NFS, `close` also flushes to the server (close-to-open consistency, Section 14.9).

Definition 14.10 (Lease). A *lease* is a right that a server grants to a client for a limited period of time, such as the right to use cached data or to hold a lock. When the period expires, the right is lost unless the client has renewed it.

A lease on cached data need not be exclusive: several clients can hold leases on the same file at the same time. Two further approaches avoid inconsistent views altogether.

Definition 14.11 (Immutable file). An *immutable file* is never modified after it has been created. To change its contents, a new file is created, and the old one may be deleted.

All copies of an immutable file are identical, so sharing it cannot produce inconsistency. Services that store photos, for example, typically do not modify an uploaded photo in place; an edited photo is stored as a new file. Finally, a DFS can offer *transactions*: an application groups several updates, of one or several files, into a transaction, and the DFS applies them atomically—either all take effect or none does—as a database system does.

The same device as the copy-on-write of Lesson 8: the new version is written elsewhere and the old blocks are freed later.

Example 14.12. File F initially contains x. Client A opens F at $t = 0$ s, writes y at 4 s and closes F at 10 s. Client B opens F at 2 s, reads it at 6 s and 12 s and closes it at 14 s; it opens F again at 15 s and reads it at 16 s (Figure 14.4). For periodic updates, assume that A's client writes back dirty data every 5 s, at 5, 10, 15, … seconds, and that B's client revalidates its cache at 3, 8, 13, … seconds. Table 14.3 lists what B reads. Under session semantics B sees y only in its second session. Under periodic updates it sees y after its first revalidation that follows A's write-back, 4 s after the write; in general no write stays invisible to B for longer than the sum of the two periods, 10 s.

Table 14.3: Contents of F read by client B in Example 14.12.

Read by B at	UNIX semantics	Session semantics	Periodic updates
6 s	y	x	x
12 s	y	x	y
16 s	y	y	y

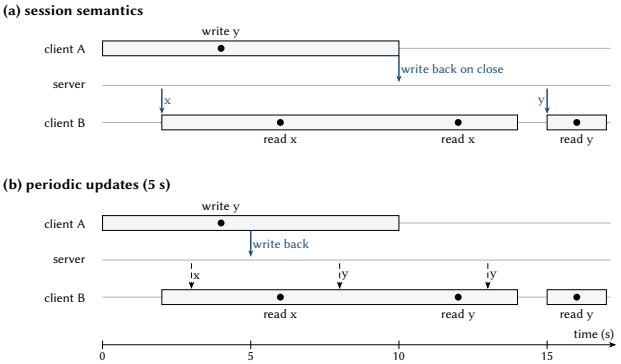


Figure 14.4: The scenario of Example 14.12. Shaded bars are sessions. (a) Under session semantics A writes back at close and B fetches the file at open. (b) Under periodic updates A writes back at 5 s and B revalidates every 5 s (dashed).

14.6 Files and directories

Source P4L2, file vs. directory service.

No single semantics is the best choice for every DFS. The right choice depends on the workload: how often files are shared, how often they are written, and how important a consistent view is to the applications. Knowing these access patterns allows the designer to optimize for the common case (Lesson 1).

A DFS serves two kinds of files whose workloads differ. A regular file is typically used by one client at a time, and a delay before other clients see its changes is rarely noticed. A directory is shared far more often—every client that creates, removes or renames a file in a shared directory modifies it—and an inconsistent view of a directory is harmful: two clients could, for instance, both create a file of the same name. A DFS can therefore apply different policies to the two, for example session semantics for files and UNIX semantics for directories, or periodic updates with less frequent write-back for files than for directories.

> **Key point**
> File sharing semantics define when one client observes the writes of another. UNIX semantics makes every write visible immediately but is expensive in a DFS; session semantics propagates changes at close and open; periodic updates bound the inconsistency by the update period; immutable files and transactions avoid inconsistent views altogether. The choice follows the workload, and files and directories may be treated differently.

14.7 Replication and partitioning

Source P4L2, replication and partitioning; Silberschatz et al. ch. 19.

A file service that runs on several machines distributes the files among them in one of two ways (Figure 14.1b and c).

Definition 14.13 (Replication). With *replication* every server machine holds a complete copy, a *replica*, of all files.

Replication balances the load, since a read can be served by any replica, and it provides availability and fault tolerance: when a machine fails, the others continue to serve all files. Writes, however, become more complex. Either every write is performed synchronously on all replicas, which makes writes slow, or it is performed on one replica and propagated to the others afterwards, in which case the replicas differ for a while. Replicas that have diverged must be reconciled, for example by voting, in which the contents held by a majority of the replicas prevail.

Definition 14.14 (Partitioning). With *partitioning* each server machine holds a subset of the files, and the subsets together make up the file system.

Partitioning improves availability compared with a single server, since the failure of a

machine affects only the files it holds. It scales with the size of the file system: more files need only more machines. A write to a single file involves only the machine that holds it, so writes remain simple. On the other hand, the portion of the data held by a failed machine is lost, and balancing the load is harder: if popular files are concentrated on a few machines, these become hot spots. Table 14.4 compares the two approaches. They can also be combined: the files are partitioned, and every partition is replicated on several machines.

Table 14.4: Replication and partitioning.

	Replication	Partitioning
Capacity	that of one machine	sum over all machines
Reads	served by any replica; load balanced	served by the machine holding the file; hot spots possible
Writes	on all replicas, synchronously or propagated later; replicas must be reconciled	on one machine; simple
Machine failure	no files lost	the files of that machine lost

Large-scale systems combine both: GFS and HDFS split files into chunks (partitioning) and keep several replicas of each (Lesson 16).

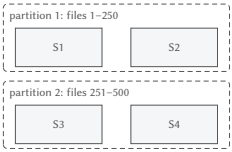


Figure 14.5: Two partitions, each replicated on two machines.

Example 14.15. Four server machines can store 250 files each. Replicated, the DFS holds 250 files, and the failure of one machine—indeed of any three—loses none of them. Partitioned, it holds 1000 files, and the failure of one machine loses $250/1000 = 25\,\%$ of them. Combined as in Figure 14.5, with two partitions of 250 files each replicated on two machines, the DFS holds 500 files; the failure of one machine loses none of them, and only the failure of both machines of the same partition loses 50 %.

14.8 The Network File System

The *Network File System* (NFS), developed at Sun Microsystems in the 1980s [54], is a widely used DFS in which clients access files that reside on a remote server. Its structure is shown in Figure 14.6. An application on the client performs ordinary file system calls. The VFS finds that the file belongs to an NFS file system and passes the operation to the *NFS client*, a file system implementation that turns the operation into a remote procedure call to the server. There the *NFS server* component receives the request, performs the operation on the server's files through the server's own VFS and local file system, and returns the result, which travels back to the application. NFS is built on Sun RPC and its data representation XDR (Lesson 13).

Source P4L2, NFS design; Sandberg et al. [54]; OSTEP ch. 49; Silberschatz et al. ch. 15.

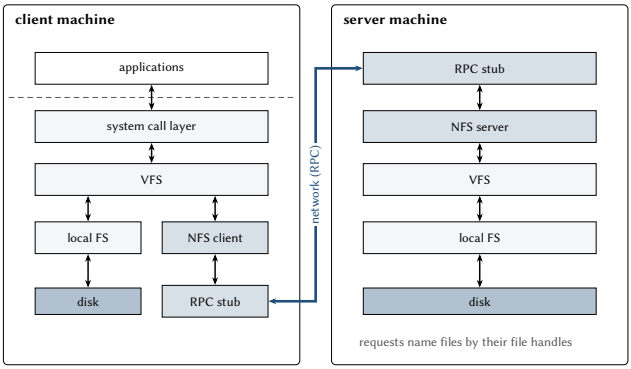


Figure 14.6: NFS. The client's VFS passes operations on NFS files to the NFS client, which sends them as RPC requests to the NFS server; the server performs them through its VFS and local file system.

Which remote files a client can reach is determined when an NFS file system is attached to the client's directory tree. A server *exports* some of its directories, and a client makes an exported directory available with the *mount* operation, which attaches it to a local directory, the *mount point*. Paths below the mount point then name files on the server. When a process opens such a file, the NFS client looks up its path at the server, and the server returns a file handle.

Definition 14.16 (File handle). A *file handle* is the byte sequence by which the NFS client and the NFS server identify a file in their requests. The server creates it, encoding the server-local identity of the file—the file system and the file within it—and the client treats it as opaque.

The handle is 32 B and of fixed size in NFSv2; later versions make it variable, at most 64 B in NFSv3 [55] and 128 B in NFSv4 [48].

The NFS client keeps the handle with its representation of the open file and passes it in every subsequent request, such as a read, a write or a request for the attributes of the file. A server typically encodes in the handle an identifier of the file system, the inode number of the file, and a *generation number* that changes whenever the inode is reused. If the file is removed on the server, by a local process or by another client, requests with the handle fail with the error *stale file handle* (`ESTALE`; reopening the path obtains a fresh handle): the handle refers to a file that no longer exists, and the generation number prevents it from silently referring to a new file that has reused the inode.

Example 14.17. The directory `/home` of a client is mounted from a server; the mount operation has given the client the file handle of the exported directory. The program below does not know this; its comments show the principal NFSv3 requests that the calls cause when nothing is cached.[2]

```
char buf[4096];
int fd = open("/home/ana/log.txt", O_RDONLY);
        /* LOOKUP "ana" in the handle of /home;
           LOOKUP "log.txt" in the handle of ana:
           the server returns the file handle   */
ssize_t n = read(fd, buf, sizeof buf);
        /* READ(file handle, offset 0, 4096)    */
close(fd);  /* read-only: no request is needed  */
```

[2] NFSv3 is really a family of RPC programs: NFS itself [55], a MOUNT protocol that turns a path into the handle of an exported directory, a lock manager, and a status monitor with which the two ends tell each other that they have rebooted. Each is registered with `rpcbind` (Lesson 13); NFSv4 folds them all into one protocol on port 2049.

14.9 NFS versions

Source P4L2, NFS versions; RFC 1813 [55]; RFC 7530 [48]; OSTEP ch. 49.

NFS has been in use since the 1980s and has gone through several versions, of which NFSv3 [55] and NFSv4 [48] are in current use. They differ fundamentally in how they treat state. NFSv3 is a stateless protocol, although in practice it is complemented by auxiliary protocols that keep state, such as a lock manager. NFSv4 is stateful: opening files, locking and the delegations described below are part of the protocol.

NFSv4.0: RFC 3530 (2003), revised as RFC 7530 (2015); NFSv4.1 (RFC 8881 [50]) and 4.2 (RFC 7862) extend it and are what is deployed today.

14.9.1 Caching

For files that are not accessed concurrently, NFS provides session-based semantics, called *close-to-open consistency*: a client writes its changes back to the server when it closes a file, and checks for a newer version when it opens one. In addition, while a file is in use, the client periodically asks the server whether the file has changed, by default every 3 s for files and every 30 s for directories. These periods are the timeouts of the client's *attribute cache*, which holds the attributes of files and directories, such as size and modification time; a changed modification time tells the client that its cached data are stale. The longer period for directories is deliberate: the contents of a directory change far less often than the data of a file, while lookups in it are frequent, so checking it rarely saves many requests and costs only that a name created or removed by another client is seen late. The policy is the opposite of the example in Section 14.6, and it is acceptable because the operations that modify a directory go to the server in any case and are serialized there.

The session-like half, close-to-open consistency, means: revalidate at open, write back at close. The `nocto` mount option turns off the check at open. `nfs(5)`.

Example 14.18. Client B last checked file F with the server at $t = 20.0$ s, so its information about F is considered current until 23.0 s. Client A closes F at 21.0 s, writing its changes back. A read by B at 22.5 s is served from B's cache and returns the old data; the first read by B after 23.0 s makes B check with the server, discover the change and fetch the new data. A file that A creates at 21.0 s in a directory that B checked at 20.0 s may remain invisible to B until 50.0 s.

On Linux these periods are the mount options `acregmin` and `acdirmin`; `acregmax` and `acdirmax`, both 60 s by default, bound the interval from above.

NFSv4 can avoid these checks. The server may grant a client a *delegation* of a file for a period of time. While the delegation lasts, the client opens the file and uses its cached data without checking with the server for updates, and the server recalls the delegation when another client needs access to the file that conflicts with it.

Note 14.19. NFS thus combines session semantics with periodic revalidation. A periodic check can bring the changes of another client into a session that is still open, which pure session semantics excludes, and between two checks a client may read data that has already

changed at the server; UNIX semantics is therefore not provided.

14.9.2 Locking

Locking in NFSv4 is based on leases. The server grants a lock for a limited period, and before this lease expires the client must either release the lock or renew the lease. If a client fails or becomes unreachable, its lease runs out, and the server can grant the lock to other clients without waiting indefinitely for the failed one (Figure 14.7). NFSv4 additionally supports *share reservations*: when a client opens a file, it declares the access it requires—reading, writing or both—and the access it denies to other clients. Reservations that share reading but deny writing to others act like a reader-writer lock (Lesson 10); a client can upgrade its reservation from reading to writing, or downgrade it again, without closing the file.

The lock manager of NFSv3 is a protocol of its own, the Network Lock Manager (NLM).

Caution Leases need no synchronized clocks, only clocks that do not drift much over one lease: the lease is a duration, and the client measures it from the moment it sent the request, so it gives up before the server does.

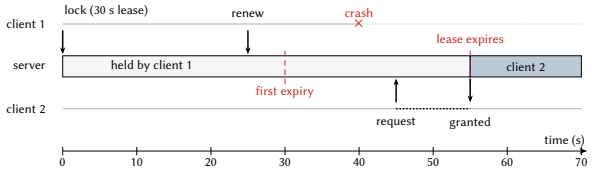


Figure 14.7: Lease-based locking. Client 1 renews its lease at 25 s and crashes at 40 s; when the lease expires at 55 s, the server grants the lock to client 2.

Example 14.20. A server grants leases of 30 s. Client 1 acquires a lock at $t = 0$ and renews the lease at 25 s, extending it to 55 s; it crashes at 40 s. Client 2 requests the lock at 45 s and waits. At 55 s the lease of client 1 expires without renewal, and the server grants the lock to client 2 (Figure 14.7). Had client 1 not crashed but merely lost contact with the server, it would have to stop using the lock shortly before 55 s by its own clock, because it could not know whether the lock had been granted to another client.

The protocol fixes no duration: all leases of one server have the same length, which a client reads from the server's `lease_time` attribute [48]. The Linux NFS server defaults to a 90 s lease (`/proc/fs/nfsd/nfsv4leasetime`); the 30 s here is for the example.

Key point
NFS clients reach remote files through the VFS, the NFS client and RPC, and identify files by server-generated file handles. NFSv3 is stateless and NFSv4 stateful. NFS combines session-based caching with periodic checks, by default every 3 s for files and every 30 s for directories; NFSv4 adds delegations, lease-based locking and share reservations.

14.10 The Sprite distributed file system

The distributed operating system *Sprite* was developed at the University of California, Berkeley, in the 1980s; its file system let the workstations of a network share files stored on servers. Nelson, Welch and Ousterhout [53] describe how its file system caches data. The design process is the point of the paper: each design decision is derived from, and justified by, an analysis of how files are actually accessed. Table 14.5 lists the observations and the conclusions drawn from them.

Source P4L2, Sprite distributed file system, Sprite DFS access pattern analysis, Sprite DFS from analysis to design; Nelson et al. [53].

Table 14.5: Access patterns and their consequences for the design of Sprite, measured on time-shared 4.2BSD systems (the BSD trace study [56] cited by Nelson et al.).

Observation	Consequence
33 % of all file accesses are writes	Caching pays off for the majority of accesses, which are reads; but propagating every write at once, as UNIX semantics requires, is too expensive.
75 % of files are open for less than 0.5 s, 90 % for less than 10 s	Session semantics still has too much overhead: with such short sessions, updates at every open and close would reach the server very frequently.
20 % to 30 % of new data are deleted within 30 s, 50 % within 5 min	Writing data back at close is not necessary; much of the data are deleted before any other client needs them.
Concurrent access to a file is rare	Concurrent access need not be optimized, but it must be supported.

From these observations the design of Sprite follows:

- Clients cache file blocks and write modified blocks back to the server with a delay of 30 seconds, as described below.
- When a client opens a file that another client has been writing, the server first collects the dirty blocks from that client.
- Every open goes to the server, and directories are not cached at the clients.
- When a file is written concurrently, caching of that file is disabled and all its operations go to the server.

Linux does the same for its page cache: flusher threads wake every 5 s and write back data that has been dirty for more than 30 s (`dirty_expire_centisecs`).

The first point defines the write policy. Keeping modified blocks in the client cache and writing them back only after a delay, rather than on every write or close, is called *delayed write* or delayed write-back. In Sprite, every 30 seconds a client writes back the dirty blocks that have not been modified during the last 30 seconds. Blocks that are still being modified are therefore not transferred repeatedly, and data deleted within 30 seconds of being written are never transferred at all. A block reaches the server between 30 and 60 seconds after its last modification.

Classic UNIX had the same rhythm: the `update` daemon ran `sync` every 30 s.

Caution Delayed write trades durability for traffic: a client crash loses every block not yet written back, up to a minute of work. Sprite accepts this because the delay keeps writes fast and removes traffic; the server adds another 30 s to 60 s before writing to disk.

Example 14.21. A client writes back at $t = 0, 30, 60, 90, \ldots$ seconds, t counted from the first flush; a block is written back at the first flush at which it has been unmodified for at least 30 s. Block b_1 is modified at 12 s; at 30 s it has been unmodified for only 18 s, so it is written back at 60 s. Block b_2 is modified at 12 s and again at 50 s; at 60 s it has been unmodified for 10 s, so it is written back at 90 s. Block b_3 belongs to a temporary file that is written at 20 s and deleted at 45 s; it is never sent to the server.

The semantics Sprite provides depend on how a file is shared.

Definition 14.22 (Sequential and concurrent write sharing). A file is subject to *sequential write sharing* when one client modifies it and closes it and another client opens it later: the clients use the file one after another. It is subject to *concurrent write sharing* when it is open on several clients at the same time and at least one of them has it open for writing.

Under sequential write sharing Sprite caches the file, and every client that opens it sees the data written by the previous writer. Under concurrent write sharing Sprite does not cache the file; the server executes all reads and writes in order, which yields UNIX semantics.

Caution *Sequential* here describes the clients, not the access pattern within the file: one client after another, not blocks read in order.

14.11 File access operations in Sprite

Source P4L2, file access operations in Sprite; Nelson et al. [53].

In the following, a file is read by several clients and written by one or more. All opens go through the server, all clients cache blocks while the file is cacheable, and a writer keeps a timestamp for every block it modifies, as delayed write requires. Clients and server keep the following state for each file (Figure 14.8):

Client whether caching is enabled for the file; the cached blocks; the time of the last modification of each dirty block; and the version number of the cached data.

Server the clients that have the file open for reading and for writing; the client that wrote the file last; the current version number; and whether the file is cacheable.

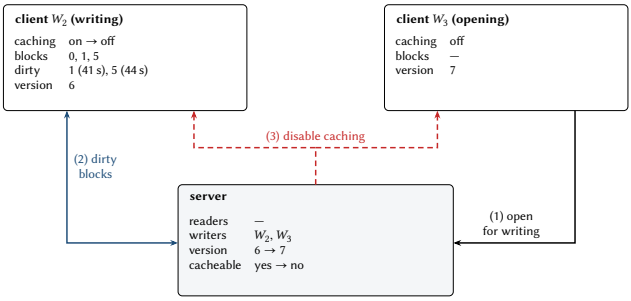


Figure 14.8: Per-file state in Sprite. When W_3 opens the file that W_2 has open for writing (1), the server collects W_2's dirty blocks (2) and disables caching on both clients (3).

Sequential write sharing. Each time a client opens the file for writing, the server increments the version number. A client that opens the file compares the version number returned by

the server with that of its cached blocks, and discards the blocks if they are older. Because of delayed write, the client that wrote the file last may still hold dirty blocks of it even after closing it. The server therefore asks that client to write them back before another client opens the file, so that the new client sees the latest data. Caching remains enabled.

Concurrent write sharing. When a client opens the file for writing while another client has it open, or for reading while another client has it open for writing, the server again collects the dirty blocks of the writer, marks the file as not cacheable, and notifies all clients that have it open. From then on no client caches the file, and every read and write goes to the server. Once the concurrent write sharing has ended, the file becomes cacheable again for later opens; clients that already have it open continue without caching it.

Example 14.23. Table 14.6 traces the server state for file F, initially at version 4, as clients R_1, W_1, W_2 and W_3 open and close it. In step 5 W_1 has closed F but still holds dirty blocks, which the server collects before W_2 proceeds; the sharing is sequential, and W_2 may cache F. In step 6 W_3 opens F while W_2 still has it open for writing (Figure 14.8), and caching is disabled. In step 7 R_1 finds that its cached blocks of version 4 are outdated; it discards them and reads from the server.

Tip Caching survives sequential sharing because there is always a last writer whose blocks the server can collect first; under concurrent sharing there is none, so the server must serve every access itself.

Table 14.6: Server state for file F in Sprite. Version and cacheability are shown after each step.

	Event	Server action	Open by	Ver.	Cache
1	R_1 opens for reading	—	R_1	4	yes
2	R_1 closes	—	—	4	yes
3	W_1 opens for writing	increments version	W_1	5	yes
4	W_1 writes, closes	— (dirty blocks stay at W_1)	—	5	yes
5	W_2 opens for writing	collects W_1's dirty blocks; increments version	W_2	6	yes
6	W_3 opens for writing	collects W_2's dirty blocks; increments version; disables caching	W_2, W_3	7	no
7	R_1 opens for reading	tells R_1 not to cache	R_1, W_2, W_3	7	no
8	W_2, W_3, R_1 close	enables caching	—	7	yes

Key point — Lesson summary

A distributed file system serves files from networked machines through the ordinary file system interface, with one server, replicated or partitioned servers, or peers. Practical designs lie between the upload/download and remote access models: clients cache blocks and interact frequently with a stateful server, which must keep the cached copies coherent. File sharing semantics—UNIX, session, periodic updates with leases, immutable files, transactions—define what clients observe, and may differ for files and directories. Replication improves availability and load balance at the cost of complex writes; partitioning scales with the size of the file system. NFS uses file handles over RPC, stateless NFSv3 or stateful NFSv4, session-based caching with periodic checks, and lease-based locking. Sprite derived its design from measured access patterns: delayed write with a 30-second delay, caching under sequential write sharing, and no caching under concurrent write sharing.

Lesson 15

Distributed Shared Memory

Lecture	P4L3
Textbook	OSTEP [1] ch. 48; Protić et al. [57]; Nitzberg and Lo [58]; Tanenbaum and Bos [6] ch. 8; Tanenbaum and van Steen [59] ch. 7
Keywords	distributed shared memory, RDMA, sharing granularity, false sharing, access algorithm, migration, replication, consistency management, page fault, consistency model, strict consistency, sequential consistency, causal consistency, weak consistency

Lesson 14 studied distributed file systems, in which servers own the files and serve them to clients that cache parts of them. In many distributed applications, however, every node both owns a part of the state and serves it to the others. This lesson examines distributed shared memory, which lets processes on different machines share memory as if they ran on a single shared memory multiprocessor, as an example of how such distributed state is managed. It discusses the design choices of a DSM system—granularity, access patterns, migration versus replication and consistency management—how a DSM locates its pages and uses the MMU to intercept accesses, and the consistency models that define what processes observe when they share memory.

15.1 Peer distributed applications

Source P4L3, lesson preview, reviewing distributed file systems, peer distributed applications; OSTEP [1] ch. 48; Tanenbaum and van Steen [59] ch. 2.

In the distributed file systems of Lesson 14 the roles of the machines are clearly divided. Servers own and manage the state, the files, and provide the service, file access; clients send requests to the servers. Clients cache parts of the state, which lowers the latency of their accesses and lets a server support more clients, but the state itself belongs to the servers.

Lesson 16 examines such applications in the datacenter, where a framework sends each computation to the node that already holds its input.

In many distributed applications no such division exists. In big data analytics, web search or content sharing, the state of the application is distributed over all of its nodes: every node owns a portion of the state and provides service on that portion to the other nodes, while it also uses the state owned by others. Every node is thus both a client and a server, and all nodes are peers; we call such applications *peer distributed applications*. The word "peer" is used loosely here. In a peer-to-peer system (Lesson 14) even the control and management of the overall system are carried out by all peers, whereas a peer distributed application may assign management tasks, such as keeping track of which node holds which part of the state, to designated nodes.

Managing state that is spread over peers raises three issues, which recur throughout this lesson:

The first two arise inside one machine as well: NUMA-aware scheduling (Lesson 7) moves the computation to its memory instead.

Placement The node on which each part of the state resides. Access is fastest when the state resides on the node of the processes that access it most.

Migration How and when state that resides on another node is brought to the node of the process that accesses it, by moving it or by copying it.

Sharing rules The rules that determine the order in which concurrent operations on the same state take effect, and when each node observes the updates made by the others.

Caution The migration of this lesson moves *memory* between nodes. It is not the process migration of Lesson 8, which transfers a whole process to another machine.

Distributed shared memory, a peer distributed application whose state is memory, is the example on which the lesson studies these issues.

15.2 Distributed shared memory

Source P4L3, distributed shared memory; Protić et al. [57]; Nitzberg and Lo [58]; OSTEP ch. 48; Tanenbaum and Bos ch. 8.

A shared memory multiprocessor (Lesson 7) lets threads on different CPUs cooperate through memory that all of them access, but its memory is limited to what one machine can hold. Distributed shared memory extends the model to many machines.

Definition 15.1 (Distributed shared memory). A service that manages memory across the nodes of a network, so that the applications running on top of it have the illusion of running on a single shared memory machine, is called *distributed shared memory* (DSM). Processes on any node read and write one shared address space, whose contents reside in the physical memories of the different nodes.

Like every peer distributed application, each node of a DSM system (Figure 15.1)

- owns state: a portion of the physical memory of the system, namely the part of its own memory that it contributes to the DSM;
- provides service on that state: read and write operations on its memory, which may be issued by processes on any node;
- takes part in consistency protocols with the other nodes, which ensure that accesses to the shared memory remain meaningful when several nodes access the same memory—for example, that a read returns the value of the most recent write.

Caution This is not the shared memory IPC of Lesson 9: there processes share the memory of one machine, and the hardware keeps their views consistent.

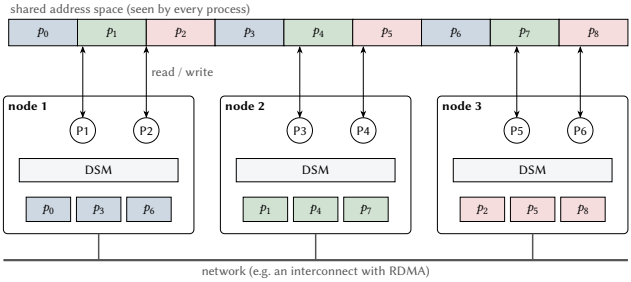


Figure 15.1: Distributed shared memory. Every page p_i of the shared address space resides in memory contributed by one of the nodes; the DSM layer of each node serves accesses to local pages and obtains remote pages over the network.

DSM permits applications to scale beyond the memory limits of a single machine: an application that needs more memory than one machine holds obtains it from other machines. It also provides shared memory at a lower cost, because a single machine with a very large memory is disproportionately more expensive than several commodity machines with moderate memories. The price is latency. An access to memory that resides on another node crosses the network, which is orders of magnitude slower than an access to local memory. Commodity interconnects reduce this cost by supporting remote direct memory access.

Local memory takes some 100 ns, a round trip over datacenter Ethernet tens of microseconds, a one-sided RDMA read (a round trip) on 400 Gbit s^{-1} NDR InfiniBand about 2 µs (perftest `ib_read_lat`); vendors quote sub-microsecond one-way latency.

Definition 15.2 (Remote direct memory access). With *remote direct memory access* (RDMA), a network interface transfers data directly between the memory of its own machine and the memory of a remote machine, without involving the CPU or the operating system of the remote machine in the transfer.

RDMA extends direct memory access (Lesson 11) across the network. It is typically combined with operating system bypass (Lesson 11), so that the kernel of neither machine lies on the data path.

InfiniBand provides RDMA natively; RoCE and iWARP are RDMA protocols carried over Ethernet.

15.3 Hardware and software DSM

DSM can be provided by hardware, by software, or by a combination of both.

Source P4L3, hardware vs. software DSM; Protić et al.; Nitzberg and Lo [58].

Hardware-supported DSM relies on a special-purpose interconnect. The operating system of each node is given the impression that it manages a physical memory much larger than the node's own, spanning several nodes. An access to an address that resides on another node is translated by the network interface card (NIC) into a message to that node, and the NICs take part in every aspect of managing the shared memory, including atomic operations on remote memory, without involving the CPU of the remote node. Such hardware is expensive; it is used in high-end systems where performance is paramount, as in high-performance computing.

Software-supported DSM performs everything in software: locating the node that holds the memory, fetching it, detecting accesses and keeping copies consistent. The software is part of the operating system or of the runtime of a programming language; the first shares pages, the second objects (Section 15.4).

The Scalable Coherent Interface (IEEE 1596-1992), implemented by the IQ-Link of the Sequent NUMA-Q, is such an interconnect: it keeps the memories of the nodes coherent in hardware.

Hybrid DSM divides the work between the two. Hardware, the MMU in particular, performs what must happen on every access: it translates addresses and raises an exception when an access is not permitted by the current mapping. Software makes the decisions that require knowledge of the system as a whole, for example which pages to prefetch, that is, to obtain from other nodes before they are accessed.

15.4 Sharing granularity

Source P4L3, DSM design: sharing granularity; Protić et al.; Nitzberg and Lo; Tanenbaum and Bos ch. 8.

The first design decision of a DSM is the unit of sharing: the unit of memory that the DSM transfers between nodes, replicates and keeps consistent as a whole. On a shared memory multiprocessor the hardware keeps the CPU caches coherent at the granularity of cache lines (Lesson 10), units of a few tens of bytes; this is affordable because the hardware coordinates the caches within nanoseconds. In a DSM every act of coordination costs messages over the network, which makes such a fine granularity far too expensive. The candidates are:

Variables Sharing individual variables is too fine as well. A variable occupies only a few bytes, so the messages needed to coordinate each access would cost far more than the data are worth.

A 2 MiB large page (Lesson 8) would cover 512 base pages at once: fewer and larger transfers, and correspondingly more false sharing.

Pages A page, commonly 4 KiB, is the unit that the operating system already manages. A page-based DSM is implemented by the operating system in cooperation with the MMU (Section 15.9); it is transparent to applications and independent of the language they are written in.

Objects A language runtime knows the objects that a program creates. An object-based DSM, implemented in the runtime, shares units that correspond to the program's data structures and needs no change to the operating system, but only programs that use that runtime benefit from it.

A coarser granularity amortizes the cost of a transfer or of an act of coordination over more data, but it also places unrelated data in the same unit.

Definition 15.3 (False sharing). The situation in which data that are not shared—each accessed by processes on a different node—reside in the same unit of sharing, so that the DSM coordinates the accesses to that unit as if the data were shared, is called *false sharing*.

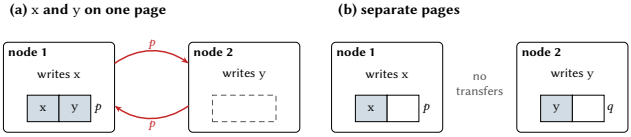


Figure 15.2: False sharing. (a) Node 1 writes x and node 2 writes y; since both reside on page p, the page moves back and forth. (b) On separate pages p and q, both accesses remain local.

Example 15.4. Variables x and y reside on the same page p. A process on node 1 writes only x, a process on node 2 writes only y; each writes its variable every 10 ms, and the writes of the two processes alternate. The processes share no data, yet a page-based DSM sees two nodes writing page p (Figure 15.2a). Every write finds the current contents of p on the other node, so p is transferred every 5 ms, 200 times per second, which amounts to $200 \times 4\,\text{KiB} = 800\,\text{KiB}$ of network traffic per second. If a transfer takes 0.5 ms, each process waits 0.5 ms of every 10 ms, 5 % of its time. With x and y on different pages (Figure 15.2b), no page is transferred after the first access.

Caution False sharing also occurs on a single machine, when two variables share a cache line and the coherence protocol (Lesson 10) moves the line back and forth. A 4 KiB page holds 64 lines of 64 B, and each move of it is a network transfer rather than a cache miss.

False sharing is avoided by placing data that different nodes access in different units, which the programmer, the compiler or a memory allocator can do when the access pattern is known.

15.5 Access algorithms

Source P4L3, DSM design: access algorithm; Protić et al.; Nitzberg and Lo.

What a DSM must provide depends on the access algorithm of the applications that run on it. The access algorithm describes how many nodes read and write the same unit of memory at the same time; three algorithms are distinguished.

SRSW Under *single reader/single writer* (SRSW) access, only one node at a time reads and writes a unit. The main task of the DSM is to make memory that resides on other nodes accessible; since no two nodes use a unit at the same time, there are no concurrent accesses to coordinate.

MRSW Under *multiple readers/single writer* (MRSW) access, several nodes may read a unit at the same time, but only one node writes it. The DSM must ensure that every read returns the value of the most recent write.

MRMW Under *multiple readers/multiple writers* (MRMW) access, several nodes may both read and write a unit. In addition, the DSM must order the writes correctly, so that all

nodes observe them in the same order and reads return the most recent write.

Caution The access algorithm states what the applications do, not a discipline that the DSM enforces: under MRMW the DSM must cope with concurrent writers, not prevent them.

15.6 Migration and replication

The key performance metric of a DSM is access latency, the time that a read or write of shared memory takes. Since a remote access is orders of magnitude slower than a local one, even a small fraction of remote accesses raises the average sharply: one remote access in ten thousand already adds half as much again, and one in a hundred multiplies the average by about fifty.

Source P4L3, DSM design: migration vs. replication; Protić et al.; Nitzberg and Lo.

Example 15.5. A local memory access takes 100 ns, and an access to a page that must first be obtained from another node takes 0.5 ms. If a fraction f of the accesses are remote, the average access latency is

$$\bar{t} = (1 - f) \times 100\,\text{ns} + f \times 0.5\,\text{ms}. \tag{15.1}$$

For $f = 10^{-4}$, $\bar{t} \approx 100\,\text{ns} + 50\,\text{ns} = 150\,\text{ns}$, 1.5 times the latency of local memory; for $f = 10^{-2}$, $\bar{t} = 99\,\text{ns} + 5000\,\text{ns} \approx 5.1\,\mu\text{s}$, about 51 times.

The 0.5 ms of Example 15.5 is a conservative figure. A 4 KiB page is on the wire for 0.33 µs at 100 Gbit s^{-1}; with the fault, the request and the protocol overhead, a fetch over such an interconnect costs a few microseconds.

Low latency therefore requires locality: the memory that a process accesses should reside on the process's own node. Two techniques bring memory to the processes that access it (Figure 15.3).

Migration The DSM keeps a single copy of each unit and moves it to the node that accesses it. With one copy there is nothing to keep consistent. Migration suits SRSW access, where one node uses a unit for a stretch of accesses before another node needs it. It transfers the unit whenever a different node accesses it, which is expensive when nodes take turns for short periods, and it cannot serve several nodes at the same time.

Replication The DSM copies a unit to several nodes, each of which accesses its local replica; caching is a form of replication. Replication is the more general technique: it lets several nodes read, or even write, a unit at the same time, as MRSW and MRMW access require. Its price is consistency management (Section 15.7): when one replica is written, the others must be invalidated or updated. This overhead grows with the number of replicas, and the DSM can control it by limiting the number of replicas of each unit.

Tip Count messages: with n replicas a write costs $n - 1$ invalidations, and the next access on each invalidated node costs a fetch. Replication pays while the local reads it saves outnumber them.

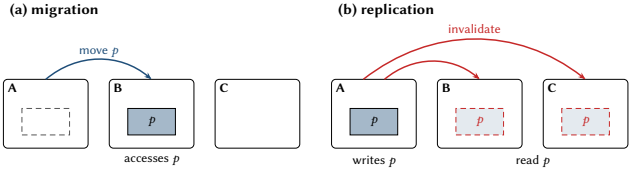


Figure 15.3: Bringing page p to the nodes that access it. (a) Migration moves the single copy. (b) Replication keeps copies on several nodes, which read p locally; a write by the owner A invalidates the replicas on B and C (dashed).

Example 15.6. Page p initially resides on node A, and nodes A, B and C access it in the sequence of Table 15.1. A page transfer takes 0.5 ms and an invalidation message 0.05 ms. Migration moves p on every access by a node that does not hold it: 7 transfers, 3.5 ms. With replication, B and C fetch replicas once and read them locally until A's write in step 6 invalidates them: 4 transfers and 2 invalidations, 2.1 ms. Had A made all eight accesses, neither technique would transfer anything. Had A and B instead alternated writes, every write would invalidate the other node's replica and every next access would fetch it again, so that replication would cost as many transfers as migration plus the invalidations.

An invalidation carries only the identifier of a page, a few tens of bytes, so its cost is that of a message exchange and not of the data.

Neither technique is best for every workload. Migration is appropriate when a unit is used by one node at a time for long stretches. Replication is necessary when several nodes access a unit concurrently, and it pays off when reads dominate. When replicated units are written often, the consistency management that every write triggers can cost more than the locality of the replicas saves.

Replication with invalidation on write is the write-invalidate strategy of Lesson 10 in software: a page instead of a cache line, a message instead of a bus transaction.

Table 15.1: Accesses to page p under migration and under replication with invalidation on write.

		Migration		Replication	
Step	Access	p at	Transfer	Copies at	Messages
0	(initial)	A	—	A	—
1	A writes	A	—	A	—
2	B reads	B	A→B	A, B	fetch from A
3	C reads	C	B→C	A, B, C	fetch from A
4	B reads	B	C→B	A, B, C	—
5	C reads	C	B→C	A, B, C	—
6	A writes	A	C→A	A	invalidate B, C
7	B reads	B	A→B	A, B	fetch from A
8	C reads	C	B→C	A, B, C	fetch from A
Total			7 transfers		4 fetches, 2 invalidations

Key point
DSM performance is governed by access latency, and low latency requires locality. Migration moves a single copy and suits SRSW access; replication keeps several copies, supports concurrent access, and pays for it with consistency management that grows with the number of replicas and the frequency of writes.

15.7 Consistency management

Source P4L3, DSM design: consistency management; Protić et al.; Nitzberg and Lo.

Once memory is replicated, the DSM must keep the replicas consistent. On a shared memory multiprocessor, software orders its accesses with locks, and the hardware keeps the copies of memory in the CPU caches coherent (Lesson 10). A DSM has no such hardware; its mechanisms resemble those with which a distributed file system keeps client caches consistent (Lesson 14):

Push When a replica is written, the DSM immediately pushes invalidations to the nodes that hold other replicas of the same unit. This approach is *proactive* and *eager*; it is also *pessimistic*, because it assumes that a stale replica would be read and cause harm, and removes stale replicas as soon as they arise. Instead of invalidations, the DSM may also push the new contents to the replicas: write-invalidate versus write-update, as for the caches of Lesson 10.

Caution Consistency management in a DSM is not cache coherence: it is carried out by software, at the granularity of a page or an object, and only at the points the consistency model requires.

Pull Nodes pull information about modifications from the owner of a unit, periodically or when they need it. This approach is *reactive* (on demand) and *lazy*; it is also *optimistic*, because it hopes that using a stale replica for a while does no harm, and it saves the messages of eager propagation.

When the DSM actually triggers these mechanisms—on every write, on every access, periodically, or only at points that the application designates—is determined by the consistency model that the DSM provides for the shared state (Section 15.10).

Push and pull are the DSM counterparts of the server-driven and client-driven cache consistency of Lesson 14.

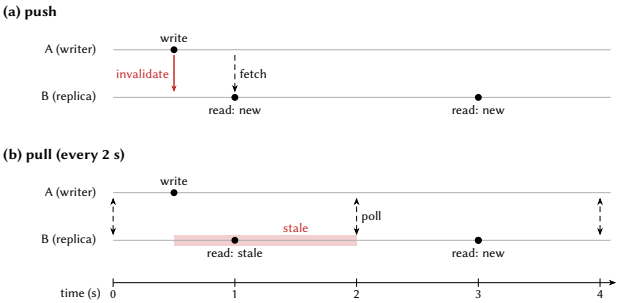


Figure 15.4: Node A writes page p at 0.5 s; node B holds a replica. (a) A pushes an invalidation, and B fetches the new contents on its next read. (b) B polls every 2 s, and its replica is stale until the next poll.

Example 15.7. Node A owns page p, and node B holds a replica. A writes p at $t = 0.5$ s, and B

reads p at 1 s and at 3 s (Figure 15.4). With push, A sends B an invalidation at 0.5 s; B's read at 1 s finds no valid replica and fetches the new contents. With pull, B asks A every 2 s whether p has changed. B's read at 1 s returns the stale contents; the poll at 2 s reveals the modification, and the read at 3 s returns the new contents. A replica may thus remain stale for up to one polling period. In exchange, pulling costs messages at a fixed rate: 10 replicas that poll every 2 s make 5 polls (10 messages) per second, whether p is written a hundred times per second or not at all. Pushing sends no message while p is not written, and at most one invalidation per replica for each write.

Tip Push costs messages in proportion to the writes, pull in proportion to time: compare the rate of writes with the rate of polling to see which is cheaper.

15.8 DSM architecture

Source P4L3, DSM architecture, indexing distributed state; Nitzberg and Lo.

We now consider a page-based DSM implemented by the operating system. Its organization follows from the preceding design choices.

- Every node contributes part of its main memory, a number of page frames, to the DSM. The contributed pages form a system-wide pool of shared memory, accessible from all nodes.
- Every node keeps local copies of pages that its processes access, a cache of replicas, to lower the access latency.
- Management is distributed as well: every node is responsible for part of the shared memory.

For each page the DSM distinguishes the following roles.

Home node The node that manages the page, also called its *manager*. It maintains the state of the page: which nodes access it, whether it has been modified, whether caching of it is enabled, whether it is locked, and where its replicas reside. The address of a page names the node that manages it initially; management can be handed to another node without changing the address (Section 15.8.1).

The home node plays the part that the server plays in a distributed file system (Lesson 14), except that every node is the home node of a portion of the memory.

Owner The node that currently holds the authoritative copy of the page and controls its updates, for example the node that most recently obtained the right to write it. Ownership changes as the page is accessed, so the owner need not be the home node; the home node keeps track of the current owner.

Explicit replicas Copies that the DSM creates deliberately, rather than as a side effect of access, for load balancing, for performance, or for reliability; in datacenters, for instance, data are commonly kept in three copies. The home node controls where these replicas are placed and how they are managed.

Three is the default replication factor of HDFS, which places two replicas in one rack and the third in another rack (Hadoop documentation).

15.8.1 Indexing distributed state

To serve an access, a node must find the metadata of the page, which is maintained by the page's manager. A DSM organizes this metadata as follows (Figure 15.5).

- Every page is identified by an address composed of a node ID and a physical frame number (PFN, Lesson 8): the node ID names the node that contributes the frame and manages the page initially, and the PFN the frame on that node.
- A global map from page ID to the ID of the manager node is replicated on every node, so that any node finds the manager of any page locally, without exchanging messages.
- The metadata of the pages themselves are partitioned: each manager keeps the metadata of the pages it manages.

Rather than deriving the manager directly from bits of the page ID, the global map can be implemented as a *mapping table*: part of the page ID serves as an index into the table, and the entry names the manager node. The manager of a set of pages can then be changed, for load balancing or after a failure, by updating a table entry on all nodes, while the page IDs, and every reference that uses them, remain unchanged. Once the table is interposed, the node-ID field is no more than an index into it: the manager of a page is the node that the entry names, which need no longer be the node the field denotes.

Replicating the map is affordable because it is small: the mapping table indexed by the 8-bit node-ID field of the example below has 256 entries. The per-page metadata, far more numerous, stay partitioned.

Tip The mapping table is to the manager of a page what a page table is to a frame: a level of indirection that lets what a name refers to change while the name stays the same.

Example 15.8. A DSM identifies each page by a 32-bit address: an 8-bit node ID followed by a 24-bit PFN, so that each of up to 256 nodes can contribute up to 2^{24} frames of 4 KiB, 64 GiB of memory. Page `0x050012A3` is frame `0x0012A3` of node 5, which manages it initially. The mapping table is indexed by the node-ID field and maps every index i to node i to begin with. When node 5 becomes overloaded, the management of its pages is handed to node 9 by setting entry 5 to 9 on all nodes and moving the page metadata from node 5 to node 9 (Figure 15.5); the frames themselves stay on node 5, and no page ID changes. Had the manager been taken directly from the node ID field, every one of these pages would have needed a new ID, and

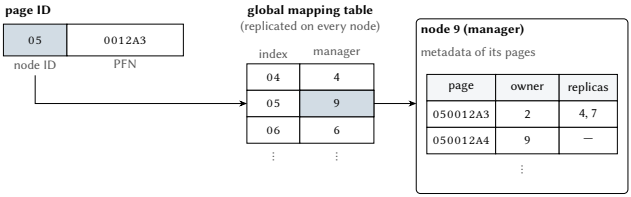


Figure 15.5: Indexing distributed state. The node-ID field of the page ID indexes the global mapping table, which names the manager of the page; the manager keeps the metadata of its pages. Entry 05 has been changed to node 9, which therefore manages the pages whose node-ID field is 05, although their frames reside on node 5.

every reference to it would have had to be updated.

15.9 Implementing DSM

Source P4L3, implementing DSM; Nitzberg and Lo; Tanenbaum and Bos ch. 8.

A DSM must intercept accesses to the shared memory for two reasons. When a process accesses memory that is not present on its node, the DSM must send messages to the node that holds it, requesting access. When a process updates memory of which other nodes hold replicas, the DSM must detect the update and perform the consistency operations it requires. At the same time, accesses to memory that is present locally and needs no coordination must involve neither the DSM nor the operating system: an intervention by software in every memory access would forfeit the performance that locality is meant to provide.

A page-based DSM resolves this conflict with the hardware support of the MMU (Figure 15.6). The MMU traps into the operating system with a page fault (Lesson 8) whenever a virtual address has no valid mapping, or the mapping does not permit the kind of access attempted; every other access completes in hardware. The DSM sets up the mappings so that exactly the accesses it must intercept fault.

Caution Such a page fault is not served from swap space: the DSM fetches the page from the memory of another node.

- A page that is not present on the node has no valid mapping. An access to it traps, and the operating system passes the fault to the DSM, which requests the page from its owner and installs a mapping for the received copy before it resumes the process: read-only for a read; for a write, the DSM performs the consistency operations below as well and maps the page read-write, so that a write to an absent page costs one fault and not two.
- A page of which the node holds a replica is mapped read-only through its protection bits (Lesson 8). Reads proceed without traps. A write traps, and the DSM performs the consistency operations—it invalidates the other replicas and takes over ownership—before it enables writing and resumes the process.[1]

Other information recorded by the MMU is useful as well. The dirty bit of a page shows whether the page has been modified, and hence whether modifications must be propagated; the access bit shows whether a local replica has been used recently, which helps to decide which replicas to evict when memory runs short, as in page replacement (Lesson 8). To propagate only what changed, the DSM may keep a copy (twin) of a write-protected page and send the difference (diff) at the consistency operation (as in TreadMarks [61]). An object-based DSM intercepts accesses in the language runtime instead, which mediates the program's operations on shared objects.

[1] IVY [60], the first page-based DSM, works in this way and provides sequential consistency under MRSW access; TreadMarks [61] later implemented a DSM entirely in user space, on unmodified UNIX systems, with a weaker model. These MMU operations are not free: Appel and Li [62] measured about 1200 µs of fault and re-protection overhead per page on the machines of 1991.

Tip Per node the mapping of a page is a three-state machine—none, RO, RW—and only the transitions fault: a read takes none to RO, a write none or RO to RW; another node's access downgrades it to RO or to none.

Example 15.9. Page p initially resides only on node 1, its owner, which maps it read-write. Table 15.2 traces accesses by three nodes under MRSW access with invalidation on write. When node 2 obtains a replica in step 1, node 1's mapping is downgraded to read-only, so that a later write by node 1 would fault and invalidate the replica. Of the seven accesses, four fault; the reads in steps 2 and 3 and the write in step 5 complete in hardware, because the mappings already permit them.

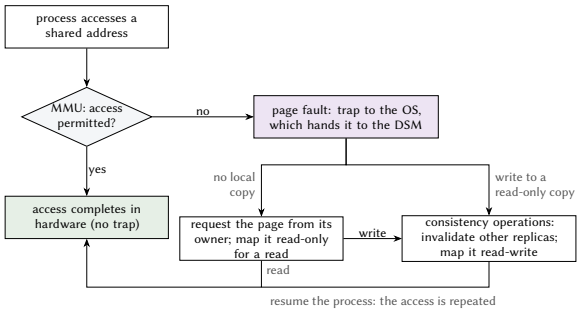


Figure 15.6: Intercepting accesses in a page-based DSM. Accesses permitted by the current mapping never leave the hardware; the others fault, and the DSM fetches the page or performs consistency operations before the access is repeated.

Table 15.2: Accesses to page p and the faults they cause. The last column shows the mappings of p on nodes 1/2/3 after each step (RW read-write, RO read-only, — none).

Step	Access	Fault	DSM action	Maps
0	(initial)	—	—	RW/—/—
1	node 2 reads	page fault	fetch p from owner node 1; downgrade node 1 to RO	RO/RO/—
2	node 2 reads	none	—	RO/RO/—
3	node 1 reads	none	—	RO/RO/—
4	node 2 writes	protection	invalidate node 1's copy; node 2 becomes owner	—/RW/—
5	node 2 writes	none	—	—/RW/—
6	node 3 reads	page fault	fetch p from owner node 2; downgrade node 2 to RO	—/RO/RO
7	node 2 writes	protection	invalidate node 3's copy	—/RW/—

Key point
A page-based DSM places the shared pages in memory contributed by all nodes, finds each page's manager through a replicated global map, and uses the MMU to intercept exactly the accesses that need it: accesses to pages that are not present, and writes to replicated pages. All other accesses proceed at hardware speed.

15.10 Consistency models

Source P4L3, consistency model; strict, sequential, causal and weak consistency; Protić et al.; Tanenbaum and van Steen [59] ch. 7.

The mechanisms of Section 15.7 can be triggered at different times, and the resulting guarantees differ. For a distributed file system these guarantees are the file sharing semantics of Lesson 14; for memory, and for shared state in general, they are stated by a consistency model.

Definition 15.10 (Consistency model). A *consistency model* is an agreement between memory, or any shared state, and the software layers above it: the memory guarantees to behave correctly—with respect to the ordering of accesses and to the propagation and visibility of updates—provided that the software follows specific rules.

File sharing semantics are thus consistency models for files. To describe consistency models for memory we write $R_m(v)$ for a read that returns the value v from memory location m, and $W_m(v)$ for a write of v to m. Operations are drawn on one timeline per process, with time increasing to the right; all locations initially hold 0.

In the timelines m_a and m_b are two locations of the shared address space, and $t_1 < t_2 < \ldots$ are instants of real time as an outside observer sees them.

15.10.1 Strict consistency

Definition 15.11 (Strict consistency). Under *strict consistency* every update is visible everywhere immediately: every read, on any node, returns the value of the write to that location that is the most recent in real time, and all nodes observe the updates in the order in which they were made.

Strict consistency is what programmers intuitively expect of memory, but it is a theoretical model. Even on a single shared memory multiprocessor, concurrent updates on different CPUs take effect in no guaranteed order unless the software adds locking and synchronization. In a distributed system, updates travel over a network with non-zero latency, messages may be reordered or lost, and no global clock defines which of two writes on different nodes is the more recent; strict consistency cannot be guaranteed.

15.10.2 Sequential consistency

Definition 15.12 (Sequential consistency). Under *sequential consistency* the result of every execution is the same as if the operations of all processes were executed in some sequential order, in which the operations of each process appear in the order in which the process issued them, every read returns the value of the most recent write to its location in that order, and all processes observe this same order [63].

Updates from different processes may thus be interleaved arbitrarily, and the interleaving need not match the real-time order, but all processes must see the same interleaving.

Caution A sequentially consistent execution need not agree with real time: a read may return a value older than the write that most recently preceded it in real time, provided that every process observes the same order.

15.10.3 Causal consistency

A write *causally depends* on an earlier write if the same process issued the earlier write, or if the process that issued it had observed the earlier write before—for instance by reading the value it wrote—directly or through a chain of such dependencies. Writes of which neither depends on the other are *concurrent*.

Definition 15.13 (Causal consistency). Under *causal consistency* all processes observe causally related writes in their causal order, and the operations of each process in the order in which the process issued them; concurrent writes may be observed in different orders by different processes.

Vector timestamps are a common way to record these dependencies (Tanenbaum and van Steen ch. 6).

To provide causal consistency, the DSM must detect possible causal relationships, that is, record which updates a process had observed before each of its writes. In exchange, it needs to coordinate only the order of causally related updates.

15.10.4 Weak consistency

Weak consistency introduces *synchronization points*: operations, such as `sync`, that the DSM makes available to the software above it.

Definition 15.14 (Weak consistency). Under *weak consistency* a node is guaranteed to observe the updates that another node made before that node's synchronization point only after the observing node itself performs a later synchronization operation; between synchronization points no guarantee is given about which updates a node observes.

A DSM may offer a single synchronization operation for the entire shared state, or separate operations for subsets of the state, such as individual pages or objects, so that an application synchronizes only the state it needs. It may also split the operation in two: an *acquire*, performed before a process uses shared state, which makes the updates of the other processes visible to it, and a *release*, performed after it has finished updating, which propagates its own updates. Weak consistency limits data movement and coordination to the synchronization points, at which the updates made since the previous point can be propagated together. In exchange, the DSM must maintain extra state to record and aggregate these updates, and between synchronization points processes may observe stale data.

Session semantics (Lesson 14) is the file counterpart of weak consistency: `open` and `close` are the synchronization points.

15.10.5 Comparing the models

Strict, sequential and causal consistency form a hierarchy: every strictly consistent execution is sequentially consistent, and every sequentially consistent execution is causally consistent. A weaker model admits more executions and therefore requires less coordination, but it gives the applications fewer guarantees.

Caution Real hardware is not sequentially consistent either: x86 lets a load pass an earlier store to another location (total store order), ARM and POWER reorder more, and fences take the part of synchronization points (Intel SDM vol. 3A, memory ordering).

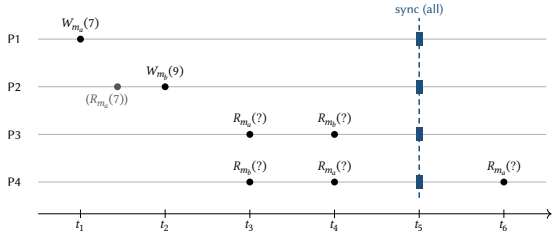


Figure 15.7: Operations of Example 15.15. P2's read of m_a (grey) occurs only in the variant in which P2's write depends on P1's. The dashed line marks a synchronization operation by every process.

Example 15.15. In Figure 15.7, P1 writes 7 to m_a at t_1 and P2 writes 9 to m_b at t_2. P3 and P4 read both locations at t_3 and t_4, in opposite orders. Table 15.3 lists three possible outcomes of these reads and the models they satisfy.

- Outcome (a) is strictly consistent: every read returns the most recent write in real time.
- Outcome (b) is not: P4 reads 0 from m_b at t_3, after 9 was written at t_2. It is sequentially consistent, because the order $R_{m_a}(0)$ by P3, $R_{m_b}(0)$ by P4, $W_{m_a}(7)$, $W_{m_b}(9)$, $R_{m_b}(9)$ by P3, $R_{m_a}(7)$ by P4 explains every read and preserves the order of each process's operations.
- Outcome (c) is not sequentially consistent: P3 observes $W_{m_a}(7)$ before $W_{m_b}(9)$, P4 observes them the other way round, and no single order explains both. If the two writes are concurrent, (c) is causally consistent. If P2 read 7 from m_a before writing m_b, however, $W_{m_b}(9)$ causally depends on $W_{m_a}(7)$, and P4, which reads 9 from m_b and afterwards 0 from m_a, observes the effect before its cause.

If all four processes execute `sync` at t_5, all three outcomes are weakly consistent, because no guarantee applies before t_5; the read of m_a by P4 at t_6, however, must return 7.

Tip To test sequential consistency, look for one merged list of the operations that keeps each process's own order and explains every read.

Caution A model constrains the outcomes; it does not determine them. A DSM that provides sequential consistency may produce (a) on one run and (b) on the next, and a weakly consistent one may produce any of the three.

Table 15.3: Outcomes of the reads at t_3 and t_4 in Example 15.15. The causal column gives the result for concurrent writes / for P2 reading m_a before writing m_b.

	P3	P4	Strict	Sequential	Causal	Weak
(a)	$R_{m_a}(7)$, $R_{m_b}(9)$	$R_{m_b}(9)$, $R_{m_a}(7)$	yes	yes	yes / yes	yes
(b)	$R_{m_a}(0)$, $R_{m_b}(9)$	$R_{m_b}(0)$, $R_{m_a}(7)$	no	yes	yes / yes	yes
(c)	$R_{m_a}(7)$, $R_{m_b}(0)$	$R_{m_b}(9)$, $R_{m_a}(0)$	no	no	yes / no	yes

Key point — Lesson summary

Distributed shared memory is a peer distributed application in which every node contributes memory, serves accesses to it, and takes part in consistency protocols, so that processes on many machines share one address space. Its design is determined by the granularity of sharing (pages or objects, with the risk of false sharing), the access algorithm (SRSW, MRSW, MRMW), the choice between migration and replication to achieve locality, and consistency management by pushing invalidations or pulling modifications. A page-based DSM finds each page's manager through a replicated mapping table and relies on MMU faults to intercept only the accesses that need coordination. Consistency models state what processes observe: strict consistency is unattainable in practice; sequential consistency requires one common order of all operations; causal consistency orders only causally related writes; and weak consistency guarantees visibility only at synchronization points.

Datacenter Technologies

Lecture	P4L4
Textbook	Silberschatz et al. [2] ch. 1; OSTEP [1] ch. 48; Armbrust et al. [64]; Mell and Grance [65]
Keywords	Internet service, multi-tier architecture, scale-out, load balancer, homogeneous architecture, heterogeneous architecture, cloud computing, elasticity, law of large numbers, utility computing, IaaS, PaaS, SaaS, multi-tenancy

The preceding lessons presented mechanisms with which processes on different machines cooperate: remote procedure calls, distributed file systems and, in Lesson 15, distributed shared memory. Together with the threads, servers, scheduling and virtualization of the earlier lessons, these mechanisms are deployed on the largest scale in datacenters, which run the services that are used over the Internet. This final lesson gives a brief, high-level overview of datacenter technologies: how Internet services are structured and scaled out over many machines, what cloud computing is, why it works and which models it offers, what it requires of the underlying systems, and which technologies make it possible.

16.1 Internet services

Many applications in everyday use do not run on the machines of their users. They run on machines in a *datacenter*, a facility that houses a large number of networked computers together with their storage, power supply and cooling, and operates them to provide computing services. The users reach these applications over the Internet.

Source P4L4, lesson preview, Internet services.

Definition 16.1 (Internet service). An *Internet service* is any service that is provided to its users over the Internet through a web interface.

Web mail, online shops, search engines and social networks are Internet services. Most Internet services consist of three components (Figure 16.1):

Presentation interacts with the users. It receives their requests, returns web pages, and provides the *static content*, such as page layouts and images, which is the same for every request.

Business logic implements the functionality of the service. It processes each request and generates the *dynamic content*, such as the results of a search or the contents of a shopping cart.

Database stores the data of the service, such as its users, products and orders, and retrieves them for the business logic.

A common open-source realization is the LAMP stack: Linux, Apache (presentation), PHP (logic), MySQL (database).

Definition 16.2 (Multi-tier architecture). A service that is structured as a sequence of components, each of which uses the services of the next, has a *multi-tier architecture*. Each component is a *tier*; the three components above form a three-tier architecture.

The tiers are logical components. They need not be separate processes, and separate processes need not run on separate machines. Many open-source and proprietary middleware products—web servers, application servers, database servers—implement individual tiers and the interaction between them, and they can be combined in different configurations. When the tiers run as separate processes, the processes communicate through some form of IPC: message passing or shared memory on the same machine (Lesson 9), and RPC or RMI across machines (Lesson 13).

A tier boundary is also a cost boundary: the same call costs nanoseconds within one process, a few microseconds through local IPC (Lesson 9), and about 0.5 ms by RPC to another machine in the datacenter—Dean's "numbers everyone should know" [66].

16.2 Scale-out architectures

A single machine serves only a limited number of requests per second, however efficiently its server software is designed (Lesson 6). A service that must handle more requests therefore runs as many processes on many machines, the *nodes* of the service.

Source P4L4, Internet service architectures, homogeneous architectures, heterogeneous architectures.

Definition 16.3 (Scale-out). An architecture that increases the capacity of a service by adding nodes, rather than by replacing a node with a more powerful machine (*scale-up*), is a *scale-out* architecture.

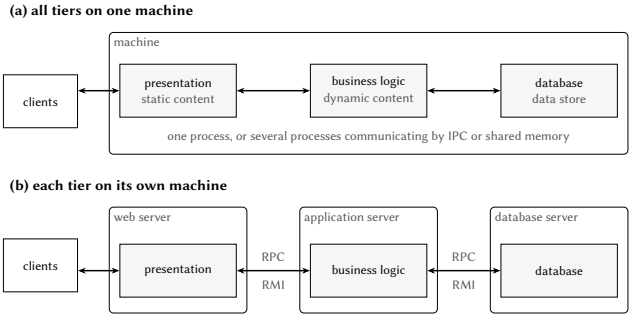


Figure 16.1: A three-tier Internet service. (a) All tiers run on one machine, in one process or in several processes. (b) Every tier runs on its own machine, and the tiers communicate by RPC or RMI.

Scale-out architectures commonly apply the boss-workers pattern of Lesson 3 to machines instead of threads: a front-end node receives all requests and passes each of them to one of the nodes that process requests.

Definition 16.4 (Load balancer). A *load balancer* is a front end that distributes the incoming requests of a service among the nodes that process them.

Typical dispatch policies are round robin and least connections, both offered by NGINX and HAProxy (their documentation).

Just as the workers of the boss-workers pattern are either all equal or specialized, the nodes of a scale-out architecture can be organized in two ways (Figure 16.2):

All equal Every node can execute every step of the processing of every request. The nodes are functionally homogeneous.

Specialized Each node executes only some steps of the processing, or only requests of some types. The nodes are functionally heterogeneous.

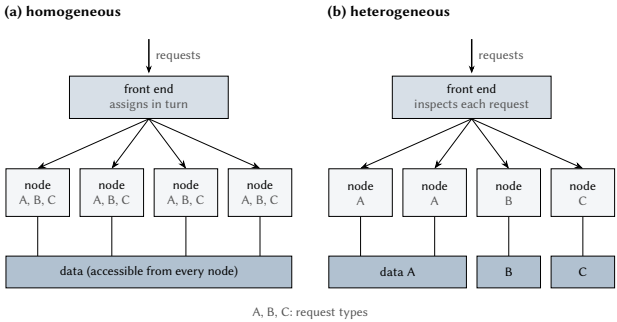


Figure 16.2: Scale-out architectures. (a) Every node processes requests of every type, and every node can access all data. (b) Nodes are specialized for request types, and the front end must inspect each request to choose a node.

16.2.1 Homogeneous architectures

Definition 16.5 (Homogeneous architecture). In a *homogeneous architecture* all nodes are functionally identical: each of them can perform every processing step for every request.

Which node holds which data is usually decided by hashing a key—a user, a session, a file name. Consistent hashing is the common choice: adding the Nth node then moves only about $1/N$ of the keys [67].

The front end of a homogeneous architecture is simple. It need not examine the requests; it can assign them to the nodes in turn, in round-robin order, or to the node with the least load. Functional homogeneity does not require every node to store all data. It requires only that every node can *access* all data, for instance through a distributed file system or a shared database tier, which may itself be replicated or partitioned (Lesson 14). To increase the capacity, nodes are simply added.

The price is that the service benefits little from caching. The front end does not know which node has recently processed a similar request and holds the data it needs in a hot cache. A request that one node could serve from its cache is therefore likely to be sent to a node that must first fetch the data—the lack of locality that Lesson 3 noted for equal workers.

16.2.2 Heterogeneous architectures

Definition 16.6 (Heterogeneous architecture). In a *heterogeneous architecture* the nodes are functionally specialized: different nodes perform different processing steps or serve different types of requests.

Data then need not be uniformly accessible from all nodes; each node needs only the data of its own processing steps or request types. The advantage is locality. All requests of a given type reach the same few nodes, which keep the code and data these requests need in hot caches, as the specialized threads of Lesson 3 do. The front end, however, becomes more complex: it must inspect every request, for example parse its URL, to determine which nodes can process it. The management of the service becomes more complex as well.

A front end that only forwards connections needs to look no further than the transport headers; one that chooses a node by URL or cookie must terminate the connection and parse the request itself.

16.2.3 Scaling and management

The two architectures differ most when the load grows. To scale a homogeneous architecture, it suffices to add nodes, together with the processes, machines and access to storage that they need. Every new node takes its share of every type of request, and no knowledge of which requests cause the load is required. To scale a heterogeneous architecture, the operator must know which types of requests are in demand and add nodes of these types, which requires profiling the workload. As the demand for the different request types changes over time, the mix of nodes must follow it. Otherwise the nodes of a popular type become *hot spots* with long queues of pending requests, while the nodes of other types are idle. Table 16.1 summarizes the comparison.

Scale-out and scale-up are also called horizontal and vertical scaling. Either raises the throughput of the service; neither shortens the processing of a single request.

The same imbalance arises inside one machine in Lesson 7, where load balancing migrates threads between per-CPU runqueues. Specialized nodes cannot take over each other's work, because they lack the data.

Table 16.1: Homogeneous and heterogeneous scale-out architectures.

	Homogeneous	Heterogeneous
Nodes	functionally identical	specialized by processing step or request type
Front end	simple; need not inspect requests	must inspect every request
Data	accessible from every node	needed only on some nodes
Caching and locality	little benefit	requests of a type reach the same nodes
Scaling	add nodes	profile the demand; add nodes of the types needed
Change in request mix	absorbed without intervention	hot spots unless nodes are reassigned

Example 16.7. An online shop receives two types of requests. On a node of a homogeneous architecture, a search takes 5 ms of processing and a checkout 20 ms. A load of 1200 searches and 200 checkouts per second requires $1200 \times 0.005 + 200 \times 0.020 = 6 + 4 = 10$ seconds of processing per second, the capacity of ten nodes. When the mix changes to 600 searches and 350 checkouts per second, the requirement is $3 + 7 = 10$ again: the same ten nodes suffice, and no intervention is required.

In a heterogeneous architecture, suppose that hot caches let the specialized nodes process requests in 20 % less time, 4 ms per search and 16 ms per checkout. The original load needs $\lceil 1200 \times 0.004 \rceil = 5$ search nodes and $\lceil 200 \times 0.016 \rceil = 4$ checkout nodes, nine in total. After the change, $\lceil 2.4 \rceil = 3$ search nodes and $\lceil 5.6 \rceil = 6$ checkout nodes are needed—still nine, but in a different mix. Until the operator moves two nodes from search to checkout, the four checkout nodes complete only $4/0.016 = 250$ checkouts per second, the queue of pending checkouts grows by 100 requests every second, and the search nodes are only 48 % busy, 2.6 nodes' worth of capacity idle.

Tip This is Little's law of Lesson 6: the number of nodes busy at once is the arrival rate times the service time, so the load in nodes is $\sum_i \lambda_i t_i$ whatever the mix of requests.

Key point
An Internet service is organized in tiers and scaled out over many nodes behind a load balancer. Homogeneous nodes keep the front end and scaling simple but gain little from caching; heterogeneous nodes gain locality at the cost of a front end that inspects requests and of management that must follow the mix of requests.

16.3 The motivation for cloud computing

Source P4L4, cloud computing poster child: Animoto, cloud computing requirements; Armbrust et al. [64].

According to the lecture, Amazon, the online retailer, had provisioned its hardware for the peak load of the holiday shopping season, and for the rest of the year much of this hardware was idle. Amazon opened access to it through web-based APIs, so that third parties could run their own workloads on Amazon's hardware for a fee. The result was Amazon Web Services (AWS). Its Elastic Compute Cloud (EC2), launched in 2006, rents out *compute instances*: virtual machines (Lesson 12) that customers start and stop through the API and that were paid for by the hour at the time (per second, with a 60 s minimum, since 2017).

Caution The story is disputed: Werner Vogels, Amazon's CTO, has publicly called the excess-capacity story a myth; AWS grew out of the shared infrastructure Amazon had built for its own teams.

Animoto, a service that produces videos from users' photographs, rented its compute instances from EC2. In April 2008 Animoto became available to the users of Facebook, and its demand surged: within three days the number of servers it used grew from 50 to 3500, after which its traffic fell to well below the peak [64]. Growth by almost two orders of magnitude within days is impossible when machines must be bought, installed and configured; with rented resources it required only calls to an API.

EC2 opened as a limited public beta in August 2006 and became generally available in October 2008.

16.3.1 Provisioning for variable demand

Traditionally, a service buys and configures its own resources. Their capacity is determined in advance from the expected demand, usually from its expected peak, and it can later be increased only in large steps, each delayed by a purchase. Since demand varies over time, a fixed capacity is wrong most of the time (Figure 16.3a):

- while the demand is below the capacity, the idle resources are wasted: they have been paid for but are not used;
- when the demand exceeds the capacity, requests are dropped, and the opportunity to serve these users, and the revenue they would have brought, is lost.

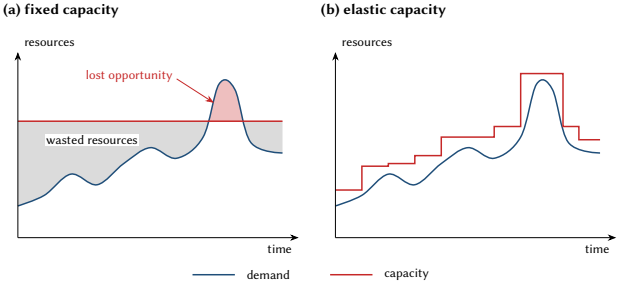


Figure 16.3: Capacity and demand over time. (a) A fixed capacity wastes resources while demand is low and loses requests during a surge. (b) An elastic capacity follows the demand.

Ideally, the capacity of a service would instead follow its demand (Figure 16.3b):

- capacity scales elastically with the demand, both up and down;
- scaling is instantaneous;
- the cost is proportional to the demand, and thus to the revenue opportunity;
- all of this happens automatically, without special expertise;
- the resources can be accessed at any time, from anywhere.

Caution "Instantaneous" is the ideal, not the mechanism: a virtual machine needs tens of seconds to boot and a container about a second, and an autoscaler that watches one-minute metrics waits for several of them, so capacity follows demand with a lag of minutes.

Definition 16.8 (Elasticity). The ability of a system to acquire and release resources quickly and automatically as its demand rises and falls, so that its capacity follows the demand, is called *elasticity*.

The price of this ideal is that a service no longer owns the resources on which it runs, and

its data reside on machines that others operate.

Example 16.9. The demand of a service is 20 servers during 16 hours of every day and 100 servers during the remaining 8 hours, $20 \times 16 + 100 \times 8 = 1120$ server-hours per day. An owned server costs c per hour, whether it is used or not. Provisioned for the peak, the service pays for $100 \times 24 = 2400$ server-hours, $2400c$ per day, and $1 - 1120/2400 \approx 53\,\%$ of them are wasted. Provisioned for the average demand of $1120/24 \approx 47$ servers, it still wastes $27 \times 16 = 432$ server-hours while the demand is low, and during the peak it drops $53/100 = 53\,\%$ of the requests. With elastic capacity rented at a 50 % higher price, $1.5c$ per server-hour, the service pays $1120 \times 1.5c = 1680c$ per day, 30 % less than with peak provisioning, and drops no requests.

Caution Elasticity is not scalability: a scalable system can grow; an elastic one also shrinks again, automatically.

Caution Renting capacity does not remove the provisioning decision; it only shortens its horizon. How many instances to run must still be decided, now every few minutes rather than every few years.

16.4 Cloud computing

Cloud computing approximates the ideal described above. Its essential elements are the following.

Shared resources A provider offers infrastructure—compute, storage and network—as well as software and services, and shares them among many customers.

APIs for access and configuration Customers request, configure and release resources through web-based APIs, through libraries for programming languages, or with command-line tools.

Billing and accounting Usage is metered and billed. Many pricing models coexist, for example on-demand prices, lower prices for capacity reserved for a longer period, and *spot* prices for spare capacity that vary with supply and demand, so that the provider in effect runs a marketplace. Resources are offered in discrete sizes, such as small, medium and extra-large instances with different numbers of CPUs and amounts of memory.

Management by the provider The provider operates the datacenters, the hardware, and the software that implements the services.

Source P4L4, cloud computing overview, why cloud computing works, cloud computing definitions; Armbrust et al. [64]; Mell and Grance [65]; Silberschatz et al. ch. 1.

Definition 16.10 (Cloud computing). The provision of computing resources as services over a network, from resources that are shared among many customers, acquired and released on demand through APIs, billed according to use and managed by the provider, is called *cloud computing*.

Real prices, for scale: EC2's first instance type, `m1.small`—one virtual core and 1.7 GB of memory—cost $0.10 per hour on demand [64], and AWS advertises spot capacity at discounts of up to 90 %.

Note 16.11. Published definitions agree with these elements. The NIST definition [65] names five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service. Armbrust et al. [64] use the term for both the applications delivered as services over the Internet and the hardware and systems software in the datacenters that provide these services; the latter they call a *cloud*. They exclude private clouds from cloud computing, and they avoid the "X as a service" labels for the layers below the application, such as IaaS and PaaS (Section 16.5).

16.4.1 Why cloud computing works

A provider must own enough resources for the demand of all its customers, and still charge them less than they would spend on resources of their own. Two principles make this possible.

Definition 16.12 (Law of large numbers). By the *law of large numbers*, the average of N independent random quantities with the same distribution approaches their expected value as N grows.

The resource needs of a single customer vary widely over time, but different customers reach their peaks at different times, and the demand averaged over many customers is roughly constant. If the demands of N customers are independent, each with mean μ and standard deviation σ, their sum has mean $N\mu$ and standard deviation $\sigma\sqrt{N}$, so that its variation relative to its mean, $\sigma/(\mu\sqrt{N})$, shrinks as N grows. A provider can therefore provision for the aggregate demand, which is far smaller than the sum of the individual peaks.

Tip The relative variation falls only as $1/\sqrt{N}$: ten times as many independent customers reduce it by a factor of about 3, a hundred times by a factor of 10.

Example 16.13. A provider serves 100 customers. Each customer needs 10 servers most of the time and 100 servers during its peaks, which occupy 10 % of the time and occur independently of the peaks of other customers. Provisioning for every customer's peak would take $100 \times 100 = 10\,000$ servers. The number K of customers at their peak at a given moment, however, has mean $100 \times 0.1 = 10$ and standard deviation $\sqrt{100 \times 0.1 \times 0.9} = 3$, and the total demand, $1000 + 90K$ servers, averages 1900. With 2710 servers, enough for $K \leq 19$, three standard deviations above the mean, the demand exceeds the capacity with a probability of only about 0.2 %. The provider

Caution Independence is the assumption that does the work. Real demands share a daily cycle, holidays and live events, so peaks do arrive together; that is what reserve capacity is for, and why spot instances are reclaimed at two minutes' notice.

needs fewer than 30 % of the servers that its customers would need to own.

The second principle is that of *economies of scale*: the unit cost of providing a resource falls as the quantity provided grows. A provider buys hardware, power and network capacity in bulk, spreads the fixed costs of a datacenter over many machines, and lets each administrator manage many machines.

16.4.2 Computing as a utility

The vision of cloud computing is older than its technology. In 1961, John McCarthy suggested that computing might one day be organized as a public utility, as the telephone system is, and that such a computer utility could become the basis of an important new industry.

Definition 16.14 (Utility computing). In *utility computing*, computing resources are provided in the way electricity or telephone service is: as a metered, fungible commodity—any unit is as good as any other—that customers consume on demand and pay for according to use.

In the terminology of Armbrust et al. [64], utility computing is what a cloud sells when it is made available to the general public on a pay-as-you-go basis. Current clouds realize the vision only in part, because computing has not become a fully fungible utility. The limitations are these:

API lock-in Every provider has its own APIs, and an application written for one provider cannot move to another without changes.

Hardware dependence Applications may depend on particular hardware, such as specific processors or accelerators, which not every provider or instance type offers.

Latency The resources reside in a remote datacenter, and every interaction with them incurs network latency.

Privacy and security Data and computations reside on machines that the customer neither owns nor controls and that are shared with other customers.

> **Key point**
> Cloud computing offers shared, provider-managed resources through APIs, elastically and billed by use. It works because the aggregate demand of many customers varies far less than the demand of each of them (law of large numbers) and because unit costs fall with scale (economies of scale). It realizes the vision of computing as a utility only in part.

16.5 Deployment and service models

Source P4L4, cloud deployment models, cloud service models; Mell and Grance [65]; Silberschatz et al. ch. 1.

Clouds differ in who may use them and in what they provide. The *deployment models* describe who uses a cloud.

Public cloud A *public cloud* is operated by a provider for third-party customers, its *tenants*, who share its resources.

Private cloud In a *private cloud* an organization applies cloud technology, such as virtualization, APIs and automated provisioning, to its own infrastructure, for its own applications. By the NIST definition, the infrastructure may also be operated by a third party and be located off premises.

Hybrid cloud A *hybrid cloud* combines two or more distinct clouds (private, community or public), typically a private cloud with a public one. An organization runs its applications in its private cloud and uses the public cloud for failover, to absorb spikes in demand that exceed its own capacity, or for testing.

Community cloud A *community cloud* is provisioned for the exclusive use of a specific community of organizations with shared concerns, such as mission, security requirements, policy or compliance; government agencies or research institutions are examples. It may be owned and operated by one or more of these organizations, by a third party, or by both, and be located on or off premises.

Caution The lecture describes a community cloud as a public cloud restricted to certain users; NIST [65] does not treat it as a kind of public cloud.

The *service models* describe how much of the stack of hardware and software the provider manages (Figure 16.4). In an on-premises installation the organization manages the entire stack itself, from the network, storage and servers through virtualization, the operating system, middleware and runtime to its data and applications.

With *infrastructure as a service* (IaaS) the provider manages the physical infrastructure, that is, the network, storage and servers, and its virtualization. Customers rent virtual machines

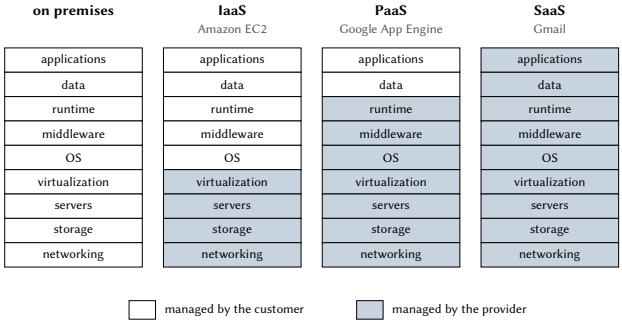


Figure 16.4: Cloud service models, with an example of each. The shaded layers are managed by the provider, the others by the customer.

and install and manage the operating system and everything above it. Amazon EC2 is an example.

With *platform as a service* (PaaS) the provider also manages the operating system, the middleware and the runtime environment. Customers supply only their applications, written against the interfaces of the platform, and their data, and the platform deploys and runs the applications. Google App Engine is an example.

With *software as a service* (SaaS) the provider manages the entire stack, the application included, and customers merely use the application, as the users of Gmail do.

NIST SP 800-145 [65] lists exactly three service models (IaaS, PaaS, SaaS) and four deployment models (public, private, community, hybrid).

Example 16.15. The operator of the three-tier online shop of Section 16.1 can run it on IaaS by renting virtual machines, installing an operating system, a web server, an application server and a database server on them, and managing all this software, its updates and its scaling. On PaaS the operator uploads only the code of the business logic and the static content; the platform runs them, provides a managed database, and adds instances as the load grows. The employees of the shop may at the same time read their e-mail through a SaaS offering, for which they manage nothing but their accounts and messages.

16.6 Requirements for the cloud

From the provider's point of view, cloud computing places a number of requirements on the underlying technology.

Source P4L4, requirements for the cloud, cloud failure probability; OSTEP ch. 48.

1. **Fungible resources.** Resources must be interchangeable and easy to repurpose for customers with different requirements, such as different operating systems and software stacks. Without fungibility every customer would need dedicated resources, and the economic opportunity of the cloud, which rests on sharing (Section 16.4), would not exist.
2. **Elastic, dynamic resource allocation.** Mechanisms and policies must allocate resources on demand and reclaim them when they are released.
3. **Scale.** Management and resource allocation must work for datacenters of many thousands of machines and for many customers.
4. **Dealing with failures.** At this scale failures are the normal case, and the system must continue to operate despite them.[1]
5. **Multi-tenancy.** Tenants must be isolated from each other, and the performance of each of them must be guaranteed.
6. **Security.** The data and computations of every tenant must be protected from other tenants and from attackers outside the cloud.

[1] Google's study of more than 100 000 of its own disks measured an annualized failure rate of 1.7 % for drives in their first year and above 8 % for three-year-old ones [68]. At 5 % a year, a fleet of 100 000 disks loses about 14 of them every day—before any machine, switch or power supply is counted.

Definition 16.16 (Multi-tenancy). The sharing of the same physical resources by several customers, each of which uses its own, isolated set of resources, is called *multi-tenancy*.

Multi-tenancy requires isolation of the tenants' state, as a VMM provides it for its VMs (Lesson 12), but also isolation of their performance.

Definition 16.17 (Performance isolation). A system provides *performance isolation* when the

performance that a tenant obtains does not depend on the workloads of the other tenants that share its resources.

Without performance isolation, a tenant whose VM shares a machine with a VM that saturates the disk or the memory observes unpredictable response times, and the provider cannot keep the service level agreements (Lesson 6) it has promised.

Tip The performance interference of a co-located tenant is the "noisy neighbour" problem; performance isolation is what prevents it.

16.6.1 Failures at scale

Failures become frequent simply because of the number of components. If each of N components fails independently with probability p within a given period, the probability that at least one of them fails is

$$P_{\text{fail}} = 1 - (1-p)^N, \tag{16.1}$$

which approaches 1 as N grows, while the expected number of failed components, Np, grows linearly with N. A component that rarely fails on its own therefore fails somewhere in a datacenter all the time. Systems in the cloud must detect failures, for instance with timeouts, and continue despite them, with the mechanisms of the preceding lessons: replicas that take over for failed ones (Lesson 14), checkpoints from which computations resume (Lesson 8), and migration of VMs away from failing machines (Lesson 12).

Tip For small p, $(1-p)^N \approx e^{-Np}$, so $P_{\text{fail}} \approx 1 - e^{-Np}$: about Np while $Np \ll 1$, and above 95 % once $Np \geq 3$.

Caution Equation (16.1) assumes that the components fail independently. Correlated failures—a loss of power or cooling, a network partition, a faulty software update—take down many machines at once and are the ones that cause outages.

Example 16.18. A machine fails on a given day with probability $p = 0.001$, independently of the other machines; on its own, it fails on average once in about three years. By Equation (16.1), the probability that at least one machine fails on a given day is 1.0 % for 10 machines, 9.5 % for 100 machines and 63 % for 1000 machines. For 10 000 machines it is practically 100 %, and on average 10 machines fail every day.

> **Key point**
> A cloud requires fungible resources, elastic allocation, management at scale, tolerance of failures, and isolation of the state and the performance of its tenants. With many components, some component is almost always failing (Equation (16.1)).

16.7 Cloud enabling technologies

Source P4L4, cloud enabling technologies; Hindman et al. [69]; Dean and Ghemawat [70]; Zaharia et al. [71]; McKeown et al. [72].

Several technologies together make cloud computing possible.

Virtualization. Virtualization (Lesson 12) makes resources fungible. A physical machine can host the VMs of any tenants, with any operating systems and software stacks, and the VMs isolate the tenants from each other. VMs can be created, destroyed and migrated as the demand changes.

Resource provisioning. Resource managers for clusters, such as Mesos [69] and Hadoop YARN, allocate, or in Mesos offer, the resources of many machines to the frameworks that run on them, which place their tasks on these resources (two-level scheduling).

Hadoop is the open-source counterpart of the stack Google described: HDFS corresponds to the Google File System, Hadoop MapReduce to MapReduce, HBase to Bigtable.

Big data processing. Frameworks such as Hadoop MapReduce and Spark process large data sets in parallel on many machines.

Definition 16.19 (MapReduce). *MapReduce* [70] is a programming model, and a runtime system that implements it, for processing large data sets on a cluster. The programmer writes a *map* function, which transforms each input record into intermediate key/value pairs, and a *reduce* function, which combines all intermediate values with the same key. The runtime splits the input, runs map and reduce tasks in parallel on many machines, groups the intermediate pairs by key, and re-executes the tasks of machines that fail.

To count how often every word occurs in a collection of documents, for instance, the map function emits the pair $(w, 1)$ for every occurrence of a word w, and the reduce function sums the values for each word. MapReduce stores intermediate results on disk; Spark [71] keeps them in memory, which speeds up iterative algorithms and interactive queries that reuse the same data.

Dean and Ghemawat [70] report 29 423 MapReduce jobs at Google in August 2004, reading 3288 TB in total.

Storage. Clouds store data in distributed file systems (Lesson 14) designed for very large files that are mostly appended to rather than overwritten; in NoSQL databases, which scale out by giving up the relational data model and some of the consistency guarantees of traditional databases; and in distributed in-memory stores and caches, which keep data in the memory of many machines.

Such systems work in large pieces: GFS used chunks of 64 MB and HDFS uses blocks of 128 MB, each stored on three machines by default, so that a scan is sequential and one lost machine costs no data.

Software-defined networking, storage and datacenters. Networks, storage and eventually entire datacenters are configured by software through APIs rather than by hand.

Definition 16.20 (Software-defined networking). In *software-defined networking* (SDN), the *control plane* of a network, which decides how traffic is forwarded, is separated from the *data plane* of the switches, which forwards the packets. A logically centralized controller, implemented in software, programs the forwarding tables of the switches through an open interface such as OpenFlow [72].

OpenFlow, specified in 2008 and the first widely implemented such interface, lets the controller install match-and-act rules in the forwarding tables of a switch.

With SDN a cloud can reconfigure its network by software, for example to give every tenant a virtual network of its own that is isolated from those of other tenants. Software-defined storage and software-defined datacenters apply the same principle to storage and to all resources of a datacenter.

Monitoring. Thousands of machines and the services running on them continuously produce logs and measurements. Real-time log processing systems, such as Flume, CloudWatch or Log Insight, collect and analyze them as they are produced, in order to detect failures, anomalies and overloads, and to trigger elastic scaling.

16.8 The cloud as a big data engine

Many applications analyze data sets that no single machine can hold, such as the logs of user activity of a large Internet service, web pages to be indexed, or scientific measurements.

Source P4L4, the cloud as a big data engine, example big data stacks.

Definition 16.21 (Big data). Data sets whose size exceeds what a single machine can store and process, so that they must be stored and analyzed on many machines, are called *big data*.

The cloud provides both the storage for such data and the many machines that process them in parallel. The software that does so forms a *big data engine* with the following layers (Figure 16.5).

Data storage stores the data, distributed and replicated over many machines.
Data processing executes computations over the data in parallel on many machines.
Caching keeps frequently used data in memory, avoiding the latency of the storage layer.
Language front-ends let users query and analyze the data in high-level languages, for instance SQL-like query languages, instead of programming the processing layer directly.
Analytics libraries provide algorithms built on the processing layer, for instance for machine learning, graph analysis or statistics.
Streaming data such as logs, clicks or sensor readings, enter the engine continuously and must be processed as they arrive, not only after they have been stored.

The layers are orders of magnitude apart: a memory reference costs about 100 ns and a disk seek about 10 ms [66]; a random 4 kB read from a flash SSD takes about 100 µs to 150 µs (later extensions of the same list).

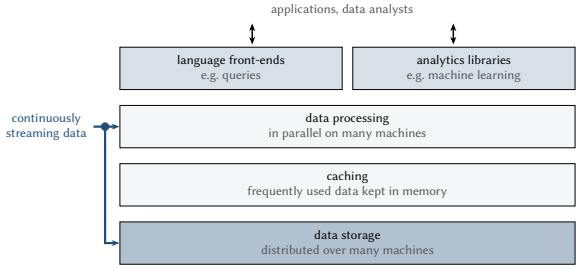


Figure 16.5: The layers of a big data engine. Streaming data enter the engine continuously; they are processed as they arrive and stored.

Two widely used open-source stacks implement these layers (Table 16.2). The *Hadoop* stack is built around the Hadoop Distributed File System (HDFS) and MapReduce. The *Berkeley Data*

Analytics Stack (BDAS), developed at the University of California, Berkeley, is built around Spark and the cluster resource manager Mesos, and it can use HDFS and other storage systems below it.

Some components have since been renamed or replaced: Tachyon became Alluxio in 2016, and Shark was superseded by Spark SQL. Table 16.2 keeps the original names.

Table 16.2: Components of two big data stacks, by layer.

Layer	Hadoop	BDAS
Data storage	HDFS (file system); HBase (column-oriented store)	HDFS, Amazon S3 and other storage systems
Caching	—	Tachyon (in-memory storage)
Resource management	YARN	Mesos, YARN
Data processing	MapReduce	Spark
Streaming data	Flume (log collection)	Spark Streaming
Language front-ends	Hive (SQL), Pig (data flow)	Shark (SQL), BlinkDB (approximate queries)
Analytics libraries	Mahout (machine learning), R connectors (statistics)	MLlib, MLbase (machine learning), GraphX (graphs)
Coordination	ZooKeeper (coordination), Oozie (workflows), Sqoop (data exchange with relational databases)	—

Key point — Lesson summary

Internet services consist of presentation, business logic and database tiers, and scale out over many nodes behind a load balancer; homogeneous nodes are simple to manage, while heterogeneous nodes exploit locality but need request-aware front ends and demand-driven management. Cloud computing provides shared resources elastically through APIs, billed by use and managed by the provider. It works because of the law of large numbers and economies of scale, and it approaches the vision of utility computing. Clouds are public, private, hybrid or community clouds, and offer infrastructure, platforms or software as a service. They require fungible resources, elastic allocation at scale, tolerance of frequent failures and isolation of their tenants, which virtualization, cluster resource management, big data frameworks such as MapReduce, distributed storage, software-defined networking and monitoring provide. On these technologies the cloud serves as a big data engine.

Bibliography

[1] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. 1.10. Arpaci-Dusseau Books, 2023. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[2] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. 10th ed. Wiley, 2018.

[3] Livio Soares and Michael Stumm. "FlexSC: Flexible System Call Scheduling with Exception-Less System Calls". In: *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2010, pp. 33–46.

[4] Gerwin Klein et al. "seL4: Formal Verification of an OS Kernel". In: *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*. 2009, pp. 207–220.

[5] Jochen Liedtke. "Improving IPC by Kernel Design". In: *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*. 1993, pp. 175–188.

[6] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. 4th ed. Pearson, 2015.

[7] Chuanpeng Li, Chen Ding, and Kai Shen. "Quantifying the Cost of Context Switch". In: *Proceedings of the 2007 Workshop on Experimental Computer Science (ExpCS)*. San Diego, CA: ACM, June 2007.

[8] Andrew D. Birrell. *An Introduction to Programming with Threads*. SRC Research Report 35. Palo Alto, CA: Digital Equipment Corporation, Systems Research Center, Jan. 1989.

[9] Ulrich Drepper and Ingo Molnar. *The Native POSIX Thread Library for Linux*. White paper. Red Hat, Inc., Feb. 2005.

[10] Butler W. Lampson and David D. Redell. "Experience with Processes and Monitors in Mesa". In: *Communications of the ACM* 23.2 (1980), pp. 105–117.

[11] Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco: No Starch Press, 2010.

[12] *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications, Issue 7*. IEEE Std 1003.1-2017. IEEE and The Open Group. 2018.

[13] Ulrich Drepper. *Futexes Are Tricky*. Revision 1.6. 2011. URL: https://www.akkadia.org/drepper/futex.pdf.

[14] J. R. Eykholt et al. "Beyond Multiprocessing: Multithreading the SunOS Kernel". In: *Proceedings of the Summer 1992 USENIX Technical Conference*. San Antonio, TX, June 1992.

[15] Dan Stein and Devang Shah. "Implementing Lightweight Threads". In: *Proceedings of the Summer 1992 USENIX Technical Conference*. San Antonio, TX, June 1992.

[16] Thomas E. Anderson et al. "Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism". In: *ACM Transactions on Computer Systems* 10.1 (Feb. 1992), pp. 53–79.

[17] Vivek S. Pai, Peter Druschel, and Willy Zwaenepoel. "Flash: An Efficient and Portable Web Server". In: *Proceedings of the 1999 USENIX Annual Technical Conference*. Monterey, CA: USENIX Association, June 1999, pp. 199–212.

[18] Jeffrey Dean and Luiz André Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (Feb. 2013), pp. 74–80.

[19] Alexandra Fedorova et al. "Chip Multithreading Systems Need a New Operating System Scheduler". In: *Proceedings of the 11th ACM SIGOPS European Workshop*. 2004.

[20] Fernando J. Corbató, Marjorie Merwin-Daggett, and Robert C. Daley. "An Experimental Time-Sharing System". In: *Proceedings of the AFIPS Spring Joint Computer Conference*. 1962.

[21] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. 3rd ed. O'Reilly Media, 2005.

[22] Alexandra Fedorova et al. "Performance of Multithreaded Chip Multiprocessors and Implications for Operating System Design". In: *Proceedings of the USENIX Annual Technical Conference*. 2005.

[23] Jeff Bonwick. "The Slab Allocator: An Object-Caching Kernel Memory Allocator". In: *Proceedings of the USENIX Summer 1994 Technical Conference*. 1994, pp. 87–98.

[24] Christopher Clark et al. "Live Migration of Virtual Machines". In: *Proceedings of the 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '05)*. 2005, pp. 273–286.

[25] Edsger W. Dijkstra. *Cooperating Sequential Processes*. Technical Report EWD-123. Eindhoven, The Netherlands: Technological University Eindhoven, 1965.

[26] C. A. R. Hoare. "Monitors: An Operating System Structuring Concept". In: *Communications of the ACM* 17.10 (1974), pp. 549–557.

[27] Carl E. Hewitt and Russell R. Atkinson. "Specification and Proof Techniques for Serializers". In: *IEEE Transactions on Software Engineering* SE-5.1 (1979), pp. 10–23.

[28] Roy H. Campbell and A. Nico Habermann. "The Specification of Process Synchronization by Path Expressions". In: *Operating Systems, International Symposium*. Vol. 16. Lecture Notes in Computer Science. Springer, 1974, pp. 89–102.

[29] Maurice Herlihy. “Wait-Free Synchronization”. In: *ACM Transactions on Programming Languages and Systems* 13.1 (1991), pp. 124–149.

[30] Paul E. McKenney and John D. Slingwine. “Read-Copy Update: Using Execution History to Solve Concurrency Problems”. In: *Proceedings of the International Conference on Parallel and Distributed Computing and Systems (PDCS)*. Las Vegas, NV, 1998.

[31] Thomas E. Anderson. “The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors”. In: *IEEE Transactions on Parallel and Distributed Systems* 1.1 (1990), pp. 6–16.

[32] Gerald J. Popek and Robert P. Goldberg. “Formal Requirements for Virtualizable Third Generation Architectures”. In: *Communications of the ACM* 17.7 (1974), pp. 412–421.

[33] Mendel Rosenblum and Tal Garfinkel. “Virtual Machine Monitors: Current Technology and Future Trends”. In: *Computer* 38.5 (2005), pp. 39–47.

[34] Luiz André Barroso and Urs Hölzle. *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2009.

[35] Paul Barham et al. “Xen and the Art of Virtualization”. In: *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. Bolton Landing, NY, 2003, pp. 164–177.

[36] Avi Kivity et al. “kvm: the Linux Virtual Machine Monitor”. In: *Proceedings of the Linux Symposium*. Vol. 1. Ottawa, Canada, 2007, pp. 225–230.

[37] John Scott Robin and Cynthia E. Irvine. “Analysis of the Intel Pentium’s Ability to Support a Secure Virtual Machine Monitor”. In: *Proceedings of the 9th USENIX Security Symposium*. Denver, CO, Aug. 2000.

[38] Edouard Bugnion et al. “Bringing Virtualization to the x86 Architecture with the Original VMware Workstation”. In: *ACM Transactions on Computer Systems* 30.4 (2012), 12:1–12:51.

[39] Keith Adams and Ole Agesen. “A Comparison of Software and Hardware Techniques for x86 Virtualization”. In: *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. San Jose, CA, 2006, pp. 2–13.

[40] *Single Root I/O Virtualization and Sharing Specification, Revision 1.0*. PCI-SIG. 2007.

[41] Rusty Russell. “virtio: Towards a De-Facto Standard for Virtual I/O Devices”. In: *ACM SIGOPS Operating Systems Review* 42.5 (2008), pp. 95–103.

[42] Rich Uhlig et al. “Intel Virtualization Technology”. In: *Computer* 38.5 (2005), pp. 48–56.

[43] Advanced Micro Devices. *AMD-V Nested Paging*. White paper. Advanced Micro Devices, Inc., July 2008.

[44] Andrew D. Birrell and Bruce Jay Nelson. “Implementing Remote Procedure Calls”. In: *ACM Transactions on Computer Systems* 2.1 (Feb. 1984), pp. 39–59.

[45] Robert Thurlow. *RPC: Remote Procedure Call Protocol Specification Version 2*. RFC 5531. Internet Engineering Task Force, May 2009.

[46] Mike Eisler. *XDR: External Data Representation Standard*. RFC 4506. Internet Engineering Task Force, May 2006.

[47] R. Srinivasan. *Binding Protocols for ONC RPC Version 2*. RFC 1833. Internet Engineering Task Force, Aug. 1995.

[48] Thomas Haynes and David Noveck. *Network File System (NFS) Version 4 Protocol*. RFC 7530. Internet Engineering Task Force, Mar. 2015.

[49] Mike Eisler, Alex Chiu, and Lin Ling. *RPCSEC_GSS Protocol Specification*. RFC 2203. Internet Engineering Task Force, Sept. 1997.

[50] David Noveck and Charles Lever. *Network File System (NFS) Version 4 Minor Version 1 Protocol*. RFC 8881. Internet Engineering Task Force, Aug. 2020.

[51] US-CERT. *UDP-Based Amplification Attacks*. Alert TA14-017A. Jan. 2014. URL: https://www.cisa.gov/news-events/alerts/2014/01/17/udp-based-amplification-attacks.

[52] Ann Wollrath, Roger Riggs, and Jim Waldo. “A Distributed Object Model for the Java System”. In: *Proceedings of the 2nd USENIX Conference on Object-Oriented Technologies and Systems (COOTS)*. Toronto, Canada, June 1996.

[53] Michael N. Nelson, Brent B. Welch, and John K. Ousterhout. “Caching in the Sprite Network File System”. In: *ACM Transactions on Computer Systems* 6.1 (Feb. 1988), pp. 134–154.

[54] Russel Sandberg et al. “Design and Implementation of the Sun Network Filesystem”. In: *Proceedings of the Summer 1985 USENIX Conference*. Portland, OR, June 1985, pp. 119–130.

[55] Brent Callaghan, Brian Pawlowski, and Peter Staubach. *NFS Version 3 Protocol Specification*. RFC 1813. Internet Engineering Task Force, June 1995.

[56] John K. Ousterhout et al. “A Trace-Driven Analysis of the UNIX 4.2 BSD File System”. In: *Proceedings of the Tenth ACM Symposium on Operating Systems Principles (SOSP)*. Orcas Island, WA, Dec. 1985, pp. 15–24.

[57] Jelica Protić, Milo Tomašević, and Veljko Milutinović. “Distributed Shared Memory: Concepts and Systems”. In: *IEEE Parallel & Distributed Technology: Systems & Applications* 4.2 (1996), pp. 63–79.

[58] Bill Nitzberg and Virginia Lo. “Distributed Shared Memory: A Survey of Issues and Algorithms”. In: *Computer* 24.8 (Aug. 1991), pp. 52–60.

[59] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms*. 2nd ed. Pearson Prentice Hall, 2007.

[60] Kai Li and Paul Hudak. “Memory Coherence in Shared Virtual Memory Systems”. In: *ACM Transactions on Computer Systems* 7.4 (Nov. 1989), pp. 321–359.

[61] Pete Keleher et al. “TreadMarks: Distributed Shared Memory on Standard Workstations and Operating Systems”. In: *Proceedings of the Winter 1994 USENIX Conference*. 1994, pp. 115–131.

[62] Andrew W. Appel and Kai Li. “Virtual Memory Primitives for User Programs”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*. 1991, pp. 96–107.

[63] Leslie Lamport. “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs”. In: *IEEE Transactions on Computers* C-28.9 (Sept. 1979), pp. 690–691.

[64] Michael Armbrust et al. *Above the Clouds: A Berkeley View of Cloud Computing*. Technical Report UCB/EECS-2009-28. EECS Department, University of California, Berkeley, Feb. 2009.

[65] Peter Mell and Timothy Grance. *The NIST Definition of Cloud Computing*. Special Publication 800-145. National Institute of Standards and Technology, Sept. 2011.

[66] Jeffrey Dean. *Designs, Lessons and Advice from Building Large Distributed Systems*. Keynote, 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware (LADIS), Big Sky, MT. Oct. 2009.

[67] David Karger et al. “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web”. In: *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC)*. El Paso, TX, May 1997, pp. 654–663.

[68] Eduardo Pinheiro, Wolf-Dietrich Weber, and Luiz André Barroso. “Failure Trends in a Large Disk Drive Population”. In: *Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST)*. San Jose, CA, Feb. 2007, pp. 17–28.

[69] Benjamin Hindman et al. “Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center”. In: *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. Boston, MA, 2011.

[70] Jeffrey Dean and Sanjay Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters”. In: *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. San Francisco, CA, Dec. 2004, pp. 137–150.

[71] Matei Zaharia et al. “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing”. In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. San Jose, CA, 2012.

[72] Nick McKeown et al. “OpenFlow: Enabling Innovation in Campus Networks”. In: *ACM SIGCOMM Computer Communication Review* 38.2 (Apr. 2008), pp. 69–74.

Index

A
abstraction, 1
access bit, 83, 168
active array, 73
adaptive mutex, 46, 104
address space, 9
address translation, 80
allocation on first touch, 82
AMPED, *see* asymmetric multi-process event-driven model
AMTED, *see* asymmetric multithreaded event-driven model
Apache, 61
arbitration, 2
asymmetric multi-process event-driven model, 59
asymmetric multithreaded event-driven model, 59
asynchronous I/O, 59, 120
asynchronous signal, 48
atomic instruction, 109
attach, 97

B
back-end driver, 136
bare-metal hypervisor, 130
barrier, 107
baseline, 64
benchmark, 64
big data, 181
big-endian, 140
binary semaphore, 104
binary translation, 133
binding, 142
block device, 116
block group, 123
boss-workers pattern, 29, 174
bottom half, 49
bound thread, 28
bounded buffer, 38
broadcast operation, 22
buddy allocator, 88
buffer cache, 125
busy waiting, 103

C
cache affinity, 75
cache coherence, 111, 154, 164
causal consistency, 170
CFS, *see* Completely Fair Scheduler
character device, 116
checkpointing, 92, 153, 180
child process, 13
chip multithreading, 76
clone, 52
close-to-open consistency, 158
cloud computing, 177
CMT, *see* chip multithreading
cold cache, 12
community cloud, 178
compare-and-swap, 109
Completely Fair Scheduler, 74
concurrency, 20
concurrent write sharing, 160
condition variable, 22, 37
consistency model, 170
consolidation, 129
context switch, 12
copy-on-write, 92
counting semaphore, 104
COW, *see* copy-on-write
CPI, *see* cycles per instruction
CPU scheduler, 15
CPU utilization, 55
CPU-bound task, 70
critical section, 21
cycles per instruction, 78

D
data segment, 9
datacenter, 173
datagram socket, 96
deadlock, 26
deadlock detection and recovery, 27
deadlock prevention, 27
delayed write, 160
delegation, 158
demand paging, 90
dentry, *see* directory entry
dentry cache, 123
detach, 98
detached thread, 33
device driver, 116
device file, 116
device register, 115
DFS, *see* distributed file system
direct memory access, 118, 163
direct pointer, 124
directory entry, 123
dirty bit, 83, 168
dispatch, 15
distributed file system, 151
distributed shared memory, 162
DMA, *see* direct memory access
dom0, 130
double indirect pointer, 124
DSM, *see* distributed shared memory

E
economies of scale, 178
elasticity, 176
elevator algorithm, 126
entry procedure, 106
EPT, *see* extended page table
event dispatcher, 57
event handler, 57

event-driven model, 57
`exec`, 14
execution time, 55
expired array, 73
ext2, 123
extended page table, 137
external data representation, 144
external fragmentation, 88

F
false sharing, 164
FCFS, *see* first-come first-served scheduling
fidelity, 129
file descriptor, 122
file handle, 158
file system, 121
first-come first-served scheduling, 67
Flash, 60
`fork`, 14
`fork`, 91
front-end driver, 136
full virtualization, 133

G
generation number, 158
generic block layer, 121
guest operating system, 128

H
hard process state, 42
hardware performance counter, 78
hardware-assisted virtualization, 137
hashed page table, 85
heap, 9
helper, 59
heterogeneous architecture, 175
homogeneous architecture, 174
host operating system, 130
hosted hypervisor, 130
hot cache, 12, 175
huge page, 87
hybrid cloud, 178
hypercall, 133
hyperthreading, *see* simultaneous multithreading
hypervisor, *see* virtual machine monitor
hypervisor-direct model, 136

I
I/O port, 117
I/O scheduling, 126
I/O-bound task, 71
IaaS, *see* infrastructure as a service
IDL, *see* interface definition language
immutable file, 155
indirect pointer, 124
infrastructure as a service, 178
`init`, 13
inode, 122
inter-process communication, 16, 94
interface definition language, 141
internal fragmentation, 87
Internet service, 173
interrupt, 46
interrupt handler, 47
interrupt handler table, 47
interrupt mask, 48
inverted page table, 85
`ioctl`, 121
IPC, *see* inter-process communication
isolation, 2, 179

J
Java RMI, *see* remote method invocation
join, 20
joinable thread, 32
journal, 126
journaling, 126

K
kernel, 3
kernel mode, 3
kernel module, 6
Kernel-based Virtual Machine, 130
kernel-level thread, 27
KVM, *see* Kernel-based Virtual Machine

L
large page, 87
law of large numbers, 177
layered pattern, 30
lease, 155
least recently used, 91
light process state, 42
lightweight process, 42
little-endian, 140
load balancer, 174
load balancing, 76
local procedure call, 97
lock contention, 112
lock ordering, 27, 37
LPC, *see* local procedure call
LRU, *see* least recently used
LWP, *see* lightweight process

M
machine address, 134
many-to-many model, 28
many-to-one model, 28
MapReduce, 180
marshalling, 142
mechanism, 2
memory allocator, 87
memory management unit, 81
memory-bound task, 77
memory-mapped file, 99
memory-mapped I/O, 117
message passing, 16, 94
message queue, 95
message signaled interrupt, 46
microkernel, 7
MLFQ, *see* multi-level feedback queue
MMU, *see* memory management unit

mode bit, 3
modular operating system, 6
monitor, 106
monolithic operating system, 6
mount, 157
MRMW, *see* multiple readers/multiple writers
MRSW, *see* multiple readers/single writer
MSI, *see* message signaled interrupt
multi-level feedback queue, 72
multi-level page table, 84
multi-process model, 56
multi-tenancy, 179
multi-tier architecture, 173
multicore processor, 75
multiple readers/multiple writers, 164
multiple readers/single writer, 164
multithreaded model, 57
multithreading, 18
mutex, 21, 36
mutual exclusion, 21

N
named pipe, 95
Native POSIX Thread Library, 52
network device, 116
Network File System, 157
new state, 13
NFS, *see* Network File System
nice value, 73
non-root mode, 131
non-uniform memory access, 76
NPTL, *see* Native POSIX Thread Library
NUMA, *see* non-uniform memory access
NUMA-aware scheduling, 76

O
O(1) scheduler, 73
offset, 82
ONC RPC, *see* Sun RPC
one-shot signal, 49
one-to-one model, 28
operating system, 1
operating system bypass, 119, 136, 163
optimize for the common case, 3, 156
OS, *see* operating system
ostrich algorithm, 27

P
PaaS, *see* platform as a service
page, 80
page cache, 125
page directory, 84
page fault, 83, 168
page frame, 80
page replacement, 91
page table, 10, 81
page table base register, 82
page table entry, 82
page-out daemon, 91
paging, 80
parallelization, 19
paravirtualization, 133
parent process, 13
partial failure, 143
partitioning, 156
passthrough model, 136
path expression, 107
PC, *see* program counter
PCB, *see* process control block
PCI, *see* Peripheral Component Interconnect
peer-to-peer, 151, 162
performance isolation, 179
performance metric, 55
Peripheral Component Interconnect, 116
PFN, *see* physical frame number
physical address, 10
physical frame number, 82, 167
pinned page, 90, 118
PIO, *see* programmed I/O
pipe, 95
pipeline pattern, 29
platform as a service, 179
policy, 2
polling, 117
port, 94
Portable Operating System Interface, 31
portmapper, *see* rpcbind
POSIX, *see* Portable Operating System Interface
POSIX shared memory, 99
POSIX threads, 31
preemption, 15
preemptive scheduling, 67
prefetching, 126, 163
priority, 68
priority aging, 68
priority inheritance, 69
priority inversion, 68
priority scheduling, 68
private cloud, 178
privileged instruction, 3
process, 9
process control block, 11
process migration, 92
process scope, 28, 32
process state, 13
process-shared attribute, 99
producer-consumer problem, 22, 38
program counter, 11
programmed I/O, 118
protection, 2
protection bits, 83, 168
protection ring, 131
proxy variable, 23
pseudo device, 117
PTE, *see* page table entry
Pthreads, *see* POSIX threads
public cloud, 178

Q
QEMU, 130
queueing lock, 113

R
race condition, 20, 35
RCU, *see* read-copy-update
RDMA, *see* remote direct memory access
read-copy-update, 108
reader-writer lock, 105, 159
readers-writers problem, 23
ready queue, 15
ready state, 13
real-time signal, 49
reaper thread, 46
red zone, 43
red-black tree, 74
registry, 142
remote access model, 152
remote direct memory access, 163
remote method invocation, 148
remote procedure call, 139
rendezvous point, 107
replication, 156, 165
response time, 55
RMI, *see* remote method invocation
root mode, 131
round robin scheduling, 69
RPC, *see* remote procedure call
RPC runtime, 140
rpcbind, 146
rpcgen, 145
run-to-completion scheduling, 67
running state, 13
runqueue, 66

S
SaaS, *see* software as a service
scale-out, 173
SDN, *see* software-defined networking
second chance algorithm, 91
segment, 80
segmentation, 80
semaphore, 104
separation of mechanism and policy, 3
sequential consistency, 170
sequential write sharing, 160
serialization, *see* marshalling
serializer, 107
service level agreement, 55, 180
service VM, 130
session semantics, 155
shadow page table, 134
shared memory, 16, 96
shared memory multiprocessor, 75, 110, 162
shared memory object, 99
shared memory segment, 97
shortest job first, 67
signal, 4
signal handler, 47
signal mask, 48
signal operation, 22
simultaneous multithreading, 76
single reader/single writer, 164
single-process event-driven model, 57
SJF, *see* shortest job first
SLA, *see* service level agreement
slab allocator, 89
SMT, *see* simultaneous multithreading
socket, 95
software as a service, 179
software-defined networking, 181
specialization, 19, 175
SPED, *see* single-process event-driven model
spinlock, 103
split-device driver model, 136
Sprite, 159
spurious wake-up, 26, 37
SRSW, *see* single reader/single writer
stack, 9
stack pointer, 11
stale file handle, 158
starvation, 68
stateful server, 153
stateless server, 153
stream socket, 96
strict consistency, 170
stub, 140
Sun RPC, 144, 157
superblock, 123
swap partition, 90
swapping, 90
synchronization, 20
synchronous I/O, 120
synchronous signal, 48
synthetic workload, 62
system call, 4
system scope, 28, 32
System V IPC, 97

T
tagged TLB, 137
task, 51
terminated state, 13
test-and-set, 109
test-and-test-and-set, 112
testbed, 55
text segment, 9
thread, 18
thread attribute, 32
thread creation, 20
thread pool, 29
throughput, 55
timesharing, 70
timeslice, 15, 70
TLB, *see* translation lookaside buffer
top half, 49
trace, 62
translation lookaside buffer, 85
trap, 3
trap-and-emulate, 131

U
UNIX semantics, 155
Unix-domain socket, 96
unmarshalling, 142
upload/download model, 151

user mode, 3
user-level driver, 119
user-level thread, 27
utility computing, 178

V
valid bit, 83
VFS, *see* virtual file system
virtual address, 10
virtual file system, 122
virtual machine, 128
virtual machine monitor, 128
virtual memory, 80
virtual page number, 82
virtual runtime, 74
virtualization, 128
VM, *see* virtual machine
VM control structure, 137
VM entry, 131
VM exit, 131
VMCS, *see* VM control structure
VMM, *see* virtual machine monitor
VPN, *see* virtual page number

W
wait graph, 26
wait operation, 22
wait time, 55
wait-free synchronization, 108
waiting state, 13
weak consistency, 171
workload, 62, 175
write-invalidate, 111
write-update, 111

X
XDR, *see* external data representation
Xen, 130

Z
Zygote, 14